



DevOps Tools Engineer

Version 2.0
English

701

Table of Contents

TOPIC 701: SOFTWARE ENGINEERING	1
701.1 Modern Software Development	2
Lesson 1	4
Introduction	4
What is Agile?	5
Agile Methodologies	6
Agile in Practice: Lifecycle and Artifacts	8
Benefits of Agile in Modern Software Development	9
What is DevOps?	10
Agile and DevOps	12
Guided Exercises	13
Explorational Exercises	14
Summary	15
Answers to Guided Exercises	16
Answers to Explorational Exercises	17
Lesson 2	18
Introduction	18
What Does “Cloud Native” Mean?	18
Service-Oriented Architecture (SOA)	22
Designing Software to be Deployed to Cloud Services	25
Guided Exercises	27
Explorational Exercises	28
Summary	29
Answers to Guided Exercises	30
Answers to Explorational Exercises	31
Lesson 3	32
Introduction	32
Microservices Architecture	32
Common API Concepts and Standards: REST and JSON	33
Data Storage, Service Status, and Session Handling in Cloud Applications	39
Guided Exercises	42
Explorational Exercises	43
Summary	44
Answers to Guided Exercises	45
Answers to Explorational Exercises	46
701.2 Standard Components and Platforms for Software	47
Lesson 1	49
Introduction	49

Features and Concepts of Object Storage	50
Features and Concepts of Relational and NoSQL Databases	51
Features and Concepts of Message Brokers and Message Queues	52
Features and Concepts of Big Data Services	53
Features and Concepts of Computing Services: IaaS	53
Features and Concepts of Application Runtimes: PaaS	54
Features and Concepts of Hosted Applications: SaaS	55
Features and Concepts of Function Applications: FaaS	55
Features and Concepts of Content Delivery Networks (CDNs)	56
Guided Exercises	57
Explorational Exercises	58
Summary	59
Answers to Guided Exercises	60
Answers to Explorational Exercises	61
701.3 Source Code Management	62
Lesson 1	63
Introduction	63
Concepts of Software Configuration Management (SCM)	63
Git's Contribution to Version Control	64
How Git Works: Remote Repository, Local Repository, and Staging Area	64
Git Structure and Internal Storage	68
Cloning a Git Repository	70
Initializing a Local Git Repository and Connecting to a Remote	72
Managing Files Within a Git Repository	74
The <code>.gitignore</code> File: Purpose and Syntax	78
Guided Exercises	80
Explorational Exercises	81
Summary	82
Answers to Guided Exercises	83
Answers to Explorational Exercises	85
Lesson 2	87
Introduction	87
Tags: Marking Specific Versions	87
Branches: Working in Parallel	90
Temporary Change Management with <code>git stash</code> and <code>git pop</code>	99
Guided Exercises	103
Explorational Exercises	104
Summary	105
Answers to Guided Exercises	106
Answers to Explorational Exercises	108

Lesson 3	110
Introduction	110
Configuring a Remote Repository	110
Sharing Changes With <code>git push</code>	111
Getting Updates With <code>git fetch</code>	111
Updating Local Branch With <code>git pull</code>	112
Force Pushing With <code>git push --force</code>	112
Force Pulling With <code>git fetch</code> and <code>git reset --hard</code>	113
Removing Untracked Files	113
Reverting to a Specific Commit Using a New Branch	114
How to Find the Commit Hash	115
Returning to a Specific Commit Without Creating a New Branch	115
Resetting the Current Branch to a Specific Commit	116
Understanding <code>git rebase</code>	116
Submodules: Managing Nested Git Repositories	120
Guided Exercises	122
Explorational Exercises	123
Summary	124
Answers to Guided Exercises	125
Answers to Explorational Exercises	126
701.4 Continuous Integration and Continuous Delivery	128
Lesson 1	130
Introduction	130
Continuous Integration and Continuous Delivery (CI/CD)	131
Components of a CI/CD Pipeline	132
Build Stage	132
Unit Testing	133
Integration Testing	133
Acceptance Testing	133
Artifact Management	133
Delivery and Deployment	133
Deployment Strategies and Best Practices	134
Blue-Green Deployment	134
Canary Deployment	134
A/B Testing	134
Preview Releases	135
Feature Toggles	135
Reconciliation Loops	135
GitOps	135
Semantic Versioning	136

Build Artifacts and Caches	136
Declarative Pipelines	137
CI/CD Tools	137
Guided Exercises	138
Explorational Exercises	139
Summary	140
Answers to Guided Exercises	141
Answers to Explorational Exercises	143
701.5 Software Composition, Licensing and Open Source	144
Lesson 1	145
Introduction	145
How an Application is Built Out of Multiple Software Components	146
The Effect of Software Licensing on Application Development	147
Proprietary and Open Source Software	148
Open Source Software and Free Software	148
Open Source Software Licenses	149
License Compatibility and Multi-Licensing	151
Dependencies and Package Management	151
Software Libraries	152
Software Bill of Materials	152
Guided Exercises	153
Explorational Exercises	154
Summary	155
Answers to Guided Exercises	156
Answers to Explorational Exercises	158
TOPIC 702: APPLICATION CONTAINER	159
702.1 Application Container Management	160
Lesson 1	162
Introduction	162
Foundations of Containerization	162
Docker and Podman Architecture	163
Using Existing Images from an OCI Registry	165
Operating and Accessing Containers	167
Guided Exercises	174
Explorational Exercises	175
Lesson 2	179
Introduction	179
Docker Networking	180
Docker Storage	188
Rootless Containers	193

Guided Exercises	195
Explorational Exercises	196
Summary	197
Answers to Guided Exercises	198
Answers to Explorational Exercises	200
702.2 Container Orchestration	202
Lesson 1	203
Introduction	203
Application Model for Docker Compose and Podman Compose	203
Defining Services, Networks, and Volumes in Docker Compose	209
Guided Exercises	217
Explorational Exercises	218
Summary	219
Answers to Guided Exercises	220
Answers to Explorational Exercises	222
702.3 Container Image Building	224
Lesson 1	226
Introduction	226
Structure of a Dockerfile	227
Guided Exercises	239
Explorational Exercises	240
Summary	241
Answers to Guided Exercises	242
Answers to Explorational Exercises	244
Lesson 2	245
Introduction	245
Building Images With <code>docker build</code>	245
OCI Image Names	246
Uploading Images to a Docker Registry	248
Image Scanners	250
Security Risks and How to Mitigate Them	251
Docker Buildx, Docker BuildKit, and Buildah	254
Guided Exercises	255
Explorational Exercises	256
Summary	257
Answers to Guided Exercises	258
Answers to Explorational Exercises	261
TOPIC 703: KUBERNETES	263
703.1 Kubernetes Architecture and Usage	264
Lesson 1	266

Introduction	266
Kubernetes Architecture	267
Control Plane	267
Worker Node Components	271
Auxiliary Services	272
Guided Exercises	273
Explorational Exercises	274
Summary	275
Answers to Guided Exercises	276
Answers to Explorational Exercises	277
Lesson 2	278
Introduction	278
Introducing <code>kubectl</code>	278
The <code>kubeconfig</code> File	279
Context in Kubernetes	279
Starting a Single Pod: <code>kubectl run</code>	280
Retrieving Resource Information: <code>kubectl get</code>	280
Detailed Resource Inspection: <code>kubectl describe</code>	282
Declarative Resource Management: <code>kubectl apply</code>	283
Imperative Resource Creation: <code>kubectl create</code>	284
Creating a Service: <code>kubectl expose</code>	285
Changing Replica Count: <code>kubectl scale</code>	285
Updating Resource Fields: <code>kubectl set</code>	286
Editing Live Resource Definitions: <code>kubectl edit</code>	286
Viewing Resource Schema: <code>kubectl explain</code>	286
Managing the <code>kubeconfig</code> File: <code>kubectl config</code>	287
Viewing Pod Logs: <code>kubectl logs</code>	287
Running Commands Inside a Container: <code>kubectl exec</code>	287
Configuring <code>kubectl</code> to Use an Existing Kubernetes Cluster	288
Kubernetes Operators	289
Guided Exercises	291
Explorational Exercises	292
Summary	293
Answers to Guided Exercises	294
Answers to Explorational Exercises	296
703.2 Basic Kubernetes Operations	298
Lesson 1	299
Introduction	299
Pods	299
Deployments: Pod and ReplicaSet Management	301

Using YAML Files to Declare Kubernetes Resources	301
How to Use Deployments in Kubernetes	304
ReplicaSets	307
Managing Application Configuration With ConfigMaps and Secrets in Kubernetes	308
Guided Exercises	313
Explorational Exercises	314
Summary	315
Answers to Guided Exercises	316
Answers to Explorational Exercises	320
Lesson 2	322
Introduction	322
The Role of Services	322
Defining a Service in Kubernetes	323
Ingresses in Kubernetes	329
Guided Exercises	333
Explorational Exercises	334
Summary	336
Answers to Guided Exercises	337
Answers to Explorational Exercises	341
Lesson 3	346
Introduction	346
Persistent Storage in Kubernetes	347
Lifecycle of a PersistentVolume and a PersistentVolumeClaim	348
Managing Persistent Volumes and Claims in Kubernetes	349
The Retain Reclaim Policy	351
Other Kubernetes Orchestration Resources	355
Guided Exercises	357
Explorational Exercises	358
Summary	359
Answers to Guided Exercises	360
Answers to Explorational Exercises	364
703.3 Kubernetes Package Management	366
Lesson 1	367
Introduction	367
Helm: Package Management for Kubernetes Applications	368
Installing, Upgrading, and Uninstalling Software Using Helm	371
Finding Charts With <code>helm search</code>	377
Kustomize	378
Flux CD and Argo CD	378
Guided Exercises	380

Explorational Exercises	381
Summary	382
Answers to Guided Exercises	383
Answers to Explorational Exercises	384
TOPIC 704: SECURITY AND OBSERVABILITY	386
704.1 Cloud Native Security	387
Lesson 1	389
Introduction	389
The Importance of IT Infrastructure in Deployment	389
Common IT Infrastructure Security Risks and Ways to Mitigate Them	391
Common Forms of Application-Level Security	393
Supply Chain Security and Dependencies on Foreign Code	394
Guided Exercises	398
Explorational Exercises	399
Summary	400
Answers to Guided Exercises	401
Answers to Explorational Exercises	402
Lesson 2	403
Introduction	403
Concepts of Asymmetric Cryptography and Digital Certificates	404
Principles of Common Standards for Authentication and Authorization	406
How to Manage User Credentials and Use Advanced Authentication Technologies	408
Guided Exercises	411
Explorational Exercises	412
Summary	413
Answers to Guided Exercises	414
Answers to Explorational Exercises	415
704.2 Prometheus Monitoring	416
Lesson 1	418
Introduction	418
IT Operational Goals and Service Monitoring	418
Metrics for Monitoring Technical Functionality	422
Metrics for Monitoring Logical Functionality	424
Guided Exercises	427
Explorational Exercises	428
Summary	429
Answers to Guided Exercises	430
Answers to Explorational Exercises	432
Lesson 2	433
Introduction	433

Prometheus Concepts and Architecture	433
Installing Prometheus	439
The prometheus.yml Configuration File	441
Starting Prometheus	443
Exploring Metrics With the Expression Browser	444
Using the Graph Interface	445
Guided Exercises	447
Explorational Exercises	448
Summary	449
Answers to Guided Exercises	450
Answers to Explorational Exercises	451
Lesson 3	453
Introduction	453
Grafana: Visualizing Time-Series Metrics	454
Monitoring Containers and Microservices Using Prometheus	455
Common Exporters in Prometheus	466
Advanced PromQL: Metrics Aggregation and Querying over Time and Labels	467
Application Instrumentation	468
Thanos	469
Guided Exercises	470
Explorational Exercises	471
Summary	472
Answers to Guided Exercises	473
Answers to Explorational Exercises	474
704.3 Log Management and Analysis	475
Lesson 1	476
Introduction	476
Application and System Logging	477
Architecture and Features of Open Source Logging Stacks	479
Open Source Logging Stacks	480
Guided Exercises	488
Explorational Exercises	489
Summary	490
Answers to Guided Exercises	491
Answers to Explorational Exercises	492
704.4 Tracing	494
Lesson 1	495
Introduction	495
Concepts of Tracing	495
Application Instrumentation	499

Distributed Tracing with Jaeger and Grafana Tempo	502
Guided Exercises	504
Explorational Exercises	505
Summary	506
Answers to Guided Exercises	507
Answers to Explorational Exercises	509
Imprint	511



**Linux
Professional
Institute**

Topic 701: Software Engineering



701.1 Modern Software Development

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 701.1

Weight

6

Key knowledge areas

- Understand and design service based applications
- Understand common API concepts and standards
- Understand aspects of data storage, service status and session handling
- Understand the properties of cloud native applications
- Design software to be run in containers
- Design software to be deployed to cloud services
- Awareness of risks in the migration and integration of monolithic legacy software
- Awareness of database schema updates and database migrations
- Understand the concept of agile software development
- Understand the concept of DevOps and its implications to software developers and operators

Partial list of the used files, terms and utilities

- REST, JSON
- Service Oriented Architectures (SOA)
- Microservices
- Immutable servers

- Loose coupling
- Test-driven development



Linux
Professional
Institute

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.1 Modern Software Development
Lesson:	1 of 3

Introduction

This lesson introduces the foundations of Agile and DevOps, two complementary approaches that have transformed modern software engineering.

DevOps — combining the terms “development” and “operations” — emerged in the early 2000s as a response to the growing need for faster, more reliable, and scalable software delivery. It promotes collaboration between developers and IT operations teams to automate processes, enhance communication, and ensure continuous integration and continuous delivery (CI/CD).

To understand how DevOps evolved, this lesson begins by exploring the traditional software development models that preceded it — most notably, the waterfall model — and the challenges they posed in dynamic, fast-paced environments. From there, it examines how agile software development introduced iterative, adaptive, and collaborative methods that align naturally with DevOps principles. This lesson introduces the principles of Agile and how they align with the demands of modern software delivery in collaborative, DevOps-oriented environments.

What is Agile?

Agile is not a single method or technology, but a mindset rooted in the values and principles of the *Agile Manifesto*, published in 2001. It emphasizes individuals and interactions, working software, customer collaboration, and responding to change over traditional documentation-heavy and plan-driven approaches.

Agile encourages iterative development, where teams deliver software in small, consumable *increments*, allowing for frequent feedback and rapid adaptation.

The four values of the Agile Manifesto are supported by 12 guiding principles. These principles shape the mindset and practices of agile teams and are directly aligned with modern DevOps workflows. Let's take a closer look at five of the principles with the most impact on software development and operations.

Deliver Working Software Frequently

Agile encourages short, regular release cycles, often ranging from one to four weeks. This practice reduces risk, accelerates feedback loops, and allows for quick validation of business value. In DevOps, these cycles are implemented through *continuous integration* (CI) and *continuous delivery* (CD) pipelines that automatically test and deploy code changes. Frequent delivery ensures that features reach users faster and that bugs can be identified and resolved before they escalate.

Welcome Changing Requirements

Unlike traditional models, where late-stage changes are seen as costly or disruptive, Agile embraces change as a natural part of the development process. Requirements are expected to evolve as users interact with the system and provide feedback.

In DevOps, *infrastructure-as-code* and *containerization* allow teams to respond to changes quickly and consistently, reducing the friction typically associated with late adjustments.

Build Projects Around Motivated Individuals

Agile promotes a people-first philosophy. Rather than relying on rigid processes or micromanagement, agile teams thrive when composed of autonomous, cross-functional individuals who are empowered to make decisions.

For DevOps teams, this means developers, sysadmins, testers, and security professionals working side by side, with shared ownership of the product and its operational performance.

Maintain a Constant Pace

Sustainable development avoids burnout and ensures long-term productivity. Agile teams strive to avoid overcommitting and instead focus on delivering steady, predictable output over time.

DevOps reinforces this principle through automation, alerting, and observability tools, which reduce manual toil and enable teams to handle incidents without disrupting overall velocity.

Reflect and Adjust Regularly

Agile encourages continuous improvement through regular *retrospectives*. These meetings allow teams to discuss what went well, what didn't, and what can be improved in the next iteration.

This aligns closely with DevOps' *post-incident reviews* and the use of metrics such as lead time, deployment frequency, and change failure rate to guide improvements across both development and operations.

Agile Methodologies

Agile is not a one-size-fits-all process. Over the years, several frameworks and methodologies have emerged to help teams implement Agile principles in practical ways. Each has its own structure, strengths, and use cases. Overviews follow of the most common ones, those that are particularly relevant for DevOps teams working in collaborative, cross-functional environments.

Scrum

Scrum is a structured framework that organizes work into short, fixed-length development cycles known as *sprints*. Each sprint typically lasts between one and four weeks and results in a potentially releasable increment of the product. Scrum introduces clearly defined roles, such as the *Scrum master*, who facilitates the process and helps remove impediments; the *product owner*, who prioritizes and manages the *product backlog*; and the *development team*, which is responsible for delivering the work.

Scrum also includes a set of recurring meetings, or *ceremonies*, that help structure team activities. These include daily *stand-up meetings* for quick synchronization, *sprint planning sessions* to define goals for the upcoming sprint, *sprint reviews* to demonstrate completed work, and *retrospectives* to reflect on the process and identify improvements. This cycle of delivery, feedback, and refinement supports continuous learning and incremental progress, which aligns well with the rapid delivery goals of DevOps teams.

Kanban

Kanban offers a more flexible and flow-based approach to Agile. Rather than working in time-boxed iterations, Kanban focuses on visualizing the team's workflow using a *board* where tasks move through various stages, such as “To Do,” “In Progress,” and “Done.” This method emphasizes the importance of limiting work in progress, to avoid overloading the team and to maintain a steady, predictable flow of tasks.

Because there are no prescribed roles or ceremonies, teams using Kanban can adopt it gradually. This trait makes it particularly effective for operations and support, where priorities can shift quickly and work items vary in complexity. The continuous delivery mindset supported by Kanban makes it highly compatible with DevOps practices, particularly when integrated with automated pipelines and monitoring tools.

Extreme Programming (XP)

Extreme programming, or XP, is a methodology that focuses on achieving technical excellence and improving software quality through rigorous engineering practices. It promotes quality-enhancing practices such as *pair programming*, where two developers work together on the same piece of code to ensure quality and share knowledge.

XP also emphasizes *test-driven development*, in which automated tests are written *before* the actual implementation code, ensuring that software behavior is continuously validated.

Another core element of XP is *continuous integration*, which involves merging code changes into a shared repository several times a day, accompanied by automated builds and tests. These practices reduce integration problems, encourage clean and maintainable code, and foster a culture of responsibility and collaboration, all of which are vital in high-performing DevOps teams.

SAFe, LeSS, Nexus

In larger organizations, where multiple teams must collaborate across products or services, scaled Agile frameworks provide the structure necessary to maintain alignment and coordination. Frameworks such as *Scaled Agile Framework* (SAFe), *Large-Scale Scrum* (LeSS), and *Nexus* address the challenges of scaling Agile practices to enterprise-level environments.

SAFe integrates principles from *Agile* and *DevOps* with *Lean*, another popular methodology that focuses on eliminating waste, optimizing processes, and maximizing value delivery. It also introduces additional roles and planning layers to synchronize work across teams and departments.

LeSS extends Scrum principles to multiple teams working on a single product, aiming to maintain simplicity and minimize overhead.

Nexus, created by Scrum.org, focuses on cross-team coordination for up to nine Scrum teams sharing a common backlog.

These scaled frameworks provide the organizational scaffolding required to manage complexity while preserving Agile's responsiveness and adaptability.

Agile in Practice: Lifecycle and Artifacts

While not as rigid or sequential as the waterfall model, Agile still follows a set of well-defined stages that guide teams through planning, execution, review, and continuous improvement. These stages ensure that development remains iterative, adaptive, and aligned with user needs throughout the software lifecycle.

Sprint Planning

In Agile environments, software is developed through a series of short, repeatable cycles known as *iterations* or *sprints*. These iterations usually span from one to four weeks, depending on the team's workflow and the complexity of the project. Each iteration follows a structured lifecycle designed to promote continuous feedback, adaptive planning, and incremental delivery of working software.

The iteration begins with a planning phase, during which the team defines the goal and scope of the sprint. This is known as *sprint planning* and involves selecting the most valuable *user stories* from the *product backlog*, estimating the effort required, and breaking the work down into actionable *tasks*.

Execution

Once the plan is established, the team moves into the execution phase, where developers code, test, and integrate the selected features. This phase emphasizes collaboration, continuous integration, and early testing to ensure quality and stability throughout the sprint.

Review

Following the execution, the team conducts a sprint review. This is a demonstration session where the team presents the completed work to stakeholders, gathers feedback, and verifies that the software meets the agreed-upon criteria.

Retrospective

During this reflective meeting, the team discusses what went well, what challenges were encountered, and how their processes can be improved for future iterations. The retrospective reinforces Agile's core principle of continuous improvement and supports a culture of learning and adaptation.

Artifacts

In addition to this iterative lifecycle, Agile teams rely on a set of core *artifacts* to manage work and maintain transparency. One of the most essential artifacts is the *user story*, which represents a specific feature or function from the end-user's perspective. User stories are intentionally lightweight and focused on value, often structured using the format: "As a user, I want [feature] so that [benefit]."

All user stories are collected and prioritized in a *product backlog*, which serves as a dynamic to-do list for the entire project. The product backlog evolves continuously as new requirements emerge and priorities shift. For each sprint, the team creates a sprint backlog, which contains the user stories that have been committed for delivery during that iteration. This backlog provides a focused view of the work in progress and serves as a reference point during the sprint.

To monitor progress, teams often use a *burndown chart*. This visual tool tracks the amount of work remaining over time, providing a clear indication of whether the team is on track to meet its sprint goals. Burndown charts support data-driven discussions during daily stand-up meetings and help teams adjust their pace or scope as needed.

Together, the Agile lifecycle and its associated artifacts form the foundation of a transparent, adaptive, and value-driven development process.

Benefits of Agile in Modern Software Development

Agile principles and frameworks bring a range of practical benefits to organizations, especially those operating in cloud-native, containerized, or microservice-based environments.

One of the most important benefits is early and continuous value delivery. Agile teams ship working features incrementally, which means stakeholders start receiving value sooner and can validate the product's direction through real-world use. This is essential in environments where requirements evolve rapidly or where user feedback drives development.

Another benefit is improved adaptability. Agile welcomes changes — even late in the development process — allowing teams to pivot quickly in response to market dynamics or shifting priorities.

Combined with DevOps automation, these changes can be propagated safely and swiftly across environments.

Agile also improves collaboration across roles. By breaking down silos between development and operations, Agile fosters better communication and shared responsibility. This results in higher quality releases and faster recovery times when issues arise.

In addition, Agile drives technical quality through practices such as automated testing, continuous integration, and code reviews. These are all essential to maintaining a reliable and maintainable system, especially when changes are deployed frequently.

Finally, Agile supports greater transparency and visibility. Stakeholders can track progress in real time, participate in sprint reviews, and influence priorities. This level of engagement helps ensure that the product meets both technical and business goals.

Agile methodologies have revolutionized the way software is developed by promoting iterative delivery, continuous feedback, and close collaboration with stakeholders. However, Agile alone does not fully address the challenges of deploying, operating, and scaling software in complex production environments. This is where DevOps becomes essential.

What is DevOps?

By integrating operations into the Agile development cycle, DevOps extends the benefits of agility beyond code creation to include deployment automation, infrastructure management, and system monitoring. This seamless collaboration between development and operations leads to faster releases, greater reliability, and a more resilient end-to-end software delivery process.

DevOps is not a specific tool or framework. Instead, it is a set of practices and a philosophy aimed at unifying software development (Dev) and IT operations (Ops).

DevOps is a cultural and technical movement that integrates people, processes, and tools across development and operations to accelerate the delivery of high-quality software. It encourages teams to collaborate from the very beginning of a project, sharing knowledge, responsibilities, and feedback throughout the product's lifecycle.

The DevOps approach is based on several key principles. These include automation of repetitive tasks such as building, testing, and deploying software, continuous integration and continuous delivery_ (CI/CD), infrastructure-as-code, proactive monitoring, and a strong emphasis on communication and accountability across teams.

Rather than handing off completed code to operations teams for deployment, DevOps promotes cross-functional teams in which developers, operators, testers, and security professionals work

together as one unit.

Implications for Software Developers

For software developers, DevOps means greater involvement in the lifecycle of an application beyond just writing code. Developers are now expected to understand how their applications will behave in real-world production environments. This includes being familiar with containerization, infrastructure provisioning, and monitoring tools.

Developers in DevOps teams often write and maintain automated pipelines for testing and deployment. They use continuous integration systems to ensure that every code change is automatically tested and integrated with the rest of the application. By embracing infrastructure as code, developers can define system configurations and dependencies alongside application code, making the deployment process more predictable and consistent.

In short, DevOps expands the developer's responsibilities and enables them to build more resilient and maintainable systems. It also improves feedback loops, enabling developers to learn from production performance and user behavior, which leads to better design decisions and faster innovation.

Implications for System Operators

For system operators or sysadmins, DevOps introduces a shift from manual configuration and maintenance to automated, programmable infrastructure. Operators work closely with developers to define infrastructure as code, manage cloud environments, monitor system performance, and ensure scalability and reliability.

Instead of reacting to issues after they arise, operations professionals in a DevOps environment focus on proactive monitoring, logging, and incident response automation. They help define and enforce *service-level objectives* (SLOs) and error budgets, collaborating with developers to balance stability with innovation.

Operators also play a critical role in security and compliance, implementing *DevSecOps* practices that integrate security checks and validations directly into CI/CD pipelines.

This new way of working requires sysadmins to be comfortable with scripting, automation tools, configuration management systems, and cloud platforms. It also encourages a mindset of continuous learning and experimentation.

The DevOps Toolchain

To support the DevOps philosophy, teams rely on a wide range of tools that automate and

streamline the software delivery process. These tools span several categories, including:

Version control systems

(e.g., Git) to manage code collaboratively.

Continuous integration servers

(e.g., Jenkins, GitLab CI, GitHub Actions) to automate build and test processes.

Configuration management tools

(e.g., Ansible, Puppet, Chef) to provision and manage infrastructure.

Containerization platforms and orchestrators

(e.g., Docker, Kubernetes) to deploy applications consistently across environments.

Monitoring and observability tools

(e.g., Prometheus, Grafana, ELK Stack) to provide real-time insights into system health and performance.

These tools play a vital role in minimizing human error, accelerating deployment, and enhancing visibility throughout the entire software delivery pipeline.

Agile and DevOps

While Agile focuses on how to develop software, DevOps focuses on how to deliver and operate it. Together, they enable:

Continuous integration/continuous delivery (CI/CD)

Agile iterations feed into automated pipelines.

Fast feedback loops

Both technical (testing, monitoring) and human (user stories, retrospectives).

Cross-functional teams

Developers, sysadmins, and testers collaborate from day one.

Agile and DevOps jointly reduce lead time, improve resilience, and support high availability and scalability.

Guided Exercises

1. What is the main difference between Agile and DevOps in terms of their focus within the software development lifecycle?

2. How does Agile encourage teams to adapt to change throughout a project?

3. What role do DevOps practices play in supporting Agile principles like frequent delivery and continuous improvement?

Explorational Exercises

1. Retrospectives are a core part of the Agile cycle, where teams reflect on the process and look for ways to improve. Research how real-world Agile or DevOps teams conduct retrospectives. What techniques or tools do they use, and how do these practices help teams improve over time?

2. Research how Scrum and Kanban are used in practice by software or DevOps teams. What are the main operational differences, and in what contexts is each more effective?

Summary

Agile is a mindset and methodology that has reshaped software development by promoting short, iterative cycles, close collaboration, and frequent delivery of working software. Rooted in the Agile Manifesto, it emphasizes adaptability, user feedback, and continuous improvement. Frameworks such as Scrum, Kanban, Extreme Programming (XP), and scaled approaches like SAFe, LeSS, and Nexus help teams of all sizes apply Agile principles in real-world settings, especially in environments that require speed, flexibility, and cross-functional cooperation.

However, Agile by itself does not address all the challenges of running software in complex production environments. That's where DevOps comes in. DevOps extends the Agile philosophy by integrating operations into the development lifecycle, and thereby bringing automation, infrastructure management, monitoring, and deployment under a unified workflow. It encourages developers and operators to work together from the start, sharing tools, responsibilities, and goals.

Answers to Guided Exercises

1. What is the main difference between Agile and DevOps in terms of their focus within the software development lifecycle?

Agile focuses primarily on how software is developed, emphasizing iterative development, continuous feedback, and collaboration between stakeholders and development teams. DevOps, on the other hand, focuses on how software is delivered and operated—integrating operations into the development process through automation, deployment pipelines, infrastructure management, and monitoring. While Agile optimizes the creation of software, DevOps ensures that this software is reliably deployed, maintained, and scaled in production environments.

2. How does Agile encourage teams to adapt to change throughout a project?

Agile encourages adaptability through short, time-boxed development cycles known as sprints, which allow teams to frequently reassess priorities and incorporate feedback. Agile principles welcome changing requirements—even late in development—and promote regular retrospectives to evaluate what's working and what needs to improve. This cycle of continuous reflection and adjustment helps teams respond effectively to evolving user needs and market conditions.

3. What role do DevOps practices play in supporting Agile principles like frequent delivery and continuous improvement?

DevOps practices operationalize Agile principles by automating critical stages of the software delivery process. Through tools like CI/CD pipelines, DevOps enables frequent and reliable delivery of working software. Infrastructure-as-code and proactive monitoring provide consistency and insight into system performance, supporting faster feedback loops. These practices ensure that each Agile iteration can be smoothly deployed, tested, and evaluated in real-world environments, reinforcing a culture of continuous improvement across both development and operations.

Answers to Explorational Exercises

1. Retrospectives are a core part of the Agile cycle, where teams reflect on the process and look for ways to improve. Research how real-world Agile or DevOps teams conduct retrospectives. What techniques or tools do they use, and how do these practices help teams improve over time?

In real-world Agile and DevOps environments, teams often conduct retrospectives at the end of each sprint using structured facilitation techniques such as “Start, Stop, Continue,” “Mad, Sad, Glad,” or “4Ls” (Liked, Learned, Lacked, Longed for). These retrospectives help teams identify recurring bottlenecks, improve communication, and evolve their workflows.

2. Research how Scrum and Kanban are used in practice by software or DevOps teams. What are the main operational differences, and in what contexts is each more effective?

Scrum is commonly used in product development teams that benefit from time-boxed planning, structured ceremonies, and clearly defined roles. It provides predictability and rhythm through sprints and regular checkpoints. Kanban, by contrast, is often preferred by teams with continuous workflows, such as IT operations or support teams, where work arrives unpredictably. Kanban allows more flexibility by focusing on flow efficiency and limiting work in progress. While Scrum requires sprint planning and retrospectives, Kanban emphasizes real-time prioritization and gradual process improvement. In practice, many DevOps teams blend both, using Scrum for planning and team coordination, while applying Kanban principles to manage incoming requests and infrastructure tasks.



Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.1 Modern Software Development
Lesson:	2 of 3

Introduction

As organizations increasingly shift their infrastructure and applications to cloud environments, a new paradigm of software design has emerged: cloud native. Cloud-native applications are built to thrive and scale in modern, dynamic environments. They are designed from the ground up to exploit the full capabilities of the cloud, including elasticity, distributed computing, automation, and continuous delivery.

This lesson explores the defining properties of cloud-native applications, focusing on three foundational characteristics: immutable servers, loose coupling, and test-driven development. Understanding these properties is essential for developers, operators, and DevOps professionals aiming to build resilient, scalable, and maintainable software systems. The lesson also covers service-oriented architectures (SOA) and common API concepts and standards.

What Does “Cloud Native” Mean?

Cloud native doesn’t mean just hosting applications in the cloud. It refers to an approach that maximizes the benefits of cloud computing by embracing *microservices architecture*, *containerization*, *orchestration*, *automation*, and *continuous integration/continuous delivery*.

(CI/CD). The *Cloud Native Computing Foundation* (CNCF) defines cloud-native technologies as those that empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds.

Cloud-native applications are designed to be resilient, manageable, and observable. They are composed of microservices packaged in containers, deployed dynamically, and orchestrated to optimize resource utilization. The shift from *monolithic* to *microservice-based* architectures supports agility and rapid deployment, making the cloud-native model ideal for today's fast-paced development cycles.

Immutable Servers

One of the key concepts in cloud-native architecture is the idea of *immutable servers*. In traditional systems, servers are often long-lived and subject to manual configuration changes over time. This approach leads to configuration drift, unpredictability, and operational complexity.

In contrast, immutable infrastructure ensures that once a server or service is deployed, it is never modified. Instead of updating an existing server, a new instance is created from a tested and versioned image.

This model brings several advantages: It enhances consistency across environments, since all instances are created from the same image. It also simplifies rollback, because previous images can be redeployed if a problem arises.

Immutable servers reduce the risk of human error and eliminate the accumulation of undocumented changes. Automation tools such as Terraform, Ansible, and Packer are commonly used to define and build immutable images. Containerization platforms like Docker and Podman and orchestration tools like Kubernetes reinforce immutability by providing *declarative configuration* and *stateless deployment patterns*.

The concept of immutability also extends to *infrastructure-as-code* (IaC). With IaC, infrastructure is defined and provisioned through machine-readable files, enabling repeatable, testable, and version-controlled deployments. This approach aligns closely with DevOps practices and ensures that environments remain consistent across development, staging, and production.

To better understand the practical benefits of immutable infrastructure, it is useful to examine real-world implementations of immutable servers across various technologies. These examples illustrate how immutability can be achieved through many different tools at different levels of detail, ranging from cloud virtual machines and containers to orchestrated deployments and infrastructure automation tools. In each case, the core principle remains the same: rather than modifying a live system, a new version is created and deployed, ensuring consistency, reliability, and predictability across all stages of the software lifecycle.

EC2 with Amazon Machine Images (AMIs)

In AWS, teams can create *Amazon Machine Images* (AMIs) that contain the full operating system (OS), dependencies, application code, and configurations for a computing instance. Every time an update is needed, a new AMI is built, and a new *Elastic Compute Cloud* (EC2) instance is launched using that AMI. The previous instance is terminated.

This ensures that every EC2 instance is identical, and the deployment process is fully automated. There is no need to SSH into a server to apply manual fixes — any fix is made at the image level and redeployed cleanly.

Docker Containers

Docker is a core technology in immutable infrastructure. A Docker image captures the complete application environment: the base operating system layer, system libraries, dependencies, and application code. Once built, the image becomes read-only and cannot be modified.

When an update is required, the image is rebuilt and redeployed as a new container. This immutability enhances portability, simplifies debugging, and guarantees consistent behavior across development, testing, and production environments.

Kubernetes Deployments

In *Kubernetes*, workloads are deployed as immutable *Pods* created from container images. Deploying a new version of an application does not involve patching existing pods; instead, they are replaced with new instances built from updated container images.

Kubernetes handles this through features like rolling updates, ensuring zero downtime while replacing old immutable pods with new ones. This model aligns with the core DevOps principle of automated, predictable deployments.

CI/CD Pipelines with Immutable Artifacts

Modern CI/CD systems such as GitLab CI, Jenkins, and GitHub Actions often generate immutable artifacts during the build process. These artifacts — such as compiled binaries, Docker images, or configuration bundles — are versioned, stored in registries, and deployed as-is to production.

Instead of modifying the live system, the pipeline always delivers a fresh version of the application, known to be good and up to date, thus ensuring that production environments are consistent and reproducible.

Loose Coupling

Another essential property of cloud-native applications is *loose coupling*. In a tightly coupled system, components are highly dependent on one another, making them difficult to change, scale, or replace. Changes to one part of the system often require corresponding changes elsewhere, increasing complexity and reducing agility.

Loose coupling addresses this problem by minimizing dependencies between services. Each component of a loosely coupled system interacts with others through well-defined *interfaces* and *contracts*, such as RESTful APIs or asynchronous messaging protocols. This decoupling allows services to evolve independently, be developed and deployed by separate teams, and scale based on their own demand.

Microservices architecture is a common implementation of loose coupling. In this model, the application is broken into independent services that communicate over the network. Service discovery, API gateways, and message brokers like Kafka or RabbitMQ are often used to facilitate communication between services without creating tight interdependencies.

Loose coupling enhances system resilience. If one service fails, the others can continue functioning, possibly with degraded performance but without complete system failure. This design supports fault isolation and graceful degradation, which are critical in distributed systems.

Moreover, loose coupling promotes reuse and composability. Services can be repurposed in different contexts, integrated with other applications, or replaced with alternative implementations without disrupting the entire system. Loose coupling also enables faster innovation, as teams can iterate on individual services without waiting for changes in other parts of the application.

To achieve effective loose coupling, developers and architects must prioritize clear API design, robust service contracts, and comprehensive documentation. Monitoring, tracing, and observability tools are also necessary to understand how services interact and to diagnose issues when they arise.

Test-Driven Development (TDD)

Test-driven development (TDD) is not exclusive to cloud-native applications, but it is particularly valuable in this context. TDD is a software development approach where tests are written *before* the actual code. The process typically follows a short cycle: write a test that naturally fails because no code is present to support it, then implement the minimal code to pass the test, and finally refactor the code for clarity and performance.

In cloud-native systems, TDD supports reliability, maintainability, and rapid iteration. Since

applications are composed of many small, independent services, testing each component in isolation is crucial. TDD ensures that every unit of functionality is backed by automated tests, which catch *regressions* and validate behavior during every deployment.

TDD also aligns well with continuous integration and continuous delivery. CI/CD pipelines rely on automated tests to validate code changes before they are merged or deployed. When TDD is adopted, the test suite becomes a safety net that allows teams to move quickly without sacrificing quality.

Several types of tests are relevant to TDD in cloud-native development:

Unit tests

validate individual functions or classes.

Integration tests

ensure that components work together correctly.

End-to-end tests

simulate real user interactions with the system.

Each layer of testing contributes to overall confidence in the application's correctness.

Modern development tools and frameworks facilitate TDD. Modern languages such as Python, Go, Ruby, Rust, C#, and JavaScript offer robust testing libraries. Containerized testing environments can simulate production-like conditions, while service virtualization can mock dependencies. These capabilities make it easier to practice TDD in distributed, containerized systems.

Beyond technical implementation, TDD encourages a mindset of intentional design. Writing tests first forces developers to clarify requirements and think about how the code will be used. It promotes modularity, reduces technical debt, and improves documentation.

Service-Oriented Architecture (SOA)

As modern applications become increasingly complex and distributed, the need for modularity, scalability, and flexibility in software design has become critical. Traditional monolithic architectures, where all components of an application are tightly interwoven and deployed as a single unit, often struggle to meet these demands. In response, *service-based* application design has emerged as a powerful architectural paradigm, enabling systems to be built from loosely coupled, independently deployable components that communicate over well-defined interfaces.

One of the foundational models in this domain is *service-oriented architecture* (SOA). SOA promotes the organization of software functionality into discrete services, each of which

encapsulates a specific business capability and interacts with others through standardized protocols. Understanding how to design applications based on services, and the principles behind SOA, is essential for professionals building modern, scalable, and maintainable systems.

What is a Service-Based Application?

A service-based application is a system built from a collection of independent services, each responsible for a specific piece of functionality. These services interact with each other over a network using well-defined interfaces, often through common web protocols such as HTTP, *Simple Object Access Protocol* (SOAP), or *gRPC Remote Procedure Calls* (gRPC).

Each service in the system represents a cohesive business unit, such as payment processing, user authentication, or inventory management. Unlike components in a monolithic application, services can be developed, deployed, and scaled independently. This decoupling allows teams to evolve parts of the system without affecting the entire application.

Service-based applications offer better fault isolation, improved scalability, and more flexibility in choosing programming languages, frameworks, or databases for individual services. They also support distributed development, where different teams can own and operate separate services.

What is Service-Oriented Architecture (SOA)?

Service-oriented architecture (SOA) is a software design approach in which application components provide services to other components via a communication protocol, typically over a network. The key principle of SOA is that services are loosely coupled and designed to be reusable and composable.

In SOA, services expose contract-based interfaces that describe what the service does, how it should be invoked, and what data formats it expects. These interfaces are often described using formal specifications, such as *Web Services Description Language* (WSDL) for SOAP-based services. Communication between services is handled via message exchange patterns, often formatting the data payloads through *Extensible Markup Language* (XML) or *JavaScript Object Notation* (JSON).

SOA emerged as a response to the challenges of tightly coupled systems and was intended to increase modularity, interoperability, and maintainability. It is particularly well-suited for enterprise systems requiring integration between multiple, heterogeneous systems.

Service-Oriented Architectures have several defining characteristics that distinguish them from monolithic or layered architectures.

First, services in SOA are autonomous. Each service runs independently and has control over the logic and data it manages. This autonomy reduces dependencies and simplifies system evolution.

Second, SOA emphasizes standardized communication. By using agreed-upon protocols and data formats, services can interact regardless of their internal implementation. This is especially important in large organizations where services may be written in different programming languages or run on different platforms.

Third, services are discoverable and reusable. SOA systems often include a service registry, where available services are published and can be discovered by consumers. This enables reuse and avoids duplication of functionality across the enterprise.

Finally, SOA promotes composability. Complex workflows or applications can be built by combining several services, using either orchestration engines or choreography patterns. Composability supports agility and makes it easier to modify business processes as requirements evolve.

SOA vs. Microservices

While SOA and microservices share many principles — such as modularity, reusability, and loose coupling — they differ in scope and implementation.

SOA tends to focus on integration at the enterprise level, often involving large, complex systems that need to communicate across organizational or technological boundaries. It commonly uses middleware such as an *enterprise service bus* (ESB) for message routing, message transformation, and orchestration.

Microservices, on the other hand, are typically more lightweight and granular, with an emphasis on autonomy and decentralized governance. In microservices, each service is deployed independently, often packaged in containers, and managed through DevOps practices such as continuous delivery, infrastructure as code, and service meshes.

Despite these differences, SOA laid the groundwork for many of the architectural patterns now widely adopted in microservices and cloud-native environments.

Benefits and Challenges of SOA

SOA offers a number of benefits for organizations aiming to improve agility and system maintainability:

Service reuse

Once a service is developed and exposed, it can be reused across multiple applications, reducing development time and redundancy.

Interoperability

Because SOA services use open standards and protocols, they can bridge systems that run on different platforms or are written in different programming languages.

Scalability and fault tolerance

Services can be deployed independently and scaled according to demand. If one service fails, others can continue functioning, reducing the risk of system-wide outages.

However, SOA also introduces challenges. Designing effective service boundaries requires careful analysis, and service orchestration can become complex. Additionally, the use of shared infrastructure such as ESBs may introduce performance bottlenecks or single points of failure.

Monitoring, logging, and tracing across services become more critical—and more difficult—as the number of services grows. Organizations adopting SOA must invest in tooling and practices that provide visibility into service interactions and health.

Designing Software to be Deployed to Cloud Services

Designing a service-based application to be deployed to cloud services begins with identifying the business capabilities the system must provide and mapping these to individual services. Each service should have a single, clearly defined responsibility and operate independently of others as much as possible. Good service design also includes well-documented APIs, versioning strategies, and robust error handling.

A common design approach builds services around business domains, a concept known as *domain-driven design*. For example, an e-commerce platform might have services such as OrderService, PaymentService, and ShippingService. Each of these services represents a domain and encapsulates its data and logic.

It's important to manage data ownership boundaries carefully. In SOA, each service typically “owns” its data and exposes only what is necessary through its interface. Shared databases between services are discouraged, as they create tight coupling and make services harder to evolve independently.

Communication between services can be *synchronous* (e.g., HTTP or gRPC) or *asynchronous* (e.g., message queues or event streams). Asynchronous communication allows services to remain decoupled and improves system resilience and scalability.

Another important consideration is *service orchestration*. In some systems, a central orchestrator is responsible for invoking services in a specific sequence. In others, services are designed to collaborate through an event-driven architecture, where services react to published events and

trigger their own logic.

In the cloud, applications are no longer bound to a specific physical server; instead, they run in *virtualized* or *containerized* environments, often orchestrated by tools such as Kubernetes and managed through infrastructure-as-code.

To design effectively for the cloud, applications must be *stateless* whenever possible. This means they should not rely on local disk or session data, because instances may be replaced, scaled, or restarted at any time. Instead, persistent data should be stored in external services like *object storage* (e.g., AWS S3) or *managed databases* (e.g., Azure SQL, Amazon RDS).

Another key consideration is scalability. Applications should be designed to scale horizontally, meaning they can handle increased load by adding more instances, rather than scale vertically by upgrading a single server. This includes designing APIs and workloads that support load balancing and auto-scaling.

Cloud-deployed software must also embrace observability and resilience. Logs, metrics, and traces should be collected and centralized using tools such as Prometheus, Grafana, ELK Stack, or CloudWatch. For resilience, patterns such as circuit breakers, retry logic, and graceful degradation are essential to handle failures gracefully.

Security is built into cloud design from the start: secrets management, identity and access control, and network segmentation (e.g., via VPCs and security groups) are critical. It's also common to integrate with CI/CD pipelines to automate builds, tests, and deployments, ensuring faster and safer delivery of updates.

By following these principles, software becomes cloud-ready: able to take full advantage of the scalability, flexibility, and automation that modern cloud platforms provide.

Guided Exercises

1. What are the key properties of cloud-native applications, and why are they important for building scalable and maintainable systems?

2. Explain the concept of immutable servers and give one example of how this concept is implemented in practice.

3. What is service-oriented architecture (SOA), and how does it support modularity and system evolution?

Explorational Exercises

1. Based on the description of SOA in this lesson, research how microservices architectures build upon or diverge from traditional SOA.

2. Test-driven development (TDD) is mentioned as an important practice for cloud-native applications. Research and list at least two popular tools or frameworks used for implementing TDD in cloud-native environments (e.g., container-based or CI/CD-enabled workflows).

Summary

As organizations embrace cloud environments, cloud-native design has become essential for building scalable, resilient, and modern applications. This lesson introduced the core properties of cloud-native systems, including immutable servers, loose coupling, and test-driven development (TDD). Through containerization, orchestration, and automation, cloud-native applications support continuous delivery and seamless scalability. Immutable infrastructure ensures that deployments are repeatable and consistent, reducing human error and simplifying rollback.

The lesson also explored the design of service-based applications, with a focus on service-oriented architecture (SOA). SOA provides a modular framework for building applications from independent, discoverable services that interact over standard protocols. This approach improves fault isolation, promotes reuse, and allows teams to iterate on individual components without affecting the whole system.

Answers to Guided Exercises

1. What are the key properties of cloud-native applications, and why are they important for building scalable and maintainable systems?

The key properties of cloud-native applications include immutable servers, loose coupling, and test-driven development (TDD). These properties are important because they support resilience, scalability, and maintainability. Immutable servers ensure consistency and reduce errors by preventing manual changes. Loose coupling allows independent development and deployment of components, enhancing agility. TDD ensures code reliability and supports rapid iteration by requiring automated tests before implementation.

2. Explain the concept of immutable servers and give one example of how this concept is implemented in practice.

Immutable servers are systems that are never modified after deployment. Instead of applying changes directly to a running server, a new instance is created from a tested, versioned image. This ensures consistency and simplifies rollbacks.

Example: In AWS, teams use Amazon Machine Images (AMIs) to build and deploy EC2 instances. When an update is required, a new AMI is created, a new EC2 instance is launched, and the old one is terminated.

3. What is service-oriented architecture (SOA), and how does it support modularity and system evolution?

SOA is a design approach where software is organized into autonomous services that communicate over standard protocols like HTTP or SOAP. Each service encapsulates a specific business capability and exposes contract-based interfaces. This modular structure supports reusability, interoperability, and scalability. Because services are loosely coupled, they can evolve independently, making the system easier to maintain and adapt to changing requirements.

Answers to Explorational Exercises

1. Based on the description of SOA in this lesson, research how microservices architectures build upon or diverge from traditional SOA.

While both SOA and microservices architectures focus on modular, reusable services, microservices tend to be more fine-grained, with each service typically managing a single function. Microservices emphasize independent deployment, lightweight communication protocols (like REST or gRPC), and decentralized governance, whereas SOA often relies on centralized components such as enterprise service buses (ESBs) and WSDL contracts for SOAP services.

2. Test-driven development (TDD) is mentioned as an important practice for cloud-native applications. Research and list at least two popular tools or frameworks used for implementing TDD in cloud-native environments (e.g., container-based or CI/CD-enabled workflows).

Test-driven development (TDD) is widely supported in cloud-native environments through a variety of tools that integrate seamlessly with containers and CI/CD pipelines. For example, Pytest is a popular Python testing framework known for its simplicity and extensibility, easily executed within Docker containers and CI tools such as GitHub Actions or GitLab CI. In the JavaScript ecosystem, Jest stands out for its speed and built-in features like mocking and snapshot testing, making it ideal for both frontend and backend services.



Lesson 3

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.3 Modern Software Development
Lesson:	3 of 3

Introduction

As software systems evolve to meet the demands of cloud-native environments, containerization has become foundational to modern application design. Containers offer a consistent and lightweight runtime that simplifies deployment, enhances portability, and enables rapid scaling across diverse infrastructure. To fully benefit from containers, software must be designed with distributed principles in mind: statelessness, modularity, and clear service boundaries.

This shift often goes hand-in-hand with adopting a microservices architecture, RESTful APIs, and cloud-optimized storage and session management strategies. This lesson explores how applications can be architected to run effectively in containers, communicate reliably through APIs, and remain robust and maintainable in dynamic, production-ready environments.

Microservices Architecture

A common architectural pattern for containerized applications is the *microservices architecture*. Instead of building one large, monolithic application, microservices break functionality into small, independent services that can be developed, deployed, and scaled separately. Each microservice is typically packaged in its own container and managed through orchestration platforms such as

Kubernetes or Docker Swarm. This approach enhances modularity, supports team autonomy, and enables faster, more reliable software delivery.

However, migrating from a monolithic legacy application to a microservices-based, containerized system presents several risks. Legacy applications are often tightly coupled, with shared databases and hidden dependencies, making it challenging to separate functions cleanly. Migration efforts can introduce data consistency issues, service duplication, or unexpected behavior during integration. Additionally, without a clear understanding of the legacy system's architecture and business rules, breaking it into services may lead to loss of functionality or degraded performance.

Organizations must approach this transition carefully, starting with modular refactoring, implementing APIs to wrap existing functionality, and introducing containerization incrementally. It's also essential to invest in observability tools, CI/CD pipelines, and automated testing to support the increased complexity of a distributed system. With proper planning, containerized microservices can bring agility and resilience. But it's critical to remain aware of migration risks to ensure a smooth and successful transformation.

We've come a long way in how we build and structure software. It all started with monolithic architectures — everything bundled together — which were simple but hard to scale or maintain as things got bigger. Then came Service-Oriented Architecture, which helped break things down into more manageable services that could talk to each other. It was a big step forward.

And now, we are in the age of microservices, taking modularity even further with small, independent services that can be developed, deployed, and scaled on their own. Unlike SOA, microservices avoid the complexity of enterprise service buses and focus on lightweight communication like REST or messaging, giving teams more autonomy and speed.

Each step in this evolution has been about making systems more flexible, scalable, and easier to work with.

To take full advantage of containerization, applications should be designed to be stateless, support environment configuration via variables, and expose well-defined network interfaces such as RESTful APIs.

Common API Concepts and Standards: REST and JSON

Modern software systems rely heavily on *application programming interfaces* (APIs) to enable communication between different services, applications, and platforms. APIs define a set of rules that allow one piece of software to interact with another, making them a foundational component in cloud-native and service-based architectures.

One of the most common API styles is *representational state transfer* (REST). REST is an architectural standard for designing stateless APIs that use standard HTTP methods such as GET, POST, PUT, DELETE, and PATCH to perform operations on resources.

In RESTful APIs, data is typically represented as resources, each identified by a unique URL. For example, an API request like GET /users/123 might return the profile information of a specific user.

REST APIs are popular because they are simple, scalable, and language-agnostic — any client that can make HTTP requests can use them. They also follow predictable conventions, making them easy to understand and consume by developers.

In RESTful APIs, different HTTP methods are used for different actions that a client wants to perform on a resource, such as retrieving, creating, updating, or deleting data, as described in the following subsections.

GET: Retrieve Data

The GET method retrieves data from a server. It is safe and idempotent, meaning that repeating the same request will always result in the same state of the resource. GET should have no side effects on the resource. In RESTful APIs, GET is commonly used to fetch a list of resources or a specific resource by ID.

Example of retrieving a list of users:

```
$ GET /api/users
```

Retrieving a single user by ID:

```
$ GET /api/users/42
```

These requests return data in formats such as JSON or XML and should not modify any data on the server.

POST: Create a New Resource

The POST method creates a new resource on the server. Unlike GET, POST is not idempotent, meaning that multiple identical requests may result in multiple resources being created. The resource to be created is specified in the body of the request, usually formatted as JSON.

Example of creating a new user:

```
$ POST /api/users
```

Request body:

```
{
  "name": "Carol",
  "email": "carole@example.com"
}
```

The server processes this request and, if successful, responds with a **201 Created** status and the URI of the new resource in the Location header.

PUT: Update or Replace a Resource

The **PUT** method completely updates or replaces an existing resource. It is idempotent.

Example of updating a user:

```
$ PUT /api/users/42
```

Request body:

```
{
  "name": "Carol Smith",
  "email": "Carol.smith@example.com"
}
```

If the user with ID 42 exists, this request replaces the user's entire record with the new data. If some fields are omitted from the request, those fields in the record may be reset or removed depending on the implementation.

DELETE: Remove a Resource

The **DELETE** method removes a resource from the server. It is also idempotent: deleting the same resource multiple times has the same effect as deleting it once.

Example of deleting a user:

```
$ DELETE /api/users/42
```

The server typically responds with a `204 No Content` status code if the deletion is successful and the resource no longer exists.

PATCH: Partially Update a Resource

The `PATCH` method applies a partial update to an existing resource. Unlike `PUT`, which replaces the entire resource, `PATCH` changes only the fields specified in the request.

Example of updating only the user's email address:

```
$ PATCH /api/users/42
```

Request body:

```
{
  "email": "new.email@example.com"
}
```

The `PATCH` method is particularly useful for applying partial updates to a resource, allowing minimal changes to be made without transmitting its entire representation. This reduces bandwidth usage and avoids unnecessary overwriting of unchanged data.

Understanding and using these methods correctly is fundamental when designing or consuming RESTful APIs, especially in DevOps environments where automation, versioning, and standardization are key. These methods help create consistent, maintainable, and predictable interfaces for service-based applications.

Formatting Data in RESTful APIs

In RESTful APIs, the most common data format used to send and receive information is *JavaScript Object Notation* (JSON). JSON is a lightweight, human-readable data format that is easy to parse and generate in virtually every programming language. For example, a response from an API might represent a user's ID, name, and email address through the following JSON:

```
{
  "id": 123,
  "name": "Henry Jobs",
  "email": "henry@example.com"
}
```

```
"email": "henryj@example.com"  
}
```

JSON structures data using *key-value pairs*, *arrays*, and *nested objects*, making it ideal for representing complex data in a compact form. It is often used alongside REST because it allows fast, efficient communication between services—especially in microservices and cloud-based environments.

Example of RESTful APIs

Instead of designing a single monolithic application, the platform is decomposed into smaller, independent *microservices*. Each service owns a specific business capability (for example, catalog, orders, payments, shipping) and exposes well-defined APIs or publishes/consumes messages via *message queues* to collaborate asynchronously.

This modular architecture improves deployment agility (services can be updated independently), fault isolation (a failure in one service does not bring down the entire system), and scalability (each service scales according to its own workload profile). Clear service boundaries, versioned contracts, and automated pipelines further streamline development and operations while preserving autonomy across teams and services.

Here's how the system might be structured:

1. User Service

- Handles user registration, authentication, and profile management.
- Offers requests like `POST /register`, `POST /login`, and `GET /profile/{id}`.
- Stores user data in its own isolated database.
- Responsible for issuing JWT tokens for authentication or API keys for secure communication.

2. Product Catalog Service

- Manages all product information: names, descriptions, prices, images, and availability.
- Provides requests such as `GET /products`, `GET /products/{id}`, and `POST /products`.
- Keeps product metadata in a separate product database.
- Allows internal updates by administrators and public browsing by customers.

3. Cart Service

- Manages shopping cart functionality for each user.

- Allows items to be added, updated, or removed through requests like `POST /cart/add` or `DELETE /cart/item/{id}`.
- Stores cart data temporarily, often in a fast-access data store like Redis.

4. Order Service

- Processes completed carts into orders.
- Handles order creation, payment status, and tracking.
- Communicates with both the Cart and Payment services.
- Stores order history and generates order confirmations.

5. Payment Service

- Integrates with external payment gateways (e.g., Stripe, PayPal).
- Validates transactions and updates payment status.
- Provides secure endpoints such as `POST /payment/process`, which forwards payment details to the selected gateway and returns the transaction result.
- Does not store sensitive payment information directly, but relies on third-party providers for PCI compliance.

6. Shipping Service

- Coordinates delivery logistics and estimated delivery times.
- Integrates with external shipping APIs (e.g., FedEx, UPS).
- Provides tracking numbers and delivery status updates.
- Can trigger notifications through the Notification Service.

7. Notification Service

- Sends email messages, SMS messages, or push notifications to users about order status, promotions, or support tickets.
- Can be triggered by other services via events or API calls.

Together, these services illustrate a modular and scalable approach to building a RESTful application using a microservice architecture. Each service is responsible for a specific domain, exposes well-defined API endpoints, and maintains its own data store. This architecture enables loose coupling, independent deployment, and improved maintainability. By separating concerns and using standard HTTP methods, the system ensures clear communication between services and provides a reliable foundation for delivering a responsive, secure, and extensible user experience.

Data Storage, Service Status, and Session Handling in Cloud Applications

Understanding how data is stored, how services manage their status, and how sessions are handled is essential when designing modern, distributed applications—especially in service-based or cloud-native environments. Each of these aspects plays a critical role in ensuring that applications remain reliable, scalable, and responsive under real-world conditions.

Data Storage

A key principle in data storage is *data ownership*. Each service in a distributed system should be responsible for its own data, maintaining its own database or data store. This approach contributes to loose coupling between services and makes it possible to scale and evolve individual components without introducing dependencies.

Additionally, services may use different types of storage based on their requirements. For example, a product catalog service might use a relational database for structured queries, while a notification service might use a key-value store or a message queue for high-throughput processing.

Modern architectures also rely on high-availability storage systems, replication, and backups to ensure data resilience in case of failure.

In cloud environments, data storage is offered through a wide range of services, each designed to meet specific requirements for performance, durability, and scalability.

One of the most widely used types is *object storage*, such as Amazon S3. It allows applications to store and retrieve unstructured data—such as images, videos, backups, or logs—using unique identifiers. S3 is highly durable, supports versioning, and integrates easily with other AWS services. Similar offerings include Azure Blob Storage and Google Cloud Storage.

For structured and relational data, cloud providers offer managed database services such as Amazon RDS (Relational Database Service), which supports engines such as MySQL, PostgreSQL, SQL Server, and Oracle Database. These services handle routine tasks like backups, patching, replication, and failover, allowing DevOps teams to focus on application logic rather than database maintenance. Google Cloud SQL and Azure Database provide similar capabilities in their respective ecosystems.

In use cases that require low latency and high throughput for semi-structured or unstructured data, *NoSQL databases* are often preferred over relational (SQL-based) databases. Amazon DynamoDB is a fully managed key-value and document database designed for high-performance

workloads. Similarly, Azure Cosmos DB and Google Firestore offer globally distributed, horizontally scalable storage solutions for applications that require real-time responsiveness at scale.

For temporary or fast-access storage, such as caching or session storage, services like Amazon ElastiCache (which supports Redis and Memcached) offer in-memory data storage with microsecond response times. These services are ideal for reducing the load on backend systems and improving application performance.

File storage is another important storage category, particularly for applications that require shared access to a file system. Services like Amazon EFS (Elastic File System), Azure Files, and Google Filestore provide scalable and secure file systems that can be mounted on virtual machines or containers across availability zones.

These examples highlight how cloud providers deliver flexible, reliable, and scalable data storage options to suit a wide variety of application requirements. Choosing the right type of storage depends on factors such as data access patterns, consistency requirements, performance expectations, and cost constraints.

Service Status

Service status refers to services communicating their health and availability to other parts of the system. Monitoring service status is critical for system reliability, automated scaling, and incident response. Health checks—both liveness and readiness probes—are commonly used in orchestration platforms such as Kubernetes to detect whether a service is functioning correctly and is ready to receive traffic. Service registries and load balancers may use these checks to route traffic away from unhealthy instances, ensuring that clients try to connect only to responsive services. Moreover, exposing service metrics, logs, and tracing information allows real-time observability and pro-active incident detection.

Session Handling

Session handling is another crucial component in distributed systems, especially for applications that require user authentication or that maintain temporary user-specific state. In monolithic applications, session data is often stored in memory. However, in distributed service-based applications, storing sessions locally can cause inconsistencies and hinder scalability, particularly when multiple service instances or containers are running behind a load balancer. To address this gap in availability, sessions are typically stored in centralized, fast-access data stores such as Redis or Memcached, allowing any service instance to retrieve session information as needed.

Alternatively, *stateless authentication methods*, such as JSON Web Tokens (JWT), can be used to

encode session data on the client side, reducing server-side dependencies and improving horizontal scalability.

Effective data storage, service status monitoring, and session management work together to form the backbone of robust service-based applications.

Database Schema Updates and Migrations

Another critical aspect of data management in service-based architectures is the handling of database schema updates and migrations.

In traditional monolithic systems, schema changes are typically applied centrally and deployed alongside application updates. However, in distributed systems—especially those built with microservices—schema management becomes more complex, as each service may have its own database and update cycle.

Database migrations must be carefully coordinated, especially when changes are applied to production environments with live traffic. A common best practice is to design backward-compatible migrations, ensuring that new schema versions can work with both the current and previous versions of the application. This allows teams to roll out database changes gradually without causing service disruptions.

In DevOps workflows, database migrations should be treated as first-class citizens within the deployment process. This means applying schema updates as part of the release pipeline, testing them in staging environments, and monitoring their post-deployment effects. In environments where downtime is unacceptable, zero-downtime migration techniques—such as additive changes, blue-green deployments, and feature toggles—are employed to ensure smooth transitions.

Being aware of the implications of schema changes, and implementing migrations safely and consistently, is essential for maintaining data integrity and minimizing operational risk in modern applications. The procedure requires collaboration between developers, database administrators, and operations teams to ensure that data and applications evolve together, without breaking functionality or causing unexpected outages.

NOTE

For readers interested in diving deeper into cloud-native design principles, it's highly recommended to explore the concept of the *12-Factor App*. It outlines foundational practices for building scalable, maintainable, and portable applications in modern cloud environments.

Guided Exercises

1. Design a simple user registration microservice that follows stateless principles and is suitable for deployment in a containerized environment. Describe the API endpoints it would expose, how it would handle session management, and how data would be stored.

2. You are managing a microservice that uses a PostgreSQL database. A new feature requires adding a column to a table used in production. Describe how you would implement this schema change safely with zero downtime.

3. You are deploying multiple instances of a microservice behind a load balancer in Kubernetes. Describe how the system should handle service health checks to ensure that traffic is routed only to healthy instances.

Explorational Exercises

1. The lesson introduces Kubernetes as an orchestration platform for managing containerized microservices. Choose a real-world company or open-source project that uses Kubernetes, and describe how it manages application deployment, scaling, and service discovery.

2. Describe a secure way to implement authentication and authorization in a RESTful API. Include how the API should validate client identity, restrict access to protected resources, and ensure that data is transmitted securely. Mention at least one standard method or technology used for this purpose.

Summary

This lesson introduced key concepts for designing modern, distributed applications that run in containers. It explored how containerization supports portability and scalability, and how a microservices architecture enables independent development, deployment, and scaling of loosely coupled services. Through the use of RESTful APIs, services communicate over well-defined interfaces using standard HTTP methods such as GET, POST, PUT, DELETE, and PATCH, enabling modular and predictable interactions between components.

Additionally, the lesson covered essential operational concerns, including data storage strategies in cloud environments, service health monitoring, and session handling in distributed systems. Special attention was given to database schema updates and migrations, highlighting the importance of careful planning and automation. Together, these practices form the foundation for building resilient, maintainable, and scalable applications in DevOps and cloud-native environments.

Answers to Guided Exercises

1. Design a simple user registration microservice that follows stateless principles and is suitable for deployment in a containerized environment. Describe the API endpoints it would expose, how it would handle session management, and how data would be stored.

A stateless user registration microservice could expose a `/register` endpoint that accepts a POST request with JSON-formatted user data, such as `name`, `email`, and `password`. Upon receiving the request, the service would validate the input, hash the password, and store the user data in its own relational database (e.g., PostgreSQL). It would return a `201 Created` status with the new user ID. The service would not store session data locally. Instead, it would issue a JSON Web Token (JWT) to the client, which the client can present in subsequent requests to other services.

2. You are managing a microservice that uses a PostgreSQL database. A new feature requires adding a column to a table used in production. Describe how you would implement this schema change safely with zero downtime.

To perform a zero-downtime migration, the new column should be added to the table using an additive schema change that does not interfere with existing operations. For example, an `ALTER TABLE` command can be used to add the column with a default value or set it as nullable. The application code should then be updated to begin writing data to the new column while continuing to read from the original structure. After confirming that the updated code is functioning correctly in production, existing data can be gradually migrated to populate the new column, potentially using background jobs or batch processes. Once all clients are verified to be using the new structure, the old column can be safely removed in a later release.

3. You are deploying multiple instances of a microservice behind a load balancer in Kubernetes. Describe how the system should handle service health checks to ensure that traffic is routed only to healthy instances.

In a Kubernetes environment, define both liveness and readiness probes in the deployment configuration for the microservice. The liveness probe checks whether the application is still running and capable of recovering, while the readiness probe determines whether the instance is ready to receive traffic. The load balancer would rely on the readiness probe to decide whether to forward requests to a given pod. If the readiness check fails—for example, due to initialization errors or dependency failures—Kubernetes would temporarily remove that instance from the service endpoints. The probes might target an internal health endpoint such as `/healthz` that performs lightweight checks on service status and database connectivity. This setup ensures that only healthy, fully initialized instances handle live traffic, reducing the risk of failed requests and improving service reliability.

Answers to Explorational Exercises

1. The lesson introduces Kubernetes as an orchestration platform for managing containerized microservices. Choose a real-world company or open-source project that uses Kubernetes, and describe how it manages application deployment, scaling, and service discovery.

Spotify is a well-known example of a company that uses Kubernetes to manage its microservices infrastructure. With hundreds of services running, Spotify uses Kubernetes to schedule containers across clusters, scale services automatically based on resource metrics, and manage rolling updates with zero downtime. Service discovery is handled through Kubernetes's built-in DNS system, allowing services to find each other by name. Spotify also integrates Kubernetes with internal CI/CD pipelines to enable frequent, automated deployments. To ensure observability, they rely on tools such as Prometheus for monitoring and Fluentd for log aggregation.

2. Describe a secure way to implement authentication and authorization in a RESTful API. Include how the API should validate client identity, restrict access to protected resources, and ensure that data is transmitted securely. Mention at least one standard method or technology used for this purpose.

A common and effective way to secure a RESTful API is to use token-based authentication with JSON Web Tokens (JWT). When a user logs in through a `POST` request to a `/login` endpoint, the server verifies the credentials and responds with a cryptographically signed JWT. This token contains encoded claims such as the user ID and roles, and is sent by the client in the authorization header of each subsequent request using the bearer scheme. The API validates the token signature on each request and checks the claims to determine whether the user is authorized to access the requested resource. For example, a user with the “admin” role may have access to `DELETE /users/{id}`, while a regular user may not. Additionally, the API should be served over HTTPS to encrypt data in transit and prevent interception or tampering. Rate limiting, input validation, and Cross-Origin Resource Sharing (CORS) policies can be used to further protect the API from abuse or malicious input. Using standardized protocols along with secure token handling ensures that RESTful APIs remain both user-friendly and resistant to common security threats.



**Linux
Professional
Institute**

701.2 Standard Components and Platforms for Software

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 701.2

Weight

3

Key knowledge areas

- Features and concepts of object storage
- Features and concepts of relational and NoSQL databases
- Features and concepts of message brokers and message queues
- Features and concepts of big data services
- Features and concepts of computing services / IaaS
- Features and concepts of application runtimes / PaaS
- Features and concepts of hosted applications / SaaS
- Features and concepts of function applications / FaaS
- Features and concepts of content delivery networks
- Awareness of identity and access management in cloud services

Partial list of the used files, terms and utilities

- Objects, Buckets, ACLs, S3
- MariaDB, MySQL, PostgreSQL,
- Redis, MongoDB, InfluxDB
- Elasticsearch and OpenSearch

- Kafka, MQTT
- IAM



**Linux
Professional
Institute**

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.2 Standard Components and Platforms for Software
Lesson:	1 of 1

Introduction

As modern IT solutions evolve to embrace cloud-native architectures, containerization, and service-oriented design, it becomes essential to understand the foundational infrastructure and service models that support this ecosystem. From persistent storage and scalable compute resources to real-time messaging and globally distributed content delivery, cloud technologies provide the building blocks for deploying secure, efficient, and resilient applications at scale.

This lesson explores the core features and concepts behind a wide range of cloud-based technologies and services. It begins with the principles of object storage. Then follows a comparison of relational and NoSQL databases, highlighting their appropriate use cases in microservice and big data architectures. The lesson then introduces message brokers and queues, which facilitate asynchronous communication in distributed systems, and examines the role of big data services in processing and analyzing high-volume datasets.

The lesson continues with the four primary cloud service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), Software as a Service (SaaS), and Function as a Service (FaaS). The lesson concludes with an overview of content delivery networks (CDNs). These topics equip professionals with the conceptual tools needed to design, deploy, and operate modern applications that are high-performing, secure, and cloud-ready.

Features and Concepts of Object Storage

Object storage is a fundamental component of modern, scalable infrastructure, particularly in cloud-native environments and distributed systems. Unlike traditional *file* or *block* storage, object storage manages data as discrete units called *objects*, each of which includes the data itself, associated metadata, and a unique identifier. This structure allows for massive scalability and efficient storage of unstructured data such as images, videos, backups, and logs.

Objects are organized within *buckets*, which serve as logical containers for grouping and managing related data. Buckets are created and managed at the storage service level, and define the *namespace* within which object keys must be unique. Each object can be accessed via a unique URL constructed from the bucket name and object key, making integration with web-based or distributed applications straightforward.

Access to object storage resources is governed by *access control lists* (ACLs) and sometimes more advanced policy mechanisms. ACLs define who can read, write, or manage specific objects or buckets. The ACLs can be applied at both the bucket and object levels, allowing fine-grained control over data access. This flexibility is essential in multi-tenant environments or in systems that require strict separation between public and private data.

A widely adopted implementation of object storage is Amazon S3 (*Simple Storage Service*), which has become a de facto standard in the industry. S3 provides a RESTful API interface that allows clients to perform operations such as uploading, retrieving, and deleting objects. It also supports features such as versioning, lifecycle policies, server-side encryption, and multi-region replication.

The following table shows some storage solutions offered by popular cloud vendors.

Table 1. Storage solutions offered by popular cloud vendors

CLOUD STORAGE			
	Object Storage	Block Storage	File/Network Storage
Amazon	Amazon S3	Amazon EBS	Amazon EFS and FSx
Microsoft	Azure Blob Storage	Azure Disk Storage	Azure Files
Google	Cloud Storage	Persistent Disk	Filestore

Features and Concepts of Relational and NoSQL Databases

In modern software architecture, understanding the strengths and use cases of both *relational* and *NoSQL* databases is essential for selecting the right data storage strategy. These two categories offer significantly different data modeling, query capabilities, performance characteristics, and consistency guarantees, making them suitable for different application requirements.

Relational databases follow the structured model of *tables* with predefined *schemas*. Data is organized in *rows* and *columns* (or *fields*), and relationships between entities are expressed through *foreign keys* and *joins*. Relational databases are well-suited for applications requiring transactional consistency, strong data integrity, and complex queries. They use *Structured Query Language* (SQL) as the primary language for data manipulation and retrieval. Common features of relational databases are ACID compliance, indexing, stored procedures, and robust replication mechanisms.

Some of the most widely used relational databases in Open Source environments are MariaDB, MySQL, and PostgreSQL. MariaDB and MySQL share a common ancestry and are often used interchangeably. MariaDB, developed as a community-driven fork of MySQL, tends to include newer features and performance enhancements.

PostgreSQL is recognized for its advanced capabilities, including full support for JSON data types, powerful indexing options, extensibility through custom functions, and strict compliance with SQL standards. PostgreSQL is often preferred in enterprise environments that require complex data modeling and analytics.

In contrast to relational databases, NoSQL databases are designed to handle large volumes of semi-structured or unstructured data with high availability and horizontal scalability. NoSQL is a broad category that includes key-value stores, document databases, wide-column stores, and time-series databases.

MongoDB is a widely adopted document-oriented database that stores data in flexible JSON-like documents. It allows dynamic schemas, making it suitable for applications whose data model evolves frequently or is not strictly defined up front. MongoDB is often used in *content management systems* (CMS), user profile stores, and mobile application backends.

Redis is a high-performance in-memory key-value store commonly used for caching, session management, and real-time analytics. Its low-latency access makes it ideal for reducing database load and improving response times in web applications.

For time-series data such as monitoring metrics, sensor readings, and financial data, InfluxDB

offers specialized features such as retention policies, downsampling, and time-based queries. It is optimized for fast ingestion and querying of chronological data and integrates well with observability tools in DevOps pipelines.

For full-text search, log analysis, or real-time data exploration, technologies like Elasticsearch and OpenSearch are commonly used. These engines index large volumes of textual data and support advanced search queries, filtering, and aggregations. Elasticsearch, originally built on the Apache Lucene library, has become a leading solution for search and analytics. OpenSearch, maintained by an open source community under the guidance of AWS, is a fork of Elasticsearch and Kibana, offering similar functionality with an open governance model. Both Elasticsearch and OpenSearch are essential components in observability stacks such as the ELK (Elasticsearch, Logstash, Kibana) or OpenSearch Dashboards.

The choice between relational and NoSQL solutions depends on application requirements; different solutions are sometimes combined in a polyglot architecture. Systems that need strict data integrity, complex joins, and transaction guarantees typically benefit from relational databases. On the other hand, applications requiring horizontal scaling, flexible schemas, low-latency reads, or full-text search capabilities are better served by NoSQL databases.

Features and Concepts of Message Brokers and Message Queues

In distributed systems and service-based architectures, *message brokers* and *message queues* play a vital role in enabling asynchronous communication, decoupling services, and increasing system resilience. Rather than making direct API calls that are synchronous (that is, the service making the call waits for the other service to respond), a message broker allows messages to be published by one component and consumed by another, often at different times, and without requiring the sender and receiver to be aware of each other's presence or state. The sender *publishes* to a channel and the recipients *subscribe* to that channel to receive the messages.

A message queue temporarily holds messages in a buffer until they are processed by consumers. This pattern is essential for handling traffic spikes, balancing loads between services, and improving fault tolerance. If a downstream service is unavailable or slow, messages can still be queued, preventing the system from failing or becoming blocked. In addition to publish-subscribe models, message brokers also support routing, message filtering, and persistence, depending on the architecture and use case.

One of the most widely used technologies in this space is Apache Kafka, a distributed event streaming platform designed for high-throughput, low-latency data pipelines. Kafka uses a log-based architecture, where messages (*events*) are written to *topics*, and consumers can read from

those topics in real time or at their own pace. Kafka is fault-tolerant, horizontally scalable, and suitable for use cases such as real-time analytics, log aggregation, metrics collection, and asynchronous communication between microservices. Its support for message retention, partitioning, and consumer groups makes it ideal for building decoupled and reactive systems that process massive volumes of data.

For lightweight communication between IoT devices or systems with constrained resources, *Message Queuing Telemetry Transport* (MQTT) has been widely adopted. MQTT is a simple, efficient, and low-bandwidth publish-subscribe messaging protocol designed for devices that have limited connectivity or computational power. It is commonly used in smart sensors, telemetry systems, and home automation.

Both Kafka and MQTT serve as examples of how message-oriented middleware can be used to create scalable, fault-tolerant, and flexible systems.

Features and Concepts of Big Data Services

Big data services are designed to process, store, and analyze extremely large and complex datasets that exceed the capabilities of traditional database systems. These services enable organizations to gain insights from vast volumes of structured, semi-structured, and unstructured data collected from a variety of sources, including logs, sensors, transactions, user behavior, and social media streams. Big data platforms typically support high-throughput data ingestion, scalable storage, and parallel processing frameworks that can handle petabytes of information in distributed environments.

One of the core concepts in big data architectures is the separation of storage and computation. Data is often stored in distributed file systems such as *Hadoop Distributed File System* (HDFS), or in object storage like Amazon S3 or Azure Data Lake. These data stores are then accessed by compute engines such as Apache Spark and Presto to perform transformations, aggregations, and analytics on demand.

These systems scale horizontally by distributing processing tasks across many computer systems or nodes, guaranteeing performance and reliability even under heavy loads. Big data services often support batch processing for historical analysis as well as stream processing for real-time analytics. Big data services also frequently integrate with machine learning frameworks and support SQL-like interfaces for accessibility.

Features and Concepts of Computing Services: IaaS

Infrastructure as a Service (IaaS) is a fundamental model of cloud computing that provides

virtualized computing resources over the internet. IaaS allows organizations to provision and manage infrastructure components such as virtual machines (VMs), storage volumes, and network configurations without the need to purchase or maintain physical hardware. This approach offers high flexibility and scalability, enabling developers and system administrators to deploy applications on demand and adapt quickly to changing workloads. Popular IaaS providers include Amazon EC2, Microsoft Azure Virtual Machines, and Google Compute Engine.

A key feature of IaaS is the ability to provision resources programmatically via APIs or infrastructure-as-code tools like Terraform, Ansible, or CloudFormation. This allows teams to automate the creation, scaling, and teardown of virtual infrastructure as part of their CI/CD pipelines. IaaS platforms also offer options for autoscaling, load balancing, and high availability across multiple zones or regions. These capabilities are critical for ensuring application resilience and performance under variable traffic conditions. Additionally, users can select from a wide range of instance types optimized for compute, memory, storage, or GPU workloads.

IaaS services often integrate with other cloud-native components such as object storage, identity management, security groups, and monitoring tools. This ecosystem enables a full-stack operational model where compute resources are just one part of a broader platform.

Features and Concepts of Application Runtimes: PaaS

Platform as a Service (PaaS) is a cloud computing model that provides a ready-to-use environment for deploying, running, and managing applications without the complexity of managing the underlying infrastructure. PaaS solutions offer developers the core runtime components—including operating systems, programming language runtimes, web servers, and application frameworks—alongside integrated services such as databases, messaging, and authentication. This abstraction accelerates development workflows and allows teams to focus entirely on writing and delivering code.

A central feature of PaaS is its support for automated deployment and scaling. Developers typically deploy applications by pushing source code or containers to the platform, which then handles the rest: building, provisioning, routing, and scaling as needed. Services such as Heroku, Google App Engine, Azure App Service, and Red Hat OpenShift allow teams to deploy web services, APIs, and background jobs with minimal configuration. These platforms often include built-in CI/CD pipelines, logging, monitoring, and environment variable management, making them ideal for modern DevOps practices.

PaaS solutions also provide strong support for runtime portability and multi-language environments, enabling teams to build polyglot systems that span multiple services and technologies. For example, a single application may use Node.js for the frontend, Python for machine learning components, and PostgreSQL as the backing store—all orchestrated within the

same PaaS environment. This makes PaaS especially useful for microservices and cloud-native applications. By abstracting infrastructure and streamlining operations, PaaS helps reduce operational overhead and improves consistency across development, staging, and production environments.

Features and Concepts of Hosted Applications: SaaS

Software as a Service (SaaS) is a cloud computing model that delivers software applications to users over the internet as fully managed, hosted services. Unlike traditional software that requires installation, configuration, and maintenance on local systems, SaaS applications are accessed through a web browser or API and maintained entirely by the provider. This eliminates the need for customers to manage infrastructure, perform updates, or handle scalability. Well-known SaaS offerings include tools like Google Workspace, Microsoft 365, GitHub, Salesforce, and Slack.

One of the key features of SaaS is multi-tenancy, where a single application instance serves multiple users or organizations while isolating their data securely. SaaS platforms also typically offer automatic updates, applying new features, bug fixes, and security patches seamlessly without user intervention. Integration with identity providers (via SAML, OAuth, or OpenID Connect) allows centralized user management and secure access control.

Features and Concepts of Function Applications: FaaS

Function as a Service (FaaS) is a serverless computing model that allows developers to deploy individual *functions*: small, self-contained units of code that are executed in response to specific *events*. Unlike traditional applications that run continuously, FaaS functions are invoked on demand, automatically triggered by events such as HTTP requests, file uploads, database changes, or scheduled timers. The cloud provider handles all aspects of infrastructure management, including provisioning, scaling, and availability, allowing developers to focus solely on writing business logic.

A defining feature of FaaS is automatic scaling based on load. When a function is triggered, the platform creates an isolated runtime environment to execute it. Horizontal scaling is achieved by running multiple instances as needed to handle concurrent invocations. This model is highly cost-efficient because billing is typically based on the actual execution time and resource consumption, rather than pre-allocated compute resources.

Popular FaaS platforms include AWS Lambda, Azure Functions, Google Cloud Functions. OpenFaaS runs functions in Kubernetes environments. These platforms support multiple programming languages and integrate with a wide range of event sources.

Function applications are ideal for event-driven workloads, such as processing webhook calls, handling asynchronous background jobs, performing data transformations, or automating operational tasks. They are often used alongside APIs, queues, and cloud storage services to build modular, loosely coupled systems. However, since FaaS functions are stateless and have short execution lifespans, state management typically relies on external services like databases or object stores.

Features and Concepts of Content Delivery Networks (CDNs)

A *content delivery network* (CDN) is a distributed system of servers strategically placed across multiple geographic locations to deliver web content, media, and other digital assets to users with improved speed, reliability, and efficiency. Instead of serving content directly from a central origin server, a CDN caches static and dynamic assets—such as images, videos, CSS, JavaScript, and even entire HTML pages—closer to the end user. This significantly reduces latency, minimizes bandwidth usage on the origin server, and improves the user experience by reducing load times.

One of the key features of a CDN is geographic distribution, which enables content to be delivered from the edge server that is physically nearest to the client. This reduces the number of network hops required and helps mitigate the impact of congestion or regional outages. CDNs also provide intelligent caching, which automatically refreshes content when updates are detected or based on specified cache control headers. Many CDN providers include features such as TLS termination, DDoS protection, origin failover, and real-time traffic analytics, making them not only performance-enhancing tools but also critical components of a secure and resilient web infrastructure.

Popular CDN services include Cloudflare, Amazon CloudFront, Akamai, and Fastly. These platforms integrate seamlessly with modern web and cloud architectures, including static site generators, container-based deployments, and serverless applications.

Guided Exercises

1. What is the primary structural unit in object storage, and how does it differ from traditional file or block storage?

2. When should a developer choose a NoSQL database like MongoDB over a relational database like PostgreSQL?

3. What role do content delivery networks (CDNs) play in improving web application performance, and how do they achieve this?

Explorational Exercises

1. Research and compare two major CDN providers. What differentiates Cloudflare from Amazon CloudFront in terms of features or integration options?

2. Investigate how a popular FaaS platform (such as AWS Lambda or Google Cloud Functions) handles function versioning and deployment. How can teams manage changes safely in a production environment?

Summary

This lesson introduced essential components of modern cloud-native and service-based architectures, covering a range of foundational technologies that support scalable, distributed, and resilient applications. It began with the concepts behind object storage, highlighting how data is organized into buckets and accessed via unique URLs, often using services such as Amazon S3. It then explored data persistence through relational and NoSQL databases, including MySQL, PostgreSQL, MongoDB, Redis, and InfluxDB, explaining how different database types serve distinct performance and flexibility needs. The lesson also examined message brokers and queues such as Kafka and MQTT, as tools to decouple services and support asynchronous workflows.

Moving into cloud computing models, the lesson explained the characteristics and use cases of IaaS, PaaS, SaaS, and FaaS. It emphasized the automation, scalability, and operational benefits each model provides across the software delivery lifecycle. Finally, it explored the role of content delivery networks (CDNs) in enhancing performance and availability for end users by distributing content geographically.

Answers to Guided Exercises

1. What is the primary structural unit in object storage, and how does it differ from traditional file or block storage?

In object storage, the primary structural unit is the object. Each object contains the data itself, associated metadata, and a unique identifier. This differs from traditional file storage, which organizes data hierarchically in files and folders, and block storage, which divides data into fixed-size blocks managed by a file system. Object storage provides better scalability and is optimized for unstructured data such as images, videos, and logs.

2. When should a developer choose a NoSQL database like MongoDB over a relational database like PostgreSQL?

A developer should consider using a NoSQL database like MongoDB when the application deals with semi-structured or unstructured data, requires dynamic schemas, or needs to scale horizontally across distributed systems. MongoDB is well-suited for use cases like content management, user profiles, and applications whose data models evolve frequently. In contrast, relational databases like PostgreSQL are preferred when strong consistency, complex queries, and strict data integrity are required.

3. What role do content delivery networks (CDNs) play in improving web application performance, and how do they achieve this?

Content delivery networks (CDNs) improve web application performance by caching static and dynamic content closer to end users through a geographically distributed network of edge servers. This reduces latency, offloads traffic from the origin server, and accelerates load times. CDNs achieve this by serving content from the edge location nearest the user, and by automatically refreshing cached assets when updates occur, thereby improving both speed and availability across global user bases.

Answers to Explorational Exercises

1. Research and compare two major CDN providers. What differentiates Cloudflare from Amazon CloudFront in terms of features or integration options?

Cloudflare and Amazon CloudFront are both leading CDN providers, but they cater to slightly different needs. Cloudflare offers a globally distributed network with built-in DDoS protection, DNS name resolution, a web application firewall (WAF), and performance optimization. It emphasizes ease of use and provides generous free-tier services. Amazon CloudFront, tightly integrated with AWS services, offers deep customization for content delivery, advanced access control via signed URLs, and real-time log streaming.

While Cloudflare focuses on simplicity and security across multiple platforms, CloudFront excels in AWS-native deployments where tight integration with services like S3, Lambda@Edge, and IAM is a priority.

2. Investigate how a popular FaaS platform (such as AWS Lambda or Google Cloud Functions) handles function versioning and deployment. How can teams manage changes safely in a production environment?

In AWS Lambda, function versioning allows teams to publish immutable versions of their functions, each identified by a version number. Once published, a version cannot be changed, which ensures stability in production environments. Lambda also supports aliases, which act as pointers to specific versions and can be used to route traffic. For example, teams can implement *traffic splitting*, such as routing 90% of traffic to version 2 and 10% to version 3 during a phased deployment. This supports canary releases (testing a new feature on a small number of production systems) and minimizes the risk of introducing bugs. Similar mechanisms exist in Google Cloud Functions, which use revisions and traffic splitting. These tools help DevOps teams deploy updates safely while allowing instant rollbacks if issues occur.



701.3 Source Code Management

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 701.3

Weight

6

Key knowledge areas

- Understand Git concepts and repository structure
- Manage files within a Git repository
- Manage branches and tags
- Work with remote repositories and branches as well as submodules
- Merge files and branches
- Awareness of SVN and CVS, including concepts of centralized and distributed SCM solutions

Partial list of the used files, terms and utilities

- `git`
- `.gitignore`



Linux
Professional
Institute

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.3 Source Code Management
Lesson:	1 of 3

Introduction

In modern software development, version control is essential for managing source code, tracking changes, and enabling collaboration across teams. A version control system (VCS) allows developers to record the history of a project, revert to previous states, and coordinate parallel work through branching and merging. This forms the foundation for continuous integration (CI), automated testing, and the collaborative coding workflows that are central to DevOps and Agile methodologies. This lesson covers using Git to manage and share source code.

Concepts of Software Configuration Management (SCM)

Version control is a core component of a broader discipline known as *software configuration management* (SCM). SCM encompasses the processes and tools used to manage changes in software artifacts, including source code, documentation, configuration files, and dependencies. Its goals include maintaining consistency, traceability, and reproducibility throughout the software lifecycle. SCM helps prevent conflicts, track contributions, and ensure that changes are properly reviewed and integrated.

There are two primary types of version control systems: *centralized* and *distributed*. Centralized

systems, such as *Subversion* (SVN) and the older *Concurrent Versions System* (CVS), rely on a single, central server that hosts the authoritative version of the codebase.

A *distributed version control system* (DVCS) such as Git allows every developer to clone the full repository, including its entire history, onto their local machine. Changes can be committed, branched, and merged locally without immediate network access. This enables powerful offline workflows, faster operations, and better support for parallel development.

Git's Contribution to Version Control

Git was created in 2005 by Linus Torvalds, the creator of the Linux kernel, in response to the need for a fast, secure, and distributed version control system that was open source and could handle the complex and highly collaborative nature of Linux kernel development.

Git was designed with performance, data integrity, and strong support for non-linear development in mind. Its decentralized model, cryptographic tracking of changes, and efficient branching and merging mechanisms quickly made it the tool of choice not only for the Linux community but for software projects of all sizes around the world. Today, Git is the backbone of modern collaborative development platforms such as GitHub, GitLab, and Bitbucket.

One of Git's most significant advantages is its ability to manage lightweight branches, making it easy to experiment, isolate features, or apply fixes without disrupting the main development line. Its integrity model—based on SHA-1 hashes—ensures that all changes are cryptographically tracked, making it extremely reliable.

How Git Works: Remote Repository, Local Repository, and Staging Area

To understand how Git functions, it is important to grasp the relationship between three file collections: the remote repository, the local repository, and the staging area.

Remote Repository

The *remote repository* is typically hosted on a platform such as GitHub, GitLab, Bitbucket, or an organisation's own Git remote repository. It acts as the central reference point for all contributors and contains the shared version of the codebase on which the team collaborates. Developers can *push* changes to the remote repository or *pull* updates made by others.

Local Repository

Once cloned, the project exists as a *local repository* on the developer's computer. This local repository contains:

Working directory

Where files can be edited, created, or deleted.

.git directory

Hidden in the project root, this stores all the repository's history and metadata.

Staging area (index)

An intermediate space where changes are prepared before being committed.

Developers can work entirely offline in a local repository, performing commits, checking logs, creating branches, and resolving conflicts. Local commits remain invisible to others until they are pushed to the remote repository.

Staging Area (Index)

The *staging area* (or *index*) is a fundamental concept in Git. It serves as an intermediate space where changes are collected and reviewed before being committed to the repository. This mechanism provides precise control over what is included in each commit, enabling work to be organized into smaller, logical units and allowing temporary or experimental edits to be left out.

Before creating a commit, changes must be *staged* — that is, explicitly added to the staging area. This process makes it possible to select specific files or even individual lines for inclusion, ensuring that each commit represents a coherent and meaningful step in the project's history.

As an example of staging and committing, suppose that a developer modifies a file:

```
$ echo "New feature" >> app.py
```

Git sees this as a modified file but doesn't track it for the next commit yet. The state of the changed file can be shown by asking for the local repository's status:

```
$ git status
On branch main

No commits yet
```

Untracked files:

(use "git add <file>..." to include in what will be committed)

app.py

nothing added to commit but untracked files present (use "git add" to track)

To stage the change, enter:

```
$ git add app.py
```

Now the file `app.py` is in the staging area. Git is tracking this version and will include it in the next commit. The Git status shows files ready for next commit:

```
$ git status
```

On branch main

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)

new file: app.py

A *commit* saves staged changes to the user's local repository with a message that describes what the commit is about. To save the staged changes to the local repository, enter a `commit` command:

```
$ git commit -m "Add new feature to app.py"
```

This creates a new snapshot in the local repository.

The `-m` option in the `git commit` command stands for “message”. It allows the user to provide a commit message directly from the command line, avoiding the need to open a text editor.

If a file was accidentally left out of a commit, the command `git commit --amend` can be used to modify the most recent commit. This option is helpful for adding forgotten changes or updating the previous commit message, keeping the commit history clean and accurate.

Common use cases for the `--amend` option include:

- A file was forgotten before committing.
- The commit message contains a typo or needs clarification.

- The contents of the most recent commit need adjustment without creating a new one. For example, after making a commit, suppose it becomes apparent that the file `forgotten_file.txt` was not included. To correct this, execute the following commands:

```
$ git add forgotten_file.txt
$ git commit --amend
```

To share the commit with the team, push it to the remote repository:

```
$ git push origin main
```

This command uploads the user's local commits to the remote repository's `main` branch.

Likewise, when teammates have pushed new updates to the remote repository, those changes can be integrated into the local copy by executing:

```
$ git pull origin main
```

This retrieves the latest commits and merges them into the developer's working branch. Workflow for a Git commit shows the typical steps for using the staging area and committing a change to the remote repository.

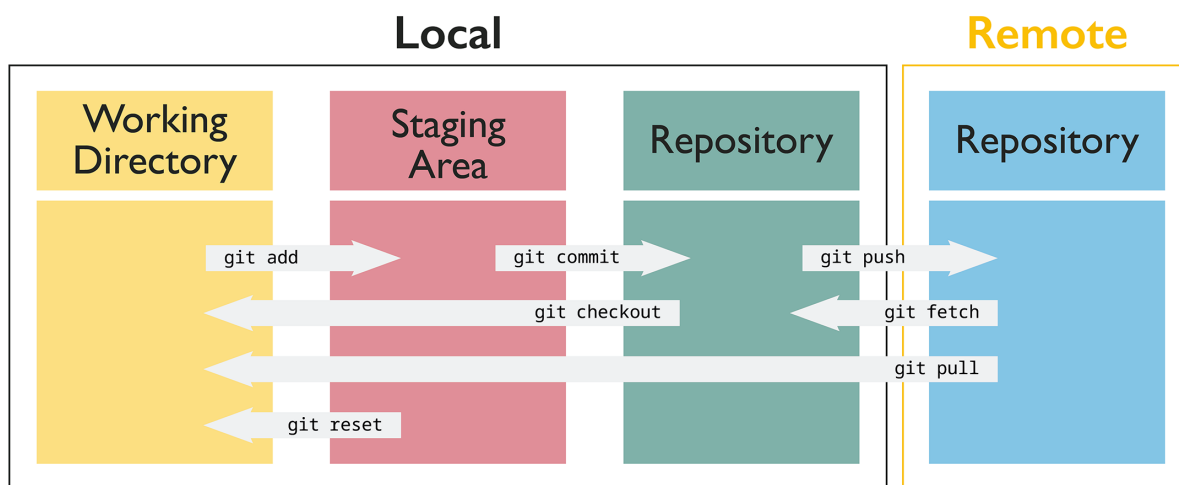


Figure 1. Workflow for a Git commit

The full Git workflow typically follows these steps:

1. Clone a remote repository once:

```
$ git clone <url>
```

2. Edit files in the working directory, e.g.

```
$ echo "New line" >> README.md
```

3. Stage changes:

```
$ git add .
```

4. Commit staged changes:

```
$ git commit
```

5. Push local commits to the remote repository:

```
$ git push
```

6. Pull updates from the remote repository:

```
$ git pull
```

Another command, `git init`, initiates a new repository locally. This option is explained later.

Git Structure and Internal Storage

At its core, Git is a *content tracker*. Unlike traditional version control systems that store differences (or *deltas*) between file versions, Git captures and stores snapshots of the entire project state over time.

When the developer commits a change, Git records a snapshot of the current contents of all tracked files and saves a reference to that snapshot. If a file hasn't changed, Git doesn't duplicate it; instead, it points to the previous identical version. This model makes Git fast, efficient, and reliable, especially when managing large repositories.

All of Git's internal information is stored inside a hidden directory called `.git`, located at the root of every local repository. This directory contains all the metadata, object data, configuration settings, logs, and references required to manage the repository. Its contents look like:

```
.git/
├── HEAD
├── config
├── description
├── hooks/
├── info/
│   └── exclude
├── objects/
│   ├── pack/
│   └── info/
├── refs/
│   ├── heads/
│   └── tags/
└── index
```

Inside `.git`, several subdirectories and files serve specific purposes:

objects

contains all the data stored by Git: blobs (binary large objects), commits, and trees, each identified by an SHA-1 hash.

- Blobs are compressed snapshots of every file version that has been committed.
- Trees are Git's version of directories: they map filenames to blob hashes and contain permissions metadata. Each commit points to a tree.
- Commits are metadata packages containing author information, commit messages, and pointers to parent commits.

refs

holds pointers to commit objects, including branches and tags.

HEAD

acts like a compass needle, always pointing to the current position where the developer is working.

config

holds repository-specific configuration settings such as the configured remotes (such as GitHub URLs), branch tracking relationships, and custom user settings that override global Git configs.

index

a binary file, is Git's staging area: a temporary loading dock where changes wait before being committed. When `git add` is issued to add files, Git updates the index with its current state.

hooks

is Git's automation toolkit. The directory contains templates for pre-commit scripts that run before a commit and post-receive scripts that are triggered after pushes.

info

contains repository-specific configurations that include attributes for path-specific Git behaviors.

logs

is Git's "black box" recorder. It maintains reference histories (`HEAD` and branches) enabling commands like `git reflog` to recover lost commits.

Cloning a Git Repository

In Git, *cloning* creates a full copy of an existing repository, including all files, branches, commit history, and metadata. This is the most common way for developers to begin working on a project that already exists on a remote server, such as GitHub, GitLab, or a self-hosted Git server. Once the remote repository is cloned, the developer has a fully functional local repository, which includes both a working directory and the full version history stored inside the `.git` folder.

When a repository is cloned, Git downloads all of the project's content and its complete history from the remote location. By default, Git also sets up a connection to the original source as a remote named `origin`, so the developer can easily fetch updates, push commits, and collaborate with others.

Here's an example of cloning a public repository:

```
$ git clone https://git.example.com/examples/project.git my-local-copy
```

This command performs the following steps:

1. Creates a new directory called `my-local-copy`.
2. Initializes a `.git` directory inside it.
3. Downloads all objects and refs (branches, tags).
4. Checks out the latest commit from the default branch (typically `main` or `master`).

After cloning, a developer can immediately begin working with the files, create branches, make commits, or push changes (if the developer has write access).

Working With Remotes After Cloning

When a repository is cloned, *Git* automatically creates a reference to the remote repository under the default name `origin`.

This alias acts as a shortcut for the remote URL, simplifying commands that interact with the original repository.

```
$ *git remote -v
origin https://git.example.com/examples/project.git (fetch)
origin https://git.example.com/examples/project.git (push)
```

To display detailed information about a specific remote repository, including its fetch and push URLs, tracking branches, and synchronization status, use the following command:

```
$ git remote show origin
* remote origin
Fetch URL: https://git.example.com/examples/project.git
Push URL: https://git.example.com/examples/project.git
HEAD branch: main
```

To fetch the latest changes from the remote (without merging):

```
$ git fetch origin
```

The `git fetch origin` command retrieves updates from the remote repository, identified by the alias `origin`, and downloads them into the local repository. Unlike `git pull`, this operation does not automatically merge or alter the working directory.

Instead, it updates the local references to the remote branches, enabling the user to inspect incoming commits, compare differences, and decide when or how to integrate those changes into the local branch.

To download the most recent commits from the remote repository and merge them into the current branch, execute the following command:

```
$ git pull origin main
```

To upload the local commits to the remote repository, use the following command:

```
$ git push origin main
```

Initializing a Local Git Repository and Connecting to a Remote

Although many developers begin by cloning an existing project, it is equally common to start entirely from scratch — particularly when creating a new application, script, or configuration repository.

In such scenarios, *Git* provides the `git init` command, which converts any local directory into a Git-enabled project.

Once the repository is initialized, changes can be tracked locally, commits can be created, and the project can later be connected to a remote repository (for example, on *GitHub* or *GitLab*) to enable collaboration, version synchronization, and offsite backups.

Step 1: Initialize a Local Repository

To create a new local repository, first navigate to the desired project directory or create a new folder for it:

```
$ mkdir my-project  
$ cd my-project  
$ git init
```

This initializes, within the `myproject` directory, a hidden folder named `.git`, which stores all of Git's internal data, including commit history, configuration files, and branch information.

At this stage, the project becomes a fully functional local Git repository, even though it is not yet linked to any remote server.

Files can now be tracked and managed using commands such as:

```
$ echo "# My Project" > README.md  
$ git add README.md  
$ git commit -m "Initial commit with README"
```

At this stage, Git is tracking the file `README.md` locally. Commits can be created, branches managed, and the full project history reviewed — all entirely offline. A network connection is only required when synchronizing with a remote repository or collaborating with others.

Step 2: Create a Remote Repository

Once the local repository is configured, the next step is to connect it to a remote repository to enable collaboration, centralized backups, and code sharing.

On *GitHub*, a new remote repository can be created as follows:

1. Navigate to <https://github.com> and sign in to an existing account or create a new one.
2. Click **New repository**.
3. Provide a name for the repository, such as `my-project`.
4. Leave the options to initialize with a `README`, `license`, or `.gitignore` file **unchecked**, since those files already exist locally.
5. Click **Create repository**.

After creation, GitHub will display a remote repository URL similar to:

```
https://github.com/<your-username>/my-project.git
```

Step 3: Connect the Local Repository to the Remote

In the terminal, link the local repository to the remote repository by executing:

```
$ git remote add origin https://github.com/<your-username>/my-project.git
```

This command configures the remote named `origin` to reference the GitHub repository.

To upload the local commits to the remote repository, execute:

```
$ git push -u origin main
```

The `-u` option establishes an *upstream tracking relationship* between the local branch and its corresponding remote branch.

Once this link is configured, subsequent operations can be simplified — future updates can be

synchronized using only the commands:

Managing Files Within a Git Repository

Some of the most common and essential operations in Git involve file management: adding new files, tracking modifications, renaming files, and removing files. Git provides a concise yet powerful set of commands that make it possible to maintain a clean, organized, and traceable project history. Understanding how these commands interact with the working directory, staging area, and repository history is key to effective version control.

Adding New Files

To begin tracking a new file, create it in the working directory and add it to the staging area using the `git add` command. Example:

```
$ echo "# Project Title" > README.md
$ git add README.md
$ git commit -m "Add project README"
```

This creates a new file `README.md`, stages it, and commits it to the local repository. The file is now tracked, and its history will be preserved by Git.

Modifying Tracked Files

When a file already tracked by Git is modified, the system automatically detects the change. However, the modification must be explicitly *staged* before the next commit:

1. Append a new line to `README.md`:

```
$ echo "More details about the project." >> README.md
```

This adds a single line at the end of `README.md`.

2. Inspect the working tree:

```
$ git status
On branch main
Changes not staged for commit:
  modified:   README.md
```

The `git status` command displays the current state of the working directory and staging area, indicating which files have been modified, which are staged for commit, and which remain untracked.

Running `git status` before committing is a best practice, helping to ensure that all intended changes are included and preventing accidental omissions or incomplete commits.

3. Review unstaged changes:

```
$ git diff
diff --git a/README.md b/README.md
index fed3d1c..3fc20df 100644
--- a/README.md
+++ b/README.md
@@ -1,2 @@
 # Project Title
+More details about the project.
```

This shows differences between the working directory and the last commit (only unstaged edits).

4. Stage the file:

```
$ git add README.md
```

Moves `README.md` from *modified* to *staged*.

5. Review staged changes:

```
$ git diff --cached
diff --git a/README.md b/README.md
index fed3d1c..3fc20df 100644
--- a/README.md
+++ b/README.md
@@ -1,2 @@
 # Project Title
+More details about the project.
```

This shows what is staged and will be included in the next commit. The `git diff` command shows working directory vs. last commit (unstaged changes). The `git diff --cached` (or `--staged`) command shows staging area vs. last commit (what will be committed).

6. Commit the staged changes:

```
$ git commit -m "Update README with project details"
```

Creates a new commit containing the staged modifications.

Deleting Files

To remove a file from both the working directory and the repository, use `git rm`:

```
$ git rm obsolete.txt
$ git commit -m "Remove obsolete file"
```

Renaming or Moving Files

Git can detect when a file has been renamed or moved, especially when using `git mv`, which stages the operation automatically:

```
$ git mv old_name.py new_name.py
$ git commit -m "Rename script from old_name.py to new_name.py"
```

Alternatively, a file can be renamed manually using standard system commands, followed by updating Git's tracking information with `git add` and `git rm`.

This approach explicitly removes the old file reference and stages the new one:

```
$ mv old_name.py new_name.py
$ git add new_name.py
$ git rm old_name.py
$ git commit -m "Manual rename of Python script"
```

Stopping the Tracking of a File

To stop tracking a file while keeping it in the working directory, use the `git rm` command with the `--cached` option.

This removes the file from version control without deleting it from the local filesystem:

```
$ git rm --cached secrets.txt
```

```
$ echo "secrets.txt" >> .gitignore
$ git commit -am "Stop tracking secrets.txt and add to .gitignore"
```

This is especially useful to stop tracking files that shouldn't be versioned, such as sensitive configuration files or generated artifacts.

Viewing the Commit History

The `git log` command displays the commit history of the current branch. Each entry in the log includes the commit hash, the author, the timestamp, and the commit message, providing detailed context for each change. Basic usage is:

```
$ git log
commit a1b2c3d4e5f6...
Author: Jane Developer <jane@example.com>
Date:   Tue Mar 26 10:15:00 2024 -0300
    Fix login bug and update error message
```

This shows the full commit history in reverse chronological order (most recent commits first).

The `commit` line in the log output contains the full *SHA-1* hash that uniquely identifies each commit.

Because this hash is a long 40-character hexadecimal string, using it directly in commands can be inconvenient.

To simplify this, *Git* automatically displays and accepts shortened versions of the *SHA-1* hash, as long as they remain unique within the repository.

These abbreviated identifiers can be used safely in most commands, such as `git show`, `git checkout`, or `git revert`, making navigation and reference to specific commits faster and more practical.

Reverting File Changes

The `git restore` command is designed to discard changes in the working directory or recover previous versions of files from the repository history.

It is particularly useful for reverting unintended modifications before they are staged or committed, allowing the working directory to return to a clean and consistent state.

Discarding Unstaged Changes

If modifications have been made to a file but not yet staged, the file can be reverted to its last committed state using:

```
$ git restore filename.txt
```

This will undo local modifications and revert the file to its last committed version.

Restoring a File from Another Branch or Commit

A file can also be restored from a specific commit or branch, allowing recovery of an earlier version or comparison between revisions.

```
$ git restore --source=my-branch README.md
```

This command copies the version of `README.md` from the branch `my-branch` into the current working directory without changing the active branch. It is a safe way to retrieve or compare specific files from another branch without altering the current workspace.

Unstaging a File

Unstaging a file removes it from the staging area while preserving its modifications in the working directory.

This action effectively tells Git not to include the file in the next commit, without discarding any local changes.

The equivalent command is:

```
$ git restore --staged filename.txt
```

The `.gitignore` File: Purpose and Syntax

In every Git project, developers often have files that should not be tracked by version control, such as temporary build files, editor backups, OS-generated artifacts, or sensitive credentials. The `.gitignore` file provides a mechanism to explicitly tell Git which files or directories to exclude from tracking.

The `.gitignore` file defines which files and directories Git should ignore when tracking changes.

It is typically placed in the root of the repository but can also exist in subdirectories to apply localized ignore rules.

Each line in the file represents a pattern that matches specific files or folders to exclude from version control.

```
*.log          # Ignore all .log files
build/         # Ignore the entire build/ directory
*.env          # Ignore environment variable files
!important.env # Do not ignore this specific file
```

Git evaluates these rules in the order they appear, and it supports globbing patterns such as:

wildcard matching

?

single-character matching

****/**

matching directories recursively

As shown in the previous example, **!** negates a pattern to explicitly include something that was previously ignored.

It is important to note that `.gitignore` affects only untracked files. If a file is already being tracked by Git, adding it to `.gitignore` will not stop Git from continuing to track it.

To remove a tracked file that should be ignored now, it must first be removed from the index:

```
$ git rm --cached another-file.txt
```

Then Git will respect the `.gitignore` rule for future changes.

Guided Exercises

1. Create a new Git repository called `demo-repo`, add a `hello.txt` file, and commit it with the message `Initial commit with hello.txt`.

2. You created a `readme.md` file but haven't staged it yet. Use Git to check the current status.

3. Display a list of commits in the current branch.

4. You made a change to `hello.txt` but want to discard the change before staging. What command do you enter to accomplish that?

5. Connect your local repository to a remote GitHub repository and push your commits.

6. What kind of information does the `git status` command display?

Explorational Exercises

1. Research how to sign Git commits using GPG or SSH keys. Enable commit signing in your local Git configuration and make a signed commit.

2. Choose a programming language or framework you use (e.g., Python, Node.js, Java) and research what should typically go into a `.gitignore` file for that environment.

Summary

Git is a powerful, distributed version control system that enables developers and DevOps professionals to track changes, collaborate effectively, and manage source code across multiple environments. Unlike centralized systems, Git allows every user to maintain a full copy of the project history, enabling offline work, rapid branching, and seamless merging. It supports workflows ranging from solo development to large-scale enterprise collaboration and integrates smoothly with modern tools like GitHub and GitLab. With features such as a staging area, commit history, and branching strategies, Git provides the foundation for reliable, scalable, and auditable software delivery.

This lesson offered the basic commands for working locally and with other developers, along with the concepts behind the commands and workflows.

Answers to Guided Exercises

1. Create a new Git repository called `demo-repo`, add a `hello.txt` file, and commit it with the message `Initial commit with hello.txt`.

```
$ mkdir demo-repo
$ cd demo-repo
$ git init
$ echo 'Hello, Git!' > hello.txt
$ git add hello.txt
$ git commit -m 'Initial commit with hello.txt'
```

2. You created a `readme.md` file but haven't staged it yet. Use Git to check the current status.

```
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  readme.md
```

3. Display a list of commits in the current branch.

```
$ git log
commit 3c20fe7c615fbfdb22931b2e61296d306593b510 (HEAD -> main)
Author: Jane Developer <jane@example.com>
Date: Thu Mar 27 19:53:33 2025 -0300

    Initial commit with hello.txt
```

4. You made a change to `hello.txt` but want to discard the change before staging. What command do you enter to accomplish that?

```
$ git restore hello.txt
```

5. Connect your local repository to a remote GitHub repository and push your commits.

```
$ git remote add origin https://github.com/<your-username>/demo-repo.git
$ git branch -M main
```

```
$ git push -u origin main
```

6. What kind of information does the `git status` command display?

The `git status` command displays the current state of the working directory and the staging area. It shows which files have been modified but not yet staged for commit, which files are staged and will be included in the next commit, and any untracked files that are not yet under version control. It does not show unchanged files or locked files, as Git does not use a locking mechanism by default.

Answers to Explorational Exercises

1. Research how to sign Git commits using GPG or SSH keys. Enable commit signing in your local Git configuration and make a signed commit.

To sign Git commits using GPG, first generate a GPG key:

```
$ gpg --full-generate-key
```

After generating the key, list it:

```
$ gpg --list-secret-keys --keyid-format=long
```

Configure Git to use it:

```
$ git config --global user.signingkey <YOUR_KEY_ID>
$ git config --global commit.gpgsign true
```

You can now sign commits with:

```
$ git commit -S -m "My signed commit"
```

2. Choose a programming language or framework you use (e.g., Python, Node.js, Java) and research what should typically go into a `.gitignore` file for that environment.

A typical `.gitignore` file for a Node.js project includes:

```
# Node modules
node_modules/

# Logs
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Runtime files
pids
*.pid
*.seed
```



```
# Environment variables
.env

# Build output
dist/
build/

# IDE settings
.vscode/
.idea/
```



**Linux
Professional
Institute**

Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.3 Source Code Management
Lesson:	2 of 3

Introduction

In Git, branches provide a flexible way to work on multiple tasks in parallel without interfering with the main codebase, while tags are useful for identifying stable points in a project's history, such as release versions. Together, branches and tags are critical for implementing feature development workflows, managing software releases, and maintaining a clean and traceable commit history.

Understanding how to create, switch, merge, and delete branches—as well as how to tag and share important releases—is a foundational skill for any Git user involved in collaborative or production-grade development.

Tags: Marking Specific Versions

Like many version control systems, Git provides the ability to tag specific commits to identify significant points in a project's history. This feature is most commonly used to mark release versions—such as v1.0, v2.0, etc.—and other milestones. Tags are immutable: they remain attached to the commit they were assigned to and do not move as new commits are added.

Creating a Tag

Git provides support for two types of tags: *lightweight* and *annotated*. A lightweight tag acts as a simple reference to a specific commit. It contains no additional metadata and is typically used for temporary or local tagging purposes.

In contrast, an annotated tag includes the tagger's name and email address, the current date, a message, and optionally, a cryptographic signature using GNU Privacy Guard (GPG). Annotated tags are also checksummed and are the preferred format for marking official release points or significant milestones in the project.

A tag must be created after a `git commit` to tag this commit with a specific mark version.

An example follows of creating a lightweight tag. First create a new commit:

```
$ git add .  
$ git commit -m "Final version 1.0"
```

Now create a tag for this commit:

```
$ git tag v1.0
```

To create an annotated tag (recommended for releases), use the `-a` option:

```
$ git tag -a v1.0.1 -m "First stable release"
```

This creates a tag with metadata such as the author, date, and message.

To view tag details, enter:

```
$ git show v1.0.1  
tag v1.0.1  
Tagger: Jane Developer <jane@example.com>  
Date:   Wed Aug 6 20:07:14 2025 -0700  
  
First stable release  
...
```

Listing Tags

To view all tags in the repository, enter `git tag`, optionally using the `-l` or `--list` option for more control over the output:

```
$ git tag
v1.0
v2.0
```

By default, this command displays all tags in alphabetical order, although the sequence has no functional significance.

To filter tags by a specific pattern, use wildcards. For example, if the Git source repository contains hundreds of tags, but you are interested only in those from the `v2.1` release series, run the following command to narrow the results:

```
$ git tag -l "v2.1*"
v2.1
v2.1-rc0
v2.1.1
```

Pushing Tags to a Remote Repository

By default, the `git push` command does not automatically transfer tags to a remote repository. To share tags with others, the developer must explicitly push them after creating them.

Use `git push origin <tagname>` to upload a specific tag to the remote server.

```
$ git push origin v1.0.0
```

To push all tags from the local repository to the remote one, enter:

```
$ git push origin --tags
```

Deleting a Tag

To remove a tag from the local repository, use the command `git tag -d <tagname>`. For example, to delete the lightweight tag created earlier locally, run:

```
$ git tag -d v1.0
```

To delete a tag from the remote repository, enter:

```
$ git push origin --delete v1.0.0
```

Branches: Working in Parallel

A *branch* in Git represents an independent line of development. Branching allows developers to diverge from the main line of development and work independently without affecting the primary codebase.

In many traditional version control systems, creating a branch can be resource-intensive. It often involves the duplication of the entire source code directory, which may be time-consuming and inefficient, especially for large projects.

Git's approach to branching is exceptionally lightweight and efficient, allowing branch creation and switching to happen almost instantly. Git is designed to encourage frequent branching and merging, even multiple times within the same day.

By default, every Git repository starts with a branch called `main` (or `master` in older repositories). Creating a new branch allows a developer to isolate some work, such as developing a feature or fixing a bug, without affecting the main codebase.

Creating a New Branch

Creating a new branch in Git means essentially creating a new pointer to a specific commit in the repository's history. For example, to create a branch named `feature-login`, use the following command:

```
$ git branch feature-login
```

This creates a branch named `feature-login`, but does not switch to it.

To see the available branches, enter:

```
$ git branch
feature-login
```

```
* main
```

The `*` symbol indicates which branch is currently checked out.

To both create and switch at once, enter:

```
$ git checkout -b feature-login
```

The `-b` option tells Git to create a new branch with the given name and switch to that branch.

Alternatively, starting from Git 2.23, the `git switch` command offers a clearer and more user-friendly way to create and switch branches.

Instead of using `git checkout -b`, the following command can be used:

```
$ git switch -c feature-login
```

The `-c` option stands for “create,” indicating that a new branch should be created and checked out immediately.

Alternatively, the full form `--create` can be used for better readability and clarity in scripts or documentation.

Switching to an Existing Branch

To switch to an existing branch, use the `git checkout` or `git switch` command followed by the branch name. For example, to switch to the `testing` branch, run:

```
$ git checkout testing
```

Or use `git switch`:

```
$ git switch testing
```

Return to your previously checked out branch by entering:

```
$ git switch -
```

Deleting an Existing Branch

To delete a branch `feature-login` that is no longer needed, enter:

```
$ git branch -d feature-login
```

If an attempt is made to delete a branch that has not yet been merged, Git displays an error message indicating that the branch is not fully merged. This safeguard prevents accidental loss of unmerged work.

To override this protection and force the deletion, use the uppercase `-D` option:

```
$ git branch -D feature-login
```

Setting Upstream Tracking for Remote Branches

Branches can also track remote counterparts:

```
$ git push -u origin feature-login
```

This remembers the upstream repository so that future pushes and pulls can be done simply with `git push` and `git pull`.

Merging Branches

Suppose you have switched to the branch called `testing` to develop a feature or fix a bug. After making the necessary changes, you stage and commit them on this branch using `git add` and `git commit`:

```
$ git switch testing
$ echo 'some changes' >> readme.md
$ git add .
$ git commit -m 'changes made on testing branch'
```

Once you determine that the changes made in the `testing` branch are ready, you can merge them back into the `main` branch after the work is complete:

```
$ git checkout main
```

```
$ git merge testing
```

Merge Conflicts

Sometimes, the merge process doesn't proceed seamlessly. If two branches have made different changes to the same lines in the same file, Git will be unable to merge them automatically. In such cases, a *merge conflict* occurs. Merge conflicts are a normal part of collaborative development and are not errors in themselves—they simply indicate that multiple people (or branches) have made changes to the same code, and Git needs help deciding how to combine them.

Conflicts commonly occur when:

- Two branches modify the same line in a file differently.
- A file is deleted in one branch but modified in another.
- A file is renamed or moved differently in both branches.

As an example, create a new repository and add the initial file:

```
$ mkdir git-conflict-demo && cd git-conflict-demo
$ git init
```

Consider a repository where the `main` branch contains a file named `app.py`.

This file represents the base version of the project before any new feature or modification is introduced.

```
def greet():
    print("Hello, world!")
```

Commit the file:

```
$ git add app.py
$ git commit -m "Initial commit with greet()"
```

Create the first branch `feature/spanish-update` from `main` and modify the same line:

```
$ git checkout -b feature/spanish-update
```


Next, open the `app.py` file and replace the greeting text `"Hello, world!"` with `"¡Hola, mundo!"`.

Once the change is saved, stage the file and commit it:

```
$ git add app.py
$ git commit -m "Update greeting to Spanish"
```

Return to `main`, create another branch `feature/english-update` from the same commit, and again modify the same line:

```
$ git checkout main
$ git checkout -b feature/english-update
```

Open the `app.py` file again and replace the greeting text `"Hello, world!"` with `"Hello, everyone!"`.

Once the change is saved, stage the file and commit it:

```
$ git add app.py
$ git commit -m "Update greeting in English"
```

Merge the Spanish branch first. This should succeed:

```
$ git checkout main
$ git merge feature/spanish-update
```

Verify the contents of the `app.py` file to confirm the update:

```
$ cat app.py
def greet():
    print("¡Hola, mundo!")
```

Now try merging the English branch. This will produce a conflict, since both branches changed the same line:

```
$ git merge feature/english-update
Auto-merging app.py
```

```
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and then commit the result.
```

Don't worry! Just view `app.py` as shown in the next section and resolve the conflict.

What a Merge Conflict Looks Like

Git marks the conflicting file like this:

```
$ cat app.py
def greet():
<<<<<<< HEAD
    print("¡Hola, mundo!")
=====
    print("Hello, everyone!")
>>>>>> feature/english-update
```

Here's what each section means:

<<<<<<< HEAD

Precedes the current content in your branch (`main`) that is in conflict with yours.

=====

A separator between conflicting changes. The lines that follow are the incoming changes from the branch you are trying to merge.

>>>>>> feature/english-update

Terminates the block containing the conflicting lines.

Viewing Changes with `git diff`

When working with Git, understanding how changes progress through the different stages of the commit process is essential.

Each stage—from editing files in the working directory to staging and finally committing—represents a snapshot of the project's evolution.

Git offers several commands to inspect these changes at every step, allowing precise tracking, validation, and control over what will be recorded in the repository's history.

Two commonly used commands for this purpose are:

git diff

Displays the differences between the working directory and the staging area (index). In other words, it shows what has been modified in the files but not yet staged with `git add`.

git diff --cached (or --staged)

Shows the differences between the staging area and the most recent commit (HEAD). This output reveals what has already been staged and will be part of the next commit.

Both options help review modifications before committing, ensuring accuracy and clarity in the project history.

Additionally, `git diff` can compare specific files, commits, or branches, providing a detailed view of what has changed—an essential practice when validating code or resolving merge conflicts.

```
$ git diff app.py
diff --cc app.py
index 5253755,1cb4cb0..0000000
--- a/app.py
+++ b/app.py
@@@ -1,3 -1,3 +1,7 @@@
     app.py in main
     def greet():
++<<<<<< HEAD
+     print("¡Hola, mundo!")
++=====
+     print("Hello everyone!")
++>>>>>> feature/english-update
```

Resolving the Conflict

To resolve the conflict, the file `app.py` must be manually edited to represent the correct and intended final version, for example:

```
def greet():
    print("Hello, everyone and ¡Hola, mundo!")
```

After editing, add your file again:

```
$ git add app.py
```

Then complete the merge with:

```
$ git commit
```

During a merge, Git may automatically generate a default commit message describing the merge operation.

This message can be accepted as-is or replaced with a custom one that better explains the purpose or context of the merge. A clear, descriptive merge message helps maintain an understandable project history and simplifies future reviews or debugging.

TIP

- Use `git status` to see which files are in conflict.
- Use `git diff` to review changes before resolving.
- Tools such as VS Code, Sourcetree, and GitKraken provide graphical merge tools that simplify resolution.
- For advanced conflict resolution, you can use `git mergetool`.

Branch Management

The `git branch` command offers more functionality than just creating and deleting branches. When run without any arguments, it displays a list of all local branches, highlighting the one currently checked out. To view the most recent commit associated with each branch, use the `-v` option:

```
$ git branch -v
  feature/english-update 91f4dc5 new file
  feature/spanish-update b1b0e81 Update greeting to Spanish
* main                   b1b0e81 Update greeting to Spanish
```

The `--merged` and `--no-merged` options are helpful for filtering branches based on their merge status relative to your current branch. To display a list of branches that have already been merged into the current branch, use:

```
$ git branch --merged
  feature/spanish-update
* main
```

Since you previously merged the `feature/spanish-update` branch, it appears in the list of

merged branches.

This makes it easy to identify which branches have already been merged and are safe to delete, as well as those that remain active or pending integration.

Branches listed without an asterisk (*) are not the currently checked-out branch. It is typically safe to delete such branches using `git branch -d`, because its changes have already been integrated elsewhere and won't be lost.

To view branches that contain work not yet merged into your current branch, use the `--no-merged` option:

```
$ git branch --no-merged
feature/english-update
```

This lists branches that still have unmerged changes, in this case `feature/english-update`.

Renaming a Git Branch Locally and Remotely

Git makes it easy to rename branches while preserving the full commit history. The process involves renaming the branch locally and updating the branch name on the remote (e.g., GitHub, GitLab, or another Git server). In the following example, a branch named `my-old-branch` will be renamed to `my-new-branch`.

Step 1: Rename the Branch Locally

Use `git branch` with the `--move` or `-m` option to rename the branch locally:

```
$ git branch --move my-old-branch my-new-branch
$ git branch
main
* my-new-branch
```

This command replaces `my-old-branch` with `my-new-branch` locally, keeping all history and commit references intact. However, this change affects only your local repository.

Step 2: Push the Renamed Branch to the Remote

To make the updated branch name visible to others, push it to the remote and set the upstream reference:

```
$ git push --set-upstream origin my-new-branch
```

This creates the new branch on the remote and links your local branch to it.

You can now verify the branch structure with:

```
$ git branch --all
main
* my-new-branch
remotes/origin/main
remotes/origin/my-new-branch
remotes/origin/my-old-branch
```

At this point, you are working on `my-new-branch`, and it has been successfully pushed to the remote. However, the old `my-old-branch` branch still exists remotely and should be removed to complete the cleanup.

Step 3: Delete the Old Branch from the Remote

To remove the incorrectly named branch from the remote repository, run:

```
$ git checkout main
$ git push origin --delete my-old-branch
```

This command permanently deletes `my-old-branch` from the remote, leaving only the correctly renamed branch.

Step 4: Clean Up Local References (Optional)

If you still see the old branch listed under `remotes/origin`, remove stale references:

```
$ git fetch --prune
```

You have now fully renamed the branch — both locally and remotely — without losing any commit history.

Temporary Change Management with `git stash` and `git pop`

When working on a feature or fixing a bug, it is common to make multiple edits before completing

a commit. However, there are times when an urgent task requires switching branches or pulling the latest updates—even though the current work is unfinished. In such situations, the `git stash` command is invaluable.

It temporarily saves local modifications (by default, only tracked files) without committing them, allowing the workspace to be safely cleared so that another task can be addressed. Once the urgent work is completed, the stashed changes can be restored exactly as they were.

Consider the following scenario:

Work is being done on the `main` branch to fix the login form, and the file `login.html` has been modified but not yet committed.

Suddenly, an urgent request arises to apply a correction on another branch named `hotfix/logo`.

Since switching branches with uncommitted changes (that affect overlapping files) is not allowed, the best approach is to temporarily stash the current work:

```
$ git add login.html
$ git stash push -m "fixing login form"
```

This sequence stages the changes to `login.html` and then saves them in a stash identified by the message `fixing login form`.

The working directory becomes clean, allowing an immediate switch to the `hotfix/logo` branch to handle the urgent issue. This will save your local modifications and revert the working directory to match the `HEAD` commit.

To confirm that the stash was created successfully, list the available stashes with:

```
$ git stash list
stash@{0}: On main: fixing login form
```

This output indicates that the current work has been safely stored under the label `fixing login form`. With the workspace now clean, it is possible to switch branches and address the urgent task.

Switch to the `hotfix/logo` branch:

```
$ git checkout hotfix/logo
```

After switching branches, perform the necessary corrections, then stage and commit the fix:

```
$ git commit -am "Fixed logo alignment"
```

The urgent issue is now resolved, and the previous work remains safely stored in the stash, ready to be reapplied later.

With the `-a` option, Git automatically stages all modifications to already tracked files before committing—saving you from the step of running `git add`. Then you can go back to your original branch.

Once you're ready to return to your previous work, use `git stash pop` to reapply the latest stashed changes and remove them from the stash list:

```
$ git stash pop
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   login.html
```

At this point, the stashed changes have been restored to the working directory, allowing the unfinished work to continue seamlessly from where it was left off.

Once a stash is successfully applied using `git stash pop`, it is automatically removed from the stash list, keeping the repository clean and avoiding duplicate entries.

To apply a specific stash, enter:

```
$ git stash pop stash@{2}
```

To reapply the changes and keep the stash saved, enter:

```
$ git stash apply
```

To delete all saved stashes, enter:

```
$ git stash clear
```

If there are conflicts when popping the stash (e.g., if some files were modified both in the stash

and in your current branch), Git will notify you of merge conflicts, and you will need to resolve them manually.

Guided Exercises

1. What happens when you run `git branch feature-x`?

2. What is the difference between a lightweight tag and an annotated tag in Git?

3. After renaming a local branch, how do you delete the old branch `my-old-branch` from the remote?

4. You have a repository with a file called `message.txt`. Two branches (`english-greeting` and `spanish-greeting`) will make conflicting edits to this file:

```
<<<<<<< HEAD
¡Hola, mundo!
=====
Hello, world!
>>>>>>> english-greeting
```

Show how to merge the branches and manually resolve the conflict.

5. How do you bring back the most recent stashed changes and remove them from the list of stashes?

Explorational Exercises

1. Explore how Git handles tags created before and after a merge; do so through the following actions:

- Create a new Git repository and commit a file.
- Create a new branch (feature) from main and make a commit there.
- Tag the commit on the feature branch with v1.1-beta.
- Switch back to main, make another commit.
- Merge feature into main.
- Tag the result of the merge as v1.1-final.
- Run `git log --oneline --decorate --graph --all` and analyze the output.

2. Based on this scenario, answer the following: Where is each tag pointing? Does the tag from the feature branch appear in the main history after the merge?

Summary

In Git, branches and tags are essential tools for navigating, isolating, and labeling different points in a project's history. They allow developers to experiment with new features, manage releases, and mark specific versions of a project without disrupting the main development workflow. Branch and tag management are fundamental to maintaining clean, collaborative, and reliable codebases.

Answers to Guided Exercises

1. What happens when you run `git branch feature-x`?

It creates a new branch called `feature-x` pointing to the current commit, but it does not switch to that branch.

2. What is the difference between a lightweight tag and an annotated tag in Git?

A lightweight tag is a simple reference to a commit, without metadata. An annotated tag includes metadata such as the tagger's name, the current date, and a message. Annotated tags are recommended for marking official releases.

3. After renaming a local branch, how do you delete the old branch `my-old-branch` from the remote?

```
$ git push origin --delete my-old-branch
```

4. You have a repository with a file called `message.txt`. Two branches (`english-greeting` and `spanish-greeting`) will make conflicting edits to this file:

```
<<<<<< HEAD
¡Hola, mundo!
=====
Hello, world!
>>>>>> english-greeting
```

Show how to merge the branches and manually resolve the conflict.

Manually edit `message.txt` to resolve the conflict, for example:

```
Hello, world! / ¡Hola, mundo!
```

Stage the resolved file:

```
$ git add message.txt
```

Complete the merge:

```
$ git commit
```

5. How do you bring back the most recent stashed changes and remove them from the list of stashes?

```
$ git stash pop
```

Answers to Explorational Exercises

1. Explore how Git handles tags created before and after a merge; do so through the following actions:

- Create a new Git repository and commit a file.
- Create a new branch (feature) from main and make a commit there.
- Tag the commit on the feature branch with v1.1-beta.
- Switch back to main, make another commit.
- Merge feature into main.
- Tag the result of the merge as v1.1-final.
- Run `git log --oneline --decorate --graph --all` and analyze the output.

Create a new Git repository and commit a file:

```
$ git init tag-merge-demo
$ cd tag-merge-demo
$ echo "Initial content" > app.txt
$ git add app.txt
$ git commit -m "Initial commit"
```

Create a new branch (feature) from main and make a commit there:

```
$ git checkout -b feature
$ echo "Feature notes" > feature.txt
$ git add feature.txt
$ git commit -m "Add feature.txt"
```

Tag the commit on the feature branch with v1.1-beta:

```
$ git tag -a v1.1-beta -m "Beta version"
```

Switch back to main and make another commit:

```
$ git checkout main
$ echo "Main notes" >> app.txt
$ git add app.txt
```

```
$ git commit -m "Append notes on app.txt"
```

Merge feature into main:

```
$ git merge feature
```

Tag the result of the merge as v1.1-final:

```
$ git tag -a v1.1-final -m "Final version"
```

Run `git log --oneline --decorate --graph --all` and analyze the output:

```
$ git log --oneline --decorate --graph --all
* 439ce20 (HEAD -> main, tag: v1.1-final) ok Merge branch 'feature'
| \
| * 9e53836 (tag: v1.1-beta, feature) Add feature.txt
* | 5651546 Append notes on main
| /
* 3e464b8 Initial commit
```

2. Based on this scenario, answer the following: Where is each tag pointing? Does the tag from the feature branch appear in the main history after the merge?

v1.1-beta stays on the feature commit; v1.1-final is on the merge commit.

The v1.1-beta tag from the feature branch does not appear in the main history (439ce20). This is because tags do not move with merges; they remain attached to the original commit.

This feature allows you to mark versions both before and after integration.



Linux
Professional
Institute

Lesson 3

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.3 Source Code Management
Lesson:	3 of 3

Introduction

The previous lessons showed how to work with local branches and manage changes within a single repository. This lesson expands the workflow to include remote repositories, feature branches, and Git submodules. These tools are essential for collaborating with others, structuring large projects, and maintaining modular codebases.

Configuring a Remote Repository

To collaborate with others or contribute to an existing project, the local repository must be connected to a remote repository. A *remote* serves as a shared version of the repository, typically hosted on platforms such as GitHub, GitLab, or Bitbucket.

If a local repository needs to be linked to a remote named `project.git` hosted on GitHub, execute:

```
$ git remote add origin https://github.com/<username>/project.git
```

This command associates the alias `origin` with the specified remote URL.

To verify that the connection was successfully established, run:

```
$ git remote -v
origin https://github.com/<username>/project.git (fetch)
origin https://github.com/<username>/project.git (push)
```

The output lists the remote URLs used for both fetching (downloading) and pushing (uploading) commits to the repository associated with `origin`. This confirmation ensures that synchronization between the local and remote repositories is properly configured.

Sharing Changes With `git push`

After committing changes locally, the next step is to share them with others by pushing them to the remote repository. This operation uploads the commits from the local branch to the corresponding branch on the remote, making the updates accessible to collaborators.

To push the current branch (`main`) to the remote named `origin`, use:

```
$ git push origin main
```

If this is the first time pushing the branch, include the `-u` option to establish an upstream tracking relationship between the local branch and its remote counterpart:

```
$ git push -u origin main
```

Once this association is created, subsequent pushes can be performed simply with:

```
$ git push
```

From that point on, Git automatically remembers which remote and branch the local branch is linked to, streamlining future synchronization operations.

Getting Updates With `git fetch`

`git fetch` downloads new data from the remote repository, such as new branches or updates to remote branches, but it does *not* automatically update your working directory:

```
$ git fetch origin
```

This updates local references to the remote branches (e.g., `origin/main`) without touching files or merging anything. To inspect what's new:

```
$ git log HEAD..origin/main
```

This shows commits that exist in `origin/main` but aren't yet in the current branch.

Updating Local Branch With `git pull`

`git pull` is the shorthand for `git fetch` followed by a `git merge`. It downloads changes from the remote and integrates them into the current working branch:

```
$ git pull origin main
```

This command synchronizes the local `main` branch with any new commits from `origin/main`.

If conflicting changes exist between the local and remote versions, Git will detect them, notify the user, and pause the merge process until the conflicts are manually resolved.

Once all conflicts are fixed and the changes are committed, the update completes successfully.

Force Pushing With `git push --force`

In certain situations, it becomes necessary to rewrite the commit history of a branch — for instance, after performing a `git rebase` or using `git commit --amend`.

These operations modify existing commits, generating new commit hashes. As a result, the local branch history no longer matches the one stored on the remote repository. To prevent accidental overwriting of shared work, a regular `git push` is rejected by default when histories diverge.

In such cases, a *force push* is required to update the remote branch with the rewritten history:

```
$ git push --force
```

Before executing it, ensure that no other collaborators are depending on the previous commits to avoid data loss or synchronization issues.

WARNING

Force pushing rewrites the history on the remote and can cause problems for collaborators if not coordinated properly. It is safest to use on branches that only you are working on.

Force Pulling With `git fetch` and `git reset --hard`

Unlike `git push --force`, Git does not have a direct `git pull --force` command. What many developers refer to as a *force pull* is actually a combination of two commands:

```
$ git fetch origin
$ git reset --hard origin/main
```

This operation discards all local changes and commits, replacing the current branch with the exact state of its remote counterpart.

It is a powerful but potentially destructive command, as it overwrites local work without the possibility of recovery unless backups or other branches exist. Therefore, it must be used with extreme caution.

A force pull may be necessary in the following scenarios:

- The local branch has diverged from the remote because its history was rewritten (for example, after someone performed a `git push --force`).
- All local changes need to be discarded to make the branch identical to the remote version.
- The local repository history is corrupted, or a clean testing environment is required.

WARNING

A *force pull* permanently deletes local commits and any uncommitted modifications. Before executing it, verify that all important local work has been backed up, merged, or is no longer required.

Removing Untracked Files

To complete a force pull by also removing untracked files and directories (those not added to Git), execute:

```
$ git clean -fd
```

where:

-f

forces the deletion of untracked files.

-d

removes untracked directories as well.

This command is particularly useful when a completely clean working directory is required—for instance, in automated build systems, deployment environments, or when resetting a repository to a pristine state for testing.

Reverting to a Specific Commit Using a New Branch

In Git, it is often necessary to revisit a previous point in a project's history—for example, to inspect an older version, investigate a bug, or begin new development based on a stable past state.

Rather than resetting or modifying existing branches, the safest method is to create a new branch from a specific commit. This approach preserves the repository's integrity while providing a clean workspace for experimentation or targeted changes.

For instance, to return to a commit identified by the hash `a1b2c3d` and begin new work from that point, first navigate to the project directory:

```
$ cd my-project
```

Create a new branch from the specific commit:

```
$ git checkout -b legacy-fix a1b2c3d
```

This command performs two actions simultaneously:

- Checks out the commit `a1b2c3d`, restoring the working directory to the exact state it had at that point in history.
- Creates a new branch named `legacy-fix`, starting from that commit.

Once the branch is created, new edits can be made, staged, and committed normally, without altering or interfering with the history of other branches. This technique is ideal for hotfixes, debugging legacy code, or developing new features from a known stable version.

How to Find the Commit Hash

If the commit hash is not known in advance, the recent commit history can be reviewed with the following command:

```
$ git log --oneline
a633ee0 (HEAD -> main, origin/main) Update login.html
fbf92c7 create the logon.html file
89bf9bb (origin/my-new-branch, my-new-branch, hotfix/logo) first commit
```

This displays a compact list of commits, showing their abbreviated SHA-1 hashes along with the corresponding commit messages, making it easier to identify the desired point in the project's history.

Returning to a Specific Commit Without Creating a New Branch

Occasionally, it may be necessary to return to an earlier state of the project—for instance, to review old code, test a previous version, or perform a temporary rollback for debugging purposes.

When creating a new branch is not required, there are two primary ways to move to a specific commit. Each method serves a different purpose depending on whether the goal is a temporary inspection or a permanent reset of the project's state.

Checkout a Specific Commit (Detached HEAD)

The `git checkout` command can be used to move directly to any commit by specifying its hash. This action places the repository in a *detached HEAD* state, meaning that the current view is based on a specific commit rather than a named branch.

```
$ git checkout a1b2c3d
```

At this point, the working directory reflects the exact state of the project as it was at commit `a1b2c3d`. Files can be inspected, tests executed, or temporary modifications made.

However, any new commits created in this state will not belong to a branch and may be lost unless they are explicitly saved by creating a new branch or tag.

To return to the most recent branch (for example, `main`), execute:

```
$ git checkout main
```

Resetting the Current Branch to a Specific Commit

To move a current branch permanently back to a previous commit — effectively discarding all commits after it — use `git reset`. There are three modes:

`--soft`

keeps all changes staged.

`--mixed`

keeps changes in the working directory but unstaged (default).

`--hard`

discards all changes.

The following example resets the branch hard to a previous commit:

```
$ git reset --hard a1b2c3d
```

This rewrites the branch history and removes all commits after `a1b2c3d`.

WARNING

Use a hard reset only if you are certain that later commits are no longer needed. If you have already pushed these commits to a remote repository, resetting will create divergence and may require a force push (`git push --force`).

Understanding `git rebase`

In Git, there are two primary methods to integrate changes between branches: *merging* and *rebasing*.

The `rebase` command is a powerful tool that allows you to reapply commits on top of another base tip. Unlike merging, which creates a new commit that combines histories, rebasing rewrites the commit history to make it appear as if changes were made in a straight line. This results in a cleaner, more linear project history, which is often easier to read and understand.

Used correctly, `git rebase` can simplify collaboration, reduce unnecessary merge commits, and clarify the evolution of a feature or bug fix. However, because rebasing rewrites commit history, it

must be used with care, especially when working with shared branches.

When to Use `git rebase`

Rebase is best suited for scenarios where a linear, easy-to-read history is desirable:

- Updating a feature branch with the newest commits from `main` before merging, so the feature is built atop the latest base.
- Tidying commit history by squashing or reordering commits prior to publishing.
- Keeping the development history straightforward, minimizing merge commits and making `git log` easier to follow.

Rebase is discouraged on branches already shared with collaborators unless all parties are aligned and ready to coordinate. Rebasing rewrites commit hashes; once published, that rewrite requires a force push and can disrupt other clones.

Suppose you are working on a branch called `feature/login`, and the `main` branch has received updates since you branched off. Before merging `feature/login` back into `main`, you want to integrate the latest changes from `main` to ensure your code is up to date.

Step 1: Check Out Your Feature Branch

```
$ git checkout feature/login
```

Step 2: Rebase Onto the Latest Main Branch

```
$ git rebase main
```

This takes the commits from `feature/login` and replays them on top of the latest `main`. If there are no conflicts, the rebase finishes automatically. Your feature branch now appears as if it was developed from the latest version of `main`.

If there are conflicts, Git will pause and allow you to resolve them manually. After fixing conflicts in a file, enter:

```
$ git add <filename>
```

Then continue the rebase:


```
$ git rebase --continue
```

To cancel the rebase at any time and return to the previous state, enter:

```
$ git rebase --abort
```

Rebase Versus Merge

Let's review the difference between the two approaches:

`git merge main (on feature/login)`

creates a merge commit, preserving the full history of both branches, including divergent paths.

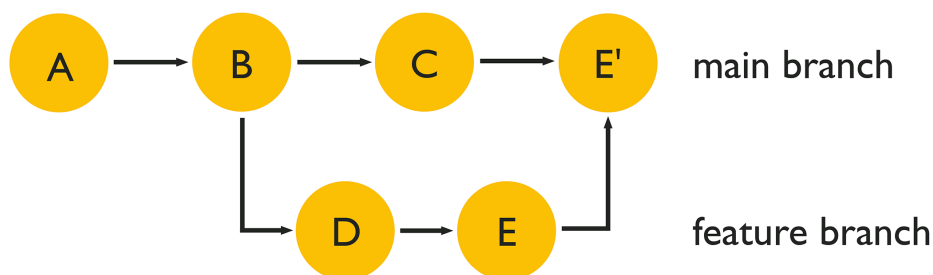


Figure 2. Merge history

`git rebase main`

moves the feature branch's commits on top of `main`, rewriting the history to produce a linear timeline.

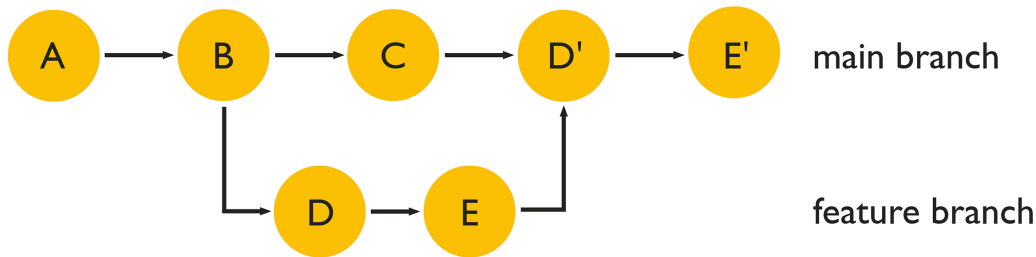


Figure 3. Rebase history

The commits D and E have been replayed as (D') and (E'), rebased on top of commit C.

But which approach is better? To decide that, let's take a step back and think about what commit history really means.

One perspective is that a repository's commit history is a factual log of what actually happened during development. It is a historical document — valuable in itself — and should not be altered. From this point of view, rewriting history using tools like rebase is almost sacrilegious. Even if the merges were messy, the repository should reflect that for future reference.

The opposing perspective views commit history as the story of how a project was built. And just as an author would not publish the rough draft of a book, developers may prefer not to expose a (sometimes messy) development process. During active work, keeping a record of mistakes and experiments can be valuable, but when it is time to share the results — especially with collaborators or the public — it often makes sense to present a cleaner, more coherent history. Developers who follow this mindset use tools like interactive rebasing to refine and organize their commits before merging into the main branch, aiming to tell a clearer and more meaningful story for future readers.

Real-World Example: Collaborating on a Shared Project

Let's say you are working on a team project hosted on GitHub. Clone the repository:

```
$ git clone https://github.com/<username>/shared-project.git
$ cd shared-project
```

Create a feature branch:

```
$ git checkout -b feature/contact-form
```

After committing changes, share them:

```
$ git push -u origin feature/contact-form
```

A teammate updates the main branch with new work. Sync the repositories with:

```
$ git checkout main  
$ git pull origin main
```

Update your feature branch with the latest changes from `main`:

```
$ git checkout feature/contact-form  
$ git rebase main
```

Finally, when your feature is complete, open a pull request or merge your branch.

Submodules: Managing Nested Git Repositories

A Git *submodule* is a repository embedded inside another repository. This is commonly used when your project depends on an external library or shared component that is managed in its own Git repo.

Adding a Submodule

To add a submodule to your repository, enter:

```
$ git submodule add https://github.com/<otheruser>/library.git lib/library
```

This creates a new folder at `lib/library`, links it to the remote repository, and records the submodule path in a `.gitmodules` file.

Cloning a Repository with Submodules

If you clone a repository that uses submodules, you must also initialize and fetch the submodules:

```
$ git clone https://github.com/<username>/project.git
$ cd project
$ git submodule update --init --recursive
```

The `--recursive` option ensures that submodules within submodules are also initialized.

Updating Submodules

To update the submodule to the latest commit on its remote, enter:

```
$ cd lib/library
$ git checkout main
$ git pull origin main
$ cd ../../
$ git add lib/library
$ git commit -m "Update submodule to latest version"
```

You can also update all submodules recursively with:

```
$ git submodule update --remote --merge
```

Removing a Submodule

To remove a submodule completely:

1. Delete the submodule entry in `.gitmodules`.
2. Delete the submodule's entry in `.git/config`.
3. Remove the submodule directory and cached reference:

```
$ git rm --cached lib/library
$ rm -rf lib/library
```

4. Commit the change:

```
$ git commit -m "Remove submodule"
```

Guided Exercises

1. After creating a local branch called `feature/login`, how do you push it to the remote repository and set it to track the remote branch?

2. What combination of commands can be used to perform a “force pull” that replaces your local branch with the remote version?

3. How can you create a new branch from a specific commit hash `a1b2c3d`?

4. What is the difference between `git merge` and `git rebase` when integrating changes from the main branch into a feature branch?

5. What is the purpose of Git submodules?

Explorational Exercises

1. Compare the user interface for handling remote repositories in GitHub and GitLab.

2. How can you ensure that submodules are cloned and updated during pipeline execution in GitHub and GitLab?

Summary

As your Git skills evolve beyond managing changes within a single local repository, it is essential to master the tools and workflows that support real-world collaboration and project structure. This lesson introduced key concepts such as remote repositories, feature branches, rebasing, and submodules — all of which play a crucial role in professional software development. The lesson covered how to track and sync with remote branches, organize your work using isolated feature branches, and apply powerful techniques like rebasing to maintain a clean and linear commit history. Additionally, the lesson explored how Git submodules enable you to include and manage external repositories as dependencies, a common requirement in modular or large-scale projects.

Answers to Guided Exercises

1. After creating a local branch called `feature/login`, how do you push it to the remote repository and set it to track the remote branch?

```
$ git push -u origin feature/login
```

2. What combination of commands can be used to perform a “force pull” that replaces your local branch with the remote version?

```
$ git fetch origin  
$ git reset --hard origin/main
```

3. How can you create a new branch from a specific commit hash `a1b2c3d`?

```
$ git checkout -b legacy-fix a1b2c3d
```

4. What is the difference between `git merge` and `git rebase` when integrating changes from the main branch into a feature branch?

`git merge` creates a new merge commit and preserves both histories.

`git rebase` rewrites history to make it appear linear by replaying your feature commits on top of the main branch.

5. What is the purpose of Git submodules?

Submodules allow you to include other Git repositories inside your project as dependencies, enabling modular project structures.

Answers to Explorational Exercises

1. Compare the user interface for handling remote repositories in GitHub and GitLab.

Both GitHub and GitLab support remote repository hosting, feature branching, pull/merge requests, and submodules, but they differ in their user interface and integration options. Some notable differences are:

Feature branching: Both platforms support Git feature branches and allow their creation through the user interface (UI) or command line. GitHub uses the term “pull request” (PR), while GitLab uses the term “merge request” (MR) for requests to upload changes to a remote branch. GitHub’s interface is slightly more streamlined for code review, while GitLab provides more CI/CD integration during the merge request process.

Forks and contributions: GitHub emphasizes a fork-and-pull model, especially in open-source collaboration. GitLab allows similar workflows but is also optimized for internal repositories using protected branches and permissions.

Submodule support: Both platforms support Git submodules, but GitHub has limited UI visibility into submodule contents. GitLab displays submodules more clearly in the file tree and includes links to the submodule repositories.

2. How can you ensure that submodules are cloned and updated during pipeline execution in GitHub and GitLab?

To ensure submodules are cloned and updated during pipeline execution, you must explicitly instruct the pipeline to initialize and update them, because most CI systems do not fetch submodules by default when cloning a repository.

In GitHub Actions:

Use the official `actions/checkout` action with the submodules' `recursive` option through this YAML snippet:

```
- name: Checkout repository including submodules
  uses: actions/checkout@v3
  with:
    submodules: 'recursive'
```

This ensures that the repository is cloned along with all submodules, including nested ones.

In GitLab CI/CD:

Manually initialize and update submodules in your `.gitlab-ci.yml` script:

```
script:  
  - git submodule init  
  - git submodule update --recursive
```

This sequence ensures that all submodules are initialized and checked out at the correct commit, as recorded in the parent repository.



701.4 Continuous Integration and Continuous Delivery

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 701.4

Weight

3

Key knowledge areas

- Understand the concepts of Continuous Integration and Continuous Delivery
- Understand the components of a CI/CD pipeline, including builds, unit, integration and acceptance tests, artifact management, delivery and deployment
- Understand the concepts of GitOps
- Understand the role of build artifacts and caches
- Understand deployment best practices
- Understand semantic versioning
- Awareness of Jenkins and Gitlab CI
- Awareness of Artifactory and Nexus

Partial list of the used files, terms and utilities

- Declarative Pipeline
- Production, Staging and Development Environments
- Feature toggles
- Preview releases
- Reconciliation loops
- A/B testing

- Blue-green and canary deployment



Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.4 Continuous Integration and Continuous Delivery
Lesson:	1 of 1

Introduction

Modern software development demands fast, reliable, and repeatable processes to build, test, and deliver applications. That's where continuous integration and continuous delivery (CI/CD) comes in: a set of practices and tools that allow development teams to automate and streamline the entire lifecycle of software delivery. The “D” in CI/CD is sometimes interpreted as “delivery” or “deployment”, depending on whether the focus is on preparing software for release or actually deploying it to production environments. In either case, the goal remains the same: to ensure that code changes can move from development to users quickly, safely, and consistently.

In this lesson, the foundational principles of CI/CD are examined in detail. The structure of a complete CI/CD pipeline is explored, including how code moves from development to production through automated build, test, and deployment stages.

The lesson also introduces modern deployment strategies such as GitOps, blue-green deployments, and feature toggles, emphasizing how these practices improve reliability and reduce downtime during releases.

Additionally, key industry tools—including Jenkins, GitLab CI, Artifactory, and Nexus—are presented to illustrate how CI/CD pipelines are implemented in real-world environments across various technology stacks.

Continuous Integration and Continuous Delivery (CI/CD)

In the context of modern software development, CI and CD are essential practices that help teams build, test, and deploy applications in a reliable and automated fashion.

Continuous integration refers to the process of automatically integrating code changes from multiple contributors into a shared repository, sometimes several times a day. Each integration typically triggers an automated build and testing sequence, ensuring that the codebase remains functional and free of defects. This process encourages developers to submit small, incremental changes frequently and makes it easier to identify bugs early in the development cycle.

Continuous delivery extends the principles of CI by ensuring that the software is always in a releasable state. It automates the release process, so that code changes that pass testing can be deployed to staging or production environments at any chosen time. The software is not only tested continuously but also prepared continuously for delivery, making deployments predictable and less risky.

The sequence of events in CI/CD is called a *pipeline*. It consists of a series of automated steps triggered by a commit or pull request to a version control system such as Git. These steps often include building the application, running unit and integration tests, packaging the application into deployable artifacts, and deploying it to target environments such as staging or production (Elements of a typical CI/CD pipeline). All of this is orchestrated by CI/CD tools that define the pipeline in configuration files, making deployments consistent and repeatable across environments.

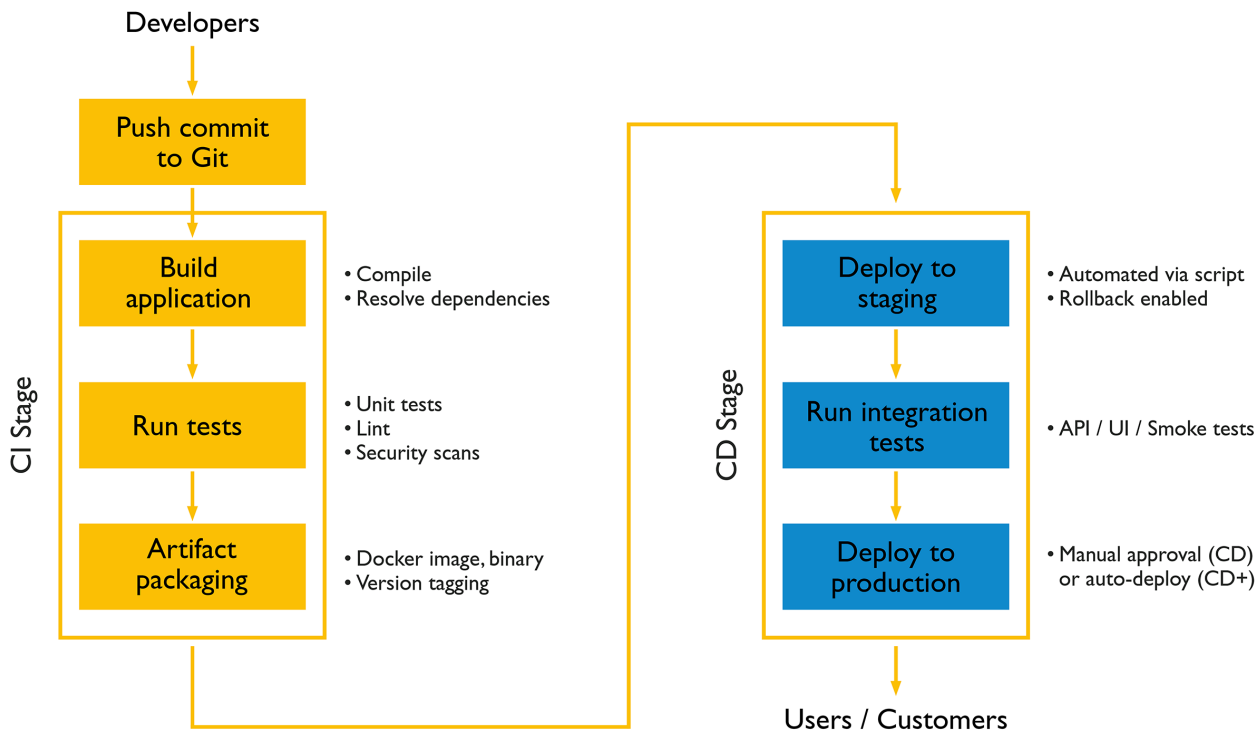


Figure 4. Elements of a typical CI/CD pipeline

Components of a CI/CD Pipeline

A well-designed CI/CD pipeline consists of several automated stages that take code from development to production with reliability and consistency. Each component plays a specific role in validating, packaging, and delivering software.

Build Stage

The *build stage* is the first automated step after code changes are committed. This phase compiles the application source code, resolves dependencies, and packages the executable program into a deployable format, such as a binary, JAR file, or Docker image.

This stage confirms that the codebase is syntactically correct and can be successfully assembled in the target runtime environment. Failures here indicate issues such as missing dependencies, syntax errors, or misconfigured build tools.

Unit Testing

Once the build is successful, the pipeline moves into the *unit testing* stage. Unit tests validate small, isolated pieces of code, such as individual functions or classes, to ensure they behave as expected. These tests are typically fast and run in parallel. They serve as the first line of defense against regressions and logic errors introduced during development. High unit test coverage helps detect bugs early and contributes to long-term maintainability.

Integration Testing

Integration tests go beyond isolated units and focus on interactions between components and modules, or with external systems such as databases or APIs. These tests ensure that different parts of the application work together as intended. Integration tests are usually slower than unit tests but are essential for catching issues that appear only during real-world usage where multiple services interact.

Acceptance Testing

Acceptance tests validate the system from the perspective of the end user or *stakeholder*. They are often driven by predefined business requirements or user stories and ensure that the application fulfills its intended purpose. Acceptance tests may involve functional user interface (UI) testing, API contract validation, or behavior-driven development (BDD) scenarios. They help determine whether the application is ready to be released to production or requires further adjustments.

Artifact Management

Once the application passes all tests, the resulting build *artifacts* are stored in a central repository. This step makes sure to preserve the exact version of the software that passed all quality checks and can be reliably deployed later. Artifact management systems—such as GitHub Packages, JFrog Artifactory, or AWS CodeArtifact handle versioning, storage, and access control for these artifacts. Proper artifact management supports rollback, traceability, and reproducible builds.

Delivery and Deployment

The final stage of the pipeline is *delivery and deployment*. In continuous delivery, the software is automatically deployed to a staging environment and awaits manual approval for production. In continuous deployment, the release is pushed all the way to production automatically after passing all stages. Deployment scripts, infrastructure as code, and container orchestration systems like Kubernetes or Docker Compose often drive this stage. Teams may use blue-green deployments, canary releases, or rolling updates to minimize downtime and risk (see below).

Deployment Strategies and Best Practices

As teams release updates more frequently through CI/CD pipelines, careful deployment strategies become essential to minimize downtime, reduce risk, and enable rapid feedback. Effective deployment is not just about copying code to servers, it is about coordinating infrastructure, application logic, and the user experience to ensure smooth transitions across versions.

A solid deployment strategy begins with an understanding of the environments involved. To manage deployments in these environments effectively, teams adopt practices such as infrastructure as code, automation, and immutable deployments, ensuring repeatability and consistency across environments. But beyond automation, strategic approaches to rolling out software play a major role in reducing risk and improving observability.

Blue-Green Deployment

A *blue-green deployment* is a strategy using two production environments: *live* (blue) and *idle* (green). The new version of the application is deployed to the idle environment (green) while the current version continues to serve users (blue). After the green deployment passes rigorous tests, traffic switches over to it, either all at once or gradually. If issues arise, rollback is as simple as reverting traffic to the original (blue) environment. This strategy minimizes downtime and allows for quick rollback to a known-good state, but it effectively doubles the infrastructure cost by requiring two parallel production environments.

Canary Deployment

A *canary deployment* rolls out new software to a small subset of users before a full-scale release. This allows teams to monitor system behavior, user experience, and error rates in real time while minimizing the blast radius of potential issues. If the release performs well, it is gradually expanded to more users. If problems are detected, the deployment can be halted or rolled back without affecting the entire user base. Canary deployments are commonly used in cloud-native platforms with routing control, such as Kubernetes with service meshes.

A/B Testing

A/B testing is useful not only for deployment but also for product experimentation. It involves releasing two or more versions (A and B) of a feature or UI element to different user segments simultaneously and measuring their impact. Unlike canary releases, which focus on technical validation, A/B testing is centered on business metrics such as click-through rates, engagement, or conversions.

In this context, *engagement* refers to how actively users interact with a feature or interface — for example, how often they click, scroll, or spend time on a page. *Conversion* measures how many users perform a desired action, such as completing a purchase, subscribing, or signing up for a service.

Proper A/B testing requires traffic splitting, telemetry collection, and statistical analysis to determine which version performs better. A related technique, known as *multi-armed bandit testing*, builds upon A/B testing by dynamically reallocating more traffic to the better-performing variations as results are collected, allowing faster optimization and less wasted exposure to underperforming options.

Preview Releases

Preview environments (also called *preview releases* or *ephemeral environments*) are short-lived, automated environments spun up for each pull request or feature branch. They allow developers, testers, and stakeholders to review changes in a real-world setup before merging those changes. These environments mirror staging or production as closely as possible and are automatically destroyed after use. Preview releases increase confidence in changes and improve collaboration across teams.

Feature Toggles

Feature toggles (or *feature flags*) allow developers to enable or disable specific features at runtime without deploying new code. This decouples deployment from release, enabling features to be hidden, rolled out gradually, or targeted to specific user groups. Feature flags also allow for safe experimentation, rollback, and staged rollouts.

Reconciliation Loops

In declarative systems like Kubernetes or tools like Argo CD, *reconciliation loops* continuously compare the desired state of the system (as declared in configuration files) with the actual state. If a drift is detected — for example, a container was modified outside of the pipeline — the controller attempts to restore the system to its declared state. This mechanism provides self-healing and enforces consistency, making the infrastructure and application state predictable and reliable. Reconciliation is a cornerstone of GitOps-based deployments, discussed next.

GitOps

GitOps is an operational model that uses Git as the single source of truth for infrastructure and application deployments. It bridges the gap between development and operations by allowing

teams to manage infrastructure and deployment configurations through version-controlled repositories, just like they manage application source code.

In a GitOps workflow, infrastructure changes are made by updating configuration files in a Git repository. Once the change is committed, an automated process (usually driven by a controller like Argo CD or Flux) detects the update and applies it to the target environment. This allows for full auditability, easy rollback, and reliable automation, since every change must be tracked, peer-reviewed, and explicitly declared in Git.

GitOps reinforces infrastructure as code, declarative configuration, and automation. It enables repeatable, secure, and scalable delivery workflows that are particularly well-suited for Kubernetes environments and cloud-native applications.

Semantic Versioning

Semantic versioning (SemVer) is a versioning system designed to clearly convey the meaning of changes in software releases. Version names follow the format `MAJOR.MINOR.PATCH`, where each component indicates the type of changes introduced.

The `MAJOR` version increases when backward-incompatible changes are made. The `MINOR` version increases when new features are added in a backward-compatible way. The `PATCH` version increases when backward-compatible bug fixes are introduced.

For example, version `2.4.1` indicates the second major version of the software, with four minor feature updates and one patch. If the next update includes a breaking API change, it would become `3.0.0`.

Semantic versioning helps teams and users understand the impact of an update before applying it. It is widely adopted in package managers, CI/CD pipelines, container tagging, and dependency management systems.

Build Artifacts and Caches

In a CI/CD pipeline, *build artifacts* are the output of a successful build process: compiled binaries, Docker images, JAR files, static assets, or zipped packages ready for deployment or distribution. These artifacts are often stored in artifact repositories (such as JFrog Artifactory, GitHub Packages, or AWS CodeArtifact) and may be versioned, signed, and reused.

Build caches, on the other hand, are intermediate results stored to speed up subsequent builds. Instead of recompiling unchanged modules or reinstalling dependencies, caching mechanisms allow reuse of previously generated outputs. This reduces build times and resource usage,

especially in large or modular projects.

Efficient use of both artifacts and caches improves CI/CD performance, reduces cost, and ensures that builds are reproducible. Many CI/CD systems allow the configuration of caching strategies and artifact retention policies to support these goals.

Declarative Pipelines

A *declarative pipeline* is a CI/CD pipeline defined using configuration files that describe “what should happen” during the software delivery process, rather than providing procedures for carrying out the changes. Declarative pipelines are written in a structured format such as YAML or a domain-specific language, which defines stages, jobs, triggers, environments, and conditions.

Declarative pipelines are commonly used in platforms like GitLab CI (where the configuration file is `.gitlab-ci.yml`), GitHub Actions (where the configuration file is `.github/workflows/*.yml`), and Jenkins (via Declarative Pipeline Syntax). These pipelines promote readability, consistency, and version control, making them ideal for teams working in collaborative and DevOps-centric environments.

By expressing the delivery process as code, declarative pipelines support automation, auditing, reuse, and integration into GitOps workflows. They also make it easier to reason about the pipeline, enforce policies, and standardize practices across multiple repositories or teams.

CI/CD Tools

DevOps engineers should be familiar with several tools for managing CI/CD pipelines: Jenkins, GitLab, Artifactory, and Nexus.

Jenkins and GitLab CI are widely used for automating CI/CD pipelines. Jenkins is an open source automation server known for its flexibility and extensive plugin ecosystem. It supports custom pipelines via Groovy-based scripts (the *Jenkinsfile*) and integrates with virtually any tool.

GitLab CI, built into the GitLab platform, offers a native YAML-based pipeline system tightly integrated with Git repositories, issue tracking, and container registries, streamlining DevOps workflows.

Artifactory and Nexus are artifact repository managers used to store, version, and distribute build artifacts such as JAR files, Docker images, and Helm charts. They act as centralized hubs in CI/CD pipelines, ensuring reproducible builds, secure artifact sharing, and dependency caching across teams and environments.

Guided Exercises

1. A developer pushes a change to the main branch of a Git repository. The project uses a full CI/CD pipeline with automated tests and deployments. Describe the sequence of stages that the code change will go through before it is deployed to production.

2. You are releasing a new version of your application and want to minimize risk while allowing for a quick rollback if something goes wrong. Which deployment strategy would you choose, and why?

3. You are releasing an update that adds a new feature to your API but does not break any existing functionality. According to semantic versioning, how should you update the version number?

4. Your team is using GitOps to manage Kubernetes deployments. A manual change is made directly in the cluster, outside of Git. Explain how the reconciliation loop will handle this situation.

5. Your team is preparing to release a new version of a payment service. The service is mission-critical and must not disrupt user transactions. You want to minimize risk, monitor real user traffic behavior, and avoid any impact on the entire user base in case of a bug. Which deployment strategy is more appropriate in this case: blue-green or canary? Justify your answer.

Explorational Exercises

1. Choose two popular CI/CD platforms (e.g., Jenkins, GitLab CI, GitHub Actions, CircleCI, Travis CI) and compare their configuration approach, ease of use, and integration with source control systems. Which one would you prefer for a containerized microservices project, and why?

2. Using a GitOps tool such as Argo CD or Flux, manually modify a deployment in a Kubernetes cluster and observe the behavior. What action does the GitOps controller take, and why?

Summary

This lesson introduced the core concepts of continuous integration (CI) and continuous delivery (CD), essential practices for automating and streamlining modern software development workflows. It also explored the concept of GitOps, which uses Git as the single source of truth for managing both infrastructure and application deployments. Learners examined the main components of a CI/CD pipeline, including build processes, unit/integration/acceptance testing, artifact management, and the stages of application delivery and deployment. The lesson emphasized the importance of *build artifacts* and *caches* for faster builds, and highlighted how *semantic versioning* ensures consistent and traceable releases.

Additionally, the lesson covered best practices for managing multiple environments such as development, staging, and production. Advanced deployment strategies were discussed — including preview releases, feature toggles, A/B testing, blue-green and canary deployments, and reconciliation loops — all designed to improve reliability and flexibility during rollouts. Finally, it introduced key CI/CD tools such as Jenkins, GitLab CI, Artifactory, and Nexus, demonstrating how declarative pipelines provide a clean, maintainable way to define automation flows across software delivery systems.

Answers to Guided Exercises

1. A developer pushes a change to the main branch of a Git repository. The project uses a full CI/CD pipeline with automated tests and deployments. Describe the sequence of stages that the code change will go through before it is deployed to production.

After the developer pushes the change, the CI/CD pipeline is triggered automatically. First, the build stage compiles the source code and resolves dependencies. Then, unit tests run to verify the correctness of individual components. If those tests pass, integration tests validate how different modules interact. Next, artifact management packages and stores the build (e.g., as a Docker image). The application is then deployed to a staging environment, where acceptance tests run. If successful, the release is finally deployed to production, either with manual approval (continuous delivery) or automatically (continuous deployment).

2. You are releasing a new version of your application and want to minimize risk while allowing for a quick rollback if something goes wrong. Which deployment strategy would you choose, and why?

A blue-green deployment would be ideal. This strategy involves deploying the new version to an idle environment (green) while the current version (blue) continues to serve users. Once the green environment is validated, traffic is switched over. If problems occur, you can quickly revert traffic to the blue environment. This minimizes downtime and provides a safe rollback mechanism.

3. You are releasing an update that adds a new feature to your API but does not break any existing functionality. According to semantic versioning, how should you update the version number?

Because the change adds a new feature without breaking existing functionality, you should increment the minor version. For example, if the current version is 2.4.1, the new version should be 2.5.0. The major version remains the same, and the patch version resets to zero.

4. Your team is using GitOps to manage Kubernetes deployments. A manual change is made directly in the cluster, outside of Git. Explain how the reconciliation loop will handle this situation.

In a GitOps setup, the reconciliation loop continuously compares the cluster's actual state with the desired state defined in the Git repository. When the loop detects that the actual state has diverged (due to the manual change), the GitOps controller (e.g., Argo CD or Flux) will attempt to restore the cluster to match the Git configuration. This ensures consistency, enforces declarative configuration, and prevents configuration drift.

5. Your team is preparing to release a new version of a payment service. The service is mission-critical and must not disrupt user transactions. You want to minimize risk, monitor real user traffic behavior, and avoid any impact on the entire user base in case of a bug. Which deployment strategy is more appropriate in this case: blue-green or canary? Justify your answer.

Canary deployment is the better choice in this scenario. Because the payment service is critical and highly sensitive to errors, canary deployments allow the new version to be released gradually to a small subset of users. This enables real-time monitoring of system behavior in production, reducing the risk of widespread failures. If issues are detected, the deployment can be halted or rolled back before affecting the majority of users. While blue-green deployment also allows rollback, it shifts 100% of traffic at once, which may be too risky for high-impact services such as payments.

Answers to Explorational Exercises

1. Choose two popular CI/CD platforms (e.g., Jenkins, GitLab CI, GitHub Actions, CircleCI, Travis CI) and compare their configuration approach, ease of use, and integration with source control systems. Which one would you prefer for a containerized microservices project, and why?

Jenkins uses Groovy-based Jenkinsfile configurations and offers a highly flexible plugin ecosystem, but typically requires additional setup and maintenance. In contrast, GitHub Actions uses YAML-based configuration and integrates directly with GitHub repositories, offering a more streamlined developer experience.

For containerized microservices, GitHub Actions is often preferred due to its native support for Docker workflows, seamless integration with GitHub, and built-in tools for managing secrets, artifacts, and caching. Its simplicity and cloud-based execution model make it especially suitable for modern DevOps pipelines.

2. Using a GitOps tool such as Argo CD or Flux, manually modify a deployment in a Kubernetes cluster and observe the behavior. What action does the GitOps controller take, and why?

When a deployment managed by Argo CD or Flux is manually altered, such as by changing the number of replicas or modifying the container image, the controller detects that the live cluster state no longer matches the desired state stored in the Git repository. The GitOps tool reacts by automatically reverting the changes to restore the declared configuration.

This action demonstrates the reconciliation loop, a foundational principle of GitOps. The system continuously ensures consistency by enforcing that the actual infrastructure state aligns with the version-controlled source of truth, enabling self-healing, predictability, and operational safety.



701.5 Software Composition, Licensing and Open Source

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 701.5

Weight

2

Key knowledge areas

- Understand how an application is build out of multiple software components
- Awareness of dependency managers like NPM, gradle or composer
- Understand the concepts proprietary and open source software
- Understand the concepts of open source software licenses
- Awareness of commonly used open source licenses (GPL, LGPL, AGPL, BSD, MIT and Apache License)
- Awareness of license compatibility and multi licensing

Partial list of the used files, terms and utilities

- Software libraries
- Software Bill Of Materials
- Proprietary software
- Open Source Software and Free Software
- Copyleft open source software licenses
- Permissive open source software licenses



Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	701 Software Engineering
Objective:	701.5 Software Composition, Licensing and Open Source
Lesson:	1 of 1

Introduction

Modern software development is highly modular. Instead of writing everything from the ground up, developers rely on a rich ecosystem of software components, including third-party libraries, frameworks, and tools. These components can come from different sources. Some are proprietary, while others are open source. Understanding how an application is assembled from these various parts is essential not only for building efficient systems but also for managing legal, security, and operational aspects of the software.

In this lesson, the fundamental distinctions between proprietary software and open source software are examined, emphasizing how the principles of freedom, transparency, and collaboration have shaped modern software development practices. A central theme is open source licensing, covering widely used licenses. The concepts of copyleft and permissive licensing, license compatibility, and multi-licensing are analyzed to illustrate how legal frameworks

influence project integration and redistribution. Understanding these principles is essential when combining different licensed components within a single codebase.

The lesson also introduces modern dependency managers—including NPM, Gradle, and Composer—which automate the retrieval and management of external libraries. As software systems grow in complexity, maintaining awareness of all dependencies becomes critical for ensuring not only functionality but also compliance and security. This leads to the concept of a software bill of materials (SBOM), a comprehensive inventory of all components, libraries, and dependencies that make up an application.

How an Application is Built Out of Multiple Software Components

Modern applications are rarely monolithic. Instead, they are constructed by combining multiple software components, each designed to handle particular functions. This modular approach allows for greater scalability, maintainability, and flexibility during development and operations.

Each application integrates various layers and services. These components may be developed in different programming languages, maintained in separate repositories, or even run in separate environments such as containers, virtual machines, or serverless runtimes.

Rather than being built entirely from scratch, today's software systems are typically assembled from a collection of interoperable components and services that collaborate to deliver complete functionality. These components include:

Front end

This is the user interface (UI) layer, responsible for what users see and interact with. It is often built using frameworks like React or Angular, providing dynamic and responsive experiences across devices.

Back end

The back end handles the server-side logic of the application. It processes business rules, manages databases, and exposes APIs that are used by the front end or other parts of the back end. Common backend frameworks include Flask, Express.js, NestJS, and Symfony.

Libraries and frameworks

These are reusable code modules that simplify the development process by providing prebuilt functionality for common tasks, such as authentication, input validation, and logging.

Middleware

Acting as the bridge between the front end and back end, middleware includes API gateways, authentication layers, and message brokers that facilitate communication and control the flow of data.

Databases

These systems store and serve up persistent data. Popular database technologies include MySQL, PostgreSQL, and MongoDB, each suited for different use cases and data models.

Third-party services

Many applications call upon external services to extend functionality, such as payment gateways (e.g., Stripe, PayPal), cloud storage (e.g., AWS S3), and email delivery platforms.

This component-based design is a key principle behind microservices architectures, which divide an application into small, autonomous services that communicate over APIs. Even in more traditional monolithic applications, modularization is encouraged through layered design (e.g., separating the presentation, business logic, and data access layers).

Managing such complexity requires thoughtful architectural planning, dependency management, testing, and automation. It also introduces the need for effective integration strategies, CI/CD pipelines, and monitoring solutions that can observe and correlate the behavior of these interconnected components.

The Effect of Software Licensing on Application Development

When an application is composed of multiple components, each provided by a different vendor, open source community, or internal team, licensing becomes critical. Every software library, framework, or service integrated into the system might be governed by a distinct license.

Some licenses impose minimal restrictions, while others require derivative works to be released as free and open source. As a result, developers and system architects must consider not only components' technical compatibilities but their legal impacts. Developers need to ensure that the combination of licenses is legally compatible with the intended distribution model for the software, whether it is delivered as a commercial product, a cloud service, or a private internal tool.

When using third-party components, therefore, understanding their licenses is crucial for ensuring compliance, avoiding legal risks, and upholding the trust and obligations associated with using third-party code.

Proprietary and Open Source Software

Software can generally be divided into two main categories: *proprietary software* and *open source software* (OSS).

Proprietary software (or *closed source software*) is developed and owned by a company or individual, and its source code is usually kept confidential. Users are granted the right to use the software by purchasing a license, but they are not allowed to view, modify, or redistribute the underlying code. This type of software is typically distributed in binary form only, and its usage is strictly controlled by an *end-user license agreement* (EULA). Common examples of proprietary software include Microsoft Office, Adobe Photoshop, and Oracle Database. This type of software restricts user freedom when it comes to customization, redistribution, or access to the source code.

On the other hand, *open source software* (OSS) is distributed under licenses that make its source code publicly available. The licenses allow for more than inspection: anyone can modify and share the software, provided they comply with the terms of its open source license. OSS encourages transparency, collaboration, and community-driven development, which often lead to faster innovation and improved responsiveness to user needs. Notable examples of open source projects include the Linux operating system, the Kubernetes container orchestration platform, and the TensorFlow machine learning framework. These tools are maintained by global communities of developers and are widely adopted in both academic and commercial environments.

Open Source Software and Free Software

Although the terms *open source software* and *free software* are often used interchangeably, they reflect distinct philosophies with nuanced differences.

Free software is rooted in the idea of user freedom. Championed by the *Free Software Foundation* (FSF), this approach guarantees four fundamental freedoms: the freedom to run the software for any purpose, to study how it works, to modify it, and to share it with others. The emphasis here is ethical and ideological, focusing on user rights and autonomy over the software they use.

Although in practice *open source software* offers the same benefits, the term places more emphasis on the practical and collaborative aspects of software development. Promoted by the *Open Source Initiative* (OSI), this model encourages transparency, peer review, and distributed innovation. The focus is less on freedom and more on the technical and organizational benefits of making source code publicly accessible.

Software that qualifies as free software also qualifies as open source — but not always vice versa.

Different licenses are not always compatible, as the next sections explain.

Open Source Software Licenses

Open source software licenses grant users specific rights to use, modify, and distribute software, but the extent of these rights can vary depending on the type of license. Broadly speaking, open source licenses fall into two main categories: *copyleft* and *permissive*.

Copyleft Licenses

Copyleft licenses are designed to preserve the freedom and openness of software by requiring that any derivative works or modified versions be distributed under the same license terms. This means that anyone who alters the software must also make their changes available under an open source license, ensuring that the code—and any improvements made to it—remains freely accessible to the community.

The GNU *General Public License* (GPL), created by the Free Software Foundation, is one of the most influential and widely used licenses in the free and open source world. The GPL is known as a strong copyleft license, meaning that any derivative work—including software that incorporates or is based on GPL-licensed code—must also be released under the GPL. This ensures that the freedoms to use, study, modify, and redistribute the software are preserved in all modified versions. For example, if a developer uses a GPL-licensed module to perform calculations in their application, and then distributes their application to others, they must release the entire application under the GPL.

The GNU *Affero General Public License* (AGPL) is an extension of the GPL designed specifically for network-based software such as web applications and software as a service (SaaS). While the GPL applies only when the software is distributed, the AGPL adds an additional requirement: if users interact with the modified software over a network, the source code must be made available to them. This closes the “ASP loophole” that allowed companies to use modified open source software on their servers without sharing their changes, since no actual distribution was taking place. With the AGPL, even software used privately on servers must share its code if it is accessed by users remotely.

The GNU *Lesser General Public License* (LGPL) offers a more permissive alternative to the GPL. It is designed for software libraries, allowing them to be used within proprietary applications without forcing the entire application to adopt the LGPL. This makes it attractive to commercial developers who want to use open source libraries while keeping their core application code closed. However, any modifications to the LGPL-covered library itself must still be made available under the same license. For example, a company can use an LGPL-licensed graphics engine within a closed-source game, as long as they don’t modify the engine. If they make enhancements to the

code of the engine itself, they must release the modified engine under the LGPL, but they still don't have to release the game under that license.

Permissive Licenses

In contrast, *permissive licenses* offer much greater flexibility. While they still allow users to view, modify, and distribute the code, they do not require derivative works to be released under the same license. This means that developers can incorporate open source code into proprietary or closed-source projects without the obligation to open their own code. Software released under permissive licenses is frequently used in both open source and proprietary projects, allowing maximum flexibility for developers and organizations.

The *MIT License* is one of the most popular and widely adopted permissive open source licenses. Its appeal lies in its simplicity and flexibility. It allows anyone to use, copy, modify, merge, publish, distribute, sublicense, or sell the software with very few restrictions. The only requirement is to include the original license notice and copyright statement in all copies or substantial portions of the software. Well-known projects such as React and Node.js use the MIT License.

The *Berkeley Software Distribution (BSD) License* is another classic permissive license, originating from the Unix-based BSD operating system. Like the MIT License, this license imposes minimal restrictions on the use and redistribution of the software. There are several variants of the BSD license (such as 2-Clause and 3-Clause), but all share the same goal: to encourage broad adoption of software while requiring only basic attribution and disclaimers. The BSD License is known for being business-friendly and legally straightforward, making it a favorite in academic and corporate environments.

The *Apache License 2.0* is also permissive but includes additional explicit terms regarding patent rights and contributor agreements, which provide legal protection for users and contributors. This makes it particularly attractive for enterprise and commercial use. Unlike the MIT or BSD licenses, the Apache License ensures that users receive a grant of patent rights, preventing contributors from later suing users for patent infringement. It also requires modifications to be clearly marked, which improves transparency when sharing code. Major projects like Apache Kafka, Hadoop, and Android use the Apache License 2.0 due to its robust legal framework.

In essence, the key difference lies in how the licenses handle redistribution and modification: copyleft licenses enforce freedom and openness across all derived versions, while permissive licenses allow for more leeway in how the code is reused, including in proprietary software.

License Compatibility and Multi-Licensing

License compatibility is a crucial consideration in open source development, because not all licenses can be combined freely. Some licenses include terms that conflict with those of others, making it legally problematic to mix code governed by different licensing models. A common example is the incompatibility between the GPL and permissive licenses like the MIT License. Since the GPL imposes strict copyleft requirements, integrating GPL-licensed code into a project under a permissive license would violate the GPL's terms.

To address different user needs and scenarios, some projects adopt a strategy known as *multi-licensing*. This approach allows the same software to be distributed under multiple licenses. For instance, a database might be made available under the GPL for open source use, while also offering a separate commercial license for proprietary applications. This enables broader adoption while maintaining compliance with open source principles. Multi-licensing also allows a developer to make some money off of the software while also providing it to non-commercial development projects without cost.

Dependencies and Package Management

Dependencies are external libraries or tools that an application relies on to function correctly. Managing these dependencies effectively is crucial not only for maintainability but also for ensuring the security and stability of software.

To simplify this process, developers use *dependency managers*—tools that automate the download, installation, and updates of required libraries. Each programming ecosystem typically has its own preferred manager. For instance, JavaScript and Node.js projects commonly use NPM (*Node Package Manager*), while Pip is widely used in Python environments. In the Java world, both Maven and Gradle are popular choices, with Gradle being especially favored for Kotlin projects. PHP developers often rely on Composer to handle package management.

These tools work by reading configuration files specific to each ecosystem. For instance, NPM consults a `package.json` file, while Gradle looks for a `build.gradle` file. The configuration file defines the required dependencies and their versions. Based on this configuration, the dependency manager fetches the appropriate packages from online repositories, ensuring that all required components are present and correctly versioned.

By streamlining dependency management, these tools help developers avoid the pitfalls of manual installation, including version conflicts and inconsistent environments across development, testing, and production systems.

Software Libraries

Libraries are reusable pieces of code that provide functions for a specific purpose such as mathematical calculation or handling web requests. Libraries allow developers to integrate complex features into their applications without having to build everything from scratch. They are designed to solve common problems or offer standardized solutions across different projects and languages.

For example, in the context of logging, Java developers often use Log4j, while Winston is a popular choice in the Node.js ecosystem. When it comes to authentication, Node.js uses the Passport.js library and Java often uses Spring Security to simplify the implementation of secure login and access control. In the area of testing, tools such as JUnit for Java and Jest for JavaScript help automate the validation of code, making it easier to ensure reliability and correctness.

Relying on well-maintained libraries not only accelerates development but also reduces the likelihood of bugs and security issues, because these libraries are often tested and reviewed by large communities. This promotes code quality, consistency, and long-term maintainability.

Software Bill of Materials

A *software bill of materials* (SBOM) is a comprehensive inventory that lists all the components, dependencies, and licenses included in a software application. Much like a bill of materials in manufacturing, an SBOM provides visibility into the inner structure of the software, detailing exactly what it is made of.

This level of transparency is becoming increasingly critical for several reasons. From a security perspective, SBOMs allow organizations to identify and track vulnerable components within their codebase, making it easier to respond quickly to threats. In terms of compliance, they help ensure that all included libraries and tools are used in accordance with their respective licenses, reducing the risk of legal issues. Furthermore, SBOMs enhance supply chain transparency, allowing teams to trace the origin and integrity of each component, an essential capability in a world of growing software interdependencies.

To help generate and maintain SBOMs, various tools have been developed, including SPDX, CycloneDX, and Syft. These tools automate the process of scanning applications and producing standardized SBOM documents. As a result, many organizations are incorporating SBOMs into their DevSecOps workflows, making them a key element of modern secure software development practices.

Guided Exercises

1. What is the difference between open source software and free software in terms of philosophy and goals?

2. Explain the main difference between copyleft and permissive open source licenses.

3. Why must developers consider software licensing when assembling an application from multiple components?

4. If a developer uses a GPL-licensed library as part of a new software application, under what conditions must the new software also be licensed under the GPL?

5. What is the role of a Software Bill of Materials (SBOM), and why is it increasingly important in modern software development?

Explorational Exercises

1. Research a real-world software project that is licensed under the GNU General Public License (GPL). Identify how its licensing terms influence the way it can be used or integrated into other applications. Consider the following: Has the project been incorporated into other software? If so, how did the developers comply with the GPL terms? If not, are there any known examples where the GPL license created legal or practical challenges?

Summary

Modern applications are rarely built from scratch. They are composed of multiple software components, often combining original code with third-party libraries and frameworks. Understanding how these components come together is essential for developers and system administrators alike. This lesson explored the difference between proprietary and open source software, and how each plays a role in modern software development.

The lesson also dived into the world of software licensing, presenting widely used open source licenses such as GPL, MIT, Apache, and others. The lesson covered key distinctions between copyleft and permissive licenses, and introduced important concepts such as license compatibility and multi-licensing. It introduced dependency managers such as NPM, Gradle, and Composer, and how to track components through a software bill of materials (SBOM)—a crucial practice for ensuring security and compliance in software projects.

Answers to Guided Exercises

1. What is the difference between open source software and free software in terms of philosophy and goals?

Open source software emphasizes transparency, collaboration, and innovation. Its focus is on practical development benefits like peer review and rapid evolution of code. Free software, on the other hand, is grounded in the philosophy of user freedom. Promoted by the Free Software Foundation, it upholds ethical rights such as the freedom to run, study, modify, and share software. While both promote access to source code, free software is more ideologically driven, whereas open source prioritizes technical and collaborative advantages.

2. Explain the main difference between copyleft and permissive open source licenses.

Copyleft licenses, such as the GNU GPL and AGPL, require any derivative works to be released under the same license, preserving the open nature of the code. This means that if a developer modifies GPL-licensed software and distributes it, the entire modified version must also be free and open source. Permissive licenses, like MIT, BSD, and Apache 2.0, allow more flexibility; developers can incorporate the code into proprietary software without having to disclose their source code, provided they meet basic attribution requirements.

3. Why must developers consider software licensing when assembling an application from multiple components?

Developers must consider licensing because each component in a software project may be governed by different legal terms. Ignoring license compatibility can lead to violations, especially when combining code with conflicting requirements. For example, integrating a GPL-licensed library into a proprietary application could force the entire project to be released as free software under the GPL, which might not align with business goals. Understanding and managing these licenses ensures legal compliance and avoids unintended exposure of source code or trade secrets.

4. If a developer uses a GPL-licensed library as part of a new software application, under what conditions must the new software also be licensed under the GPL?

If the GPL-licensed library is linked to or incorporated into the new application in a way that creates a derivative work, and if the work is distributed, the entire software must be released under the GPL. This means the developer must make the source code available under the same GPL license and grant users the same freedoms defined by the GPL. The GPL enforces this to ensure that software derived from GPL code remains free and open. Simply using a GPL tool externally (like running a script that invokes a GPL program) does not trigger this requirement.

Using the derivative software solely within the organization that developed the application is also exempt from releasing the software under the GPL.

5. What is the role of a Software Bill of Materials (SBOM), and why is it increasingly important in modern software development?

An SBOM is an inventory that lists all the components, dependencies, and licenses within a software application. It provides transparency into the structure of the software, helping teams identify security vulnerabilities, ensure license compliance, and enhance supply chain visibility. As applications become more modular and depend more heavily on third-party components, SBOMs are essential for managing risk, maintaining trust, and supporting secure development practices.

Answers to Explorational Exercises

1. Research a real-world software project that is licensed under the GNU General Public License (GPL). Identify how its licensing terms influence the way it can be used or integrated into other applications. Consider the following: Has the project been incorporated into other software? If so, how did the developers comply with the GPL terms? If not, are there any known examples where the GPL license created legal or practical challenges?

An example could be VLC Media Player, which is licensed under the GPL. Because of its license, it cannot be legally bundled with software that is proprietary or distributed under an incompatible license, such as some versions of Apple's App Store policies. As a result, VLC has faced challenges distributing its app through certain platforms. Developers who wish to use VLC's libraries must also release their software under the GPL, ensuring that the resulting work remains open source. This reinforces the GPL's intention to keep derivative software free and transparent.



**Linux
Professional
Institute**

Topic 702: Application Container



702.1 Application Container Management

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 702.1

Weight

5

Key knowledge areas

- Understand the Docker and Podman architecture
- Use existing images from an OCI registry
- Operate and access containers
- Understand Docker networking concepts, including overlay networks
- Understand the concepts of DNS service discovery
- Connect container to container networks and use DNS for service discovery
- Understand Docker storage concepts
- Use Docker volumes for shared and persistent container storage
- Awareness of rootless containers

Partial list of the used files, terms and utilities

- `docker container *`
- `docker network *`
- `docker image *`
- `docker volume *`
- `podman container *`

- podman network *
- podman image *
- podman volume *



Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	702 Application Container
Objective:	702.1 Application Container Management
Lesson:	1 of 2

Introduction

Containers have revolutionized the development, testing, and deployment of applications. By providing a lightweight and portable runtime environment, containers have become foundational to modern DevOps and cloud-native development. Among the most widely used tools in the container ecosystem are Docker and Podman. This lesson provides an in-depth exploration of their architectures, core components, and operational principles. A detailed comparison highlights how Docker relies on a centralized daemon to manage containers, while Podman operates in a daemonless and rootless architecture, offering enhanced security and better integration with systemd.

The lesson also discusses container image formats, networking models, and storage mechanisms shared between the two tools, emphasizing their interoperability through the Open Container Initiative (OCI) standards.

Foundations of Containerization

Before diving into Docker and Podman, it is essential to grasp the fundamental concept of *containerization*, which lies at the heart of modern application development and deployment.

At its core, a *container* is a lightweight, portable unit that encapsulates an application together with everything it needs to run. The contents of a container include its runtime environment, configuration files, system tools, libraries, and dependencies. This packaging ensures that the application can run reliably and consistently across different computing environments, whether it is on a developer's local machine, a test environment, or a production server in the cloud.

One of the primary motivations for using containers is to solve the age-old challenge of environmental inconsistencies—the infamous “it works on my machine” problem. By standardizing the runtime environment, containers remove the dependency on underlying host configurations and eliminate surprises during deployment.

Containers differ significantly from traditional *virtual machines* (VMs). While VMs emulate entire operating systems along with hardware layers—which can lead to increased resource usage and slower startup times—containers share the host operating system's kernel, making them much more lightweight and efficient. This means multiple containers can run simultaneously on a single host without the overhead of multiple full OS instances.

Containers achieve isolation and resource control, without the overhead of virtual machines, by exploiting two key features provided by the Linux kernel: *namespaces* and *control groups* (cgroups). These are the foundational building blocks that enable containers to behave like isolated systems while sharing the same operating system kernel.

Docker and Podman Architecture

Before focusing on hands-on usage, you should understand how Docker and Podman are structured internally. While both tools serve the same purpose—managing and running containers—their architectures reflect different design philosophies, especially around security, use of background services, and root privileges.

Both Docker and Podman adhere to the specifications defined by the *Open Container Initiative* (OCI), an industry standard for container image formats and runtimes. This compliance ensures that images built with Docker can be run by Podman, and vice versa, with full interoperability. Podman uses the same low-level runtime (runc) as Docker under the hood and can work with OCI-formatted images, making it a drop-in replacement in many use cases—especially in CI/CD pipelines, Kubernetes workflows, and developer machines.

Docker Architecture

Docker, launched in 2013, revolutionized containerization by making it accessible, portable, and easy to use. It simplified existing Linux container technologies with a user-friendly toolchain and standard image format, enabling developers to package applications and their dependencies into

portable units. Docker quickly became the de facto standard for containerized development in the open source community.

Docker follows a *client-server model*, which separates the user interface (client) from the background process (server) that performs container-related operations. This architectural design allows Docker to handle complex tasks in the background while exposing a simple and intuitive interface for developers. The key components in this model are:

Docker daemon (dockerd)

The daemon is the core service of Docker, running in the background on the host system. It is responsible for creating, running, and managing Docker objects such as *containers*, *images*, *volumes*, and *networks*. The daemon exposes a REST API, which the Docker client uses to send commands. Since the daemon typically runs with root (superuser) privileges, it has full control over system resources, which enables powerful operations. But root access also introduces potential security risks if the daemon is not properly isolated or protected.

Docker client (docker)

This is the command-line interface (CLI) that users interact with. Commands like `docker run`, `docker build`, and `docker ps` are issued through the CLI, which then communicates with the Docker daemon over the REST API (usually via a Unix socket). The CLI simplifies user interaction by abstracting low-level details, allowing developers to manage containers and images efficiently.

Container registries

Docker utilizes *registries* to store and distribute *container images*. These registries act as central repositories to which images are *pushed* after being built and then *pulled* to run containers. The most widely used public registry is Docker Hub, which hosts thousands of official and community-contributed images. Organizations can also host private registries to manage internal container assets securely.

Container runtime

At the lowest level, Docker relies on a container runtime to actually start and stop containers. Initially, Docker used its own monolithic engine, but over time, it adopted a modular approach. Today, Docker uses `containerd` as its container management engine and `runc` as the low-level runtime that interacts with the Linux kernel. These runtimes make use of kernel features such as namespaces (for process isolation) and control groups or cgroups (for resource limits) to provide a secure and efficient environment for each container instance.

Podman Architecture

Podman, released in 2018 by the libpod project under Red Hat, was designed as a secure, daemonless alternative to Docker. It allows containers to run as non-root user processes, enhancing security in multi-user environments. Fully OCI-compliant, Podman supports Docker images and runtimes, making it a drop-in replacement for Docker that preserves existing workflows while offering improved security and control. Features include:

Daemonless operation

One of the most significant differences between Podman and Docker is that Podman does not require a persistent background service (daemon) running as root. Every container-related command in Podman—such as running, building, or inspecting a container—is executed as an independent process. This eliminates the need for a central service with elevated privileges continuously running in the background. Podman’s architecture enhances transparency, auditability, and security. It also means that each container is spawned directly by the user process, making it easier to trace and manage using standard Linux tools like `ps`, `top`, or `kill`.

Rootless containers

Podman also lets non-root users run containers, a feature known as *rootless mode*. This minimizes the risk of privilege escalation attacks, because the user operating the container does not require administrative access to the host system.

Pods

These are groups of one or more containers that share the same network namespace. This design mirrors the pod concept in Kubernetes, making Podman a suitable tool for developing and testing applications intended for Kubernetes deployments.

Using Existing Images from an OCI Registry

Container *images* are at the core of how containers work. An image is a lightweight, standalone, executable package that includes everything needed to run a piece of software—including the code, runtime, libraries, environment variables, and configuration files.

Container images are composed of multiple read-only layers. Each layer represents a step in the image’s build process, such as installing dependencies or copying application files. These layers are stacked on top of each other and shared across images to save disk space and improve efficiency.

Using existing images from an OCI-compliant registry is a core part of working with containerized environments. Both Docker and Podman can pull, run, and manage container images from registries such as Docker Hub, Quay.io, GitHub Container Registry, and Google Container

Registry — basically, any registry that adheres to the OCI standard.

Pulling an Image from an OCI Registry

Images are retrieved using the `pull` subcommand. This downloads the image and its layers to your local system so it can be used to start containers.

The following examples show how to use an image from a remote registry. Because Podman offers full command-line compatibility with Docker, it is unnecessary to provide separate examples for each. All `docker` commands and options work identically with `podman`.

A practical approach to ensure compatibility when running existing scripts is to define a shell alias that redirects Docker commands to Podman:

```
$ alias docker=podman
```

This simple alias allows any script or command written for Docker to run seamlessly with Podman, without requiring modifications.

```
$ docker pull nginx:latest
```

In this example, `nginx` is the image name and `latest` is the *tag*. Tags are used to assign versions to images. If no tag is specified, the most recently built image (`latest`) is assumed by default. If the image isn't found locally, Docker or Podman will look it up in the default registry (usually Docker Hub unless configured otherwise).

You can also specify full image paths:

```
$ docker pull quay.io/<username>/myimage:1.0
```

Listing Local Images

After pulling images, the locally available container images can be listed using:

```
$ docker images
```

Authentication and Private Registries

When pulling images from a private registry, authentication may be required. Use the following command to log in with valid credentials:

```
$ docker login registry.example.com
```

This stores your credentials securely and allows access to protected images.

Operating and Accessing Containers

Containers are running instances of images. They transform the static definition of an image into a live, executable environment where applications actually run.

An image serves as a blueprint, while a container represents the realized instance of that blueprint in action. Whereas an image is immutable and read-only, a container is dynamic—it executes the packaged code along with its dependencies, configuration, and runtime context.

Once launched, a container operates as an isolated process on the host system, complete with its own filesystem, network stack, and process space, ensuring reproducibility and separation between applications.

The following subsections present the most common container management operations, illustrating how to start, inspect, stop, and remove containers effectively.

Running a Container

After pulling an image from a registry, the next step is to *run* a container derived from that image. This action launches an isolated environment in which the application executes, using the configuration and dependencies defined by the image. The basic syntax is:

```
$ docker run [options] <image-name>:<tag>
```

By default, this command:

- Pulls the image if it is not already available locally.
- Creates a new container from that image.
- Runs the container in the foreground (blocking the terminal).
- Uses default settings (no port mapping, no volume, no name, etc.).

Options can control the run:

```
$ docker run -d --name webserver -p 8080:80 -v /app/html:/var/www/html nginx:latest
```

This command:

- Runs the `nginx` container in the background (`-d`).
- Names it `webserver`.
- Maps the local machine's port `8080` to the container's port `80` using the `-p` option.
- Mounts the host directory `/app/html` to the container's mount point `/var/www/html` using the `-v` option.

Running a Container in Interactive Mode

Interactive mode enables direct interaction with a container's shell in real time, similar to working in a standard Linux terminal session. This mode is particularly useful for debugging, inspecting the container's environment, or executing commands manually within the container.

To start a container in interactive mode, use the `-it` option, where:

-i

keeps the standard input (*stdin*) open, allowing real-time user interaction.

-t

allocates a pseudo-terminal (*TTY*), providing a terminal interface for command-line access.

Examples:

```
$ docker run -it ubuntu /bin/bash
```

In the `ubuntu` image, `/bin/bash` is included by default, offering a complete Bash shell environment.

In contrast, the lightweight `alpine` image does not include Bash; instead, it provides `/bin/sh`, a minimal shell suitable for basic command execution and scripting.

To terminate the interactive container, simply type:

```
# exit
```

Accessing a Running Container

Occasionally, it becomes necessary to interact with a container that is already running—for example, to inspect files, troubleshoot issues, or execute additional commands within the container environment.

The `exec` command allows the execution of a new process inside an active container without restarting it:

```
$ docker exec -it webserver /bin/bash
```

The Docker example accesses a container named `webserver` using the Bash shell.

Why use `exec` instead of `run`?

`docker run`

starts a new container.

`docker exec`

runs a command inside an existing container.

The `exec` command should be used when a container is already running and there is no need to restart it or create a new one. It provides a convenient way to access the container's environment, inspect its state, or execute maintenance and diagnostic commands without disrupting its operation.

Listing Running Containers

After launching one or more containers, it is often useful to verify which containers are currently active on the system.

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
a1b2c3d4e5f6	nginx	"/docker-entry..."	2 minutes ago	Up 2 minutes	0.0.0.0:8080->80/tcp
webserver					

This command lists all active containers and provides detailed information about their current state and configuration:

CONTAINER ID

The unique identifier assigned to each container. This is typically a shortened version of a longer SHA256 hash and can be used in other commands such as `docker stop`, `podman logs`, or `docker inspect`.

IMAGE

The name of the image used to create the container, often including a tag (for example, `:latest` or `:v1.2`) to indicate the version.

COMMAND

Displays the entry point command executed when the container started.

CREATED

Indicates how long ago the container was created or started.

STATUS

Shows the current container state — for example, `Up`, `Exited`, `Paused`, or `Restarting`.

PORTS

Lists the network ports exposed by the container and their corresponding mappings on the host machine, shown as `<host_port>:<container_port>`.

NAMES

The human-readable name assigned automatically by Docker or Podman, unless explicitly set with the `--name` option.

To view all containers — both running and stopped — include the `-a` option:

```
$ docker ps -a
```

This extended listing is useful for reviewing past container runs, debugging issues, or cleaning up unused containers.

Creating a Container

The `create` command in both Docker and Podman creates a new container but does not start it automatically. This is particularly useful when a container needs to be prepared with specific configurations — such as environment variables, mounted volumes, or port mappings — before execution.

By separating creation from startup, this approach allows fine-tuning and inspection of container settings prior to running it.

```
$ docker create --name mynginx -p 8080:80 nginx
624840fa77280c6ddb6f7b0cb69d020d5a984d1ba16018480d373cde1d9411
```

The command output shows the full contained ID.

The `run` command shown earlier is a shortcut that combines `create` and `start`.

Managing a Container's Lifecycle

After a container has been created, its execution can be managed using the `start`, `stop`, `restart`, and `rm` commands available in both Docker and Podman. These commands control the *lifecycle* of a container, allowing it to be started, stopped, restarted, or removed without the need to recreate it.

Containers can be referenced by their *name*, their full *container ID*, or a shortened version of that ID. This flexibility simplifies day-to-day management tasks and supports both manual administration and automated scripts.

The following commands are used to manage the lifecycle of containers, providing control over their execution, termination, and removal as needed:

start

Launches a container that was previously created or stopped, resuming its execution without altering its configuration.

stop

Gracefully terminates a running container, allowing applications to shut down cleanly before the process is halted.

restart

Stops and then starts the container again—a convenient way to apply changes, refresh configurations, or recover from transient errors.

rm

Deletes containers that are no longer running. To also remove any associated anonymous volumes, use the `-v` or `--volumes` option.

These commands operate on containers that have already been configured and created, providing

precise control over when and how containers are executed, paused, or removed — without losing their established setup or configurations.

Viewing Detailed Container Information With `inspect`

The `inspect` command returns low-level JSON metadata about a container, image, volume, or network. It is useful for examining configuration, networking, mount points, environment variables, IP addresses, and more:

```
$ docker inspect <container-name-or-id>
```

This command outputs a large JSON object; specific fields can be filtered by using tools such as `jq`, or by adding the `--format` option:

```
$ docker inspect --format '{{.NetworkSettings.IPAddress}}' mynginx
```

Podman uses a different data structure than Docker for network information. As a result, the `.NetworkSettings.IPAddress` field is not available. Instead, use the following command to retrieve the container's IP address:

```
$ podman inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mynginx
```

Viewing Container Output With `logs`

The `logs` command displays the standard output (stdout) and standard error (stderr) streams of a container, whether it is currently running or has already stopped. This command is essential for diagnosing issues, especially when containers terminate unexpectedly or produce no visible output:

```
$ docker logs mynginx
```

Additional options enhance log viewing flexibility:

`-f` (“follow”)

Continuously streams the container's logs in real time, similar to `tail -f`.

`--since` and `--until`

Filter logs within a specific time range.

--tail *n*

Displays only the last *n* lines of the log output.

Example:

```
$ docker logs -f --tail 50 mynginx
```


Guided Exercises

1. What is the primary difference between a Docker container and a virtual machine (VM)?

2. How do you view all containers, including stopped ones, in Docker?

3. What is the purpose of the `docker exec` or `podman exec` command?

4. What is the difference between `docker run` and `docker create`?

5. You've launched a container called `my-nginx` using either Docker or Podman. How do you find out the container's internal IP address, so that you can use it for service discovery or testing within a custom network?

Explorational Exercises

1. Containers use Linux kernel features such as cgroups to control and limit resource usage. Research and answer the following:

- What are control groups (cgroups) in Linux and how do they relate to containerization?

- Using either Docker or Podman, how can you confirm that a running container is being limited or tracked using cgroups?

2. What are the default logging drivers used by Docker and Podman?

3. How can you change or configure a different logging driver for a specific container?

4. You have created a bridge network named `isolated_network` on two separate Docker nodes using the command:

```
$ *docker network create --driver bridge isolated_network$
```

```
---
```

```
+
```

Then you started one container on each node using `--network=isolated_network`. However, the containers are unable to communicate with each other. What needs to be done to allow these containers to interact across nodes?

```
+
```

```
|=====
```

```
|
```

```
|=====
```

```
[[sec.702.1_01-SU]]
```

```
<<<
```

```
== Summary
```

This lesson explained why containers are the environment of choice for developing modern, modularized applications and for running them in production. It described how the popular Docker and Podman tools create and manage images, which are the templates for creating containers, and the differences between Docker and Podman. The lesson ended with the most common commands for creating, interacting with, and viewing information about containers.

[[sec.702.1_01-AGE]]

<<<

== Answers to Guided Exercises

. What is the primary difference between a Docker container and a virtual machine (VM)?

+

Docker containers share the host OS kernel and are more lightweight, while VMs emulate an entire OS and hardware environment.

. How do you view all containers, including stopped ones, in Docker?

+

Use the `docker ps -a` command.

. What is the purpose of the `docker exec` or `podman exec` command?

+

It runs a command inside an already running container.

. What is the difference between `docker run` and `docker create`?

+

`docker run` creates and starts a container, whereas `docker create` only creates the container without starting it.

. You've launched a container called `my-nginx` using either Docker or Podman. How do you find out the container's internal IP address, so that you can use it for service discovery or testing within a custom network?

+

Use the `inspect` command to retrieve the IP address:

+

```
docker inspect --format '{{ .NetworkSettings.IPAddress }}' my-nginx
```

+

or

+

```
podman inspect --format '{{ .NetworkSettings.IPAddress }}' my-nginx
```

[[sec.702.1_01-AEE]]

<<<

== Answers to Explorational Exercises

. Containers use Linux kernel features such as cgroups to control and limit resource usage.

Research and answer the following:

+

* What are control groups (cgroups) in Linux and how do they relate to containerization?

+

Control groups (cgroups) are a Linux kernel feature that allows administrators to group processes and to measure, limit, or isolate their system resources (such as CPU, memory, block I/O, and network). In the context of containers, cgroups ensure that containers do not exceed specified resource quotas and do not affect each other's performance on the same host.

+

* Using either Docker or Podman, how can you confirm that a running container is being limited or tracked using cgroups?

+

Use the ``inspect`` command to locate the container's ID or cgroup path:

+

```
$ docker inspect --format '{{.Id}}' my-container
```

+

Then check cgroup usage from the host:

+

```
$ cat /sys/fs/cgroup/memory/docker/<container-id>/memory.limit_in_bytes
```

+

Alternatively, use a container ``stats`` command:

+

```
$ docker stats
```

. What are the default logging drivers used by Docker and Podman?

+

By default, Docker uses the ``json-file`` logging driver, which stores logs as JSON-formatted files on disk. Podman typically uses the ``k8s-file`` or ``journald`` logging drivers, depending on system configuration and installation.

. How can you change or configure a different logging driver for a specific container?

+

To configure a different logging driver when starting a container, use the ``--log-driver`` option:

+

```
$ docker run --log-driver=syslog nginx
```

. You have created a bridge network named `isolated_network` on two separate Docker nodes using the command:

+

```
[subs="specialchars,quotes"]
```

```
$ docker network create --driver bridge isolated_network ---
```

+ Then you started one container on each node using `--network=isolated_network`. However, the containers are unable to communicate with each other. What needs to be done to allow these containers to interact across nodes?

+ To enable communication between containers running on different Docker nodes, you need to use an overlay network instead of a bridge network. The bridge driver supports container communication only within the same host. Overlay networks, on the other hand, are designed for multihost networking and allow containers to communicate across nodes in a Docker Swarm or similar clustered environment.



**Linux
Professional
Institute**

Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	702 Application Container
Objective:	702.1 Application Container Management
Lesson:	2 of 2

Introduction

This lesson introduces the foundational concepts of networking, storage, and rootless containers—three core elements for building secure, reliable, and scalable container infrastructures. It examines how Docker manages networking, explaining how containers connect to one another and how DNS-based service discovery enables containers to locate services by name. It also discusses how containers can be attached to custom user-defined networks and how overlay networks operate in distributed, multi-host environments.

The lesson then focuses on Docker storage, describing how containers handle data persistence and how volumes offer a consistent and secure mechanism for sharing and storing data between containers and the host system. It concludes with the concept of rootless containers—an execution model that allows containers to run without requiring root (superuser) privileges. Rootless execution enhances security, particularly in shared environments or when handling untrusted workloads.

All concepts, commands, and configurations described in this lesson apply equally to both Docker and Podman. Both tools share the same command-line syntax and operational principles, and any example shown for Docker can be executed identically with Podman.

Docker Networking

In containerized environments, networking is a critical component of how containers communicate with other containers and services, whether inside the same host, across multiple hosts, or with the outside world. Docker provides a powerful, pluggable networking model that allows containers to be connected in flexible ways while maintaining isolation and security.

Docker offers several *network drivers*, each designed for specific use cases. These drivers include `bridge`, `host`, `none`, `overlay`, and `macvlan`. Each container is assigned its own network namespace, including a unique IP address, routing table, and network interfaces.

When a container is started, it can be attached to one or more of these networks using the `--network` option. This controls how the container interacts with other containers and external systems.

Bridge Network (Default)

The `bridge` network is Docker's default network driver for standalone containers. It creates a *private virtual network* on the host, allowing containers on the same bridge to communicate with each other using their internal IP addresses.

When a container is started without explicitly defining a network, it is automatically attached to the default `bridge` network. It provides basic connectivity between containers on the same host and enables outbound access to external networks through *network address translation* (NAT).

```
# docker run -d --name web nginx
```

To display all available networks on the system, use the following command:

```
# docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
ddc4000a164a	bridge	bridge	local
9863af190579	host	host	local
30680f288897	none	null	local

To examine the details of the `bridge` network—including configuration settings, connected containers, and IP address assignments—use:

```
$ docker network inspect bridge
```

Bridge networks operate in isolation from the host's main network interface, meaning containers on these networks cannot be accessed externally unless specific ports are published.

The `-p` option maps a container port to a host port, allowing external access to containerized services:

```
# docker run -d -p 8080:80 nginx
```

This maps port 80 inside the container to port 8080 on the host, making it accessible externally. Clients send traffic to the host's port 8080 to reach the web port on the container.

Host Network

Whereas the `bridge` driver preserves network isolation between the container and the host, the `host` driver removes this isolation. The container shares the host's networking stack, having direct access to its IP address and interfaces:

```
# docker run --network host nginx
```

This is useful for high-performance or low-latency use cases, but by sacrificing isolation it can lead to port conflicts.

None Network

The `none` network disables all networking for the container. It is useful for debugging or scenarios where network access is not needed.

The following example demonstrates how the `none` network mode completely isolates a container from all external and internal networks:

```
# docker run -it --network none alpine
```

Inside the container, network access is unavailable. Attempts to resolve hostnames or connect to the internet fail:

```
/ # ping lpi.org  
ping: bad address 'lpi.org'
```

Similarly, utilities that rely on DNS or external connectivity, such as `wget`, cannot reach external

resources:

```
/ # wget http://lpi.org
wget: bad address 'lpi.org'
```

Inspecting the network interfaces confirms that only the local loopback interface (lo) is active, with no external network interfaces available:

```
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
```

User-Defined Bridge Networks

By default, Docker containers on the same host can communicate with each other only when they are connected to the same user-defined network.

Creating a custom bridge network allows containers to automatically discover each other by name through Docker's built-in DNS service, simplifying inter-container communication.

To create a user-defined bridge network, execute:

```
# docker network create mynet
```

After creating a user-defined bridge network, confirm its presence by listing all available networks:

```
# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
2b0a1c0f12c3        bridge              bridge              local
e91d1b10cdd1        host                host                local
f4c2a7c2ff01        none                null                local
a9b3f84b12df        mynet               bridge              local
```

To attach a container to the new mynet network, specify the `--network` option when starting it.

The following example launches a container named db57 running a MariaDB database instance connected to the mynet network:

```
# docker run -d --name db57 \
--network mynet \
-e MYSQL_ROOT_PASSWORD=secret \
mariadb
```

This container listens on the default port 3306 within the `mynet` network, making it accessible to other containers connected to the same network.

Next, start another container attached to the same `mynet` network to verify that containers connected to the same user-defined bridge can communicate with each other.

The test confirms network connectivity between containers, regardless of which images are used.

```
$ docker run -it --rm \
--network mynet \
alpine sh
```

Run the following commands inside the `alpine` container to test communication with the MariaDB container `db57` through the `mynet` network.

This confirms that containers on the same user-defined bridge can reach each other by name using Docker's built-in DNS service.

```
/ # apk add --no-cache mariadb-client
/ # mariadb -h db57 -u root -psecret -e "SHOW DATABASES;"
+-----+
| Database          |
+-----+
| information_schema |
| mysql             |
| performance_schema |
| sys               |
+-----+
/ # exit
```

The Alpine container can reach `db57` via DNS resolution automatically provided by Docker.

From the host system, verify that both containers are attached to the `mynet` network by inspecting its configuration details.

The `docker network inspect` command displays all containers connected to the network, along

with their assigned IP addresses and metadata.

```
$ docker network inspect mynet
[
  {
    "Name": "mynet",
    "Driver": "bridge",
    "Containers": {
      "a1b2c3d4e5f6": {
        "Name": "db",
        "IPv4Address": "172.18.0.2/16"
      },
      "b2c3d4e5f6g7": {
        "Name": "app",
        "IPv4Address": "172.18.0.3/16"
      }
    }
  }
]
```

After completing the connectivity test, clean up the environment by removing the containers and the user-defined network. This ensures that no unnecessary resources remain allocated:

```
$ docker rm -f db57
$ docker network rm mynet
```

DNS Service Discovery

DNS service discovery allows containers to find and communicate with each other using human-readable names rather than hardcoded IP addresses.

In Docker and Podman, when containers are connected to the same custom, user-defined bridge network, they are automatically registered in an internal DNS system. This means that one container can resolve the hostname of another container by its container name, making service-to-service communication more flexible and reliable.

For example, if the host is running a database container named `db`, a web application container can connect to it simply by using `db` as the hostname. This dynamic and automatic discovery process simplifies configuration and enables scalable, microservices-based architectures where services can be added, removed, or updated without manual IP management.

Overlay Networks

The `overlay` network is designed for multihost communication. It enables containers running on different Docker hosts (typically part of a Swarm cluster) to communicate securely when encryption is enabled. Docker automatically routes each network packet to the appropriate daemon host and destination container, managing all communication transparently.

Overlay networks are essential in production environments where services need to scale across nodes.

Overlay networks require *Swarm* mode, which is Docker's native solution for container orchestration. Swarm mode allows you to deploy and manage a cluster of Docker nodes as a single virtual system. With Swarm, you can scale services across multiple machines, ensuring high availability, load balancing, and fault tolerance.

To set up a Docker Swarm environment, begin by initializing the swarm with the following command:

```
$ docker swarm init
```

Next, create an `overlay` network to enable communication between services across multiple nodes in the swarm:

```
$ docker network create --driver overlay my_overlay
```

Once the network is created, deploy services that utilize it.

For example, the following command launches an Nginx-based service named `webapp` connected to the `my_overlay` network:

```
$ docker service create --name webapp --network my_overlay nginx
```

The `overlay` network in Docker Swarm automatically manages several critical aspects of container communication across a distributed cluster. It enables cross-node communication, allowing containers running on different hosts to seamlessly interact as if they were on the same local network.

It also provides built-in service discovery, so containers can locate and communicate with services using simple DNS names. Additionally, the overlay network encrypts the traffic between containers, enhancing security by protecting data in transit.

The `overlay` network supports load balancing across service replicas, distributing incoming requests evenly to maintain performance and reliability across the cluster.

Inspecting Container Networking

To view the network settings of a running container, enter:

```
$ docker inspect <container_name_or_id>
```

Look for the `Networks` section of the output to see IP addresses, MAC addresses, and connected networks.

Alternatively, enter:

```
$ docker exec -it container_name ip addr
```

This shows the network interfaces and assigned IP addresses from inside the container.

Connecting and Disconnecting Networks

Containers can be dynamically attached to or detached from networks at runtime, allowing flexible reconfiguration of communication paths without stopping or recreating containers.

```
$ docker network connect mynet app
```

```
$ docker network disconnect bridge app
```

This capability simplifies multi-network setups, enabling containers to join or leave networks as required for different operational or security scenarios.

Removing Docker Networks

The `docker network rm` command removes one or more Docker networks from the system. This is particularly useful for cleaning up unused or obsolete networks that are no longer in use by containers.

Only *custom* (non-default) networks can be removed. Built-in networks such as `bridge`, `host`, and `none` cannot be removed, as they are managed by Docker itself. The syntax of the command is:

```
$ docker network rm [<network-name-or-id> ...]
```

If a network is still in use by any container, the command will fail with an error.

To enable removal, list the containers that are using the custom network:

```
$ docker network inspect my_custom_net
```

At this stage, containers connected to the network can be stopped or disconnected before the network itself is removed.

This ensures a clean and error-free removal process, as Docker does not allow the deletion of a network that still has active container attachments.

Cleaning Up Unused Docker Networks

The `docker network prune` command removes all unused Docker networks in a single step. An “unused” network is any custom network not currently connected to a running container. The syntax is:

```
$ docker network prune
```

When the command runs, Docker will prompt for confirmation before deleting. Default networks (`bridge`, `host`, `none`) are not affected by this command.

Macvlan Networks

A *Macvlan* network allows each container to appear as a physical device directly connected to the host network, with its own unique MAC address and IP address.

Unlike the traditional `bridge` driver, which performs network address translation (NAT), `macvlan` enables containers to communicate directly with other devices on the physical network—just like separate hosts.

This approach is useful in scenarios where containers must be accessible as independent devices on the same Layer 2 network as the host, such as when integrating with legacy systems, monitoring tools, or DHCP-based environments that expect unique MAC addresses per node.

To create a Macvlan network, specify the parent interface of the host and the subnet configuration:

```
$ docker network create -d macvlan \  
  --subnet=192.168.1.0/24 \  
  --gateway=192.168.1.1 \  
  -o parent=eth0 my_macvlan
```

In this example:

-d macvlan

defines the driver type.

--subnet and --gateway

configure the Layer 3 network settings.

-o parent=eth0

specifies the host's physical network interface used by the Macvlan network.

my_macvlan

is the name of the newly created network.

Containers attached to this network receive their own IP addresses within the specified subnet and can communicate with external devices on the same LAN without NAT.

However, direct communication between the host and its containers is not possible by default; a workaround involves creating a separate *Macvlan interface* on the host to act as a peer on the same subnet.

Macvlan networking is commonly used in environments that require high performance, isolation, or compatibility with systems expecting traditional network interfaces — such as IoT gateways, network appliances, or on-premises server integrations.

Docker Storage

Storage is a critical aspect of containerized applications. Although containers are often perceived as ephemeral and stateless, real-world applications frequently require *persistent data*: database files, uploaded media, configuration artifacts, etc. Docker provides several mechanisms to handle storage, allowing developers and operators to manage how data is stored, shared, and preserved across container lifecycles. Each of these mechanism—*volumes*, *bind mounts*, and *tmpfs mounts*—serves a different purpose and is suited for different scenarios.

When a container is running, it generates a temporary read-write (RW) layer on top of the image's immutable read-only layers, capturing all changes made during execution. This RW layer persists

even after the container is stopped with `docker stop`, and is removed from disk only when the container is deleted using `docker rm`. To streamline cleanup, `docker run --rm` automatically discards this layer after the container exits, while `docker container prune` removes all stopped containers and their associated RW layers.

Because this layer is inherently ephemeral, Docker provides persistent storage mechanisms to preserve data beyond the container's lifecycle, ensuring that important information is not lost when the container is removed.

Volumes

Volumes are the most robust and preferred mechanism for persisting data in Docker and Podman. Volumes are stored on the container's host machine. Volumes keep the data that is generated and used by containers in persistent storage, so that it lasts even as containers come and go. Unlike the container's RW layer, data in a volume remains intact even if the container is removed.

Docker and Podman manage volumes in specific directories on the host (e.g., `/var/lib/docker/volumes/` or `/var/lib/containers/storage/volumes/`), and volumes can be mounted into containers to store databases, configuration files, logs, or any other data the application needs to make persistent. This allows containers to be stateless while ensuring that important data is safely stored on the host.

Creating Volumes

Volumes are created using:

```
$ docker volume create mydata
```

Mounting or Attaching a Volume in a Container

Volumes are portable. Thus, they can be shared between multiple containers, making them ideal for production use. They are also decoupled from the container lifecycle, so deleting a container won't remove the associated volume unless its deletion is explicitly called for.

To attach a volume to a container, use the `-v` or `--volume` option:

```
$ docker run -v mydata:/app/data myimage
$ docker run --volume mydata:/app/data myimage
```

Alternatively, the `--mount` option can be used with the `docker run` command to provide a

clearer and more structured syntax for defining volume mounts:

```
$ docker run --mount type=volume,src=<volume-name>,dst=<mount-path>
```

While both options achieve similar results, `--mount` is generally recommended for its clarity and flexibility. It provides a more structured syntax and supports additional options, such as `readonly` or `ro`, which makes the volume read-only inside the container. You can simultaneously mount a single volume as read-write for some containers and as read-only for others.

The `readonly` option mounts a volume in read-only mode, preventing the container from modifying its contents. This is particularly useful for serving static files or protecting configuration data that should not be altered by running processes:

```
$ docker run -d \  
  --name=nginxtest \  
  --mount source=nginx-vol,destination=/usr/share/nginx/html,readonly \  
  nginx:latest
```

The same behavior can be achieved using the shorthand `-v` syntax by appending `:ro` (read-only) to the mount specification:

```
$ docker run -v ../project:ro nginx:latest
```

This maps a local directory `.` into `/project` inside the container in read-only (`ro`) mode.

Some special behaviors of volumes include:

- When a container starts with a volume that doesn't yet exist, Docker automatically creates a new, empty volume. This behavior is particularly useful for prepopulating shared data that multiple containers might later access.
- If an empty volume is mounted to a directory inside the container that already contains files or subdirectories, Docker automatically copies the existing contents of that directory into the new volume. This allows initial application files or configurations to be preserved in persistent storage.
- If a non-empty volume is mounted to a directory that already contains data, the original contents of that directory become temporarily hidden by the volume's contents. This behavior is similar to mounting an external drive (for example, `/mnt`) in Linux, where existing files are hidden until the volume is unmounted.

Listing Volumes

To list all available volumes on the host, use the following command:

```
$ docker volume ls
DRIVER      VOLUME NAME
local       mydata
```

Removing Unused Volumes

The `docker volume rm` command removes one or more volumes from the host. Multiple volumes can be deleted in a single command by listing them sequentially:

```
$ docker volume rm my-vol1 my-vol2
```

A volume cannot be removed while it is still being used by a container. To delete it, the container must first be stopped and removed:

```
$ docker stop mycontainer
$ docker rm mycontainer
$ docker volume rm myvolume
```

To remove all *unused* volumes in one step, use the `docker volume prune` command. This cleans up any volumes not currently attached to containers, freeing disk space:

```
$ docker volume prune
```

Shared and Persistent Container Storage

Volumes are particularly useful for sharing and persisting data between containers. For instance, one container can write log files or generated content to a shared volume, while another container reads, processes, or analyzes that same data—all without requiring manual file transfers or complex networking.

To create a shared volume:

```
$ docker volume create shared-data
```

This command initializes a named volume called `shared-data`, which can be mounted by

multiple containers:

```
$ docker run -d --name writer -v shared-data:/app/data my-writer-image
$ docker run -d --name reader -v shared-data:/app/data:ro my-reader-image
```

In this example:

- The `writer` container writes data to `/app/data` inside the shared volume.
- The `reader` container accesses the same directory in read-only mode (`:ro`), ensuring data consistency and preventing accidental overwrites.

This setup allows containers to share files seamlessly, even when isolated from each other.

Volumes are also valuable for backups, data inspection, and restoration tasks. Because volumes exist independently from container images, their contents persist even after containers are deleted — ensuring that essential data remains safe and reusable across deployments.

Bind Mounts

Bind mounts provide a direct link between a directory or file on the host system and a path inside a container. Unlike volumes, which are managed entirely by Docker, bind mounts rely on existing host filesystem paths. This creates a tight coupling between host and container, granting the container direct access to the source content:

```
$ docker run -v /path/on/host:/path/in/container myimage
$ docker run --mount type=bind,source=/home/user/code,target=/app/code myimage
```

Bind mounts are extremely useful in development environments, where real-time file updates are needed.

For example, when editing application source code or configuration files on the host, changes are instantly reflected inside the running container — no rebuilds required.

However, bind mounts should be used with care:

- They depend on the host's directory structure, reducing portability.
- They can expose sensitive host files if paths are not controlled properly.
- They provide containers with direct read/write access to host data, which can pose security risks in production environments.

For stable and portable deployments, volumes are generally preferred. Bind mounts are best suited for local development, testing, and troubleshooting scenarios.

Tmpfs Mounts

A `tmpfs` mount stores data directly in the host system's memory (RAM) instead of on disk. This makes it ideal for temporary or sensitive data that should not persist once the container stops. Because it never touches disk storage, access is extremely fast, and data automatically disappears when the container is removed or restarted.

To create a `tmpfs` mount with the `docker run` command, either syntax can be used:

```
$ docker run --mount type=tmpfs,dst=/app/cache myimage
```

or:

```
$ docker run --tmpfs /app/cache myimage
```

The two options behave equivalently, mounting a temporary in-memory filesystem at `/app/cache` inside the container. Any data written there is ephemeral and lost when the container stops, providing a secure and transient workspace.

Similar to volumes, if a `tmpfs` mount overlays an existing directory inside a container, the original contents of that directory become hidden until the mount is removed.

Two important restrictions apply:

- `tmpfs` mounts cannot be shared between containers. Each mount exists only within the container where it was created.
- This feature is supported only on Linux hosts.

`tmpfs` mounts are commonly used for caching, storing session data, or handling temporary files that must never persist on disk, balancing performance and security.

Rootless Containers

Rootless containers are a security-enhancing feature that allow containers to run without requiring root (administrator) privileges on the host system.

Traditionally, container engines like Docker required elevated permissions for essential tasks

such as creating namespaces, setting resource limits, and managing network interfaces. However, this posed security risks, especially in multitenant environments, where privilege escalation or container breakout could compromise the host.

With rootless containers, these operations are performed entirely in user space, using Linux features such as user namespaces. This means that a non-privileged user can create and manage containers without needing `sudo` access or interacting with system-level daemons.

Podman was designed to support rootless mode, and Docker has also added support for it in recent versions.

Rootless containers improve isolation and reduce the attack surface on shared systems. They are particularly useful in environments where users need to run containers securely without full administrative control. Such environments include CI/CD runners, development machines, and academic labs. While rootless mode may have some limitations (for instance, restricted access to certain networking modes or file systems), it offers a compelling tradeoff between functionality and security.

Guided Exercises

1. When you run a Docker container without specifying a network, which network driver is used by default?

2. You need two containers to communicate using container names. What kind of network should you create, and how?

3. How can you remove all Docker volumes that are no longer used by any container?

4. How can you make two containers share a common data directory?

5. How do you create a volume called `mydata`, use it in a container, and then remove it?

Explorational Exercises

1. You have two containers. One is a Node.js API server named `api-server` and the other is a PostgreSQL database named `pg-db`. You placed both containers on Docker's default bridge network without exposing any ports. Why might `api-server` be unable to connect to the host `pg-db` using its hostname? What solution would enable seamless communication between these two containers?

2. You created a named volume called `myvol` and used it in a container that writes logs to `/app/logs`. After stopping and removing the container, you run a new container with the same volume but see the old log files still there. Why do the log files persist even though the original container was deleted? How can you inspect what's inside the volume from outside the container?

3. You have run an NGINX container using the following command:

```
$ docker run -d --name web nginx
```

The container starts successfully, but when you open a browser and navigate to `http://localhost`, you get a connection error. Why can't you access the NGINX web server from your browser? What should you change to fix the issue?

Summary

Docker networking and storage are foundational to building reliable containerized applications. Docker offers several network drivers, including `bridge`, `host`, and `overlay`. Each is suited for specific scenarios, from isolated local setups to multihost communication in Swarm clusters. Features such as DNS-based service discovery and encrypted overlay networks enhance connectivity and scalability.

On the storage side, Docker supports volumes, bind mounts, and tmpfs mounts. Volumes provide persistent, portable data storage ideal for production use, while bind mounts are useful during development. tmpfs mounts store data in memory for sensitive information needed during the lifetime of the container. Combined with rootless containers, these features ensure flexibility, security, and efficient resource use.

Answers to Guided Exercises

1. When you run a Docker container without specifying a network, which network driver is used by default?

The default network is the `bridge` network. Docker automatically connects containers to the default `bridge` network if no other network is specified.

2. You need two containers to communicate using container names. What kind of network should you create, and how?

You should create a user-defined `bridge` network using:

```
$ docker network create mynet
```

Then run containers with:

```
$ docker run -d --name app1 --network mynet myapp1
$ docker run -d --name app2 --network mynet myapp2
```

3. How can you remove all Docker volumes that are no longer used by any container?

```
$ docker volume prune
```

4. How can you make two containers share a common data directory?

First, create a shared volume:

```
$ docker volume create shared-data
```

Then mount it into both containers:

```
$ docker run -d --name app1 -v shared-data:/app/data my-image1
$ docker run -d --name app2 -v shared-data:/app/data my-image2
```

5. How do you create a volume called `mydata`, use it in a container, and then remove it?

Create and attach the volume:

```
$ docker volume create mydata  
$ docker run -d --name myapp -v mydata:/data myimage
```

Before removing the volume, first stop and delete the container:

```
$ docker stop myapp  
$ docker rm myapp  
$ docker volume rm mydata
```

Answers to Explorational Exercises

1. You have two containers. One is a Node.js API server named `api-server` and the other is a PostgreSQL database named `pg-db`. You placed both containers on Docker's default bridge network without exposing any ports. Why might `api-server` be unable to connect to the host `pg-db` using its hostname? What solution would enable seamless communication between these two containers?

On the default `bridge` network, Docker does not provide automatic DNS resolution using container names. Therefore, `api-server` cannot resolve the hostname `pg-db` unless DNS is configured manually.

A better solution is to create a user-defined `bridge` network, which enables automatic DNS-based service discovery. Therefore, fix the problem as follows:

```
$ docker network create app-network
$ docker run -d --name pg-db --network app-network postgres
$ docker run -d --name api-server --network app-network my-api-image
```

Now the `api-server` container can connect to the host `pg-db` using its hostname, thanks to Docker's internal DNS system for user-defined networks.

2. You created a named volume called `myvol` and used it in a container that writes logs to `/app/logs`. After stopping and removing the container, you run a new container with the same volume but see the old log files still there. Why do the log files persist even though the original container was deleted? How can you inspect what's inside the volume from outside the container?

Named volumes are persistent and decoupled from container lifecycles. Deleting a container does not remove its associated volumes unless explicitly instructed to do so. That's why the old log files still exist.

To inspect the volume from the host, you can use another temporary container to mount the volume and access its contents:

```
$ docker run --rm -v myvol:/data busybox ls /data
```

3. You have run an NGINX container using the following command:

```
$ docker run -d --name web nginx
```

The container starts successfully, but when you open a browser and navigate to `http://localhost`, you get a connection error. Why can't you access the NGINX web server from your browser? What should you change to fix the issue?

By default, Docker containers on the `bridge` network do not expose their internal ports to the host unless explicitly mapped. In this case, although NGINX is running and listening on port 80 inside the container, the host has no way to access it, because no port mapping was configured.

To fix the issue, map the container's internal port to a host port using the `-p` option:

```
$ docker run -d --name web -p 80:80 nginx
```

Now, visiting `http://localhost` will work, as port 80 inside the container is mapped to port 80 on your host machine.



702.2 Container Orchestration

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 702.2

Weight

3

Key knowledge areas

- Understand the application model of Docker Compose and Podman Compose
- Create and run Docker Compose Files (version 3 or later)
- Define services, networks and volumes, along with their commonly used properties, in Docker Compose files
- Use Docker Compose to update running containers to newer images

Partial list of the used files, terms and utilities

- `docker compose`
- `podman-compose`
- `docker-compose.yml`



Linux
Professional
Institute

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	702 Application Container
Objective:	702.2 Container Orchestration
Lesson:	1 of 1

Introduction

Containerized environments need efficient management of multi-container applications. Docker Compose and Podman Compose simplify management by consulting YAML configuration files to define, run, and manage applications.

In this lesson, you will explore the application model of Docker Compose and Podman Compose, learn how to create and run Docker Compose files, define services, networks, and volumes along with their commonly used properties, and use Docker Compose to update running containers with newer images. It also describes key files and tools such as `docker compose`, `podman compose`, `compose.yaml`, and `docker-compose.yaml`, which play a central role in orchestrating containerized applications with ease.

Application Model for Docker Compose and Podman Compose

Modern applications are rarely built as single containers. They typically consist of multiple services—such as databases, APIs, message brokers, and front end web servers—that need to run together and communicate with each other. This is where tools such as *Docker Compose* and *Podman Compose* come in. They provide a simplified, declarative way to define and manage multi-

container applications, making development and orchestration much more efficient.

Both Docker Compose and Podman Compose allow developers to describe a complete application stack in a single file called `compose.yaml`, specifying how each containerized service should be built, configured, networked, and started. Instead of launching each container manually with long `docker run` commands, Compose automates the process using a *configuration-driven* model.

NOTE This lesson refers to version 3 or later of Docker Compose.

The Compose configuration file should be located in the project's working directory—that is, the directory where the application's source code and related configuration files reside. The preferred file name is `compose.yaml`, although `compose.yml` is also accepted. For backward compatibility with older setups, Docker Compose will also recognize the legacy names `docker-compose.yaml` and `docker-compose.yml`.

If multiple Compose files exist in the same directory, Docker Compose gives priority to the canonical `compose.yaml` file.

YAML

YAML (YAML Ain't Markup Language) is a human-readable data serialization format often used for configuration files. For instance, YAML appears in Docker Compose, Kubernetes manifests, GitHub Actions, Ansible playbooks, and many CI/CD systems. Its simple and clean syntax makes it easy to write and understand, compared to other data formats such as XML or JSON.

YAML uses indentation to represent data structure, meaning that whitespace is syntactically significant—much like how Python relies on indentation. Incorrect indentation can cause Docker Compose to misinterpret or reject the file entirely.

TIP To avoid such issues, it is highly recommended to configure the text editor with proper visual cues. In Vim, for instance, enabling cursor highlighting (both horizontally and vertically) helps detect indentation problems early.

A YAML parser can detect only syntax problems, such as missing colons or unmatched brackets. However, `yamllint` goes further—it is a static analysis tool for YAML files that checks for both syntax and style issues, ensuring clean, consistent, and well-formatted YAML.

Beyond detecting errors, `yamllint` also enforces best practices and formatting rules, such as indentation depth, line length, and key alignment, which are especially important in collaborative projects or automated pipelines.

Docker Compose

Docker Compose is a tool developed by Docker that lets you define and manage multi-container applications using a YAML file. Each *service* in the YAML file represents a container. You can specify image names, build contexts, environment variables, mounted volumes, ports to expose, and even network configurations.

The easiest and recommended way to install Docker Compose is by using *Docker Desktop*, which includes all necessary components such as Docker Compose, Docker Engine, and the Docker command-line interface (CLI). This all-in-one solution simplifies the setup process and ensures compatibility across systems. Docker Desktop is available for Linux, macOS, and Windows.

To verify that Docker Compose is installed and available on the system, check its version with:

```
$ docker compose version
```

After Docker Compose is installed and verified, the next step is to define a `compose.yaml` file that specifies the services, networks, and volumes required by the application.

This YAML file acts as a blueprint for a multi-container environment, describing how different components interact and start together.

For instance, the following `compose.yaml` defines a simple setup with two services—a web server running NGINX and a PostgreSQL database. Both services can be launched simultaneously with a single command: `docker compose up`.

```
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"

  db:
    image: postgres:14
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
```



```
db-data:
```

This file defines two services: an NGINX web server and a PostgreSQL database.

The `version` field specifies that the configuration uses version 3.8 of Docker Compose. It determines the format and features available in the file, along with the syntax and capabilities Docker Compose will understand when parsing the file.

However, as of Compose specification v3.9+ (and Docker Compose v1.27.0+), the `version` field is now optional and obsolete, because Docker Compose has moved toward a unified Compose specification maintained by the Compose Specification project. This new approach makes Compose files more consistent across platforms like Docker and Podman.

The first service, called `web`, uses the latest Nginx image and maps port 80 inside the container to port 8080 on the host machine. This means you can access the Nginx server by opening `http://localhost:8080` in your browser.

The second service, `db`, uses version 14 of the PostgreSQL image. It sets an environment variable called `POSTGRES_PASSWORD` with the value `example`, which defines the password for the default PostgreSQL user.

The application is using a named volume called `db-data` to persist PostgreSQL data. The first part (typically inside a service named `db`) mounts this volume to the container path `/var/lib/postgresql/data`, which is where PostgreSQL stores its database files. This ensures that the data inside the database is stored outside the container, in a persistent volume managed by Docker.

The second part at the root level of the Compose file — near the bottom of the example — declares the `volumes`. This tells Docker Compose to create the `db-data` volume automatically if it doesn't already exist. This setup is crucial for maintaining data across container restarts, rebuilds, or even system reboots. Without this, any data stored in the database would be lost when the container is removed.

This configuration is useful for quickly running a web server with a database using containers, and it is a common starting point for many web applications.

To start the multi-container application defined in the `compose.yaml` file, navigate to the project directory and run:

```
$ cd myapp
$ docker compose up -d
[+] Running 22/22
```

```

❑ db Pulled 12.2s
  ❑ 6e909acdb790 Pull complete 2.5s
  (...)
❑ web Pulled 10.3s
  ❑ 5eaa34f5b9c2 Pull complete 7.5s
  (...)
[+] Running 3/3
❑ Network myapp_default Created 0.1s
❑ Container myapp-web-1 Started 0.9s
❑ Container myapp-db-1 Started 0.9s

```

The `-d` option starts the containers in *detached mode*, allowing them to run in the background.

Docker Compose automatically creates a dedicated network (`myapp_default`) and launches each service defined in the configuration file.

Once the setup completes, the application is fully operational and accessible according to the defined port mappings.

These commands automatically create a network, pull the required images, start containers, and link them so they can communicate using service names (`web`, `db`).

By default, Docker Compose assigns the project name based on the directory containing your `compose.yaml` file. This project name serves as a prefix for resources such as containers, networks, and volumes, ensuring isolation between different environments.

Enter `http://localhost:8080/` in a browser to see the application running. Alternatively, you can retrieve the URL using `curl` in the terminal:

```

$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
(...)

```

To run your services in the background, pass the `-d` option (for “detached” mode) to `docker compose up` and use `docker compose ps` to see what is currently running:

```

$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE  CREATED        STATUS
PORTS

```

```

myapp-db-1      postgres:13      "docker-entrypoint.s..." db      5 seconds ago    Up 4 seconds
5432/tcp
myapp-web-1     nginx:latest      "/docker-entrypoint...." web      5 seconds ago    Up 4 seconds
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

```

This output lists all containers managed by the Compose project, showing their assigned names, corresponding images, commands, services, creation times, current statuses, and exposed ports. The table confirms that both the Nginx web service and the PostgreSQL database are running successfully.

To monitor the output of all running services in real time, use the `logs` command with the `-f` (“follow”) option:

```
$ docker compose logs -f
```

This continuously streams the combined logs of all services, helping with troubleshooting and real-time monitoring.

When the application is no longer needed, stop and remove all containers, networks, and volumes created by the `up` command with:

```
$ docker compose down
```

Docker Compose also allows assigning a custom project name. By default, the project name is derived from the directory name, but it can be overridden with the `-p` option. This is useful when managing multiple Compose environments on the same system:

```

$ docker compose -p myproject up -d
[+] Running 3/3
  □ Network myproject_default Created           0.1s
  □ Container myproject-db-1 Started            0.4s
  □ Container myproject-web-1 Started           0.5s

```

Using the `-p` flag keeps container, network, and volume names organized and easily identifiable by project.

Podman Compose

Podman Compose provides functionality equivalent to Docker Compose but operates in Podman-

based environments.

It reads the same `compose.yaml` file format and orchestrates containers, networks, and volumes using Podman instead of Docker.

Like Docker Compose, it automates the process of building, configuring, and launching multiple containers that form a single application stack—managing inter-service communication, environment variables, and persistent data storage.

Starting with Podman version 4.7.0 (and in *Podman Desktop*), the `podman compose` command is included by default, so no additional installation is required:

```
$ podman compose up -d
```

For older versions of Podman, the standalone `podman-compose` package must be installed separately, as it is not bundled with the core Podman utility. In such environments, the equivalent command becomes:

```
$ podman-compose up -d
```

Both tools are compatible with the same Compose file format, providing a consistent workflow across Docker and Podman ecosystems.

Defining Services, Networks, and Volumes in Docker Compose

Docker Compose defines three essential building blocks for containerized applications: *services*, *networks*, and *volumes*.

Each section of the `compose.yaml` file can be customized with specific properties that describe how containers interact, communicate, and persist data.

Together, these elements form the declarative model that allows Docker Compose to deploy and manage complex, multi-container environments with a single configuration file.

Services

A *service* represents an abstract definition of a computing resource within an application. Each service can be scaled or replaced independently of other components. Each service is backed by one or more containers that run the same image and are managed by the platform based on replication rules and placement constraints. Since services rely on containers, they are defined

using a Docker image along with a set of runtime parameters. All containers within a service are created identically using these parameters.

In a Compose file, services are defined under the top-level `services` key, which is a map where each key represents the *name* of a service and its corresponding value defines the service *configuration*. This configuration ensures uniformly to all containers launched by the service.

Optionally, a service can include a `build` section that specifies how to build the Docker image for that service. Compose supports image building directly from this definition.

You can configure how the container behaves using properties such as:

image

Docker image to use

build

path to a Dockerfile to build the image

ports

mapping of ports between the container and the host

volumes

volumes or bind mounts to mount on the host

environment

environment variables and their settings

depends_on

startup order

networks

network for communication

restart

restart policy (`always`, `on-failure`, `unless-stopped`, etc.)

The following example illustrates the structure of a `compose.yaml` file and explains the main configuration elements of a simple web service:

```
services:
  web:
```

```
image: nginx:latest
ports:
  - "8080:80"
volumes:
  - web-data:/usr/share/nginx/html
environment:
  - NGINX_HOST=localhost
networks:
  - frontend
restart: unless-stopped

volumes:
  web-data:

networks:
  frontend:
```

This configuration demonstrates how Docker Compose ties together multiple configuration aspects:

Service (web)

describes how the container runs — including image source, port mappings, and restart policy.

Volume (web-data)

ensures data persistence between container restarts.

Network (frontend)

provides an isolated communication channel between containers that belong to the same application stack.

This modular design allows applications to remain portable, consistent, and easy to manage across environments.

Networks

The top-level `networks` section in a `compose.yaml` file defines named networks that can be shared among multiple services. Containers attached to the same network can communicate directly using *service names* as hostnames, leveraging Docker's built-in DNS resolution.

Each service that participates in a custom network must explicitly declare its connection to it under the `networks` attribute. This explicit association ensures that only authorized services can communicate within that network, improving isolation and security.

The top-level `networks` element also provides advanced configuration options for more granular control of network behavior. If no custom network is defined, Docker Compose automatically creates a default network, but defining custom ones offers greater flexibility and control over communication patterns.

Common network properties include:

driver

specifies which network driver to use (for example, `bridge`, `overlay`, or `macvlan`).

ipam

allows customization of IP address management, including subnet, gateway, and static IP assignments.

external

indicates that the network already exists outside the current Compose file and should not be created by Compose.

```
networks:
  db-data:
    driver: overlay
```

In this example, the network named `db-data` uses the `overlay` driver, suitable for distributed environments such as Docker Swarm or multi-host setups.

Only services connected to this network can communicate with each other, providing isolation and predictable connectivity between application components.

Volumes

Volumes provide a mechanism for persistent data storage managed by the container engine. Docker Compose offers a portable and consistent way to define and attach these volumes to services, ensuring that important data—such as logs, databases, or uploaded files—remains available even after containers are stopped or removed.

The top-level `volumes` section in a `compose.yaml` file is used to define and name reusable volumes.

Each service that requires access to persistent data must explicitly reference one of these defined volumes under its own `volumes` attribute. This structure provides a clean separation between *volume declaration* and *volume usage*, making the setup easier to maintain and reuse across

different environments.

Common properties for volumes include:

driver

specifies the volume driver to use (for example, `local`, `nfs`, or a cloud driver).

external

indicates that the volume was created outside the current Compose file and should not be automatically managed by Compose.

name

assigns a custom name to the volume, allowing consistent identification across multiple projects.

Basic example of defining a volume:

```
volumes:
  db-data:
```

To use the defined volume in a service, reference it in the service configuration as follows:

```
services:
  db:
    volumes:
      - db-data:/var/lib/postgresql/data
```

In this setup, all PostgreSQL data stored inside the container's `/var/lib/postgresql/data` directory is persisted in the `db-data` volume.

Even if the container is stopped or removed, the database files remain intact, ensuring data durability across container restarts or image updates.

Full Compose File Example

The following `compose.yaml` file demonstrates a complete multi-service application that includes a web front end, an application backend, and a database. Each service is defined with its own configuration, dependencies, networks, and persistent storage:

```
services:
```



```
web:
  image: nginx:alpine
  ports:
    - "8080:80"
  volumes:
    - web-data:/usr/share/nginx/html
  networks:
    - frontend
  restart: always

app:
  build: ./app
  depends_on:
    - db
  environment:
    - FLASK_ENV=development
  ports:
    - "5000:5000"
  networks:
    - backend
  restart: always

db:
  image: postgres:14
  volumes:
    - db-data:/var/lib/postgresql/data
  environment:
    - POSTGRES_USER=admin
    - POSTGRES_PASSWORD=admin123
    - POSTGRES_DB=maindb
  networks:
    - backend
  restart: always

volumes:
  web-data:
  db-data:

networks:
  frontend:
  backend:
```

This `compose.yaml` file defines a typical multi-container application architecture offering three services: web, app, and db.

The `web` service uses the lightweight `nginx:alpine` image to serve static content, mapping port 80 inside the container to port 8080 on the host. It mounts a named volume called `web-data` to the Nginx HTML directory and connects to a custom network named `frontend`.

The `app` service builds its image from a local `./app` directory — presumably containing a Flask application — and defines the environment variable `FLASK_ENV=development`. The configuration includes a `depends_on` attribute referring to the `db` service, to ensure that the database starts first. It also exposes port `5000` and is connected to the `backend` network.

The `db` service uses the official `postgres:14` image and mounts a volume named `db-data` to persist data. It is configured with standard environment variables for username, password, and database name, and is also connected to the `backend` network.

All services have a restart policy set to `always`, ensuring they automatically restart if they stop unexpectedly.

Two named volumes, `web-data` and `db-data`, and two networks, `frontend` and `backend`, are declared at the top level to provide persistent storage and structured communication between services.

Using Docker Compose to Update Running Containers to Newer Images

Using Docker Compose, you can efficiently update your running containers to newer versions of their images, ensuring that your application stays current with the latest features, fixes, and security patches. This process involves pulling the updated image and recreating the containers based on your existing `compose.yaml` configuration.

Imagine you have PostgreSQL version 13 running as a service defined in your `compose.yaml` file, like this:

```
services:
  db:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: example
```

To update the services defined in a `compose.yaml` file to their latest available image versions, first ensure that the newest images are downloaded from their respective registries. This step refreshes the local cache with the most recent builds of each image used by the project:

```
$ docker compose pull
```

The `docker compose pull` command downloads the latest versions of the images defined in the Compose file without restarting any running containers. After updating the images, the containers must be recreated to apply the new versions:

```
$ docker compose up -d
```

When executed after a `pull`, the `up` command automatically detects image updates and replaces the currently running containers with new instances built from the latest versions. This ensures that services remain current without requiring a complete teardown of the environment.

For more granular control, the following options can be used:

--no-deps

recreates only the specified service containers, leaving their dependencies untouched.

--force-recreate

forces the recreation of containers even if the configuration or image has not changed.

After the update, to remove outdated and unused image layers from the system and free up disk space, execute:

```
$ docker image prune
```

Following this workflow keeps the deployment environment up to date, lightweight, and secure, avoiding unnecessary accumulation of obsolete images and containers.

Guided Exercises

1. Write a `compose.yaml` file that defines a service named `web` using the image `nginx:latest` and that exposes port 8080 on the host, mapped to port 80 in the container.

2. Extend the `compose.yaml` file to include a `db` service using the image `mariadb:latest`, set a password using an environment variable, and mount a volume to persist data.

3. Update the `web` service to include the `unless-stopped` restart policy and set the environment variable `NGINX_HOST` to the value `localhost`.

Explorational Exercises

1. Create a `compose.yaml` file that runs a basic WordPress site with a MariaDB back end. The wordpress service should use the image `wordpress:latest`, expose port 8080 on the host, and connect to the database via environment variables. The db service should use the image `mariadb:latest`, set the root password, and persist data using a named volume. Both services must be on the same custom network called `wp-network`.

2. What happens when you run `docker compose down` with additional parameters such as `--volumes` or `--rmi all`?

3. You have a `compose.yaml` file that defines a simple web service using the `nginx:alpine` image. Your goal is to run three replicas of the web service using Docker Compose. Create a basic `compose.yaml` with the web service and use a Docker Compose command to scale the service to three instances.

Summary

Docker Compose and Podman Compose provide a powerful and declarative way to define and run multi-container applications using a single YAML file. This file describes how different services—like web servers, databases, and backend apps—should be built, configured, connected through networks, and launched.

Each service in a Compose file can be finely controlled through properties such as image and version, ports, environment variables, volumes, and dependencies. Services can be scaled independently and linked together securely through custom networks. Podman Compose offers the same experience in Podman-based environments, with support for the same file format and similar commands. Compose promotes efficiency, consistency, and maintainability in containerized development workflows.

Answers to Guided Exercises

1. Write a `compose.yaml` file that defines a service named `web` using the image `nginx:latest` and that exposes port 8080 on the host, mapped to port 80 in the container.

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
```

2. Extend the `compose.yaml` file to include a `db` service using the image `mariadb:latest`, set a password using an environment variable, and mount a volume to persist data.

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"

  db:
    image: mariadb:latest
    environment:
      - MARIADB_ROOT_PASSWORD=example
    volumes:
      - db-data:/var/lib/mysql

volumes:
  db-data:
```

3. Update the `web` service to include the `unless-stopped` restart policy and set the environment variable `NGINX_HOST` to the value `localhost`.

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    environment:
      - NGINX_HOST=localhost
    restart: unless-stopped
```

```
db:
  image: mariadb:latest
  environment:
    - MARIADB_ROOT_PASSWORD=example
  volumes:
    - db-data:/var/lib/mysql

volumes:
  db-data:
```


Answers to Explorational Exercises

1. Create a `compose.yaml` file that runs a basic WordPress site with a MariaDB back end. The wordpress service should use the image `wordpress:latest`, expose port 8080 on the host, and connect to the database via environment variables. The db service should use the image `mariadb:latest`, set the root password, and persist data using a named volume. Both services must be on the same custom network called `wp-network`.

```
services:
  wordpress:
    image: wordpress:latest
    ports:
      - "8080:80"
    environment:
      - WORDPRESS_DB_HOST=db
      - WORDPRESS_DB_USER=root
      - WORDPRESS_DB_PASSWORD=example
      - WORDPRESS_DB_NAME=wordpress
    networks:
      - wp-network
    restart: unless-stopped

  db:
    image: mariadb:latest
    environment:
      - MARIADB_ROOT_PASSWORD=example
      - MARIADB_DATABASE=wordpress
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - wp-network
    restart: unless-stopped

volumes:
  db-data:

networks:
  wp-network:
```

2. What happens when you run `docker compose down` with additional parameters such as `--volumes` or `--rmi all`?

When you run `docker compose down`, Docker Compose stops and removes all the containers defined in your `compose.yaml` file, as well as the default network it created.

With additional options, the behavior becomes more destructive by removing additional artifacts:

`--volumes` removes any named volumes declared in the configuration file (such as databases or persistent storage volumes). Without this option, volumes are preserved.

`--rmi all` removes all images used by the services defined in the configuration file, including those pulled from Docker Hub or built locally. (Use it carefully if you want to clean up completely.)

`--remove-orphans` removes containers not defined in the current configuration file but still part of the same project/network (often left over from previous runs).

3. You have a `compose.yaml` file that defines a simple web service using the `nginx:alpine` image. Your goal is to run three replicas of the web service using Docker Compose. Create a basic `compose.yaml` with the `web` service and use a Docker Compose command to scale the service to three instances.

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
```

Scale the service using the CLI:

```
$ docker compose up --scale web=3 -d
```

The `--scale` option tells Docker Compose to run three identical containers for the web service. Each container is created from the same image and configuration defined in the Compose file. The `-d` option runs them in detached mode. Note: If multiple replicas of the same service attempt to expose the same host port (for example, `8080:80`), a port conflict occurs, preventing additional containers from starting.



702.3 Container Image Building

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 702.3

Weight

5

Key knowledge areas

- Create Dockerfiles and build images from Dockerfiles
- Understand OCI image names
- Upload images to a Docker registry
- Understand the principles of image scanners
- Understand security risks of container virtualization and container images and how to mitigate them
- Awareness Docker buildx, Docker Buildkit, Podman build and Buildah

Partial list of the used files, terms and utilities

- `docker image *`
- `docker login`
- `Dockerfile`
- `Containerfile`
- `.dockerignore`
- `FROM`
- `COPY`
- `ADD`

- RUN
- VOLUME
- EXPOSE
- USER
- WORKDIR
- ENV
- ARG
- CMD
- ENTRYPOINT



Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	702 Application Container
Objective:	702.3 Container Image Building
Lesson:	1 of 2

Introduction

With Dockerfiles, developers can define custom environments, install dependencies, copy application code, and configure runtime behavior in a clear, repeatable, and automated way.

Dockerfiles not only streamline the process of building consistent environments across development, testing, and production, but also promote version control and transparency. Whether you are deploying a simple web app or a complex microservice, mastering Dockerfiles is an essential step in automating your infrastructure and ensuring reproducibility across different systems.

In this lesson, you will learn how to write Dockerfiles from scratch, understand key instructions like `FROM`, `RUN`, `COPY`, and `CMD`, and apply best practices to keep your images clean, secure, and efficient.

Although the files described in this chapter bear the name `Dockerfile` traditionally used by Docker, other container engines such as Podman and Buildah prefer the more generic and open term `Containerfile`. Both formats are identical — only the filename differs.

Structure of a Dockerfile

Each instruction in a Dockerfile follows a specific format: an instruction *keyword* followed by its *arguments*. Lines that begin with a `#` character are treated as comments and ignored during the build process. While instruction keywords are not case-sensitive, it is common practice to write them in uppercase to easily distinguish them from the arguments that follow.

Docker processes the instructions in the Dockerfile in the exact order they appear, which means that the sequence is important. The Dockerfile must start with a `FROM` instruction, which specifies the base image on which your image will be built. This is the foundation for the rest of the instructions. Although comments, parser directives, and globally scoped `ARG` instructions can appear before the `FROM` line, no other Docker instructions are allowed to precede it.

In some cases, you might want to use build-time arguments to define the base image dynamically. Declare those arguments using `ARG` *before* the `FROM` instruction. This allows Docker to resolve those variables correctly when parsing the Dockerfile and selecting the base image.

The escape character `\` in a Dockerfile serves two purposes. When located in the middle of a line, it escapes the next character, just as it does in the Bash shell and other common tools. However, this escape behavior does not apply inside a `RUN` instruction.

Second, when placed at the end of a line, a `\` character continues the instruction to the next line of the file, which can make the file easier to read. This is especially useful when writing long commands.

FROM Instruction

The `FROM` instruction is always the first line in a Dockerfile, except for comments and global `ARG` lines. `FROM` tells Docker which *base image* to use as the starting point when building its custom image. Every Docker image is built on top of a base image, whether it is a minimal Linux distribution like Alpine, a full operating system like Ubuntu, or a preconfigured runtime like Node.js or Python.

The `FROM` instruction is essential: it defines the foundation of everything that follows in the Dockerfile. Choosing the right base image can improve performance, reduce image size, and simplify maintenance. You can use official images from the Docker Hub <https://hub.docker.com/> or even custom images stored in a private registry.

A basic example starts the build using Ubuntu 22.04 as the base image:

```
FROM ubuntu:22.04
```

The following example uses a prebuilt language runtime: the official Python image at version 3.11. At the time of writing, this is the standard image for Python-based projects:

```
FROM python:3.11
```

The following example uses multiple `FROM` instructions to create a *multistage build*:

```
FROM golang:1.20 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

FROM alpine:latest
COPY --from=builder /app/myapp /usr/local/bin/myapp
CMD ["myapp"]
```

This build takes place in two stages. The application is compiled in the first stage using the Go image. The compiled binary is copied into a lightweight Alpine image for the final build, omitting the libraries and utilities required during the compilation stage.

ARG Instruction

The `ARG` instruction defines a build-time variable that can be passed to the Docker build process using the `--build-arg` option. These variables are available only during the image build and do not persist in the final image like `ENV` variables do.

`ARG` is useful for customizing an image's build without hardcoding values directly into the Dockerfile. Because `ARG` variables are not encrypted and can be viewed in the image's history using `docker history`, they should *not* be used to pass secrets or sensitive data.

```
ARG BASE_IMAGE=alpine:latest
FROM ${BASE_IMAGE}
```

In this example, the base image is determined by a build-time argument, which makes the Dockerfile more flexible. You can specify the base image when running the build using the argument `BASE_IMAGE`:

```
$ docker build --build-arg BASE_IMAGE=debian:bullseye -t myimage .
```

The `command-line` option overrides the `ARG` in the Dockerfile, and the build uses `debian:bullseye` instead. So the command-line argument always takes precedence over the Dockerfile default.

COPY and ADD Instructions

The `COPY` and `ADD` instructions transfer files and directories from your local machine into the Docker image during the build process. Both instructions are commonly used to include source code, configuration files, scripts, and other assets inside a container.

COPY Instruction (Preferred)

The `COPY` instruction is simpler and more predictable. It copies files and folders from the host into the image without doing anything extra: no unpacking or downloading:

```
COPY ./app /usr/src/app
```

This copies the local `./app` directory into the image at `/usr/src/app`.

To copy individual files:

```
COPY requirements.txt /app/requirements.txt
```

To set ownership and permissions (when using Docker v1.31+):

```
COPY --chown=appuser:appgroup ./app /app
```

ADD Instruction

The `ADD` instruction does everything `COPY` does, plus two extra features:

- It can extract compressed files (e.g., `.tar`, `.gz`) automatically into the container.
- It can download files from a URL.

```
ADD source-code.tar.gz /usr/src/app/
```

This will automatically unpack the archive into `/usr/src/app/`.

An example of downloading from a URL follows:


```
ADD https://example.com/config.json /etc/config.json
```

This downloads the file from `https://example.com/` directly into the image at build time.

Both `COPY` and `ADD` help you bring external files into your image, but `COPY` is the safer, cleaner choice for most use cases. `ADD` should be used only when you specifically need its advanced features.

RUN Instruction

The `RUN` instruction executes commands inside the image at build time. It is commonly used to install software packages, configure the environment, or perform setup tasks that are needed before the container runs. You can use shell syntax, like:

```
RUN apt install -y curl
```

The *shell form* provides the command as a single string, similar to a shell command typed in a terminal. This form runs the command through `/bin/sh -c`, allowing for shell features like environment variable expansion or command chaining.

Another format, called *exec form* uses a JSON array syntax. The following instruction is equivalent to the previous example:

```
RUN ["apt", "install", "-y", "curl"]
```

The *exec form* is the recommended format because it does not invoke a shell, resulting in more predictable and secure behavior.

Each `RUN` command creates a new *layer* in the image, and the results of those commands become part of the final image filesystem.

To optimize builds, it is good practice to combine related commands into a single `RUN` instruction using logical operators such as `&&`. This reduces the total number of layers and keeps the image smaller and more efficient.

For example, instead of using multiple `RUN` commands to update the package manager and install a package, combine them into one:

```
RUN apt update && apt install -y curl
```

The instructions in `RUN` do not execute when the container starts — for runtime behavior, the `CMD` or `ENTRYPOINT` instructions should be used instead.

Efficient use of the `RUN` instruction ensures that images remain clean, compact, and fast to build, improving both security and performance in containerized environments.

Cache Invalidation for `RUN` Instructions

By default, Docker caches the results of each `RUN` instruction to speed up future builds. This cache is not automatically invalidated when the base image or remote resources change. For example, if you have a `RUN apt-get dist-upgrade -y` instruction, Docker may reuse the package cached by a previous run when running the next build, even if newer package versions are available.

To ensure that Docker executes all `RUN` instructions freshly — without using cached layers — you can use the `--no-cache` option when building the image:

```
$ docker build --no-cache .
```

This forces Docker to skip all cache layers and run every step from scratch, which is useful when working with frequently updated dependencies or when debugging builds.

VOLUME Instruction

The `VOLUME` instruction is used to declare a mount point within the container that is linked to a volume managed by Docker. This means that any data written to that location inside the container will be persisted outside the container's writable layer, surviving container restarts, rebuilds, or even deletions. Volumes are especially useful for storing data that should not be lost, such as database files, user uploads, or configuration data.

When you define a volume using `VOLUME`, Docker automatically creates an anonymous volume unless you specify a named one later through `docker run` or `docker compose`. For example, the instruction `VOLUME /data` tells Docker that the `/data` directory inside the container should be treated as a persistent storage location. Any content written there won't be lost when the container is removed.

It is important to note that `VOLUME` is a build-time declaration, and it does not copy existing files from the container image into the volume when the container is started. If the directory already contains files in the image, they may be hidden by the new, empty volume unless handled carefully during the build process.

The value of the `VOLUME` instruction can be written in two ways: as a JSON array like `VOLUME`

`["/var/log/"]`, or as a space-separated list of paths, such as `VOLUME /var/log` or `VOLUME /var/log /var/db`. Both formats define one or more directories inside the container that should be treated as Docker-managed volumes.

Using `VOLUME` is a best practice for separating an application's logic from its data, ensuring better flexibility, portability, and data safety in containerized environments.

The following example declares a volume in a Dockerfile:

```
FROM ubuntu:22.04
VOLUME /mydata
```

This Dockerfile declares `/mydata` as a volume. Anything written to that directory will be stored in a Docker-managed volume, making it persistent across container runs.

WORKDIR Instruction

The `WORKDIR` instruction sets the working directory for any `RUN`, `CMD`, `ENTRYPOINT`, `COPY`, and `ADD` instructions that follow it in the Dockerfile. It behaves similarly to the `cd` command in a shell, changing the current directory context within the image during the build process.

If the directory specified does not exist, Docker will automatically create it, so there is no need to use `mkdir` beforehand. This makes `WORKDIR` a cleaner and more reliable way to manage directory navigation inside your image than manually chaining `cd` commands in a `RUN` instruction.

Using `WORKDIR` also improves readability and maintainability. Instead of repeating full paths throughout the Dockerfile, you can set the working directory once and refer to relative paths afterwards.

```
FROM node:18
WORKDIR /usr/src/app
COPY . .
RUN npm install
CMD ["node", "app.js"]
```

In this example, all subsequent commands like `COPY`, `RUN`, and `CMD` operate inside `/usr/src/app`, making the Dockerfile clean and concise.

The following example chains together multiple `WORKDIR` levels:

```
WORKDIR /var
```

```
WORKDIR log
```

This is equivalent to:

```
WORKDIR /var/log
```

Docker resolves relative paths by appending them to the previous `WORKDIR`.

USER Instruction

The `USER` instruction sets the user, and optionally the group, under which the following instructions run inside the container. By default, Docker containers execute commands as the root user, which has full administrative privileges. However, running containers as root can pose a security risk, especially in production environments.

The `USER` switch to a non-root user for improved security. This is particularly useful after installing packages or setting up an application during the build phase, where administrative access is needed initially, but not during runtime.

You can specify the user by *username* (e.g., `USER node`) or by *UID/UID:GID* (e.g., `USER 1001:1001`). If the specified user doesn't exist in the base image, it may need to create it using a `RUN` instruction and tools such as `useradd`.

The following example creates a custom user `myuser` and instructs the Dockerfile to run all subsequent instructions under that user:

```
FROM ubuntu:22.04
RUN useradd -ms /bin/bash myuser
USER myuser
```

EXPOSE Instruction

The `EXPOSE` instruction makes the container listen on specific network ports at runtime. It acts as a kind of documentation within the image, letting others outside the container know which ports are intended to be published or connected to. However, it is important to understand that `EXPOSE` does not actually publish the port — it only defines which port the application inside the container will use.

To make a port accessible from outside the container, you still need to use the `-p` or `--publish` option when running the container (e.g., `docker run -p 8080:80`) or define port mappings in a

`compose.yaml` file.

The following basic example declares a port that will be exposed:

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["npm", "start"]
```

In this example, the image declares that the application listens on port 3000. This helps developers or orchestrators understand how to interact with the container.

The following example exposes multiple ports:

```
EXPOSE 80 443
```

This tells Docker that the container listens on both HTTP (80) and HTTPS (443) ports.

While EXPOSE is optional and doesn't affect container behavior on its own, it helps make your image more readable, self-documenting, and compatible with container orchestration tools that can auto-detect exposed ports.

ENV Instruction

The `ENV` instruction defines environment variables that will be available in the Docker image, both during the build and at run time. These variables can influence the behavior of software inside the container, provide configuration settings, or store default values that can later be overridden when running the container.

Once an environment variable is defined using `ENV`, it becomes available to all subsequent instructions in the Dockerfile, such as `RUN`, `CMD`, and `ENTRYPOINT`. It can also be used inside scripts or application code running in the container:

```
ENV APP_PORT=3000
```

This sets the variable `APP_PORT` to the value `3000`. The variable can be used in later steps, for example:

```
EXPOSE $APP_PORT
```

The following example defines multiple variables in a single instruction:

```
ENV NODE_ENV=production VERSION=1.0.0
```

The following example refers to an environment variable inside a `RUN` command:

```
ENV USER_NAME=developer  
RUN echo "Hello, $USER_NAME"
```

The value of `USER_NAME` is expanded during the build process and used in the `RUN` instruction.

Even though `ENV` sets default values in the image, you can override them when starting a container through the `-e` option:

```
$ docker run -e APP_PORT=8080 myimage
```

CMD Instruction

The `CMD` instruction defines the default command that should be executed when a container is started from the built image. It is the command that runs unless you override it when running the container with `docker run`.

There can only be one `CMD` instruction in a Dockerfile. If you define multiple instructions, only the last one will be used. The instruction is often used to run an application, a script, or a server when the container starts:

```
FROM node:18  
WORKDIR /app  
COPY . .  
RUN npm install  
CMD ["node", "app.js"]
```

When the container is run, this image will execute `node app.js` by default.

The default command defined in a Dockerfile can be overridden at runtime. When starting a container with the `docker run` command, simply append a new command at the end of the line.

This replaces the instruction specified by `CMD` in the image:

```
$ docker run myapp node another-script.js
```

ENTRYPOINT Instruction

The `ENTRYPOINT` instruction defines the main command that always runs when the container starts. Unlike `CMD`, the command specified in `ENTRYPOINT` cannot be easily overridden, making it ideal for containers where the command should never change (like running a specific binary or entry script):

```
FROM ubuntu
ENTRYPOINT ["echo"]
```

When you run the container:

```
$ docker run myimage Hello World
```

it will output:

```
Hello World
```

Here, `ENTRYPOINT` is fixed as `echo`, and whatever you pass as arguments is appended to it.

The `exec` form is the preferred way to write the `ENTRYPOINT` instruction, because it avoids using a shell and provides more predictable behavior. The form looks like this:

```
ENTRYPOINT ["executable", "param1", "param2"]
```

Alternatively, you can use the shell form, which runs the command through the shell (`/bin/sh -c` on Linux):

```
ENTRYPOINT command param1 param2
```

Like `CMD`, only the last `ENTRYPOINT` defined in a Dockfile will be used.

Combining ENTRYPOINT and CMD

You can combine both of these commands to create a flexible default behavior. In this setup, `ENTRYPOINT` defines the main command, and `CMD` provides default arguments:

```
FROM ubuntu
ENTRYPOINT ["echo"]
CMD ["Hello from Docker!"]
```

This allows the container to run with the default message, or override it:

```
$ docker run myimage
```

prints `Hello from Docker!`.

```
$ docker run myimage Goodbye
```

prints `Goodbye`.

The `.dockerignore` File

The `.dockerignore` file tells Docker which files and directories to exclude from the build context when building an image. This is similar to how `.gitignore` works with Git. By excluding unnecessary files—such as source control metadata, temporary files, local build artifacts, or large files not needed inside the image—you can make your Docker builds faster, cleaner, and more secure.

When you run `docker build`, Docker sends the entire build context (everything in your current directory, by default) to the Docker daemon. If you don't use `.dockerignore`, that context might include files you don't need, slowing down the build and potentially leaking sensitive data into the image.

An example of a basic `.dockerignore` file follows:

```
.git
node_modules
*.log
.env
Dockerfile~
```


This example tells Docker to ignore:

- The entire `.git` folder (version control metadata).
- The `node_modules` directory (which can be rebuilt inside the container).
- All `.log` files (such as `debug.log`).
- The `.env` file (commonly used for secrets or configs).
- Any backup copy of the Dockerfile (the `Dockerfile~` produced by Emacs).

Never include sensitive files such as `.env`, SSH keys, or config files with credentials in your image. The `.dockerignore` file is a great way to make sure they are never even sent to the Docker daemon.

Guided Exercises

1. Create a Dockerfile that uses `ubuntu:22.04` as the base image and prints `Hello from Ubuntu!` when the container runs.

2. Copy `index.html` into `/usr/share/nginx/html/` using the `nginx:alpine` image.

3. Expose port 8080.

4. Set the working directory to `/app`, copy files, and run `npm install`.

5. If a Dockerfile refers to the container's base image without a specific version tag, which tag of that image is used to create the container?

Explorational Exercises

1. You have the following Dockerfile:

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
CMD ["npm", "start"]
```

Now, imagine you change a line of JavaScript in a `.js` source file (not in `package.json`). When you rebuild the image, will `npm install` run again? Why or why not?

Summary

To create Docker images, developers write Dockerfiles, which are text files containing step-by-step instructions that define how the image should be built. These instructions include `FROM` to set the base image, `COPY` and `ADD` to bring files into the image, and `RUN` to execute commands for purposes such as installing packages. The `.dockerignore` file helps optimize builds by excluding unnecessary files from the build context, improving performance and security.

Other essential instructions include `VOLUME` to declare persistent storage, `EXPOSE` to document which ports the container listens on, and `USER` to switch from the root user to a non-privileged user for security. You can also set the working directory with `WORKDIR`, define environment variables with `ENV`, and use `ARG` to define build-time variables. Finally, `CMD` and `ENTRYPOINT` control what runs when the container starts: `ENTRYPOINT` defines the main command, while `CMD` in this case provides default arguments to the main command.

Answers to Guided Exercises

1. Create a Dockerfile that uses `ubuntu:22.04` as the base image and prints `Hello from Ubuntu!` when the container runs.

```
FROM ubuntu:22.04
CMD ["echo", "Hello from Ubuntu!"]
```

2. Copy `index.html` into `/usr/share/nginx/html/` using the `nginx:alpine` image.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
```

3. Expose port 8080.

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 8080
CMD ["npm", "start"]
```

4. Set the working directory to `/app`, copy files, and run `npm install`.

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```

5. If a Dockerfile refers to the container's base image without a specific version tag, which tag of that image is used to create the container?

If a Dockerfile refers to a base image without specifying a version tag, Docker uses the image tagged as `latest` by default. For instance, given the following instruction:

```
FROM node
```

Docker pulls `node:latest`, which points to the most recent stable release of the Node.js image at the time of building.

“Latest” does not mean “always up-to-date” — it simply refers to the image tagged as `latest` by the maintainer, which can change over time. For reproducible builds, it is recommended to specify explicit version tags like `node:18` or `node:18.17.1`.

Answers to Explorational Exercises

1. You have the following Dockerfile:

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
CMD ["npm", "start"]
```

Now, imagine you change a line of JavaScript in a `.js` source file (not in `package.json`). When you rebuild the image, will `npm install` run again? Why or why not?

No, `npm install` will not run again, because the `COPY package.json .` and `RUN npm install` layers are cached. Since `package.json` hasn't changed, Docker reuses the cache up to that point. Only the `COPY . .` instruction and following layers will be rebuilt. This is why it is a best practice to copy `package.json` and run `npm install` before copying the rest of the source code — it helps use the build cache effectively.



Linux
Professional
Institute

Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	702 Application Container
Objective:	702.3 Container Image Building
Lesson:	2 of 2

Introduction

This lesson goes deeper into how container images are named, built, uploaded, and secured in modern DevOps workflows. We will begin by exploring the Open Container Initiative (OCI) image naming conventions, which define how images are identified and pulled from registries. You will then learn how to upload images to a Docker registry, making them accessible for deployment and collaboration across environments.

Security is a crucial part of any container strategy. You will discover how image scanners help detect vulnerabilities and gain insight into the risks of container virtualization and container images, along with practical strategies to mitigate those risks.

Finally, the lesson introduces the advanced build tools Docker Buildx, Docker BuildKit, Podman build, and Buildah. These offer greater flexibility, performance, and security during image builds.

Building Images With `docker build`

The `docker build` command creates a Docker image from a Dockerfile and a build context. It is one of the most essential commands in container development, because it turns your application

code and dependencies into a reusable, portable image.

When you run `docker build`, Docker reads the instructions from a file named `Dockerfile`, processes each step in order, and assembles an image. The build context is the set of files in your current directory (or a specified path), which Docker sends to the Docker daemon to use during the build.

The basic syntax is:

```
$ docker build [OPTIONS] PATH
```

The `PATH` points to a directory containing your `Dockerfile` and build context (usually `.` (dot) for the current directory). The `-t` option tags the image with a name and optional version, e.g., `-t myapp:1.0`. The `--file` or `-f` option allows you to specify a custom `Dockerfile` name or location, for example:

```
$ docker build -t myapp:1.0 -f Dockerfile.dev .
```

This builds an image using the `Dockerfile Dockerfile.dev` in the current directory and tags the image as `myapp:1.0`.

Each instruction in the `Dockerfile` (like `FROM`, `COPY`, `RUN`, `CMD`) creates a *layer* in the image. Docker caches these layers to speed up future builds. If the content or instruction hasn't changed, Docker reuses the cached layer, which makes builds more efficient.

OCI Image Names

An *OCI image name* is the standardized way to refer to container images according to *Open Container Initiative* (OCI) specifications. This naming format allows container tools such as Docker, Podman, and Buildah to pull, tag, push, and identify images consistently across different platforms and registries.

Understanding OCI image naming is essential for managing container images in CI/CD pipelines, in Kubernetes deployments, and across various cloud-native environments. It ensures portability, version control, and traceability of images across systems and registries.

An OCI-compliant image name has the following structure. Any element within `[]` brackets in the format is optional, so the `image` is the only required element:

```
[registryhost[:port]/][namespace/]image[:tag][@digest]
```

Let's break the format down:

registryhost

The domain name of the registry (e.g., `docker.io`, `quay.io`, `registry.example.com`). If omitted, the registry in Docker defaults to Docker Hub (`docker.io`).

port

Specifies the network port used to connect to a custom container registry.

namespace

Often represents a user or organization. On Docker Hub, the default is `library` for official images.

image

The name of the image itself, such as `nginx`, `ubuntu`, or `myapp`.

tag

Identifies a specific version of the image (e.g., `:latest`, `:1.0.3`). The tag defaults to `:latest` if not specified.

digest

A unique SHA256 hash that represents a specific, immutable version of the image (e.g., `@sha256:...`).

A registry is a remote server where Docker images are stored and from which the images can be shared, downloaded, and deployed. Popular registries include Docker Hub, GitHub Container Registry, GitLab Container Registry, and private registries like Harbor or AWS ECR.

Examples of image names follow:

nginx

Interpreted as `docker.io/library/nginx:latest` after defaults are filled in.

docker.io/<username>/myapp:1.2

Pulls version `1.2` of `myapp` from Docker Hub under user `<username>`.

registry.example.com/app/backend@sha256:abc123...

Pulls a specific digest from a private registry.

Uploading Images to a Docker Registry

After you have built a Docker image using `docker build`, the next step is often to upload (*push*) that image to a Docker registry. Uploading images to a Docker registry allows you to share, deploy, and reuse your containers across environments and teams.

To upload an image, you first need to tag it with the registry name, username or namespace, and repository name using the `docker tag` command:

```
$ docker tag myapp:1.0 docker.io/<username>/myapp:1.0
```

This creates a new tag for an existing Docker image named `myapp:1.0`. In this case, you are tagging the local image so that it is ready to be pushed to Docker Hub under the account or namespace `<username>`.

Let us break the format of the tag down:

`myapp:1.0`

The name and tag of the local image that was built using `docker build`.

`docker.io/<username>/myapp:1.0`

The fully qualified name for the image on Docker Hub. The name consists of:

`docker.io`

The hostname of Docker Hub (can be omitted as it is the default).

`<username>`

The Docker Hub username or namespace.

`myapp`

The repository under that account.

`1.0`

The tag, representing a version or variant of the image.

Before uploading an image to a registry, authentication is required for registries that enforce access control, such as Docker Hub.

Use the `docker login` command to authenticate with the registry:

```
$ docker login
```

The command prompts for a username and a password or access token, depending on the registry's authentication method.

Once the image has been properly tagged and authentication completed, it can be uploaded to Docker Hub using the `docker push` command:

```
$ docker push docker.io/<username>/myapp:1.0
```

This publishes the image to the specified repository, making it available for pulling and deployment on other systems or environments.

After the push completes, the image will be available online and can be pulled down to any other machine with Docker installed.

A full example follows for uploading an image to Docker Hub:

```
$ docker build -t myapp:1.0 .  
[...]  
$ docker tag myapp:1.0 yourdockerhubusername/myapp:1.0  
[...]  
$ docker login  
[...]  
$ docker push yourdockerhubusername/myapp:1.0  
[...]
```

In a Podman environment, you can use `podman build` to build container images from a Dockerfile, just as you would with `docker build`. It is part of the Podman toolset: an open-source, daemonless container engine that is compatible with Docker but designed with enhanced security and flexibility, especially for rootless environments.

Unlike Docker, which relies on a background daemon (`dockerd`), Podman executes each container as an independent user process, making it particularly suited for multi-user systems and environments with strict privilege separation, for example:

```
$ podman build -t myapp:1.0 -f Dockerfile .
```

The resulting image can then be managed, tagged, and pushed using the same syntax and

workflow as Docker, ensuring a seamless experience across both ecosystems.

Just like Docker, `podman build` processes the instructions in the Dockerfile line by line to create an image layer by layer. The syntax, options, and behavior are nearly identical to Docker, making it easy for users to transition between the two. The basic syntax is:

```
$ podman build [OPTIONS] PATH
```

Image Scanners

Image scanners are tools designed to analyze container images for security vulnerabilities, outdated packages, misconfigurations, and even embedded secrets. These scanners play a vital role in securing your containerized applications by identifying risks before images are deployed into production environments.

A Docker image can include operating system packages, libraries, and dependencies. Over time, these components might develop known security flaws, usually tracked as *Common Vulnerabilities and Exposures* (CVEs). Image scanners compare the contents of an image against public lists of vulnerabilities such as the U.S. National Vulnerability Database (NVD), and report any known issues.

The Scanning Process

Image scanners work by unpacking the image layers to inspect the files, libraries, and packages contained within the image. After extracting the contents, the scanner identifies the installed software and their versions, then compares this information against known vulnerability databases to detect any security issues. The scanner then reports the findings, usually assigning severity levels such as *low*, *medium*, *high*, or *critical*. Some advanced scanners may also provide suggestions for remediation or recommended updates, helping developers address the vulnerabilities before deploying the image to production.

Scanners can be run locally, integrated into CI/CD pipelines, or provided by container registries (like Docker Hub, GitHub Container Registry, or Harbor).

Why Use Container Image Scanners?

One of the primary functions of container image scanners is vulnerability detection. Since container images typically bundle third-party libraries, dependencies, and operating system components, they may contain known vulnerabilities. Scanners analyze each layer of the image to detect outdated or risky packages, allowing developers to address these issues early in the

development lifecycle.

Scanners also support compliance and best practices. In many industries, organizations must meet strict regulatory standards such as GDPR, HIPAA, or PCI-DSS, which require regular security audits and documentation. Image scanners help ensure that containers comply with these policies and follow security best practices automatically, improving audit readiness and reducing manual effort.

The benefits of scanning are a form of supply chain security. Today's applications depend on a complex web of dependencies, including base images and third-party code. Image scanners help verify the origin and integrity of each component, mitigating the risk of supply chain attacks where malicious actors inject vulnerabilities into trusted code.

Image scanners contribute to a reduced attack surface. By continuously inspecting and improving container images, organizations can eliminate unnecessary risks, making it more difficult for attackers to exploit weaknesses. This proactive approach to container security enhances overall resilience and confidence in the deployment pipeline.

Security Risks and How to Mitigate Them

While containers are lightweight and efficient, they introduce unique security risks that differ from a traditional virtual machine (VM). One key concern is that containers share the host kernel, which means that a vulnerability in the container runtime or a misconfigured container could potentially affect the host system. Unlike VMs, containers rely more heavily on namespaces and cgroups for isolation, which — if poorly configured — can lead to privilege escalation or container breakout.

Another major risk lies in the container images themselves. Images could contain outdated packages, known vulnerabilities (CVEs), embedded secrets, or malware — especially when built from untrusted or unofficial sources. If developers pull images directly from public registries without verification, they might unknowingly introduce insecure components into production.

Shared Kernel Vulnerabilities

Containers share the host operating system's kernel, unlike virtual machines (VMs), which have isolated kernels. A vulnerability introduced into the kernel could allow an attacker to exploit one container to compromise others or even the host system.

To reduce security risks in containerized environments, it is essential to use a minimal and hardened host operating system, such as CoreOS or RancherOS, which are designed specifically for running containers securely.

Keeping the host kernel up to date and regularly applying security patches is also critical, because many container vulnerabilities stem from kernel-level exploits. Additionally, containers should be run with reduced privileges whenever possible.

Privileged Containers

Running containers with elevated privileges (e.g., using the `--privileged` option) grants them unrestricted access to the host system, making them a prime target for attackers.

To enhance container security, it is important to avoid running containers in privileged mode unless absolutely necessary. Instead of granting full privileges, use fine-grained Linux capabilities to provide only the specific permissions required by the application.

For example, if a container needs to modify network settings, you can run it with only the `NET_ADMIN` capability by using a command like:

```
$ docker run --cap-add=NET_ADMIN my-container
```

This approach limits the container's power while still allowing it to perform its intended function, aligning with the principle of least privilege.

Running Containers as Root

Running containers as non-root users is a fundamental security best practice that significantly reduces the potential impact of a container compromise. By specifying a non-privileged user in the Dockerfile using the `USER` instruction, you limit what an attacker can do if they gain access to the container. Containers running as root can interact with the host system more freely, increasing the risk of privilege escalation or unauthorized access.

Runtime Hardening

It is crucial to apply runtime security controls such as Seccomp, AppArmor, or SELinux. These kernel security modes provide fine-grained restrictions on system calls, file access, and process behavior, helping to enforce the principle of least privilege. These tools create a strong security boundary around your containers, protecting both the application and the host environment from malicious actions.

Resource Isolation Issues

Containers share resources including CPU, memory, and disk space. Misconfigured resource limits can lead to resource exhaustion attacks (e.g., one container consuming all available memory).

To prevent containers from consuming excessive system resources and potentially affecting other workloads, it is important to set resource limits. In Docker, this can be done using options such as `-m` to limit memory and `--cpus` to restrict CPU usage.

Network Exposure

By default, containers can expose unnecessary ports or use insecure networking configurations, leaving them vulnerable to network-based attacks.

To minimize exposure and reduce the risk of network-based attacks, it is recommended to use internal networks and avoid exposing containers directly to the internet unless absolutely necessary. Services that require external access should be placed behind load balancers, proxies, or API gateways.

Vulnerable Base Images

Base images often include outdated packages or libraries with known vulnerabilities. Attackers can exploit these vulnerabilities to gain unauthorized access.

To reduce the risk of vulnerabilities in container images, it is important to use minimal and trusted base images such as `alpine`, or to create *distroless* images. These images have a smaller attack surface because they include only the essential components needed to run an application.

Hardcoded Secrets

Developers sometimes embed sensitive information such as API keys, passwords, or certificates directly into container images. If these images are compromised, the secrets are exposed. Always avoid including secrets in Dockerfiles or image layers.

Unnecessary Packages

Including unnecessary packages in container images increases the attack surface and introduces potential vulnerabilities. Use multi-stage builds to create lean images that include only essential components.

Malicious or Tampered Images

Downloading images from untrusted registries can introduce malicious code or tampered images into your environment. Pull images only from trusted sources (e.g., official Docker Hub repositories or private registries). Use image signing and verification tools like *Docker Content Trust* (DCT) to ensure image integrity.

Insecure Default Configurations

Default configurations in orchestration platforms may not be secure out-of-the-box. Follow security best practices for your orchestration platform (e.g., Kubernetes CIS benchmarks).

Docker Buildx, Docker BuildKit, and Buildah

Modern container development goes far beyond the traditional `docker build` command. Developers now have access to advanced, flexible tools that include Docker Buildx, Docker BuildKit, Podman build, and Buildah, each designed to meet the evolving demands of secure, scalable, and high-performance image creation. These tools empower teams to carry out more efficient builds, automate complex workflows, and support diverse environments and platforms.

Docker BuildKit is a powerful backend builder that enhances Docker's native build capabilities. It brings significant improvements such as faster and more efficient builds, parallel execution of independent layers, support for build-time secrets (e.g., SSH keys or API tokens), and advanced caching mechanisms that reduce redundancy across builds. These features are particularly valuable in large-scale CI/CD pipelines. BuildKit is now the default builder in modern Docker versions. It works seamlessly with Dockerfiles while improving security, performance, and developer productivity.

Docker Buildx is a command-line extension that sits on top of BuildKit and enables cross-platform builds. Buildx can build images for multiple architectures — such as `linux/amd64`, `linux/arm64`, or `linux/ppc64le` — on a single host using emulators such as QEMU. Buildx also supports multi-node build clusters, remote cache exports, and advanced driver configurations, making it ideal for cloud-native workflows and distributed build environments. It is a must-have for teams targeting multiple deployment platforms (e.g., cloud, edge, and IoT).

Buildah is a lower-level, scriptable image builder developed by the same team behind Podman. Unlike Docker or Podman, Buildah allows you to build images from scratch or from existing layers without even using a Dockerfile. It provides complete control over each step of the build process via a command-line interface or shell scripts. This design makes it highly suitable for automation in CI/CD pipelines, minimal container builds, and systems that prioritize fine-grained security controls and rootless execution.

These tools represent the next generation of container image builders, designed to be faster, more secure, and more flexible than ever before. Whether you are building on-premises, in the cloud, or across multiple architectures, understanding and employing these tools gives you a powerful advantage in the container ecosystem.

Guided Exercises

1. Create a Dockerfile that uses the `node:18` image, copies the current directory into `/app`, installs dependencies with `npm install`, and runs `npm start`. Then build and tag the image as `myapp:1.0`.

2. Tag the `myapp:1.0` image to match your Docker Hub namespace and push it to the registry.

3. Create a simple Dockerfile using the `alpine` image and build it using `podman`.

4. Modify the following Dockerfile to create a non-root user and run the container as that user instead of the default root user.

```
FROM python:3.11
WORKDIR /app
COPY . .
CMD ["python", "app.py"]
```

5. Run a container with limited CPU and memory to prevent resource exhaustion attacks.

Explorational Exercises

1. A developer accidentally commits a `.env` file containing database credentials into the source code repository. Later, the file is included in a Docker image via `COPY . .` and the image is pushed to a public registry. What are the security implications of this mistake, and what steps should be taken to prevent and remediate it?

2. A developer builds an image using the `node:18` base image for a simple static site and includes many tools for local development (e.g., `curl`, `vim`, `git`). The image is pushed to production. Why is this a potential security risk, and how could the image be optimized for production use?

Summary

This lesson explored how to create, tag, and manage container images using tools like `docker build`, `podman build`, and advanced builders such as Docker Buildx, BuildKit, and Buildah. You learned how the `docker build` command transforms application code and dependencies into reusable images, and how OCI-compliant image names ensure portability and consistency across different registries and platforms. You also saw how to upload images to remote registries like Docker Hub, using commands such as `docker tag`, `docker login`, and `docker push`, enabling collaboration and deployment across teams and environments.

Beyond building and distributing images, the lesson focused on critical security practices, including the use of image scanners to detect vulnerabilities, and strategies to mitigate common container risks, such as avoiding privileged containers, running as non-root users, limiting resource usage, and isolating containers from the network.

The lesson concluded with an overview of modern image-building tools such as Docker Buildx and Buildah, which offer improved performance, multiplatform support, and better integration into secure, automated CI/CD workflows. Altogether, this foundation empowers you to build, secure, and manage container images with confidence and efficiency.

Answers to Guided Exercises

1. Create a Dockerfile that uses the `node:18` image, copies the current directory into `/app`, installs dependencies with `npm install`, and runs `npm start`. Then build and tag the image as `myapp:1.0`.

The Dockerfile is:

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```

The build command is:

```
$ docker build -t myapp:1.0 .
```

2. Tag the `myapp:1.0` image to match your Docker Hub namespace and push it to the registr:

Tag the image:

```
$ docker tag myapp:1.0 <yourdockerhubusername>/myapp:1.0
```

Log in to Docker Hub:

```
$ docker login
```

Push the image:

```
$ docker push <yourdockerhubusername>/myapp:1.0
```

Now your image is available in your Docker Hub repository.

3. Create a simple Dockerfile using the `alpine` image and build it using `podman`.

The Dockerfile is:

```
FROM alpine
CMD ["echo", "Hello from Podman"]
```

The build command is:

```
$ podman build -t hello-podman .
```

Run the container:

```
$ podman run hello-podman
```

The expected output is:

```
Hello from Podman
```

This confirms that Podman built and executed the container successfully.

4. Modify the following Dockerfile to create a non-root user and run the container as that user instead of the default root user.

Original Dockerfile:

```
FROM python:3.11
WORKDIR /app
COPY . .
CMD ["python", "app.py"]
```

The following rewritten Dockerfile reduces the risk of privilege escalation in case the container is compromised:

```
FROM python:3.11
WORKDIR /app
COPY . .

# Create a user and switch to it
RUN useradd -m appuser
USER appuser
```

```
CMD ["python", "app.py"]
```

5. Run a container with limited CPU and memory to prevent resource exhaustion attacks.

The following command limits the container to 512MB of memory and 1 CPU, preventing it from consuming all host resources:

```
$ docker run -m 512m --cpus="1" nginx
```

Answers to Explorational Exercises

1. A developer accidentally commits a `.env` file containing database credentials into the source code repository. Later, the file is included in a Docker image via `COPY . .` and the image is pushed to a public registry. What are the security implications of this mistake, and what steps should be taken to prevent and remediate it?

Including sensitive files like `.env` in a Docker image exposes secrets to anyone who can pull or inspect the image. Even if the image is later deleted, if it was pushed to a public registry, the credentials could have already been leaked or cached elsewhere.

Implications include unauthorized access to production databases or APIs, compromises of customer data or internal systems, and violations of security and compliance policies.

To prevent this, create a `.dockerignore` file to exclude `.env`, `.git`, and other sensitive files from the build context:

```
.env
.git
node_modules
```

After sensitive data exposure, immediate remediation is critical:

- *Rotate all exposed credentials* — database passwords, API tokens, certificates.
- *Delete compromised images and tags* from registries, but assume leaks are permanent.
- *Audit access logs* for suspicious connections or data exfiltration.
- *Clean the repository history* to remove `.env` traces.
- *Rebuild and redeploy* using new secrets, ensuring none are stored in Docker layers.
- *Implement secret management* (Vault, Docker/Kubernetes secrets, BuildKit secrets).

These actions contain damage, restore security, and prevent recurrence.

2. A developer builds an image using the `node:18` base image for a simple static site and includes many tools for local development (e.g., `curl`, `vim`, `git`). The image is pushed to production. Why is this a potential security risk, and how could the image be optimized for production use?

Including unnecessary packages such as `vim` and `git` in a production image increases the attack surface. More software means more potential vulnerabilities. Development tools should not be shipped with production code.

A better approach is to use a multi-stage build where one stage installs dependencies and builds the application, and a second stage uses a minimal base image (such as `alpine` or a distroless image) to include only the final output, for example:

```
# Stage 1: Build
FROM node:18 AS builder
WORKDIR /app
COPY . .
RUN npm install && npm run build

# Stage 2: Runtime
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
```

This way, the final image is smaller, faster to deploy, and more secure.



**Linux
Professional
Institute**

Topic 703: Kubernetes



703.1 Kubernetes Architecture and Usage

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 703.1

Weight

4

Key knowledge areas

- Understand the major components and services in a Kubernetes cluster
- Configure kubectl to use an existing Kubernetes cluster
- Use kubectl to get information about Kubernetes resources
- Use kubectl to create, modify and delete resources
- Awareness of Kubernetes Operators

Partial list of the used files, terms and utilities

- API-Server, etcd, Controller Manager, Scheduler
- `~/.kube/config`
- `kubectl get`
- `kubectl describe`
- `kubectl apply`
- `kubectl create`
- `kubectl run`
- `kubectl expose`
- `kubectl scale`

- `kubectl set`
- `kubectl edit`
- `kubectl explain`
- `kubectl config`
- `kubectl logs`
- `kubectl exec`



Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	703 Kubernetes
Objective:	703.1 Kubernetes Architecture and Usage
Lesson:	1 of 2

Introduction

Kubernetes has become the de facto standard for container orchestration, enabling developers and operators to deploy, scale, and manage containerized applications efficiently. This lesson explores the core concepts and essential tools needed to work effectively within a Kubernetes environment.

Kubernetes, often abbreviated to “K8s,” was originally developed by Google, drawing on their years of experience running large-scale containerized workloads on an internal system called Borg. Released as an open source project in 2014, Kubernetes was donated to the *Cloud Native Computing Foundation* (CNCF), where it rapidly gained adoption across the tech industry. Today, it is backed by a vibrant ecosystem of contributors and used by organizations of all sizes to power modern, cloud-native applications.

At the heart of Kubernetes architecture is a well-defined set of components that work together to manage the deployment, scaling, and operation of applications across clusters of machines. This lesson covers the major building blocks of a Kubernetes cluster: the control plane (with components like the API Server, Scheduler, Controller Manager, and etcd) and the worker nodes (which run the `kubelet`, Kube-Proxy, and container runtime). Understanding how these

components interact is key to mastering how Kubernetes maintains the desired state of its applications and ensures high availability, resilience, and automation at scale.

Kubernetes Architecture

A Kubernetes *cluster* is composed of a centralized set of components known as the *control plane*, along with one or more worker nodes where applications are actually deployed and run (Kubernetes components).

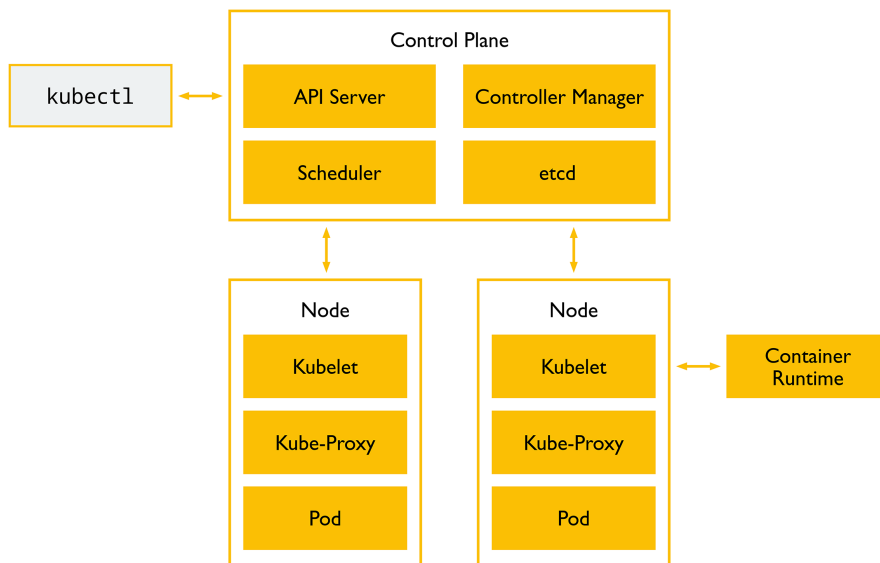


Figure 5. Kubernetes components

The control plane acts as the brain of the cluster, managing the overall state and orchestration of workloads. It handles scheduling decisions, monitors the health of components, and ensures that the desired state of applications is maintained across the environment.

The worker nodes are responsible for running the actual containerized workloads. Each node contains essential services that enable it to communicate with the control plane, manage containers, and provide the computing resources necessary to execute applications.

Together, the control plane and worker nodes form a scalable and resilient platform for managing containerized applications at scale. The following sections describe the make-up and operations of the control plane, worker nodes, and auxiliary services.

Control Plane

In a standard Kubernetes setup, especially in distributions such as kubeadm, Minikube, or K3s,

each component of the control plane—such as the API Server, Controller Manager, Scheduler, and etcd—typically runs as an individual container. This design promotes modularity, ease of deployment, and isolation between services.

API Server

The *API server* acts as the front end for the Kubernetes control plane. It exposes the Kubernetes API and serves as the central point of communication between users, automation, and other components. It is typically deployed as a containerized process named `kube-apiserver`.

The `kube-apiserver` is built to support horizontal scaling, meaning that its capacity can be increased by running multiple instances in parallel. By deploying several instances of `kube-apiserver` and distributing incoming traffic among them, you can improve the availability and performance of the control plane.

Users, administrators, and other Kubernetes components (such as the scheduler and controller manager) communicate with the cluster through this API. Any command you run using `kubectl` or an automation tool interacts with the API server.

The API Server has several key responsibilities within a Kubernetes cluster. The server validates and processes each request before passing it to the appropriate component. As the entry point for resource operations, it handles the creation, updating, and deletion of Kubernetes objects such as pods, deployments, and services.

Additionally, the API Server plays a crucial role in enforcing security by applying authentication, authorization, and admission control policies to ensure that only legitimate and authorized actions are performed within the cluster.

etcd

etcd is a distributed, consistent key-value store that acts as the single source of truth for the cluster's configuration data and state. It stores information about nodes, pods, configurations, secrets, and more, making it a critical component of the Kubernetes control plane.

etcd ensures that the cluster's state is consistent and recoverable, even in the event of failures. Misconfigurations or corruption in etcd can bring down the entire cluster, so understanding its role and backup strategies is vital.

etcd is known for being highly reliable and fault-tolerant. It uses the Raft consensus algorithm to maintain consistency across multiple etcd instances within a cluster, ensuring that all nodes agree on the current state of the data. Additionally, etcd stores versioned data, which allows administrators to roll back to previous states if necessary—an essential feature for recovering

from misconfigurations or failures.

Because of its critical role, etcd is often isolated and highly protected, and many Kubernetes distributions and cloud providers offer managed etcd instances with automated backups and high availability features.

The software that runs on etcd is simply the etcd server itself, a Go-based daemon that runs as a standalone process in a container. It can operate in single-node mode for basic setups or in multi-node clustered mode for high availability.

Internally, etcd stores data in a hierarchical key-value structure that resembles a filesystem (e.g., `/registry/pods/`, `/registry/nodes/`). The Kubernetes API Server interacts directly with etcd to persist and retrieve all cluster state, such as pod definitions, node status, or service configurations.

Controller Manager

The *controller manager* is responsible for maintaining the cluster's desired state by running various controllers: for example, the node controller, replication controller, and endpoints controller. All of these are bundled into a single `kube-controller-manager` binary, usually running inside a container.

Key Controllers

Controllers automate many operational tasks, reducing manual intervention. For example, if a pod crashes, the replication controller automatically spins up a replacement. Understanding these controllers helps in troubleshooting and optimizing cluster behavior:

The *node controller* is responsible for monitoring the status of each node in the cluster. It continuously checks whether nodes are healthy and responsive by listening for heartbeat messages sent by the `kubelet` running on each node. If a node fails to report within a configured timeout, the Node Controller marks it as `NotReady`. If the node remains unreachable for an extended period, the controller may evict all pods scheduled on that node to allow the workload to be rescheduled elsewhere. This behavior ensures that workloads remain available even in the face of hardware or network failures.

The *replication controller* ensures that a specified number of pod replicas is running and healthy at all times. If a pod fails or is deleted, the controller creates a new one to replace it.

The *endpoint controller* is responsible for managing the endpoint objects that link Kubernetes Services to the actual pods backing them. Every time a pod with matching labels is created or removed, the endpoint controller updates the corresponding endpoints resource to reflect the set

of IP addresses associated with that pod in that service. This allows Kubernetes Services to do automatic load balancing on traffic across the correct set of pods, ensuring reliable and dynamic service discovery within the cluster.

Service account and *token controllers* work together to automate the management of *service accounts* and their associated *secrets*. When a new namespace is created or a pod is scheduled, the service account controller ensures that a default service account is present. The token controller generates and attaches long-lived authentication tokens (stored as secrets) to service accounts, which are then mounted into pods as files. These tokens are used for secure communication between pods and the Kubernetes API, enabling *role-based access control* (RBAC) and authenticated API calls from within containers.

Together, these controllers form a key part of the Kubernetes control loop architecture, constantly observing the current state of the system and taking actions to ensure it matches the desired state.

Cloud Controller Manager

The *cloud controller manager* is an optional component of the Kubernetes control plane that handles interactions between your cluster and your cloud provider's APIs. It encapsulates all cloud-specific control logic, effectively separating it from the core Kubernetes components that operate independently of the underlying infrastructure. This separation ensures a cleaner architecture and allows Kubernetes to support multiple cloud providers with minimal disruption to the core system.

This component runs only the controllers that are relevant to the cloud environment where your cluster is deployed. These typically include controllers for managing cloud load balancers, networking routes, persistent volumes, and node lifecycles based on the cloud provider's metadata. If you are operating Kubernetes in a non-cloud environment — such as on-premises or within a local development setup — your cluster won't include a cloud-controller-manager, as it is not needed.

Scheduler

The *scheduler* assigns newly created pods to nodes based on resource availability and policy constraints. Like the other components, it runs as a container process under the name `kube-scheduler`.

The Kubernetes scheduler constantly watches for pods that have been created but are not yet assigned to a node. When such pods are detected, the scheduler evaluates a wide range of factors to select the most suitable node for each pod. These factors include resource availability like CPU and memory, as well as more complex criteria such as node taints and pod tolerations, node and

pod affinity or anti-affinity rules, and even custom scheduling constraints defined by the administrator.

Once all constraints are considered, the scheduler applies a scoring algorithm to rank eligible nodes and assigns the pod to the node with the highest score. Its goal is not only to meet the pod's requirements but also to ensure a balanced and efficient distribution of workloads across the cluster. This helps prevent resource bottlenecks, maximizes performance, and supports high availability.

The scheduler does not actually run the pod; it simply updates the pod specification with the chosen node. The `kubelet` on that node then takes over and starts the pod. This separation of concerns allows Kubernetes to scale efficiently and adapt dynamically to changes in cluster state.

Worker Node Components

Worker nodes are the machines (physical or virtual) where application workloads run. Each worker node runs several key components that enable it to execute and manage containers.

Kubelet

An agent named `kubelet` runs on each worker node and ensures that containers are running in the desired state. It communicates with the control plane and the container runtime.

The kubelet plays a critical role in Kubernetes as the primary agent running on each worker node. It communicates with the API server to receive instructions about which pods should be running on that node. Upon receiving these instructions, the kubelet interacts with the container runtime to create, start, stop, or restart containers as needed. In addition to managing the container lifecycle, the kubelet continuously monitors the health and status of the running pods and reports this information back to the control plane, ensuring that the cluster's views of the node's state is up to date.

The kubelet is the bridge between the control plane and the container runtime. If the kubelet process fails, the node may become unresponsive, leading to potential downtime for applications running on that node.

Kube-Proxy

Kube-proxy is an optional network proxy that runs on each worker node and implements part of the Kubernetes service concept. It ensures that network traffic is routed correctly between pods and services.

The kube-proxy is a key networking component in a Kubernetes cluster, responsible for managing the rules that govern how network traffic is routed to services. It handles iptables or IPVS rules to implement efficient load balancing, ensuring that incoming requests are correctly forwarded to the appropriate pod endpoints. Additionally, kube-proxy supports sophisticated features such as session affinity, which helps maintain a consistent connection to the same backend pod, and connection tracking, allowing the node to handle large volumes of network traffic reliably.

Container Runtime

The *container runtime* is a fundamental component that enables Kubernetes to execute and manage containers on each node. Its key functions include pulling container images from registries, launching containers based on those images, and managing their entire lifecycle from start to stop. It communicates with the kubelet through the *container runtime interface* (CRI), an abstraction layer that allows Kubernetes to support different container runtimes such as containerd, CRI-O, and Docker (via shim). This modular design allows for flexibility in choosing or switching runtimes while maintaining consistent functionality across the cluster.

Auxiliary Services

In addition to the core components, Kubernetes relies on several auxiliary services to enhance functionality and scalability.

Networking

Kubernetes requires a flat, consistent network model to allow pods to communicate seamlessly across nodes. This is typically implemented using networking plugins like Calico, Flannel, or Cilium.

Storage

Kubernetes provides abstractions like *persistent volumes* (PVs) and *persistent volume claims* (PVCs) to decouple storage from individual pods. Persistent storage ensures that data survives the termination, restart, and rescheduling of each pod. Properly configuring storage is essential for stateful applications such as databases.

NOTE

To complete the proposed exercises related to Kubernetes, you are advised to install Minikube by following the step-by-step instructions available at [Minikube Docs](#). Minikube is an excellent option for practicing with Kubernetes.

Guided Exercises

1. Explain the purpose of the Kubernetes API Server in the control plane. Why is it considered the “front door” of the cluster?

2. What kind of data does etcd store, and why is it critical for cluster reliability?

3. What are some factors the scheduler considers when assigning a pod to a node?

4. Describe the responsibilities of the kubelet. How does it interact with the rest of the Kubernetes system?

5. Define a Kubernetes pod and explain what resources are shared between its containers.

Explorational Exercises

1. Imagine you are managing a Kubernetes cluster, and you notice that one of your applications becomes unavailable after a node unexpectedly crashes. Describe how Kubernetes is designed to handle this situation, and which components are responsible for maintaining the application's availability. Would any manual action be required?

Summary

Kubernetes is the leading platform for orchestrating containerized applications. It provides developers and operators with powerful tools to deploy, scale, and manage software efficiently across distributed systems. Originally created by Google and now maintained by the Cloud Native Computing Foundation (CNCF), Kubernetes has evolved into a robust, open source ecosystem used across many industries. This lesson explored the major components of a Kubernetes cluster, including the control plane and worker nodes, and offered a foundation for understanding how Kubernetes maintains the desired state of applications at scale.

The lesson also covered essential services such as the API server, scheduler, controller manager, and etcd, which coordinate application workloads and ensure high availability and fault tolerance. On the worker side, the lesson explained the roles of the kubelet, kube-proxy, and container runtime. Kubernetes abstractions such as pods and services were introduced.

Answers to Guided Exercises

1. Explain the purpose of the Kubernetes API Server in the control plane. Why is it considered the “front door” of the cluster?

The API Server acts as the central communication hub for all components in the Kubernetes cluster. It exposes the Kubernetes API and handles all requests to create, update, delete, or read cluster resources. It also enforces security through authentication, authorization, and admission control. Because all interactions with the cluster go through it—including commands from `kubectl`—it is often called the “front door” of the cluster.

2. What kind of data does etcd store, and why is it critical for cluster reliability?

etcd stores all the persistent state data of the Kubernetes cluster, including node information, pod configurations, secrets, and service definitions. It is a distributed key-value store that ensures data consistency using the Raft consensus algorithm. Because all cluster state is saved in etcd, its availability and integrity are critical. If etcd is lost or corrupted, the cluster may stop functioning correctly.

3. What are some factors the scheduler considers when assigning a pod to a node?

The scheduler evaluates nodes based on several criteria: available CPU and memory, node taints and tolerations, affinity or anti-affinity rules, and custom constraints. It scores all eligible nodes and assigns the pod to the one that best satisfies the cluster’s scheduling policies and resource needs. The goal is to balance workloads and ensure optimal resource utilization.

4. Describe the responsibilities of the kubelet. How does it interact with the rest of the Kubernetes system?

The kubelet is an agent that runs on every worker node. It communicates with the API Server to get instructions about which pods to run. It then uses the container runtime to create and manage those containers. The kubelet also monitors the health of the containers and reports their status back to the control plane.

5. Define a Kubernetes pod and explain what resources are shared between its containers.

A pod is the smallest deployable unit in Kubernetes. It can contain one or more containers that need to run together. Containers in the same pod share the same network namespace (including IP address and ports), shared storage volumes, inter-process communication (IPC), and optionally the same PID namespace. This allows them to collaborate closely, like components of a single application.

Answers to Explorational Exercises

1. Imagine you are managing a Kubernetes cluster, and you notice that one of your applications becomes unavailable after a node unexpectedly crashes. Describe how Kubernetes is designed to handle this situation, and which components are responsible for maintaining the application's availability. Would any manual action be required?

In Kubernetes, the node controller detects when a node fails to send heartbeats and marks it as `NotReady`. After a grace period, it considers the node unreachable and evicts the pods that were running on it. The scheduler then looks for new nodes with enough resources and reschedules the pods on those nodes. The replication controller or deployment ensures that the desired number of pod replicas are running, so that, if a pod is lost due to node failure, a new one will be created automatically. This design allows Kubernetes to self-heal and maintain application availability without requiring manual intervention in most cases.



Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	703 Kubernetes
Objective:	703.1 Kubernetes Architecture and Usage
Lesson:	2 of 2

Introduction

This lesson shows how to interact with a Kubernetes cluster using `kubectl`, the powerful command-line tool that serves as the primary interface to Kubernetes. The lesson starts with querying and inspecting various Kubernetes resources, offering a clear view of what is running in the cluster and how it is behaving. From there, the lesson moves on to creating, updating, and deleting resources. When examples are run in a local Minikube environment, the lesson provides hands-on experience with the key commands needed to manage workloads, services, and infrastructure.

Introducing `kubectl`

`kubectl` is the command-line interface (CLI) that conveys administrator and developer commands to the Kubernetes API server. It provides a unified syntax for issuing HTTP requests to the API, enabling users to perform declarative and imperative operations on cluster resources. Through `kubectl`, administrators and developers can manage the full lifecycle of Kubernetes objects — such as pods, deployments, services, ConfigMaps, and more — using YAML manifests or direct commands.

`kubectl` also supports advanced functions such as label and field selectors, a choice of output formats, and resource patching, making it a critical tool for low-level interaction and automation in Kubernetes environments.

The kubeconfig File

Kubernetes uses a configuration file, commonly referred to as the *kubeconfig* file, to store information about clusters, users, and contexts. By default, this file is located at `~/.kube/config`. It contains the following high-level elements:

clusters

Information about the API server endpoint (e.g., IP address or DNS name) and certificate details.

users

Authentication details such as client certificates, tokens, or username/password combinations.

contexts

A combination of a cluster, a user, and an optional namespace, that together define how `kubectl` interacts with the cluster.

Developers with access to an existing Kubernetes cluster might find that someone has already provided a kubeconfig file. Otherwise, one should be created from scratch.

Context in Kubernetes

A *context* in Kubernetes is a configuration element within the kubeconfig file that defines access parameters for a specific cluster, including the `cluster`, the `user`, and optionally the `namespace` to be used by `kubectl`. Contexts allow users to quickly switch between multiple Kubernetes environments, such as development, staging, and production, without modifying the underlying connection details.

Each context is identified by a name and is composed of three main components:

cluster

The Kubernetes cluster to communicate with.

user

The credentials used for authentication.

namespace (optional)

The default namespace for `kubectl` commands.

The active context determines which cluster and user `kubectl` interacts with when executing each command.

Understanding and effectively using `kubectl` commands is essential for managing Kubernetes clusters. A detailed explanation follows of each command, including practical examples to demonstrate their use in real-world scenarios.

Starting a Single Pod: `kubectl run`

The `kubectl run` command is a quick way to create and run a single-instance pod. It is often used for testing or debugging. For example, to run an Nginx pod:

```
$ kubectl run nginx-pod --image=nginx
pod/nginx created
```

To specify additional options, such as environment variables, resource limits, or commands to execute inside the container, enter:

```
$ kubectl run busybox --image=busybox --command -- sleep 3600
pod/busybox created
```

Retrieving Resource Information: `kubectl get`

The `kubectl get` command retrieves basic information about Kubernetes resources in a cluster. It returns concise status details such as name, readiness, age, and current state. This command is typically used for monitoring or scripting.

To get a list of nodes, enter:

```
$ kubectl get nodes
NAME          STATUS    ROLES          AGE    VERSION
minikube      Ready    control-plane  145m   v1.32.0
```

The output in the previous example shows the format used by default to display different fields, known as *tabular format*. Other formats, such as JSON, are also available. These are useful for feeding output to other tools that process the formats.

To list all pods in the default namespace, enter:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
busybox	1/1	Running	0	31m
nginx-pod	1/1	Running	0	9s

To display additional details, such as pod IP addresses and node assignments, use the `-o wide` option:

```
$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
busybox	1/1	Running	0	47s	10.244.0.4	minikube	<none>	<none>
nginx-pod	1/1	Running	0	68s	10.244.0.3	minikube	<none>	<none>

To retrieve resources from a specific namespace, enter:

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-668d6bf9bc-stbck	1/1	Running	0	146m
etcd-minikube	1/1	Running	0	146m
kube-apiserver-minikube	1/1	Running	0	146m
kube-controller-manager-minikube	1/1	Running	0	146m
kube-proxy-n4tn5	1/1	Running	0	146m
kube-scheduler-minikube	1/1	Running	0	146m
storage-provisioner	1/1	Running	1 (145m ago)	146m

The `kube-system` namespace hosts critical system components and control plane pods managed by Kubernetes, such as the `kube-apiserver`, `kube-controller-manager`, `kube-scheduler`, `etcd`, and `coredns`.

To see all namespaces, use the `--all-namespaces` options:

```
$ kubectl get pods --all-namespaces
```

To list a single replication controller with a specified name, enter:

```
$ kubectl get replicationcontroller <replication-controller-name>
```

To list all deployments, enter:

```
$ kubectl get deployments --all-namespaces
```

NAMESPACE	NAME	READY	UP-TO-DATE	AVAILABLE	AGE
kube-system	coredns	1/1	1	1	15m

To list a single pod in JSON output format:

```
$ kubectl get pod nginx-pod -o json
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "creationTimestamp": "2025-10-23T15:07:07Z",
    "generation": 1,
    "labels": {
      "run": "nginx-pod"
    },
    "name": "nginx-pod",
    "namespace": "default",
    (...)
  }
```

To list all replication controllers and services together in tabular format, enter:

```
$ kubectl get rc,services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	151m

Detailed Resource Inspection: `kubectl describe`

The `kubectl describe` command provides a detailed view of a specific resource, including its configuration, status, and events. This is particularly useful for troubleshooting. For instance, to inspect a pod named `nginx-pod`, enter:

```
$ kubectl describe pod nginx-pod
```

The output includes information such as container specifications, the assigned node, conditions, and recent events. Other resources, such as services or nodes, can also be described:

```
$ kubectl describe service nginx-service
$ kubectl describe node minikube
```

Declarative Resource Management: `kubectl apply`

The `kubectl apply` command creates or updates resources declaratively by applying a YAML or JSON manifest file. This is a key feature of GitOps workflows.

For example, to deploy an application using a manifest file named `deployment.yaml`, enter:

```
$ kubectl apply -f deployment.yaml
```

The previous command makes any necessary changes to bring the cluster into conformance with in a configuration file named `deployment.yaml`, written in YAML. An example of a configuration file for running Nginx follows:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

If the resource already exists, `apply` updates it to match the desired state defined in the manifest.

The use of configuration files allows administrators to define the desired state of a resource and let Kubernetes reconcile the actual state to match it. This is the preferred method for managing

resources in production environments because it supports idempotency, version control, and GitOps workflows. If the resource already exists, Kubernetes performs a three-way merge and applies only the changes, preserving unmanaged fields.

To delete resources defined in the manifest, use the `--prune` option or manually delete them:

```
$ kubectl delete -f deployment.yaml
```

Imperative Resource Creation: `kubectl create`

The `kubectl create` command creates resources imperatively, either directly via the command line or from a manifest file. For example, to create a new namespace called `test-namespace`, enter:

```
$ kubectl create namespace test-namespace
```

The `kubectl create` command can be used to explicitly create a new resource, such as a deployment:

```
$ kubectl create deployment nginx --image=nginx
```

To create resources from a manifest file, enter:

```
$ kubectl create -f deployment.yaml
```

Unlike `apply`, the `create` command does not update existing resources. If the resource already exists, the command fails — unless you use the `--dry-run=client` option.

This option simulates the creation process locally: it validates and shows what would happen, without actually sending any change to the cluster.

Resources can be modified in several ways using `kubectl`.

To an updated manifest (a declarative update), enter:

```
$ kubectl apply -f updated-deployment.yaml
```

To “patch” specific fields using the imperative form, enter:

```
$ kubectl patch deployment nginx --type=merge -p '{"spec":{"replicas":4}}'
```

Resources can be deleted using either their name:

```
$ kubectl delete pod nginx-pod
```

or a manifest file:

```
$ kubectl delete -f nginx-deployment.yaml
```

Creating a Service: `kubectl expose`

The `kubectl expose` command creates a service to expose a deployment, replica set, or pod to external traffic. For example, to expose the web HTTP port of a pod named `nginx-pod` on port 8080, enter:

```
$ kubectl expose pod nginx-pod --port=80 --target-port=8080 --name nginx-service
service/nginx-service exposed
```

This command creates a service that maps the pod `nginx` 80 port to an 8080 port, making it accessible externally.

To verify the service details using `kubectl get service`:

```
$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	155m
nginx-service	ClusterIP	10.101.41.66	<none>	80/TCP	53s

Changing Replica Count: `kubectl scale`

The `kubectl scale` command adjusts the number of replicas in a deployment, replica set, or stateful set. For example, to scale a deployment named `nginx-deployment` to five replicas, enter:

```
$ kubectl scale pod nginx-deployment --replicas=5
```


This command dynamically increases or decreases the number of running pods, enabling horizontal scaling based on workload demands.

Updating Resource Fields: `kubectl set`

The `kubectl set` command modifies specific attributes of a resource without editing the entire manifest. For example, to update the image of a container in a deployment, enter:

```
$ kubectl set image deployment/nginx-deployment nginx=nginx:1.21
```

This command updates the container image to version 1.21 without requiring a full redeployment. `kubectl set` can also configure environment variables, resource limits, or other properties.

Editing Live Resource Definitions: `kubectl edit`

The `kubectl edit` command opens the live configuration of a resource in the default text editor, allowing to make changes interactively. For example, to edit a deployment named `nginx-deployment`, enter:

```
$ kubectl edit deployment nginx-deployment
```

After the changes are saved, Kubernetes applies the updated configuration immediately. This is useful for making quick adjustments without the need for a separate manifest file.

Viewing Resource Schema: `kubectl explain`

The `kubectl explain` command provides detailed documentation about the structure and fields of a Kubernetes resource. For example, to view the schema of a deployment, enter:

```
$ kubectl explain deployment
```

To drill down into specific fields to understand their purposes and valid values, enter something such as:

```
$ kubectl explain deployment.spec.template.spec.containers
```

This command is invaluable for understanding the required and optional fields when writing manifests.

Managing the kubeconfig File: `kubectl config`

The `kubectl config` command manages kubeconfig files, which store cluster, user, and context information. For example, to view the current configuration, enter:

```
$ kubectl config view
```

To get the current context, enter:

```
$ kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	minikube	minikube	minikube	default

To change the context:

```
$ kubectl config use-context <context-name>
$ kubectl config set-context --current --namespace=<namespace>
```

This command is essential for working with multiple clusters or namespaces.

Viewing Pod Logs: `kubectl logs`

The `kubectl logs` command retrieves logs from a specific pod or container. For example, to view the logs of a pod named `nginx-pod`, enter:

```
$ kubectl logs nginx-pod
```

It is also possible to stream logs in real-time using the `-f` option:

```
$ kubectl logs nginx-pod -f
```

Running Commands Inside a Container: `kubectl exec`

The `kubectl exec` command executes commands inside a running container. For example, to start an interactive shell in a pod named `busybox`, enter:

```
$ kubectl exec -it busybox -- /bin/sh
```

```
/ # exit
```

To run non-interactive commands, such as listing files in a directory, omit the `-it` options:

```
$ kubectl exec busybox -- ls /etc
group
hostname
passwd
shadow
```

This command is widely used for debugging and interacting with the applications running in containers.

Configuring `kubectl` to Use an Existing Kubernetes Cluster

To configure `kubectl` to use an existing Kubernetes cluster, the cluster needs a `kubeconfig` file. Once the `kubeconfig` file has been created, follow these steps to configure `kubectl`.

Place the `kubeconfig` File in the Default Location

By default, `kubectl` looks for the `kubeconfig` file at `~/.kube/config`. If the `kubeconfig` file is named differently or located elsewhere, either rename and move it to the default location, or specify its path explicitly.

```
$ mkdir -p ~/.kube
$ cp /path/to/your/kubeconfig ~/.kube/config
```

Verify the Configuration

After setting up the `kubeconfig` file, verify that `kubectl` is correctly configured:

```
$ kubectl config view
```

This command displays the current `kubeconfig` settings, including clusters, users, and contexts.

Test Connectivity to the Cluster

To ensure that `kubectl` is properly connected to the cluster, run the following commands:

```
$ kubectl cluster-info
$ kubectl get nodes
```

The output should display the API server endpoint and other cluster details.

Kubernetes Operators

A *Kubernetes operator* is a method of packaging, deploying, and managing a Kubernetes application using custom controllers. An operator extends the Kubernetes API by defining a *custom resource definition* (CRD) and implementing logic to manage the full lifecycle of complex, stateful applications.

While standard Kubernetes resources (like deployments and stateful sets) handle stateless applications well, operators automate tasks such as installation, upgrades, backups, scaling, and failure recovery of stateful or complex applications (e.g., databases, message brokers, etc.).

Operators are essentially controllers written in Go (or other languages using client libraries) that watch for changes to custom resources and act accordingly. They encapsulate domain-specific knowledge of how to operate a particular application.

For example, a PostgreSQL Operator might:

- Deploy a database cluster.
- Monitor the cluster's health and automatically replace failed nodes.
- Perform automated backups and restores.
- Handle version upgrades with zero downtime.

Operators bring automation, consistency, and reliability to managing complex applications in Kubernetes environments. They align with the Kubernetes philosophy of declarative configuration and reconcile loops.

Real-world examples include:

Prometheus operator

Manages Prometheus monitoring stacks.

Elastic Cloud on Kubernetes (ECK) operator

Automates the Elasticsearch lifecycle.

MariaDB/Galera operator

Manages highly available database clusters.

Guided Exercises

1. Using `kubectl run`, create a simple Nginx pod called `nginx-pod` and a temporary pod named `busybox` that sleeps for one hour.

2. List all pods running in the default namespace, showing detailed node and IP info.

3. Describe the `nginx-pod` to see detailed configuration and events.

4. Stream logs from the `nginx-pod` pod in real time.

5. Execute an interactive shell inside the `busybox` pod.

Explorational Exercises

1. An Ingress is an API object that defines rules allowing external access to services in a cluster. Enable the Ingress controller in Minikube using the `minikube addons enable ingress` command.

Now deploy a “Hello World” app using the `gcr.io/google-samples/hello-app:1.0` image in a deployment called `web`. After that, expose the deployment using the `NodePort` type to port 8080. Use the command `minikube service web --url` to access the app using `curl`.

Summary

This lesson provided a comprehensive introduction to managing Kubernetes clusters using `kubectl`, the command-line interface for interacting with the Kubernetes API server. The lesson showed how to configure access to a cluster using the `kubeconfig` file and how to switch between multiple contexts for different environments. Through practical examples, the lesson provided guidance in creating, inspecting, and managing Kubernetes resources such as pods, deployments, and services. It explored both imperative commands like `kubectl run` and `kubectl create`, and declarative management using `kubectl apply` with YAML manifests.

Answers to Guided Exercises

- Using `kubectl run`, create a simple Nginx pod called `nginx-pod` and a temporary pod named `busybox` that sleeps for one hour.

```
$ kubectl run nginx-pod --image=nginx
$ kubectl run busybox --image=busybox --command -- sleep 3600
```

The first command creates a pod named `nginx-pod` with the `nginx` image, and the second runs a BusyBox container in sleep mode, useful for testing.

- List all pods running in the default namespace, showing detailed node and IP info.

```
$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
busybox	1/1	Running	0	97s	10.244.0.21	minikube	<none>
nginx-pod	1/1	Running	0	2m29s	10.244.0.20	minikube	<none>

The output displays the pod's IP address, node, and scheduling information.

- Describe the `nginx-pod` to see detailed configuration and events.

```
$ kubectl describe pod nginx-pod
```

The output displays container specifications, conditions, and recent events.

- Stream logs from the `nginx-pod` pod in real time.

```
$ kubectl logs -f nginx-pod
```

This command performs live monitoring on the container logs, which is useful for debugging.

- Execute an interactive shell inside the `busybox` pod.

```
$ kubectl exec -it busybox -- /bin/sh
/ # ls
```

```
bin    dev    etc    home   lib    lib64  proc   root   sys    tmp    usr    var
/ # exit
```

Answers to Explorational Exercises

1. An Ingress is an API object that defines rules allowing external access to services in a cluster. Enable the Ingress controller in Minikube using the `minikube addons enable ingress` command.

Now deploy a “Hello World” app using the `gcr.io/google-samples/hello-app:1.0` image in a deployment called `web`. After that, expose the deployment using the `NodePort` type to port 8080. Use the command `minikube service web --url` to access the app using `curl`.

To activate the Nginx Ingress controller in your cluster, execute the following command:

```
$ minikube addons enable ingress
```

Verify that the Nginx Ingress controller is running by checking the pods:

```
$ kubectl get pods -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS	AGE
ingress-nginx-admission-create-swgkr	0/1	Completed	0	11m
ingress-nginx-admission-patch-pvltk	0/1	Completed	0	11m
ingress-nginx-controller-56d7c84fd4-qlzlb	1/1	Running	0	11m

To create a deployment named `web` using the `gcr.io/google-samples/hello-app:1.0` image, run the following command:

```
$ kubectl create deployment web --image=gcr.io/google-samples/hello-app:1.0
deployment.apps/web created
```

Verify that the deployment is in a `READY` state:

```
$ *kubectl get deployment web *
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
web	1/1	1	1	12m

Now expose the `web` deployment using `--type=NodePort` to port 8080:

```
$ kubectl expose deployment web --type=NodePort --port=8080
service/web exposed
```

Verify if the `web` service is exposed:

```
$ kubectl get service web
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
web	NodePort	10.105.62.222	<none>	8080:32621/TCP	15m

Use the `minikube service` command to open the service through its NodePort:

```
$ minikube service web --url
```

```
http://192.168.49.2:32621
```

Navigate to the URL displayed in the previous output to test the application using `curl`:

```
$ curl http://192.168.49.2:32621
```

```
Hello, world!
```

```
Version: 1.0.0
```

```
Hostname: web-75995f7dbf-pbxhf
```



703.2 Basic Kubernetes Operations

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 703.2

Weight

7

Key knowledge areas

- Understanding the use of YAML files to declare Kubernetes resources
- Understanding the principle of a Pod
- Understanding how to use Deployments, including scaling and rolling updates
- Understanding how to make services accessible using Services and Ingress
- Understanding how to use storage using PersistentVolumeClaims
- Awareness of other Kubernetes orchestration resources, i.e. DaemonSets, StatefulSets, Jobs and CronJobs

Partial list of the used files, terms and utilities

- Pods
- ReplicaSets
- Deployments
- Services
- Ingress
- PersistentVolumeClaims
- ConfigMaps
- Secrets



Linux
Professional
Institute

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	703 Kubernetes
Objective:	703.2 Basic Kubernetes Operations
Lesson:	1 of 3

Introduction

Kubernetes has emerged as the de facto standard for container orchestration, enabling developers and operations teams to deploy, manage, and scale applications with unprecedented efficiency. At its core, Kubernetes operates on a declarative model, where resources such as pods, deployments, and services are defined using files in YAML format. These files provide a human-readable and version-controlled way to specify the desired state of applications and infrastructure.

Pods

Central to Kubernetes's architecture is the concept of a *pod*, which acts as the basic execution unit in the ecosystem. A pod represents a single application instance in a cluster, and may include one or more tightly coupled containers that share the same network namespace, storage, and lifecycle.

A *container* is a lightweight, portable, and self-sufficient software unit that packages an application along with its dependencies, libraries, and configuration files.

Containers within a pod share the same network namespace, storage, and lifecycle, making them ideal for tightly coupled applications (Main elements of a pod).

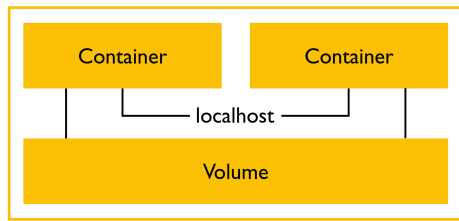


Figure 6. Main elements of a pod

While pods are ephemeral by nature (they can be created, destroyed, or replaced dynamically), they form the foundation for higher-level abstractions such as deployments, ReplicaSets, and StatefulSets.

Containers inside the same Pod share:

Network namespace

All containers in a pod share the same IP address and ports. That means they can talk to each other over the *localhost* host name.

Volumes (shared storage)

Containers in a pod can use the same volume. This is useful for exchanging files or keeping shared logs.

Interprocess communication (IPC)

Containers can communicate with each other in the pod using signals and shared memory. This simplifies process communication.

PID namespace

In some configurations, containers can see and interact with each other's process identifiers.

Even though each container runs its own binary, sharing these resources helps the binaries collaborate closely, especially when they are part of the same application.

Beyond the foundational concept of pods, Kubernetes introduces a variety of abstractions that streamline application deployment, scalability, and networking.

Kubernetes provides a rich set of tools and resources to address diverse application requirements, from exposing services to managing storage and scheduling tasks.

Deployments: Pod and ReplicaSet Management

A *deployment* is a higher-level abstraction that manages a group of pods through a ReplicaSet. Deployments allow declarative updates for pods and ReplicaSets, meaning that the administrator or developers describe the desired state (e.g., “I want three replicas of this pod”), and Kubernetes continuously works to maintain that state.

One of the key advantages of deployments in Kubernetes is support for *rolling updates*, which allow developers to release new versions of applications without any downtime. During a rolling update, Kubernetes gradually replaces old pods with new ones, ensuring that the application remains available throughout the process.

If something goes wrong during an update, deployments also support *rollbacks*. This means that developers can quickly revert to a previous, stable version of the application with a simple command, minimizing risk and recovery time.

In addition, deployments allow applications to scale up or down easily by adjusting the number of replicas. To handle a sudden spike in traffic or, in contrast, to reduce resource use during periods of low demand, scaling can be done dynamically and without interrupting service.

Deployments include *self-healing capabilities*. If a pod crashes or is accidentally deleted, Kubernetes automatically detects the issue and creates a new pod to maintain the desired state, keeping the application resilient and available.

Deployments are ideal for managing stateless applications in production, such as web servers, APIs, and front-end apps.

Using YAML Files to Declare Kubernetes Resources

Kubernetes resources—pods, deployments, services, ConfigMaps, and more—are defined using YAML files. This practice enables declarative configuration, which is a cornerstone of Infrastructure as Code (IaC).

A typical Kubernetes YAML resource contains four fields at the top level (described in the following sections). The `metadata` and `spec` fields contain most of the resource’s specifications, for example:

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
  namespace: production
```



```
labels:
  app: myapp
  tier: backend
annotations:
  prometheus.io/scrape: "true"
  prometheus.io/port: "9090"
  description: "This pod runs the backend service for myapp and is monitored by Prometheus"
spec:
  containers:
    - name: myapp-container
      image: myregistry/myapp:latest
      ports:
        - containerPort: 9090
```

This file is usually called a *manifest* in Kubernetes. It defines the desired state of one or more cluster resources. Each manifest describes how a resource should behave—for example, which containers a pod should run, how many replicas a deployment should maintain, or which configuration data a ConfigMap or secret should contain.

A manifest acts as a declarative specification. Instead of issuing imperative commands, the manifest expresses what the system should look like, and the Kubernetes control plane ensures that the actual cluster state matches this desired state.

apiVersion: Which Version of the Kubernetes API to Use

Every Kubernetes resource is handled by a specific version of the API. The `apiVersion` field tells Kubernetes which version of the API should be used to interpret the resource that the file is declaring. Example values are:

v1

for basic resources such as pods, services, and ConfigMaps.

apps/v1

for more complex resources such as deployments and StatefulSets.

API versions change as Kubernetes evolves. Using the correct `apiVersion` ensures compatibility with the cluster.

kind: What Type of Resource is being Declared

The `kind` field specifies the type of Kubernetes object the file is defining—in other words, what it

wants created. Common values (note the initial capital letters) include:

Pod

A basic unit that runs one or more containers.

Deployment

Manages pods and ReplicaSets for rolling updates.

Service

Exposes pods to the network.

ConfigMap

Stores configuration data.

metadata: Identifying and Organizing the Resource

The `metadata` section contains identifiers and organizational information about the resource being defined in the file. Kubernetes consults the `metadata` section to track and manage resources across the cluster. Common fields include:

name

A name for the pod, which must be unique within its namespace.

namespace

Scope for the pod, putting it in a specific logical grouping in the cluster. Namespaces allow resource isolation and management.

labels

Key-value pairs used for selecting and organizing resources. Controllers such as deployments, services, and NetworkPolicies rely on labels to target specific pods.

annotations

A powerful and flexible way to attach non-identifying metadata to pods, deployments, services, Ingresses, and more. Annotations are defined as key-value pairs under the `metadata.annotations` field and can hold arbitrary structured or unstructured data for use by tools, controllers, and external systems. Annotations are not indexed by the Kubernetes API server, so there is no constraint on storing large or complex values, such as full JSON blobs, URLs, or configuration data.

An example `metadata` section follows:

```
metadata:
  name: my-app
  namespace: production
  labels:
    app: frontend
    tier: web
```

spec: Desired State of the Resource

The `spec` field (short for “specification”) defines the configuration and behavior to be enforced by Kubernetes. Its structure varies depending on the resource type. For example, the field for a pod could specify:

```
spec:
  containers:
    - name: my-container
      image: nginx
```

This configuration tells Kubernetes how to create, run, and manage an Nginx container.

How to Use Deployments in Kubernetes

A deployment is one of the most common resources in Kubernetes. It provides a declarative way to manage pods and ReplicaSets. A deployment defines the desired state of the application, including how many instances (or replicas) should be running. Kubernetes ensures that this state is maintained continuously. If any pod fails or is deleted, the deployment automatically replaces it. This makes deployments a reliable and scalable way to run containerized applications in production environments.

Defining a Deployment

To create a deployment, write a YAML file that declares the resource. For example, the following file with the name `deployment.yaml` runs three replicas of an Nginx container:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
```

```

selector:
  matchLabels:
    app: nginx
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80

```

Under `spec`, the file defines the number of replicas, the `selector` that matches the pods that the deployment manages, and the `template` that describes how the pods should be created—including the container `image`, `name`, and `ports`.

This YAML file can be used to start a cluster through the following command:

```

$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created

```

The command causes Kubernetes to create the appropriate `ReplicaSet` and pods according to the specification. View the `ReplicaSet` by entering:

```

$ kubectl get replicaset

```

NAME	DESIRED	CURRENT	READY	AGE
nginx-deployment-55d67f7b54	3	3	3	16m

Scaling a Deployment

Scaling a deployment means increasing or decreasing the number of replicas running in the cluster. This can be done manually by editing the `replicas` field in the YAML file and reapplying the file.

Alternatively, scaling can be accomplished using the command line with a `kubectl scale deployment` command, specifying the new number of replicas:

```

$ kubectl scale deployment nginx-deployment --replicas=5

```

```
deployment.apps/nginx-deployment scaled
```

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-55d67f7b54-5qf78	1/1	Running	0	6s
nginx-deployment-55d67f7b54-5tm82	1/1	Running	0	6s
nginx-deployment-55d67f7b54-kbpb6	1/1	Running	0	3m10s
nginx-deployment-55d67f7b54-qbsv6	1/1	Running	0	3m10s
nginx-deployment-55d67f7b54-stjdh	1/1	Running	0	3m10s

The output shows the “old replicas” that were created three minutes ago, and the two “new” replicas that were created just six seconds ago.

When you scale up, Kubernetes adds new pods; when you scale down, it removes the excess pods to match the desired replica count.

Performing Rolling Updates

One of the most powerful features of deployments is *rolling updates*. This allows you to update the application—for example, to use a new container image—with zero downtime. When you run a command such as `kubectl set image` to update the image version, Kubernetes gradually replaces the “old” Pods with new ones:

```
$ kubectl set image deployment nginx-deployment nginx=nginx:1.26
deployment.apps/nginx-deployment image updated
```

This causes Kubernetes to create new pods with the updated image, and gradually replaces old pods. Kubernetes ensures that some pods always remain available during the process.

The behavior of rolling updates can also be customized using a `strategy` field in the YAML configuration. This field can define, for example, how many pods may be unavailable during the update (`maxUnavailable`) and how many extra pods can be temporarily created (`maxSurge`). These settings help balance availability with the speed of the rollout:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

Rollbacks and Version History

If something goes wrong during an update, an administrator can roll the cluster back to the previous working version using the `kubectl rollout undo deployment nginx-deployment` command:

```
$ kubectl rollout undo deployment nginx-deployment
deployment.apps/nginx-deployment rolled back
```

It is also possible to view the history of previous rollouts to understand what changes were made:

```
$ kubectl rollout history deployment nginx-deployment
deployment.apps/nginx-deployment
REVISION  CHANGE-CAUSE
2          <none>
3          <none>
```

ReplicaSets

A *ReplicaSet* ensures that a specific number of identical pods are running at all times. If any of the pods stop working or are deleted, the ReplicaSet automatically creates new ones to maintain the desired state.

The following ReplicaSet ensures that three Nginx pods are always running. If one pod crashes or is deleted, the ReplicaSet creates a new one automatically:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend-replicaset
  labels:
    app: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
```

```
containers:
  - name: nginx
    image: nginx:1.25
    ports:
      - containerPort: 80
```

To view existing ReplicaSets, enter:

```
$ kubectl get replicasets
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-deployment-55d67f7b54	5	5	5	16m
nginx-deployment-65dd7588fc	0	0	0	8m21s

In practice, deployments are more commonly used than ReplicaSets because deployments offer advanced features such as rolling updates and rollbacks.

Managing Application Configuration With ConfigMaps and Secrets in Kubernetes

Kubernetes encourages applications to be configured separately from container images. Hardcoding configuration values—such as environment-specific settings or sensitive credentials—into containers is highly discouraged because it makes them inflexible and insecure. To avoid this hardcoding, Kubernetes provides two key resources: ConfigMap and secret.

ConfigMaps

A *ConfigMap* is a key-value store that injects configuration information into applications. The information can come from files, literal string values, or YAML definitions.

The following file with the name `configmap.yaml` creates a ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  index.html: |
    <html>
    <head><title>Hello from ConfigMap</title></head>
    <body>
      <h1>This HTML page was loaded from a ConfigMap!</h1>
```

```
</body>
</html>
```

The idea here is to mount the HTML content as a file into the Nginx web root inside the container, replacing the default index page.

Create the ConfigMap from the file with the command:

```
$ kubectl apply -f configmap.yaml
configmap/nginx-config created
```

One way to use this ConfigMap in an Nginx pod is to refer to it in an environment variable. The following configuration file is named `my-pod.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-with-configmap
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      volumeMounts:
        - name: html-volume
          mountPath: /usr/share/nginx/html/index.html
          subPath: index.html
  volumes:
    - name: html-volume
      configMap:
        name: nginx-config
```

Enter the following command to mount the `index.html` file from the `nginx-config` ConfigMap directly into the container at the exact location where Nginx expects it (`/usr/share/nginx/html/index.html`):

```
$ kubectl apply -f my-pod.yaml
pod/nginx-with-configmap created
```

Access the Nginx server (e.g., via `kubectl port-forward` or a service) to see the custom HTML page from the ConfigMap, instead of the default Nginx welcome page. The following command

exposes the pod temporarily:

```
$ kubectl port-forward pod/nginx-with-configmap 8080:80 &
```

Now open a browser and visit `http://localhost:8080`, or use `curl` to retrieve the web page. The custom HTML page served from the ConfigMap should be displayed:

```
$ curl http://localhost:8080
Handling connection for 8080
<html>
<head><title>Hello from ConfigMap</title></head>
<body>
  <h1>This HTML page was loaded from a ConfigMap!</h1>
</body>
</html>
```

Secrets

Secrets typically store credentials needed by the system to interact with external services on behalf of the user. The secrets can be mounted into a pod as data volumes or injected as environment variables, making them accessible to containers during runtime. Additionally, secrets can be used internally by other components of the Kubernetes system without being directly exposed to the pod.

Secrets store sensitive data in a Base64-encoded format. This encoding exists only to ensure safe textual representation within YAML manifests and does not provide any real security benefit. Anyone with access to the encoded data can easily decode it. To properly secure Secrets, enable encryption at rest (for example, using a cloud-based KMS solution) and ensure they are transmitted only over secure channels such as TLS.

To encode values using Base64, enter commands such as:

```
$ echo -n 'yourvalue' | base64
eW91cnZhbHVl
```

The following file with the name `secret.yaml` creates a secret:

```
apiVersion: v1
kind: Secret
metadata:
```

```

name: mariadb-secret
type: Opaque
data:
  username: bWFyaWFkYg==          # base64 for "mariadb"
  password: c2VjdXJlcGFzcw==     # base64 for "securepass"
  root-password: cm9vdHBhc3MxMjM= # base64 for "rootpass123"

```

Apply the file to create the secret with:

```

$ kubectl apply -f secret.yaml
secret/mariadb-secret created

```

Alternative, create the secret directly via the command line:

```

$ kubectl create secret generic mariadb-secret \
  --from-literal=username=mariadb \
  --from-literal=password=securepass

```

Now, use the secret in the definition of a pod to inject the values as environment variables into a MariaDB container:

```

apiVersion: v1
kind: Pod
metadata:
  name: mariadb-pod
spec:
  containers:
    - name: mariadb
      image: mariadb:11.3
      ports:
        - containerPort: 3306
      env:
        - name: MARIADB_USER
          valueFrom:
            secretKeyRef:
              name: mariadb-secret
              key: username
        - name: MARIADB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mariadb-secret

```

```
        key: password
-   name: MARIADB_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mariadb-secret
        key: root-password
```

This setup injects all three environment variables (MARIADB_USER, MARIADB_PASSWORD, and MARIADB_ROOT_PASSWORD) into the MariaDB container from the secret named mariadb-secret. If the previous configuration file is named `maria.yaml`, the pod can be created using:

```
$ kubectl apply -f maria.yaml
pod/mariadb-pod created
```

Guided Exercises

1. Write a YAML file to create a pod named `my-pod` that runs a busybox container executing the command `sleep 3600`.
2. Create a deployment called `nginx-deployment` that runs two replicas of the `nginx:1.25` image.
3. You have a deployment named `nginx-deployment` running two replicas of the `nginx:1.25` image. Perform a rolling update to change the image version from `nginx:1.25` to `nginx:1.26`.
4. You have a deployment named `nginx-deployment` currently running version `nginx:1.26`. Assume that this new version introduced a problem and you need to roll back the deployment to the previous stable version (`nginx:1.25`). What command can you use?
5. Create a secret containing `APP_USER` and `APP_PASS`, then inject them into a pod as environment variables. How can you test results in the shell?

Explorational Exercises

1. Create a simple pod running a container with the `busybox` image. Configure the container to run a short-lived command such as `sleep 5`. Observe what happens after the container exits. Then change the `restartPolicy` from `Always` to `OnFailure` and then to `Never`, and compare the behavior in each case. What happens to the pod after the container finishes executing?

2. Configure a pod running the `nginx` image with a liveness probe on port 80 and a readiness probe that waits for five seconds before checking. What is the difference between `livenessProbe` and `readinessProbe`?

Summary

Kubernetes is a powerful container orchestration platform that enables developers and operations teams to define, deploy, scale, and manage containerized applications with consistency and efficiency. At its core, Kubernetes uses a declarative model driven by YAML files, which describe the desired state of resources such as pods, deployments, services, and secrets. By defining Infrastructure as Code, teams can version, audit, and reproduce their environments more reliably across development, staging, and production.

To build and operate real-world applications in Kubernetes, it is essential to understand not only the foundational unit—the pod—but also higher-level abstractions such as deployments and StatefulSets, which manage application lifecycles.

Answers to Guided Exercises

1. Write a YAML file to create a pod named `my-pod` that runs a `busybox` container executing the command `sleep 3600`.

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: busybox
      image: busybox
      command: ["sleep", "3600"]
```

To apply the file, enter:

```
$ kubectl apply -f my-pod.yaml
```

2. Create a deployment called `nginx-deployment` that runs two replicas of the `nginx:1.25` image.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
```

```
- containerPort: 80
```

To apply the file, enter:

```
$ kubectl apply -f nginx-deployment.yaml
```

3. You have a deployment named `nginx-deployment` running two replicas of the `nginx:1.25` image. Perform a rolling update to change the image version from `nginx:1.25` to `nginx:1.26`.

Perform the rolling update:

```
$ kubectl set image deployment/nginx-deployment nginx=nginx:1.26
```

Monitor the rollout status:

```
$ kubectl rollout status deployment/nginx-deployment
```

Verify the new image version:

```
$ kubectl get pods -l app=nginx -o wide
```

4. You have a deployment named `nginx-deployment` currently running version `nginx:1.26`. Assume that this new version introduced a problem and you need to roll back the deployment to the previous stable version (`nginx:1.25`). What command can you use?

To roll back to the previous version, enter:

```
$ kubectl rollout undo deployment/nginx-deployment
```

To monitor the rollback status, enter:

```
$ kubectl rollout status deployment/nginx-deployment
```

To verify the image version, enter:

```
$ kubectl get pods -l app=nginx -o wide
```


To view the rollout history, enter:

```
$ kubectl rollout history deployment/nginx-deployment
```

5. Create a secret containing APP_USER and APP_PASS, then inject them into a pod as environment variables. How can you test results in the shell?

The configuration file for the secret, named `secret.yaml`, follows:

```
apiVersion: v1
kind: Secret
metadata:
  name: app-credentials
type: Opaque
data:
  APP_USER: YWRtaW4=          # "admin"
  APP_PASS: c2VjdXJlcGFzcw== # "securepass"
```

The configuration file for the pod, named `secret-pod.yaml`, follows:

```
apiVersion: v1
kind: Pod
metadata:
  name: app-secret-pod
spec:
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "env && sleep 3600"]
      env:
        - name: APP_USER
          valueFrom:
            secretKeyRef:
              name: app-credentials
              key: APP_USER
        - name: APP_PASS
          valueFrom:
            secretKeyRef:
              name: app-credentials
              key: APP_PASS
```

To apply the configurations, enter:

```
$ kubectl apply -f secret.yaml
secret/app-credentials created

$ kubectl apply -f secret-pod.yaml
pod/app-secret-pod created
```

To check that the secret was incorporated into the pod, enter:

```
$ kubectl exec -it app-secret-pod -- printenv APP_USER APP_PASS
admin
securepass
```

Answers to Explorational Exercises

1. Create a simple pod running a container with the `busybox` image. Configure the container to run a short-lived command such as `sleep 5`. Observe what happens after the container exits. Then change the `restartPolicy` from `Always` to `OnFailure` and then to `Never`, and compare the behavior in each case. What happens to the pod after the container finishes executing?

Start by creating a simple pod with a `busybox` container that runs `sleep 5`, then exits:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
    - name: sleeper
      image: busybox
      command: ["sleep", "5"]
  restartPolicy: Always
```

Observation 1: `restartPolicy: Always`

After applying this configuration, you can observe that the container keeps restarting continuously, because the `sleep 5` command finishes quickly, and the pod is instructed to always restart the container regardless of whether it exited successfully or failed.

Observation 2: `restartPolicy: OnFailure`

After the YAML is changed to `restartPolicy: OnFailure` and the file is re-applied, when the container completes (which it does successfully, since `sleep 5` has exit code 0), the container does not restart. That's because this policy restarts only containers that fail (i.e., exit with non-zero status). The pod status becomes `Completed`.

Observation 3: `restartPolicy: Never`

After the YAML is changed to `restartPolicy: Never`, regardless of whether the container exits with success or failure, Kubernetes will not attempt to restart it. The pod simply finishes, and its status becomes either `Completed` or `Failed` depending on the container's exit code.

2. Configure a pod running the `nginx` image with a liveness probe on port 80 and a readiness probe that waits for five seconds before checking. What is the difference between `livenessProbe` and `readinessProbe`?

The configuration file for the pod, named `probes.yaml`, follows:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-probes
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 3
```

+ To apply the file, enter:

+

```
$ kubectl apply -f probes.yaml
pod/nginx-probes created
```

+ A *liveness probe* checks whether the application inside the container is alive and should continue to run. If the liveness probe fails, Kubernetes automatically restarts the container, assuming it is in a bad state (e.g., frozen, unresponsive, or stuck).

+ A *readiness probe* checks whether the application is ready to handle requests. If the readiness probe fails, Kubernetes removes the pod from the service load balancer until it becomes ready again — but won't restart the pod.



Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	703 Kubernetes
Objective:	703.2 Basic Kubernetes Operations
Lesson:	2 of 3

Introduction

Kubernetes is a powerful platform for orchestrating containers, but managing real-world applications involves more than just running pods. Systems in production need to expose applications to users, route traffic intelligently, and ensure that data persists even if containers are restarted or moved. This is where services, Ingresses, and PersistentVolumes come into play.

The Role of Services

When using a deployment to manage your application, Kubernetes automatically handles the creation and deletion of pods to match the desired state. These pods are ephemeral: they can be terminated and replaced at any time. Their number and identities (IP addresses, names) can change frequently, making it unreliable to connect to them directly.

Each pod receives its own unique IP address, and Kubernetes relies on network plugins to provide stable networking across the cluster. However, due to the dynamic behavior of pods, the set of pods running at one moment may be completely different the next.

This presents a challenge: if one group of pods (e.g., backends) provides services to another group

of pods (e.g., frontends), how can the frontends discover and consistently connect to the correct backend pods, whose IPs and identities are constantly changing?

This is exactly the problem that Kubernetes *services* solve. A service provides a stable network endpoint and automatically routes traffic to the appropriate set of backend pods — no matter how often those pods change — based on labels and selectors. It acts as a reliable gateway, abstracting pod lifecycle and network changes from the application logic.

Defining a Service in Kubernetes

In Kubernetes, a service is an API object, just like a pod, ConfigMap, or deployment. Developers and administrators can create, inspect, and update service definitions using the Kubernetes API. In practice, most users interact with services using tools such as `kubectl`, which make those API calls under the hood.

Imagine a group of pods that all listen on TCP port 3030 and share the label `app.kubernetes.io/name=MyApp`. A service can expose those pods and forward traffic to that specific port.

Here's a sample manifest `service/simple-service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3030
```

When you apply this manifest, Kubernetes creates a new service named `my-service` of type `ClusterIP` by default. This service will route incoming traffic on port 80 to the `targetPort` 3030 on any pod that has the label `app.kubernetes.io/name=MyApp`.

This setup allows clients within the cluster to reach your pods through a stable network endpoint, without needing to track individual pod IP addresses or lifecycle changes.

There are several types of services. The following sections describe `ClusterIP`, `NodePort`, `LoadBalancer`, and `ExternalName`.

ClusterIP

ClusterIP is the default Kubernetes service type, providing a stable internal IP address that exposes the service only within the cluster. Because this address is not accessible from outside the cluster, it is ideal for internal microservices communication.

To demonstrate ClusterIP in practice, start by creating a ClusterIP service that exposes an Nginx deployment within the cluster. Name the file `nginx-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

Apply the configuration as follows:

```
$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
```

Then create a ClusterIP service to expose the port internally. Name this file `clusterip.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
```

```

type: ClusterIP
selector:
  app: nginx
ports:
  - protocol: TCP
    port: 80
    targetPort: 80

```

Apply this manifest as follows:

```

$ kubectl apply -f clusterip.yaml
service/nginx-service created

```

The service `nginx-service` now will route traffic to all pods with the label `app: nginx`. Other pods in the cluster can reach the service using the name `nginx-service` on port 80. For example, a frontend pod can connect using:

```

$ kubectl run frontend-pod --rm -i -t --image=busybox --restart=Never -- sh
/ #

```

Inside the interactive shell, enter:

```

/ # wget -qO- http://nginx-service
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
(...)

```

NodePort

NodePort is a type of service that exposes an application on a specific port across all nodes in the cluster. When a service is defined as a **NodePort**, Kubernetes opens a static port on each node, allowing external traffic to reach the application using the node's IP address and the assigned port.

A **NodePort** service allows external traffic to access your application via a URL in the form

```
http://<NodeIP>:<NodePort>
```


Kubernetes opens the same port on all cluster nodes and routes traffic to the matching pods.

In the following example of NodePort, any request sent to port 30001 of a node will be forwarded to port 80 of the matching pods (based on the app: nginx selector). Name the file `nodeport.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80          # internal service port
      targetPort: 80    # container port in the pod
      nodePort: 30001   # external port exposed on the Node (must be between 30000-32767)
```

To apply this manifest, enter:

```
$ kubectl apply -f nodeport.yaml
service/nginx-nodeport created

$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	28h
nginx-nodeport	NodePort	10.104.66.104	<none>	80:30001/TCP	56s

This service selects pods with the label `app: nginx` (from `nginx-deployment`). Port 80 is exposed on the service and mapped to port 80 on the container. The configuration also opens NodePort 30080 on every node in the cluster.

At this point, the Nginx service is exposed outside the cluster. It can be accessed using the IP address of any node in the cluster, along with the port defined in the service configuration.

To find the IP address of a node, enter:

```
$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE

KERNEL-VERSION		CONTAINER-RUNTIME					
minikube	Ready	control-plane	28h	v1.32.0	192.168.49.2	<none>	Ubuntu
22.04.5 LTS	6.11.0-21-generic	docker://27.4.1					

The node has the IP address 192.168.49.2. Using this, retrieve the home page from Nginx:

```
$ curl http://192.168.49.2:30001
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
(...)
```

LoadBalancer

LoadBalancer is a Kubernetes service type designed to expose applications to external traffic by integrating with cloud-based load balancers (e.g., AWS Elastic Load Balancer, Google Cloud Load Balancer). When a service is configured as a **LoadBalancer**, Kubernetes automatically provisions an external load balancer and assigns it a stable IP address, routing incoming traffic to the appropriate pods within the cluster.

An example follows of manifest for a **LoadBalancer** named **nginx-deployment**. Name the file **load.yaml**:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

To apply the manifest, enter:

```
$ kubectl apply -f load.yaml
```

```
service/nginx-loadbalancer created
```

To find the external IP address, enter:

```
$ kubectl get svc nginx-loadbalancer
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-loadbalancer	LoadBalancer	10.99.130.204	35.239.87.15	80:31142/TCP	28s

To test the service using `curl`, enter:

```
$ curl http://35.239.87.15
```

When you create a service of type `LoadBalancer`, Kubernetes sends a request to the cloud provider (e.g., AWS, Google Cloud, Azure) to provision a public load balancer.

However, this works only if your cluster is running on a cloud platform that supports external load balancers and has been properly configured with cloud controller integration.

If you run the previous example in a local cluster (like Minikube, MicroK8s, or Kind), or in a bare-metal environment (your own server), there is no cloud provider to fulfill the request. As a result, Kubernetes can't assign a public IP address, and you'll see something like this:

```
$ kubectl get svc nginx-loadbalancer
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-loadbalancer	LoadBalancer	10.99.130.204	<pending>	80:31142/TCP	28s

The `<pending>` status under `EXTERNAL-IP` means: "I'm waiting for the cloud provider to assign a public IP address, but no one has responded to my request." The request never succeeds because there is no integrated cloud controller (like GCP's or AWS's load balancer manager) listening and acting on this service creation.

ExternalName

A service of type `ExternalName` does not route traffic to pods, but instead acts like a DNS alias inside the cluster. It allows clients to refer to an external hostname (outside the Kubernetes cluster) as if it were a native service. This is useful to access external services (such as a third-party API, legacy database, or hosted solution) using a Kubernetes-native DNS name.

When an `ExternalName` service is defined and created, Kubernetes creates a `CNAME` record in its

internal DNS registry. When other pods query `http://my-db.default.svc.cluster.local`, for example, they are actually being redirected to an external hostname like `database.example.com`.

There is no selector, no Pod definition, no endpoint — just a DNS redirect.

An example of an `ExternalName` service follows. Name the file `external.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: example-org
spec:
  type: ExternalName
  externalName: example.org
```

This will create a DNS entry named `example-org.default.svc.cluster.local` and direct its traffic to `example.org`.

Apply the manifest as follows:

```
$ kubectl apply -f external.yaml
service/example-org created
```

Inside any pod in the cluster, a client can now connect to the `example.org` service. Since the server at `example.org` does not recognize `example-org` as a valid domain, you can force `curl` to send the correct `Host` header using the `-H` option:

```
$ kubectl run test-client --rm -it --image=curlimages/curl --restart=Never -- sh
~ $ curl -H "Host: example.org" http://example-org
<!doctype html>
<html>
<head>
  <title>Example Domain</title>
```

Ingresses in Kubernetes

While services expose pods, they are not flexible when it comes to HTTP routing, host-based access, or TLS termination. That's where *Ingresses* come in.

An Ingress is a collection of rules that allow external access to services in the cluster, typically over HTTP or HTTPS. Ingress sits in front of multiple services and acts like a smart router.

Unlike `NodePort` and `LoadBalancer`, Ingress doesn't handle network traffic directly — it relies on an *Ingress controller*, such as Nginx or Traefik, to do the actual work.

An Ingress in Kubernetes exposes HTTP and HTTPS routes from outside the cluster to services within it, allowing access through well-defined and externally reachable URLs. It supports name-based virtual hosting, meaning that different services can be reached via different hostnames under the same IP address. Ingress also supports *path-based routing*, directing traffic to different backends based on the URL.

One of the key features of Ingress is its ability to automatically redirect HTTP traffic to HTTPS, enhancing security by enforcing encrypted connections. Custom TLS certificates can also be configured in order to secure communications, and many Ingress controllers support wildcard hostnames, enabling routing for dynamic subdomains like `*.example.com`.

In most Kubernetes clusters with an Ingress controller, the IP address can be shared among multiple services through name-based virtual hosting. Therefore, Ingress needs to be deployed along with an Ingress controller, such as Nginx, HAProxy or Traefik, which handles the routing.

An example of using Ingress to route to traffic to an Nginx service named `ingress.yaml` follows. Name the file `in.yaml`:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: myapp.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx-service
                port:
                  number: 80
```

This Ingress resource defines a rule that routes HTTP traffic based on the hostname and path. The `host: myapp.local` field specifies the hostname that this rule applies to. The Ingress matches only requests with this host in the HTTP Host header. The `backend` section defines the target service and port that should receive the traffic; in this case, traffic is forwarded to the `nginx-service` service on port 80.

To apply this manifest, enter:

```
$ kubectl apply -f in.yaml
ingress.networking.k8s.io/nginx-ingress created

$ kubectl get ingress
NAME          CLASS    HOSTS          ADDRESS          PORTS    AGE
nginx-ingress  nginx    myapp.local    192.168.49.2    80       3m
```

When a request like the following hits the cluster

```
GET / HTTP/1.1
Host: myapp.local
```

the Ingress does the following:

1. Matches the host to `myapp.local`.
2. Matches the path to `/`.
3. Rewrites the path to `/` (if needed).
4. Sends the request to `nginx-service:80`.

To test the service locally (using Minikube or Kind), add the following entry to your `/etc/hosts` file:

```
192.168.49.2 myapp.local
```

Now query the service using your browser or using `curl` in the terminal:

```
$ curl http://myapp.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

(...)

Guided Exercises

1. Create a deployment named `my-app-deployment` that runs three replicas of the `httpd:2.4` image, and use the `env=testing` label.

2. Create a service named `my-app-service` that exposes the `my-app-deployment` service on port 80 using the default service type `ClusterIP`. The service should route traffic to pods labeled with `env=testing`, forwarding to container port 80.

3. Create a service named `my-app-nodeport` of type `NodePort` that allows external access to the application. The service should expose port 80 to the client, forward traffic to container port 80, and use a `NodePort` in the range 30000-32767.

4. Assume that a service named `my-app-service` exists. Create an Ingress named `my-app-ingress` that exposes it externally. The Ingress should route HTTP traffic from the path `/` to the `my-app-service`. Use host `myapp.example.com` for the host.

Explorational Exercises

1. Explore how a Kubernetes service dynamically handles changes in the underlying pods and continues to route traffic reliably. Do the following:

- Create a deployment named `http-echo` using the image `hashicorp/http-echo`, with the argument `-text="Hello from Pod"`. Add the label `http-echo` to the deployment.

- Expose the deployment using a service named `echo-service` on port 5678.

- Send multiple HTTP requests to the service using `kubectl port-forward` or by running `curl` from another pod.

- Scale the deployment to four replicas and observe how traffic is distributed among the pods.

2. Delete one of the pods in the previous exercise manually and resend requests.

- Does the service continue to route requests evenly across the remaining pods?

- Is there any interruption in traffic when a pod is deleted?

- How does the service behave when pods are added or removed?

3. Understand the difference in accessibility between a `ClusterIP` service and a `NodePort` service. Do the following:

- Create a deployment named `web-deployment` with two replicas of the `nginx:alpine` image and label it `myapp=web`.

- Expose the application using two services: one named `web-clusterip` of type `ClusterIP`, and one named `web-nodeport` of type `NodePort` with port 30090.

- From within a pod inside the cluster, test access to both services using `wget`.

- From outside the cluster (your local machine or browser), try accessing both services using the IP address of a node.

Answer the following questions:

- Which service is accessible from outside the cluster and why?

- What is the difference (if any) in how the two services handle DNS resolution?

Summary

Kubernetes offers a robust framework for managing containerized applications, but going from deployment to production requires more than just launching pods. To make applications accessible, resilient, and persistent, Kubernetes introduces key components such as services, Ingresses, and PersistentVolumes.

Ingresses add an additional layer of intelligent routing for HTTP/HTTPS traffic, allowing users to expose multiple services through a single external IP address with host-based and path-based rules. Ingress Controllers, such as Nginx, handle the actual routing and support such features as TLS termination and request rewriting.

Answers to Guided Exercises

1. Create a deployment named `my-app-deployment` that runs three replicas of the `httpd:2.4` image, and use the `env=testing` label.

The following manifest is named `httpd.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
      env: testing
  template:
    metadata:
      labels:
        app: my-app
        env: testing
    spec:
      containers:
        - name: httpd
          image: httpd:2.4
          ports:
            - containerPort: 80
```

Apply the manifest by entering:

```
$ kubectl apply -f httpd.yaml
deployment.apps/my-app-deployment created
```

To see the running pods in this service, run `kubectl get pods` and filter output by the label `env=testing`:

```
$ kubectl get pods -l env=testing
```

NAME	READY	STATUS	RESTARTS	AGE
my-app-deployment-9b985546c-cz4vv	1/1	Running	0	81s
my-app-deployment-9b985546c-d4qjm	1/1	Running	0	81s

```
my-app-deployment-9b985546c-x9ksv    1/1    Running    0    81s
```

2. Create a service named `my-app-service` that exposes the `my-app-deployment` service on port 80 using the default service type `ClusterIP`. The service should route traffic to pods labeled with `env=testing`, forwarding to container port 80.

The following manifest is named `service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    env: testing
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

Apply the manifest by entering:

```
$ kubectl apply -f service.yaml
```

Check the service:

```
$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	13h
my-app-service	ClusterIP	10.100.111.61	<none>	80/TCP	5s

Alternatively, you can use the `kubectl` command to create the service directly, without relying on a configuration file:

```
$ kubectl expose deployment my-app-deployment --name=my-app-service --port=80 --target-port=80 --type=ClusterIP
```

3. Create a service named `my-app-nodeport` of type `NodePort` that allows external access to the

application. The service should expose port 80 to the client, forward traffic to container port 80, and use a `NodePort` in the range 30000-32767.

The following manifest is named `nodeport.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-nodeport
spec:
  type: NodePort
  selector:
    env: testing
  ports:
    - protocol: TCP
      port: 80          # Port exposed by the service
      targetPort: 80    # Container port in the Pod
      nodePort: 30080   # Custom NodePort within allowed range (30000-32767)
```

Apply the manifest by entering:

```
$ kubectl apply -f nodeport.yaml
```

Check the created service:

```
$ kubectl get svc my-app-nodeport
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-app-nodeport	NodePort	10.103.97.198	<none>	80:30080/TCP	19s

- Assume that a service named `my-app-service` exists. Create an Ingress named `my-app-ingress` that exposes it externally. The Ingress should route HTTP traffic from the path `/` to the `my-app-service`. Use host `myapp.example.com` for the host.

The following manifest is named `ingress.yaml`:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
```

```
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-app-service
            port:
              number: 80
```

Apply the manifest by entering:

```
$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/my-app-ingress created
```

Verify the Ingress:

```
$ kubectl get ingress my-app-ingress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
my-app-ingress	<none>	myapp.example.com		80	34s

Answers to Explorational Exercises

1. Explore how a Kubernetes service dynamically handles changes in the underlying pods and continues to route traffic reliably. Do the following:

- Create a deployment named `http-echo` using the image `hashicorp/http-echo`, with the argument `-text="Hello from Pod"`. Add the label `http-echo` to the deployment.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: http-echo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: http-echo
  template:
    metadata:
      labels:
        app: http-echo
    spec:
      containers:
        - name: http-echo
          image: hashicorp/http-echo
          args:
            - -text=hello world
          ports:
            - containerPort: 5678
```

- Expose the deployment using a service named `echo-service` on port 5678.

```
$ kubectl expose deployment http-echo --name=echo-service --port=5678 --target
-port=5678
```

- Send multiple HTTP requests to the service using `kubectl port-forward` or by running `curl` from another pod:

```
$ kubectl port-forward service/echo-service 8080:5678 &
[1] 1251938
Forwarding from 127.0.0.1:8080 -> 5678
```



```
Forwarding from [::1]:8080 -> 5678
```

Now you can use `curl` to test the service:

```
$ curl http://localhost:8080
Handling connection for 8080
hello world
```

- Scale the deployment to four replicas and observe how traffic is distributed among the pods.:

```
$ kubectl scale deployment http-echo --replicas=4
deployment.apps/http-echo scaled
```

Check the pods:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
http-echo-7f5b99dfd8-7s9g7	1/1	Running	0	26s
http-echo-7f5b99dfd8-n5ftn	1/1	Running	0	26s
http-echo-7f5b99dfd8-n5wrb	1/1	Running	0	26s
http-echo-7f5b99dfd8-p48wn	1/1	Running	0	11m

2. Delete one of the pods in the previous exercise manually and resend requests.

```
$ kubectl delete pod http-echo-7f5b99dfd8-7s9g7
```

- Does the service continue to route requests evenly across the remaining pods?

Yes, Kubernetes automatically routes requests to all matching and ready pods.

- Is there any interruption in traffic when a pod is deleted?

No, the service continues routing to the remaining pods without downtime.

- How does the service behave when pods are added or removed?

The service immediately updates its list of endpoints. New pods are included as soon as they're ready, and terminated pods are removed instantly from the routing list.

3. Understand the difference in accessibility between a `ClusterIP` service and a `NodePort` service. Do the following:

- Create a deployment named `web-deployment` with two replicas of the `nginx:alpine` image and label it `myapp=web`.

```
$ kubectl create deployment web-deployment --image=nginx:alpine --replicas=2
deployment.apps/web-deployment created
$ kubectl label deployment web-deployment myapp=web
deployment.apps/web-deployment labeled
```

- Expose the application using two services: one named `web-clusterip` of type `ClusterIP` and one named `web-nodeport` of type `NodePort` with port 30090.

The following exposes `web-clusterip`:

```
$ kubectl expose deployment web-deployment --name=web-clusterip --port=80 --target
-port=80 --type=ClusterIP
service/web-clusterip exposed
```

The following exposes `web-nodeport`:

```
$ kubectl expose deployment web-deployment --name=web-nodeport --port=80 --target
-port=80 --type=NodePort
service/web-nodeport exposed

$ kubectl patch service web-nodeport -p '{"spec": {"ports": [{"port": 80,
"targetPort": 80, "protocol": "TCP", "nodePort": 30090}]}}'
service/web-nodeport patched
```

- From within a pod inside the cluster, test access to both services using `wget`.

```
$ kubectl run test-pod --rm -it --image=busybox -- /bin/sh
If you don't see a command prompt, try pressing enter.
/ # wget -qO- http://web-clusterip
<!DOCTYPE html>
<html>
(...)

/ # wget -qO- http://web-nodeport
<!DOCTYPE html>
```

```
<html>
(...)
/ # exit
```

- From outside the cluster (your local machine or browser), try accessing both services using the IP address of a node.

First, get the node's IP address:

```
$ kubectl get node -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
minikube	Ready	control-plane	14h	v1.32.0	192.168.49.2	<none>			
Ubuntu 22.04.5 LTS		6.11.0-21-generic		docker://27.4.1					

Access it using the URL and port exposed through the `NodePort` service:

```
$ curl 192.168.49.2:30090
<!DOCTYPE html>
<html>
(....)
```

Access it using the local URL through the `ClusterIP` service:

```
$ curl http://localhost:80
curl: (7) Failed to connect to localhost port 80 after 0 ms: Couldn't connect to server
```

- Which service is accessible from outside the cluster and why?

`web-clusterip` is not accessible from outside the cluster — it is reachable only from inside.

`web-nodeport` is accessible externally via `<Node-IP>:30090`.

- What is the difference (if any) in how the two services handle DNS resolution?

There is no difference in how DNS resolution works between a `ClusterIP` and a `NodePort` service within the Kubernetes cluster. Both services are assigned a DNS name by Kubernetes in the format:

```
<service-name>.<namespace>.svc.cluster.local
```

Applications running inside the cluster can resolve and access both services using these names or simply `web-clusterip` and `web-nodeport`.



Lesson 3

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	703 Kubernetes
Objective:	703.2 Basic Kubernetes Operations
Lesson:	3 of 3

Introduction

Kubernetes is designed to treat pods as ephemeral: they can be created, destroyed, or rescheduled at any time. This behavior is ideal for stateless workloads, but unsuitable for applications that need to persist data, such as databases, file servers, or message brokers.

To solve the challenge of persisting data in an environment where pods are short-lived and frequently rescheduled, Kubernetes introduces a decoupled storage model. Rather than embedding storage details directly into application definitions, Kubernetes cleanly separates storage provisioning from storage requests, enabling greater flexibility, reusability, and scalability.

In this model, PersistentVolumes (PVs) and PersistentVolumeClaims (PVCs) play a central role. PVs represent actual storage resources available in the cluster while PVCs are requests made by applications for specific types and amounts of storage. This abstraction is essential for supporting stateful applications that require reliable and consistent storage.

Persistent Storage in Kubernetes

When a pod is deleted or crashes, all data stored inside the container is lost. This is fine for stateless apps like web servers, but it is a problem for apps that need to store data, like databases or file-based services.

To solve this, Kubernetes provides a way to connect pods to *persistent storage* that survives restarts, rescheduling, and even node failures. This is done using PersistentVolumes (PV) and PersistentVolumeClaims (PVC).

PersistentVolume (PV)

A *PersistentVolume* (PV) is a unit of storage in the cluster, provisioned by an administrator or dynamically by a *StorageClass*. PVs abstract the underlying storage infrastructure. A PV can represent storage from a variety of sources, such as local disks, NFS, or cloud-based volumes (e.g., AWS EBS, GCE Persistent Disks).

The PV exists independently of any pod lifecycle. It can be statically provisioned (that is, manually defined by administrators) or dynamically provisioned (automatically created by Kubernetes when a PVC requests storage).

PVs don't typically appear directly in pod definitions. Instead, pods use PVCs, which Kubernetes then matches with available PVs.

PersistentVolumeClaim (PVC)

A *PersistentVolumeClaim* (PVC) is a request for storage. It specifies how much space is needed and the *access mode* (e.g., `ReadWriteOnce`, `ReadOnlyMany`, `ReadWriteMany`). PVCs act as an abstraction layer between pods and PVs, making storage management more user-friendly.

When a PVC is created, Kubernetes looks for a PV that satisfies the request. Once bound to the PV, the pod can mount it and use it like a normal directory.

In Kubernetes, access modes in a *PersistentVolumeClaim* determine how the volume can be mounted by pods:

ReadWriteOnce

The volume can be mounted as read-write by only one node at a time. This is the most common and widely supported mode and is typically used when a single pod or a group of pods on the same node needs full read/write access to the storage.

ReadOnlyMany

The volume can be mounted by multiple nodes, but only in read-only mode. This is useful for scenarios where many pods need access to the same data but don't need to modify it—for example, when serving static website content or shared reference data.

ReadWriteMany

The most flexible mode. It permits the volume to be mounted as read-write by multiple nodes simultaneously. This is essential for distributed applications that need concurrent read/write access to the same data, such as shared logging, collaboration tools, or network file systems. However, support for ReadWriteMany depends on the underlying storage system—not all volume types provide it by default.

Lifecycle of a PersistentVolume and a PersistentVolumeClaim

In Kubernetes, PersistentVolumes (PVs) are storage resources available in the cluster, while PersistentVolumeClaims (PVCs) are requests for those resources. A PVC acts as both a request and a “claim check” for a specific volume. The interaction between PVs and PVCs follows a well-defined lifecycle composed of the following stages.

Provisioning

There are two ways to provision volumes in a Kubernetes cluster: *statically* or *dynamically*.

Static provisioning

A cluster administrator manually creates a set of PVs in advance. These volumes refer to real storage (e.g., NFS, cloud disks) and are registered in the Kubernetes API, ready to be claimed by users via PVCs.

Dynamic provisioning

If no existing static PV matches a user's PVC request (in terms of size, access mode, or other attributes), Kubernetes can automatically provision a volume. This is known as dynamic provisioning, and it relies on a StorageClass.

To use dynamic provisioning the PVC must specify a valid `storageClassName` and the cluster administrator must have previously created and configured a compatible StorageClass.

If a PVC specifies `storageClassName: ""`, it explicitly disables dynamic provisioning for that claim.

To enable dynamic provisioning, the cluster must enable the `DefaultStorageClass` admission controller. This is configured via the `--enable-admission-plugins` option in the `kube-`

apiserver component.

Binding

Once a user submits a PVC — either triggering dynamic provisioning or requesting a match with existing PVs — a control loop in the Kubernetes control plane attempts to bind the PVC to a suitable PV. The following sequence applies:

- If a PV was dynamically created for the PVC, they are bound automatically.
- If matching static PVs are available, Kubernetes chooses the best fit.
- Once bound, a PVC is exclusively attached to that PV through a `ClaimRef`, forming a one-to-one relationship.

The PVC will always receive at least what it requested, although the PV may exceed those specs (e.g., a 100G PV for a 50G claim). If no matching PV exists, the claim remains unbound until a compatible PV becomes available. For example, if all available PVs are 50G and a PVC requests 100G, it won't be bound until a 100G PV is added to the cluster.

If the volume supports multiple access modes (e.g., `ReadWriteOnce`, `ReadOnlyMany`), the desired mode must be specified in the pod configuration.

After successful binding, the volume remains dedicated to the user for as long as the claim exists. Users can create and schedule pods that access the claimed volume through the PVC.

Once a PVC is successfully bound to a PV, it can be used by pods as a volume through the following process:

- The pod references the PVC in its volumes section.
- Kubernetes automatically finds the associated PV and mounts it into the pod.

Managing Persistent Volumes and Claims in Kubernetes

A simple example can demonstrate how `PersistentVolumeClaims` (PVCs) are used in practice. First, the example creates a PVC that requests 1G of storage with read-write access. It then uses this PVC in a pod that mounts the volume and writes a file to it. The example illustrates the typical workflow of claiming storage and consuming it within a containerized application.

The following file, named `pvc.yaml`, creates the a PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
```



```
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

To apply the manifest, enter:

```
$ kubectl apply -f pvc.yaml
persistentvolumeclaim/my-pvc created
```

The following manifest refers to the PVC in a pod. The file is named `pod.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-using-pvc
spec:
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "echo Hello > /data/hello.txt && sleep 3600"]
      volumeMounts:
        - mountPath: "/data"
          name: storage
  volumes:
    - name: storage
      persistentVolumeClaim:
        claimName: my-pvc
```

Create the pod and view the PVC:

```
$ kubectl apply -f pod.yaml
pod/pod-using-pvc created

$ kubectl get pvc
NAME          STATUS    VOLUME          CAPACITY   ACCESS MODES
STORAGECLASS
```

```
my-pvc    Bound    pvc-65413e21-b2f6-4aa5-bc40-5590d61deae9    1Gi    RWO
standard
```

In this example, the pod uses the PVC `my-pvc` to mount a volume at `/data`. The container writes a simple text file (`hello.txt`) to the mounted volume and then sleeps for one hour. This demonstrates how persistent storage can be attached to a running pod, enabling data to survive container restarts or recreations.

To verify whether the `hello.txt` file was actually written to the persistent volume mounted at `/data`, access the pod and navigate to the mounted directory:

```
$ kubectl exec -it pod-using-pvc -- sh

/ # cd /data
/data # ls -l
total 1
-rw-r--r--    1 root    root          6 Apr  4 13:58 hello.txt

/data # cat hello.txt
Hello

/data # exit
```

The Retain Reclaim Policy

The *reclaim policy* of a PV determines what happens to the underlying storage when the PVC is deleted. When the reclaim policy is set to `Retain`, the storage is preserved even after the PVC is removed. This means that the PV and its data remain intact, but the PV enters a `Released` state, indicating that it is no longer bound to a PVC but still contains data. It is the responsibility of the administrator to manually clean up the data or reuse the volume.

This policy is useful when you want to ensure that no data is lost unintentionally, such as in cases where data needs to be recovered or transferred to a different PVC later.

The following example creates a `PersistentVolume` with the `Retain` reclaim policy to ensure that the data remains intact and is not automatically deleted when the corresponding PVC is deleted. The manifest is named `pv.yaml`:

```
apiVersion: v1
kind: PersistentVolume
metadata:
```

```
name: retain-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: "" # <- Important for matching the PVC
  hostPath:
    path: /mnt/data-retain
```

To apply the manifest, enter:

```
$ kubectl apply -f pv.yaml
persistentvolume/retain-pv created
```

To check that the PV was created, enter:

```
$ kubectl get pv
NAME          CAPACITY   ACCESS MODES   RECLAIM POLICY
retain-pv     1Gi        RWO            Retain
```

Now that the PersistentVolume is configured with the Retain reclaim policy, create the corresponding PVC request. The following manifest is named `pvc-request.yaml`:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: retain-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: "" # <- Disables dynamic provisioning
  resources:
    requests:
      storage: 1Gi
```

To apply the PVC, enter:

```
$ kubectl apply -f pvc-request.yaml
```

```
persistentvolumeclaim/retain-pvc created
```

Verify that the `retain-pv` PV is successfully bound to the `retain-pvc` claim:

```
$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
retain-pv	1Gi	RWO	Retain	Bound	default/retain-pvc

Next, create a pod that uses the PVC to access the persistent storage. The following manifest is named `pod-retain.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: write-to-retain-pv
spec:
  containers:
    - name: writer
      image: busybox
      command: ["sh", "-c", "echo Retained Data > /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: retained-storage
          mountPath: /data
  volumes:
    - name: retained-storage
      persistentVolumeClaim:
        claimName: retain-pvc
```

To apply the manifest, enter:

```
$ kubectl apply -f pod-retain.yaml
pod/write-to-retain-pv created
```

Verify that the data has been successfully written to the volume:

```
$ kubectl exec -it write-to-retain-pv -- sh
/ # cat /data/message.txt
Retained Data
/ # exit
```

Now delete the pod and its associated PVC to observe how the volume behaves after claim deletion:

```
$ kubectl delete pod write-to-retain-pv
pod "write-to-retain-pv" deleted

$ kubectl delete pvc retain-pvc
persistentvolumeclaim "retain-pvc" deleted
```

Check the status of the PersistentVolume:

```
$ kubectl get pv retain-pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
retain-pv	1Gi	RWO	Retain	Released	default/retain-pvc

After deleting the original PVC, the PV is now in the `Released` state, indicating that the `retain-pvc` PVC has been deleted, but the data remains and the volume can be reused. To reuse it, manually remove the `claimRef` claim:

```
$ kubectl patch pv retain-pv -p '{"spec":{"claimRef": null}}'
persistentvolume/retain-pv patched
```

Prove that the PV is still present and usable by creating a new PVC to bind to the existing PV. The following manifest is named `new-pvc.yaml`:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: new-retain-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: "" # Must match the PV (empty string in this case)
  resources:
    requests:
      storage: 1Gi
```

To apply the manifest, enter:

```
$ kubectl apply -f new-pvc.yaml
persistentvolumeclaim/new-retain-pvc created
```

To verify the PV is now bound to the new PVC, enter:

```
$ kubectl get pv retain-pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
retain-pv	1Gi	RWO	Retain	Bound	default/new-retain-pvc

Other Kubernetes Orchestration Resources

To wrap up the discussion of Kubernetes, this section presents some features of Kubernetes that address important needs and are worth knowing about.

DaemonSets

A *DaemonSet* can be used when the application needs to ensure that a specific pod runs on every node in the cluster, or on a selected group of nodes. This approach is commonly used for background services such as monitoring agents, log collectors, or networking components that must be present on every node.

When new nodes are added to the cluster, the *DaemonSet* automatically creates a pod on each of them. Likewise, when nodes are removed, their associated pods are also deleted.

DaemonSets are ideal for deploying a service or agent that runs universally across the infrastructure.

StatefulSets

A *StatefulSet* handles applications that require stable identities, persistent storage, and ordered deployment or scaling. Unlike deployments, which treat pods as interchangeable units, *StatefulSets* give each pod a fixed hostname and its own persistent volume. This is particularly important for databases or distributed systems such as MongoDB, Kafka, or Zookeeper, where each node in the system needs to retain its identity and data even after being restarted or rescheduled.

StatefulSets also manage the rollout and termination of pods in a defined order, making them well suited for an application that depends on an initialization sequence.

Jobs

A *job* in Kubernetes runs a task that executes once and completes. Jobs are ideal for short-lived, run-to-completion workloads like batch processing, data migration, or report generation.

When a job is submitted, Kubernetes creates one or more pods to complete the task. The job completes successfully when the assigned pods finish their work. If any pod fails, Kubernetes can retry the task depending on the specified configuration. Once complete, the job is not restarted unless manually resubmitted or scheduled to repeat through another mechanism.

CronJobs

CronJobs build on the concept of jobs by allowing them to run on a defined time schedule. Much like a traditional *cron* job in Linux, a Kubernetes CronJob lets users run tasks at fixed times or intervals. For example, an administrator can schedule a pod to run every hour to generate system backups or to clean up temporary files each night. The CronJob resource handles the timing, while each individual run of the job behaves like a regular Kubernetes job — meaning that it starts, completes, and exits without persisting as a running service.

Guided Exercises

1. Create a PersistentVolumeClaim named `mariadb-pvc`, requesting 200M of storage. Use it to persist the data of a MariaDB container. The pod should run a MariaDB service with a custom root password. Mount the volume at `/var/lib/mysql`.

2. A PersistentVolume named `retain-pv` exists with the `Retain` reclaim policy. Create a PVC named `archive-pvc` that binds to this PV and writes a file. Then delete the pod and PVC, patch the PV to remove the `claimRef`, and create a new PVC named `recover-pvc` to reclaim the same volume.

Explorational Exercises

1. Create a pod with two containers that communicate through a shared volume. One container should write a message to a file, and the other should read and display the contents of that file.

2. What's the difference between using a volume in Kubernetes with a PersistentVolumeClaim and without a PersistentVolumeClaim?

Summary

Kubernetes treats pods as ephemeral by design, which makes them ideal for stateless applications but problematic for workloads that need persistent storage. To solve this, Kubernetes introduces a decoupled storage model using PersistentVolumes (PVs) and PersistentVolumeClaims (PVCs). PVs represent actual storage resources available in the cluster, while PVCs act as user or application requests for those resources. This separation allows for both dynamic and static provisioning of storage and ensures that stateful applications like databases, file servers, and message queues can maintain their data even when pods are rescheduled or restarted.

PVCs provide a consistent and reusable way for pods to access storage, with support for different access modes and lifecycle management. Kubernetes matches PVCs with available PVs and binds them, allowing pods to mount and write data reliably. In scenarios where data must be preserved even after a PVC is deleted, administrators can use a PV with a **Retain** reclaim policy, which holds the data and makes the volume reusable.

Answers to Guided Exercises

1. Create a PersistentVolumeClaim named `mariadb-pvc`, requesting 200M of storage. Use it to persist the data of a MariaDB container. The pod should run a MariaDB service with a custom root password. Mount the volume at `/var/lib/mysql`.

Create a manifest named `pvc.yaml` for the PersistentVolumeClaim:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mariadb-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 200MMi
```

Create a manifest named `mariadb.yaml` for the database service:

```
apiVersion: v1
kind: Pod
metadata:
  name: mariadb
spec:
  containers:
    - name: mariadb
      image: mariadb:11.3
      env:
        - name: MARIADB_ROOT_PASSWORD
          value: my-secret-password
      ports:
        - containerPort: 3306
      volumeMounts:
        - mountPath: /var/lib/mysql
          name: mariadb-storage
  volumes:
    - name: mariadb-storage
      persistentVolumeClaim:
        claimName: mariadb-pvc
```

Apply the two manifests as follows:

```
$ kubectl apply -f pvc.yaml
persistentvolumeclaim/mariadb-pvc created

$ kubectl apply -f mariadb.yaml
pod/mariadb created
```

2. A PersistentVolume named `retain-pv` exists with the `Retain` reclaim policy. Create a PVC named `archive-pvc` that binds to this PV and writes a file. Then delete the pod and PVC, patch the PV to remove the `claimRef`, and create a new PVC named `recover-pvc` to reclaim the same volume.

Create a manifest named `archive-pvc.yaml` for the PersistentVolumeClaim used by the writing application:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: archive-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ""
  resources:
    requests:
      storage: 1Gi
```

Create a manifest named `archive-writer.yaml` for the pod that writes data:

```
apiVersion: v1
kind: Pod
metadata:
  name: archive-writer
spec:
  containers:
    - name: writer
      image: busybox
      command: ["sh", "-c", "echo ArchivedData > /data/archive.txt && sleep 3600"]
      volumeMounts:
        - name: archive-vol
```

```
      mountPath: /data
volumes:
- name: archive-vol
  persistentVolumeClaim:
    claimName: archive-pvc
```

Apply the two manifests and verify that the pod wrote the data:

```
$ kubectl apply -f archive-pvc.yaml
persistentvolumeclaim/archive-pvc created

$ kubectl apply -f archive-writer.yaml
pod/archive-writer created

$ kubectl exec -it archive-writer -- cat /data/archive.txt
ArchivedData
```

Delete the pod and PVC:

```
$ kubectl delete pod archive-writer
pod "archive-writer" deleted

$ kubectl delete pvc archive-pvc
persistentvolumeclaim "archive-pvc" deleted
```

Patch the PersistentVolume:

```
$ kubectl patch pv retain-pv -p '{"spec":{"claimRef": null}}'
persistentvolume/retain-pv patched
```

Create a manifest named `recover-pvc.yaml` for the PersistentVolumeClaim used by the reading application:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: recover-pvc
spec:
  accessModes:
    - ReadWriteOnce
```

```
storageClassName: ""
resources:
  requests:
    storage: 1Gi
```

Create a manifest named `recover-reader.yaml` for the pod that recovers the data:

```
apiVersion: v1
kind: Pod
metadata:
  name: recover-reader
spec:
  containers:
    - name: reader
      image: busybox
      command: ["sh", "-c", "cat /data/archive.txt && sleep 3600"]
      volumeMounts:
        - name: archive-vol
          mountPath: /data
  volumes:
    - name: archive-vol
      persistentVolumeClaim:
        claimName: recover-pvc
```

Apply the two manifests and verify that the data is still present:

```
$ kubectl apply -f recover-pvc.yaml
persistentvolumeclaim/recover-pvc created

$ kubectl apply -f recover-reader.yaml
pod/recover-reader created

$ kubectl exec -it recover-reader -- cat /data/archive.txt
ArchivedData
```

Answers to Explorational Exercises

1. Create a pod with two containers that communicate through a shared volume. One container should write a message to a file, and the other should read and display the contents of that file.

Create a manifest named `shared-volume.yaml` for the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-volume-pod
spec:
  containers:
    - name: writer
      image: busybox
      command: ['sh', '-c', 'echo Hello from writer > /shared/message.txt; sleep 3600']
      volumeMounts:
        - name: shared-vol
          mountPath: /shared
    - name: reader
      image: busybox
      command: ['sh', '-c', 'sleep 5; cat /shared/message.txt; sleep 3600']
      volumeMounts:
        - name: shared-vol
          mountPath: /shared
  volumes:
    - name: shared-vol
      emptyDir: {}
```

This manifest defines a pod named `shared-volume-pod` with two containers. The `writer` container writes the message `Hello from writer` to the file `/shared/message.txt` and stays alive by executing `sleep 3600`. The `reader` container waits five seconds (`sleep 5`), then reads and displays the contents of the same file. Both containers share a volume named `shared-vol`, which is defined as an `emptyDir`. This type of volume provides a temporary shared directory that exists as long as the pod is running.

Apply the manifest by entering:

```
$ kubectl apply -f shared-volume.yaml
pod/shared-volume-pod created
```

Check that the pods are running:

```
$ kubectl get pod shared-volume-pod
```

NAME	READY	STATUS	RESTARTS	AGE
shared-volume-pod	2/2	Running	0	107s

Now access the `reader` container with option `-c reader` to see the message in the shared volume:

```
$ kubectl exec -it shared-volume-pod -c reader -- sh
/ # cat /shared/message.txt
Hello from writer
```

2. What's the difference between using a volume in Kubernetes with a PersistentVolumeClaim and without a PersistentVolumeClaim?

In Kubernetes, you can use volumes either directly in a pod specification or through a PersistentVolumeClaim. When you define a volume directly—such as with `emptyDir`, `hostPath`, or `configMap`—the volume is tied to the lifecycle of the pod. For example, a specification of `emptyDir` creates a temporary directory that is deleted when the pod is removed. In contrast, `hostPath` mounts a directory from the node's local filesystem, which can be risky in multinode environments. These types of volumes are simple to use but do not offer persistence or portability across pods or nodes.

Using a PVC, on the other hand, provides a more robust and flexible approach to managing storage. A PVC is an abstraction that allows you to request storage resources from the cluster without needing to know the underlying storage implementation. A PVC binds to a PersistentVolume (PV), which can be precreated (static) or dynamically provisioned using a `StorageClass`.

The PVC approach decouples storage from application logic, ensures data persistence across pod restarts or rescheduling, and supports access controls, quotas, and reclaim policies. For any stateful workload, such as databases or file servers, using a PVC is the recommended best practice.



703.3 Kubernetes Package Management

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 703.3

Weight

2

Key knowledge areas

- Understanding the concepts of Charts, Releases and Values
- Installation, upgrading and uninstalling software using Helm
- Specify custom values to configure software installed using Helm
- Awareness of Kustomize
- Awareness of Flux CD and Argo CD

Partial list of the used files, terms and utilities

- `helm install`
- `helm upgrade`
- `helm list`
- `helm uninstall`
- `values.yaml`



Linux
Professional
Institute

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	703 Kubernetes
Objective:	703.3 Kubernetes Package Management
Lesson:	1 of 1

Introduction

Kubernetes has revolutionized the way applications are deployed and managed. But as the complexity of these applications grows, so does the need for tools that simplify their lifecycle management. Helm, Kustomize, Flux CD, and Argo CD have emerged as popular solutions for packaging, deploying, and managing Kubernetes applications.

Helm provides flexibility through templates and value injection, allowing users to override default configuration values via YAML files or command-line arguments. This capability enables highly configurable and repeatable deployments.

Although Helm is widely adopted, it is part of a broader ecosystem of tools designed to support configuration and delivery in Kubernetes environments. Kustomize offers a template-free alternative that focuses on layering and patching manifests through declarative overlays. For teams embracing GitOps workflows, tools such as Flux CD and Argo CD bring automation, consistency, and auditability to application delivery by syncing Kubernetes states directly from Git repositories.

Helm: Package Management for Kubernetes Applications

Helm is the package manager for Kubernetes. Similar to tools like `apt` on Ubuntu or `yum` on Red Hat, Helm installs, upgrades, and manages Kubernetes applications through preconfigured packages called *charts*.

With Helm, deploying complex applications becomes as simple as running a single command. It customizes deployments using configurable values, making it easy to adapt applications to different needs. Helm also supports seamless upgrades and rollbacks, so teams can confidently manage application versions over time. Additionally, administrators can reuse the same configurations across multiple environments such as development, staging, and production, ensuring consistency and reducing manual effort.

Helm also provides a templating engine based on templates written in the Go language, allowing users to parameterize their configurations. This means that image versions, resource limits, or environment-specific settings can be dynamically injected into the templates at runtime, making deployments more flexible and adaptable to different environments.

Another advantage of Helm is its ability to handle dependencies between components. For instance, if an application requires a database, message queue, or other services, these dependencies can be bundled into the chart or brought in through subcharts. This ensures that all required components are deployed together in a coordinated manner.

Additionally, Helm streamlines collaboration by allowing teams to share and reuse charts across projects, reducing duplication of effort and promoting best practices.

It is important to understand Helm's architecture as well as how to install, upgrade, and uninstall software using Helm commands. These operations simplify the management of complex applications and allow for controlled versioning and rollbacks.

Installing Helm in Linux

To install Helm on Linux or macOS, simply run the official installation script provided by the Helm project. Execute the following command in the terminal:

```
$ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 11913  100 11913    0     0 31014      0 --:--:-- --:--:-- --:--:-- 31023
Downloading https://get.helm.sh/helm-v3.17.2-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
```

```
helm installed into /usr/local/bin/helm
```

Helm Charts: The Foundation of Kubernetes Packaging

A *chart* in Helm is a package that contains everything needed to deploy an application on Kubernetes. It is like a bundle of Kubernetes manifests (YAML files) that describe resources such as deployments, services, Ingresses, ConfigMaps, and more—but with the added power of templating. Charts allow reuse, customization, and versioning in Kubernetes configurations.

The standard structure of a Helm chart provides a clear and consistent way to package Kubernetes applications. It includes a set of directories and files that define the application's resources, configurations, and metadata:

```
my-chart/  
  Chart.yaml  
  values.yaml  
  templates/  
  charts/
```

The components are:

Chart.yaml

This file contains metadata about the chart, such as its name, version, description, and dependencies. It serves as the “header” for the chart and is essential for identifying and managing it.

values.yaml

This file defines the default configuration values for the chart. These values can be overridden by users at runtime when installing or upgrading the chart. It adds parameters to templates, making the chart reusable across different environments.

templates/

This directory contains the Kubernetes manifest templates written in YAML with embedded Go templating syntax. These templates are rendered into actual Kubernetes manifests during deployment by substituting values from `values.yaml` or user-provided overrides.

charts/

This optional directory contains sub-charts or dependencies that the main chart relies on. For example, if the application requires a database, include a chart for the database here or refer to it in `Chart.yaml`.

Helm Releases: Tracking and Controlling Deployments

A *release* is a specific deployment of a chart into the Kubernetes cluster — a named instance of that chart, with all the resources and configurations applied. Every time Helm installs a chart, Helm creates a release.

For example, to install the same chart twice under different release names, enter:

```
$ helm install myapp-release ./my-chart
$ helm install staging-release ./my-chart
```

This creates two separate deployments, each isolated, with its own configuration and lifecycle.

Helm Values: Dynamic Configuration Made Simple

Values in Helm are configuration parameters that control the deployment of a chart. They are defined in the `values.yaml` file, and can be overridden at install time using the `--values`, `-f`, or `--set` options on the command line.

For example, suppose this configuration is in `values.yaml`:

```
replicaCount: 2
image:
  repository: nginx
  tag: 1.25
```

The deployment can be launched with different values taken from another file as follows:

```
$ helm install my-nginx ./my-chart -f custom-values.yaml
```

The values can also be specified directly in the command through `--set`:

```
$ helm install my-nginx ./my-chart --set replicaCount=3,image.tag=1.26
```

This flexibility makes Helm highly reusable and customizable. For instance, the same chart can be used for `dev`, `staging`, and `production`, each with different values.

Installing, Upgrading, and Uninstalling Software Using Helm

Helm simplifies application management on Kubernetes by providing a streamlined way to install, upgrade, and uninstall software. These operations are performed using Helm commands that interact with charts.

Installing of Software Using Helm

To install software using Helm, begin by selecting a chart from a *repository* or by using a locally available chart. Helm repositories are centralized locations where charts are stored and versioned, making it easy to discover and deploy applications.

One of the most popular public repositories is *Bitnami*. Add the Bitnami repository (if it is not already present) as follows:

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami
"bitnami" has been added to your repositories
```

Now update the repository:

```
$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
Update Complete. ☞Happy Helming!☞
```

Check the repository list with:

```
$ helm repo list
NAME      URL
bitnami   https://charts.bitnami.com/bitnami
```

The installation process is initiated using the `helm install` command, which creates a new release in the Kubernetes cluster. For example, install an Nginx application using:

```
$ helm install my-nginx bitnami/nginx
NAME: my-nginx
LAST DEPLOYED: Thu Apr  3 20:47:58 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
```

TEST SUITE: None

NOTES:

CHART NAME: nginx

CHART VERSION: 19.0.4

APP VERSION: 1.27.4

NGINX can be accessed through the following DNS name from within the cluster:

my-nginx.default.svc.cluster.local (port 80)

This command installs the Nginx chart with the release name `my-nginx`. Helm creates all necessary Kubernetes resources, including the deployment, the service, and ConfigMaps.

During installation, the Helm client provides helpful output detailing the resources that were created, the current status of the release, and any additional configuration steps needed.

The `helm install` command is versatile and can install charts from multiple sources. A chart can be installed directly from a chart repository, as demonstrated earlier. Alternatively, use a local chart archive by specifying a `.tgz` file, such as:

```
$ helm install foo foo-0.1.1.tgz.
```

It is also possible to install a chart from an unpacked chart directory by providing the path like the following:

```
$ helm install foo path/to/foo
```

To install a chart by specifying a full URL pointing to a chart package, enter something like:

```
$ helm install foo https://example.com/charts/foo-1.2.3.tgz.
```

Check the resources with:

```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/my-nginx-64979b59-s2cqf	1/1	Running	0	117s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP

```

46m
service/my-nginx      LoadBalancer    10.96.129.116    <pending>      80:32408/TCP,443:32521/TCP
117s

NAME                  READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/my-nginx  1/1      1              1            117s

NAME                  DESIRED    CURRENT    READY    AGE
replicaset.apps/my-nginx-64979b59  1          1          1        117s

```

To see all Helm releases currently deployed, enter:

```

$ helm list
NAME            NAMESPACE    REVISION    UPDATED                               STATUS
CHART           APP VERSION
my-nginx        default       1           2025-04-03 20:47:58.927714719 -0300 -03 deployed
nginx-19.0.4    1.27.4

```

To deploy an application quickly without specifying a custom release name, Helm provides the `--generate-name` option. This option tells Helm to automatically generate a unique name for the release.

For example, running `helm install bitnami/mysql --generate-name` installs the MySQL chart from the Bitnami repository and assigns it a randomly generated name, such as `mysql-1743725550`. This is especially useful for testing or deploying multiple instances of the same chart without having to manually manage unique release names. An example follows:

```

$ helm install bitnami/mysql --generate-name
NAME: mysql-1743725550
LAST DEPLOYED: Thu Apr  3 21:12:34 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
CHART NAME: mysql
CHART VERSION: 12.3.2
APP VERSION: 8.4.4

```


Overview of the Features Included in a Chart

To get a quick overview of the features included in the MySQL chart, enter:

```
$ helm show chart bitnami/mysql
```

For more detailed information, including values, README, and chart metadata, use:

```
$ helm show all bitnami/mysql
```

Upgrading a Release

To change the configuration or update the chart version, use `helm upgrade`. For example, to change the replica count of the Nginx deployment, enter:

```
$ helm upgrade my-nginx bitnami/nginx --set replicaCount=3
Release "my-nginx" has been upgraded. Happy Helming!
```

This updates the running `my-nginx` release by applying a new value. Helm performs a rolling update and keeps track of version history.

To view the history of upgrades for a release, enter:

```
$ helm history my-nginx
```

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
1	Thu Apr 3 20:47:58 2025	superseded	nginx-19.0.4	1.27.4	Install complete
2	Thu Apr 3 20:52:03 2025	deployed	nginx-19.0.4	1.27.4	Upgrade complete

Roll back to a previous version using:

```
$ helm rollback my-nginx 1
Rollback was a success! Happy Helming!
```

Specifying Custom Values to Configure Software Installed Using Helm

One of Helm's most powerful features is the ability to customize application deployments using

configuration values. Every Helm chart comes with a file called `values.yaml` that defines default settings for the application, such as its image names, replica counts, and ports. These values can be overridden during installation or upgrades to tailor the deployment to a specific environment.

To save the default values for a chart (for example, Bitnami's Nginx chart) to a file, use a command such as:

```
$ helm show values bitnami/nginx > custom-values.yaml
```

To save the output without the comments, enter:

```
$ helm show values bitnami/nginx | grep -v "#" > custom-values.yaml
```

Once a chart is installed, view the values injected into that release using:

```
$ helm get values my-nginx --all
```

To change a configuration, not all the values from the default `values.yaml` file have to be specified. All that is needed is a simplified file containing only the fields to override. For example, create a file named `custom-values.yaml` with the following content:

```
replicaCount: 5
image:
  repository: bitnami/nginx
  tag: 1.25
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
```

This file overrides only the specified settings when used with a Helm `install` or `upgrade` command. All other values will fall back to the chart's defaults.

To install a chart for the first time while applying custom values, enter:

```
$ helm install my-other-nginx bitnami/nginx -f custom-values.yaml
NAME: my-other-nginx
LAST DEPLOYED: Thu Apr 3 21:07:31 2025
NAMESPACE: default
```

```
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 19.0.4
APP VERSION: 1.27.4
```

To upgrade an existing installation with new values, use:

```
$ helm upgrade my-nginx bitnami/nginx -f custom-values.yaml
Release "my-nginx" has been upgraded. Happy Helming!
```

Uninstalling a Release

To remove an application completely, use the `uninstall` command:

```
$ helm uninstall my-nginx
release "my-nginx" uninstalled
```

This deletes all Kubernetes resources created by the Helm release. After this operation, Helm also removes the release information from its internal history. To confirm removal, enter:

```
$ helm list
```

If nothing appears, the release has been removed.

A `helm uninstall` command with the `--keep-history` option keeps the release history:

```
$ helm uninstall my-other-nginx --keep-history
release "my-other-nginx" uninstalled
```

To request information about that release, enter:

```
$ helm status my-other-nginx
```

Finding Charts With `helm search`

Helm provides a powerful search tool that permits the discovery of charts from different sources. There are two main ways to search:

`helm search hub`

queries the *Artifact Hub*, a centralized catalog of Helm charts from a wide range of public repositories.

`helm search repo`

searches the repositories that have already been added to the local Helm client using `helm repo add`. This search is performed locally and does not require an internet connection.

For example, to find publicly available charts on Artifact Hub related to WordPress, enter:

```
$ helm search hub wordpress
```

URL	CHART VERSION	APP VERSION	
DESCRIPTION			
https://artifacthub.io/packages/helm/wordpress-...	1.0.2	1.0.0	A
Helm chart for deploying Wordpress+Mariadb st...			
https://artifacthub.io/packages/helm/kube-wordp...	0.1.0	1.1	this
is my wordpress package			
(...)			

The `helm search hub --list-repo-url` exposes the actual Helm repository URLs. This command comes in handy for adding a new repository:

```
$ helm repo add [NAME] [URL]
```

Use `helm search repo` to list available charts from the repositories that were already added to the local Helm client:

```
$ helm search repo
```

NAME	CHART VERSION	APP VERSION	DESCRIPTION
bitnami/airflow	22.7.2	2.10.5	Apache
Airflow is a tool to express and execute...			
bitnami/apache	11.3.5	2.4.63	Apache HTTP
Server is an open-source HTTP serve...			
(...)			

Kustomize

Although Helm has become a standard tool for managing Kubernetes applications through templated charts, it is essential to understand alternative and complementary tools that offer different philosophies and benefits. One such tool is *Kustomize*, which provides a template-free approach to configuration customization.

Unlike Helm, which uses a templating engine to inject values into YAML files, Kustomize allows a team to define a base set of manifests and then create overlays that modify or extend those base definitions. This makes it easier to manage environment-specific configurations (e.g., development, staging, production) in a clean and declarative manner without relying on external tooling or complex logic.

Since Kustomize is built into `kubectl` itself (starting with Kubernetes v1.14), it is widely supported and can be used out-of-the-box to apply layered configurations with minimal setup.

Kustomize encourages a structure that separates configuration from logic. Write the application manifests once and apply different patches for each environment, such as modifying labels, container images, resource limits, or ConfigMaps.

Kustomize is particularly useful in teams that prefer to stay close to raw Kubernetes YAML and prioritize transparency and simplicity in deployment definitions. Its declarative layering model is ideal for infrastructure teams managing multiple clusters or environments with similar core applications but different configurations.

Flux CD and Argo CD

For organizations embracing GitOps — a methodology where Git is the single source of truth for both infrastructure and application definitions — Flux CD and Argo CD are indispensable. Both are continuous delivery controllers for Kubernetes that synchronize the state of the cluster with what is defined in Git repositories. This means that any change committed to Git (whether a Helm chart, Kustomize overlay, or plain YAML) is automatically deployed to the cluster, creating a fully automated and auditable delivery pipeline. With GitOps, rollback is as simple as reverting a Git commit, and every change is tracked through version control.

Argo CD provides a web interface, command-line interface, and API that allow teams to visualize and manage the synchronization state of applications. It supports advanced deployment strategies such as blue/green, and can track Helm and Kustomize projects side by side.

Flux CD, on the other hand, emphasizes automation and Git-native workflows, and is tightly integrated with the Cloud Native Computing Foundation (CNCF) ecosystem.

Both tools enable drift detection, alerting the administrator if something in the cluster changes that is not reflected in Git. In this way they help to ensure operational consistency and compliance.

In practice, many teams use Helm in combination with Kustomize or GitOps tools, rather than choosing just one. For example, Helm may be used to package an application, Kustomize to define environment-specific configurations, and Argo CD to automate and enforce those definitions from Git. By understanding and integrating these tools effectively, DevOps teams can build robust, scalable, and fully automated deployment pipelines that support modern microservices and Infrastructure as Code practices.

Guided Exercises

1. Install the Bitnami Nginx Helm chart using the release name `web-server`.

2. Install the Bitnami MySQL chart and set the root password to `SuperSecret123`.

3. Upgrade the `web-server` release to use three replicas.

4. List all the Helm releases currently installed in your cluster.

5. Uninstall the `my-database` MySQL release from your cluster.

Explorational Exercises

1. Download the default `values.yaml` file for the Bitnami Nginx chart and compare it to a minimal custom version that changes only the replica count and image tag.

2. Install the Bitnami MariaDB chart, then use `kubectl` to inspect and validate how the Helm chart created Kubernetes resources.

Summary

Kubernetes offers a flexible platform for deploying and managing applications, but its complexity requires efficient tools to streamline application lifecycle management. Helm plays a central role by packaging Kubernetes resources into reusable charts, allowing developers to install, configure, upgrade, and roll back applications with ease. Through powerful templating and `values.yaml` custom definitions, Helm supports environment-specific configurations and encourages repeatable, version-controlled deployments. Combined with its support for managing dependencies and sharing charts across teams, Helm brings structure and automation to the Kubernetes ecosystem.

Beyond Helm, tools like Kustomize, Flux CD, and Argo CD expand the possibilities for configuration and delivery. Kustomize provides a declarative and template-free way to manage layered configurations, making it ideal for teams working with native Kubernetes manifests. Flux CD and Argo CD bring GitOps automation to the table, continuously synchronizing cluster state with Git repositories.

Answers to Guided Exercises

1. Install the Bitnami Nginx Helm chart using the release name `web-server`.

```
$ helm install web-server bitnami/nginx
```

2. Install the Bitnami MySQL chart and set the root password to `SuperSecret123`.

```
$ helm install my-database bitnami/mysql --set auth.rootPassword=SuperSecret123
```

To check that the password is correct, enter:

```
$ kubectl get secret --namespace default my-database-mysql -o jsonpath="{.data.mysql-root-password}" | base64 -d  
SuperSecret123
```

3. Upgrade the `web-server` release to use three replicas.

```
$ helm upgrade web-server bitnami/nginx --set replicaCount=3
```

4. List all the Helm releases currently installed in your cluster.

```
$ helm list
```

5. Uninstall the `my-database` MySQL release from your cluster.

```
$ helm uninstall my-database
```

Answers to Explorational Exercises

1. Download the default `values.yaml` file for the Bitnami Nginx chart and compare it to a minimal custom version that changes only the replica count and image tag.

Save the default values to a file:

```
$ helm show values bitnami/nginx > default-values.yaml
```

Create a new file named `custom-values.yaml`:

```
replicaCount: 2
image:
  tag: 1.27
```

Install the chart using the custom values:

```
$ helm install test-nginx bitnami/nginx -f custom-values.yaml
```

To explore the difference between the two value sets, get the actual values:

```
$ helm get values test-nginx --all > actual-values.yaml
```

Use `diff` using to see differences:

```
$ diff default-values.yaml actual-values.yaml
```

2. Install the Bitnami MariaDB chart, then use `kubectl` to inspect and validate how the Helm chart created Kubernetes resources.

To install MariaDB, enter:

```
$ helm install mariadb-release bitnami/mariadb --set auth.rootPassword=Test123
```

Use the following `kubectl` command to inspect the deployment:

```
$ kubectl get all -l app.kubernetes.io/instance=mariadb-release
```

Identify what Kubernetes resources were created (e.g., StatefulSet, Service, Secrets).



Topic 704: Security and Observability



704.1 Cloud Native Security

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 704.1

Weight

4

Key knowledge areas

- Understand core IT infrastructure components and their role in deployment
- Understand common IT infrastructure security risks and ways to mitigate them
- Understand supply chain security and dependencies on foreign code
- Understand common application security risks and ways to mitigate them
- Understand the concepts of asymmetric cryptography and digital certificates
- Understand the principles of common standard for authentication and authorization
- Understand how to manage user credentials and how to use advanced authentication technologies

Partial list of the used files, terms and utilities

- Service exploits, brute force attacks, and denial of service attacks
- Security updates, packet filtering, load balancers and application gateways
- Cross site scripting, verbose error reports
- API authentication
- Buffer overflows, SQL injections
- API access, permissions, verbosity and rate limits
- CORS headers and CSRF tokens

- Common Vulnerabilities and Exposures (CVE)
- CVE IDs and CVE scores
- Public key, private key, X.509 certificate, certificate authority
- TLS, transport encryption
- Single sign-on (SSO)
- OAuth2, OpenID Connect and SAML
- Two-factor authentication (2FA) and multi-factor authentication (MFA)
- One-time passwords (OTP), time-based one-time passwords (TOTP)
- Authenticator applications
- Password hashing and salting



Linux
Professional
Institute

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	704 Security and Observability
Objective:	704.1 Cloud Native Security
Lesson:	1 of 2

Introduction

Understanding the foundational components of IT infrastructure is crucial for successfully deploying and maintaining secure, scalable, and high-performing applications. Within traditional data centers as well as cloud-native environments, components such as load balancers, application gateways, packet filtering, and consistent security updates all serve as essential building blocks. These technologies ensure that traffic is distributed efficiently, application access is secured, and potential vulnerabilities are addressed before they can be exploited.

The Importance of IT Infrastructure in Deployment

The following sections examine some of the key infrastructure components and their roles in deployed applications.

Load Balancers: Ensuring High Availability and Reliability

One of the most critical components in modern IT infrastructure is the *load balancer*. Load balancers distribute incoming network traffic across multiple servers to ensure that no single server becomes overwhelmed. This distribution helps maintain high availability and reliability,

which are essential for delivering a consistent user experience.

A load balancer routes traffic intelligently based on factors such as server capacity, current load, and geographic location. Additionally, load balancers can monitor server health and automatically reroute traffic away from failed or underperforming servers, further enhancing system resilience.

In security, load balancers can act as a first line of defense by hiding backend servers from direct exposure to the internet and by encrypting traffic between clients and servers.

Kubernetes can expose a service using type `LoadBalancer`, which provisions a cloud-based external load balancer to distribute traffic to the application pods. Examples of such cloud services include: *AWS Elastic Load Balancer* (ELB), *Azure Load Balancer* and *Google Cloud Load Balancer*.

A load balancer has the capability to perform SSL offloading, also known as *SSL termination*, by taking over the task of managing SSL/TLS encryption and decryption. Instead of passing encrypted traffic directly to the backend servers, the load balancer handles the TLS handshake and decrypts the data before forwarding it internally. This approach significantly reduces the computational load on backend services, because they no longer need to process encrypted traffic, allowing them to operate more efficiently and scale more effectively.

Application Gateways: Managing and Securing Application Traffic

Another important component is the *application gateway*, an intermediary that manages and secures application traffic. Unlike traditional load balancers, application gateways operate at the application layer (layer 7 of the OSI model) and provide advanced features such as request inspection, content-based routing, and SSL termination.

Application gateways inspect incoming requests to ensure they comply with security policies and route them to the appropriate backend services.

Application gateways enhance security by protecting against common threats such as cross-site scripting (XSS) and SQL injection attacks. They can also cache frequently accessed content, reducing the load on backend servers and improving response times for users.

Packet Filtering: Controlling Network Traffic

At the network level, firewalls perform *packet filtering* to control traffic based on predefined rules. Packet filtering works by examining individual packets of data as they pass through the network and determining whether to allow or block them based on criteria such as source IP address, destination IP address, port numbers, and protocols.

By filtering out malicious or unnecessary traffic, packet filtering helps reduce the attack surface and reduces the risk that illegitimate traffic reaches. While packet filtering is effective for basic traffic control, it is often combined with more advanced techniques such as stateful inspection and intrusion detection to provide comprehensive network security.

Security Updates: Protecting Against Vulnerabilities

No discussion of IT infrastructure would be complete without addressing the importance of security updates, also known as patches or upgrades. Software vulnerabilities are constantly being discovered, and attackers are quick to exploit them if left unpatched. Security updates are released by vendors to address these vulnerabilities, fix bugs, and improve overall system security.

Common IT Infrastructure Security Risks and Ways to Mitigate Them

In today's interconnected digital world, IT infrastructure is constantly under threat from a variety of security risks. Successful attacks can compromise the confidentiality, integrity, and availability of systems, leading to data breaches, service disruptions, and financial losses.

Service Exploits: Gaining Unauthorized Access

One of the most prevalent security risks in IT infrastructure is *service exploits*, which occur when attackers exploit vulnerabilities in software services to gain unauthorized access. An unpatched vulnerability in a server could allow attackers to execute arbitrary commands on it, or escalate privileges to take over the whole computer or network.

Services such as web servers, databases, and email servers are frequent targets because they are exposed to external networks and often have complex configurations that can introduce weaknesses.

To mitigate this risk, it is essential to keep all software up to date with the latest security patches. Additionally, organizations should conduct regular vulnerability assessments and penetration testing to identify and address potential weaknesses before attackers can exploit them.

Brute Force Attacks: Exhaustive Trial-and-Error Methods

Brute force attacks are another common threat, where attackers attempt to guess passwords or encryption keys through exhaustive trial and error. These attacks are particularly dangerous for systems with weak or easily guessable credentials, as the attack can eventually succeed given

enough time and computational resources.

To defend against brute force attacks, organizations should enforce strong password policies. Multi-factor authentication (MFA) adds an additional layer of security by requiring users to provide two or more verification factors, making it significantly harder for attackers to gain access even if they guess a password. Rate limiting and account lockout mechanisms can also help by restricting the number of login attempts within a specific timeframe.

Denial of Service (DoS) Attacks: Overwhelming Systems

Denial of service (DoS) attacks aim to overwhelm a system or network, rendering it unavailable to legitimate users. These attacks flood the target with excessive traffic or resource requests, exhausting its capacity to respond. *Distributed denial of service* (DDoS) attacks, a variant of DoS, marshal multiple compromised systems that work together to amplify the attack.

Mitigating DoS attacks requires a combination of proactive and reactive measures. Load balancers and content delivery networks (CDNs) can help distribute traffic and absorb malicious requests. Firewalls and intrusion detection/prevention systems (IDS/IPS) can detect and block suspicious traffic patterns. Additionally, organizations should implement rate limiting and traffic filtering to reduce the impact of large-scale attacks. For critical systems, redundant infrastructure and failover mechanisms can help maintain continuity even during an attack.

Buffer Overflows: Writing Excessive Data

A *buffer overflow* occurs when excessive data is written into a buffer, exceeding its allocated size. This can overwrite adjacent memory locations, potentially allowing attackers to execute arbitrary code or crash the system. Buffer overflows are commonly exploited in applications with poor input validation or insecure coding practices.

To prevent buffer overflows, developers should adopt secure coding practices, such as using programming languages and libraries that include built-in protections against these vulnerabilities. Input validation and bounds checking are critical to ensuring that user-supplied data does not exceed expected limits. Regular code reviews and analysis tools (both static and dynamic) can help identify and fix potential buffer overflow issues before deployment.

SQL Injection: Manipulating Databases

SQL injection is a type of attack that exploits vulnerabilities in SQL queries to manipulate databases. Attackers inject malicious SQL code into input fields, tricking the application into executing unintended commands. This can lead to unauthorized access to sensitive data, data corruption, or even complete database compromise.

To mitigate SQL injection risks, developers should use parameterized queries or prepared statements, which separate SQL code from user input. Input validation and sanitization are also essential to ensure that only expected data types and formats are accepted. A web application firewall (WAF) can provide an additional layer of protection by filtering out malicious requests before they reach the application.

Cross-Site Scripting (XSS): Injecting Malicious Scripts

Cross-site scripting (XSS) is a vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users. These scripts execute in the context of the victim's browser, potentially stealing sensitive information such as session cookies, login credentials, or personal data.

To mitigate XSS risks, developers should implement robust input validation and output encoding. Input validation ensures that only expected data types and formats are accepted, while output encoding converts special characters into safe representations before rendering them in the browser. Additionally, modern frameworks often include built-in protections against XSS, such as Content Security Policy (CSP) headers, which restrict the sources of executable scripts. Regular security testing, including penetration testing and automated scanning, helps identify and address potential XSS vulnerabilities.

Common Forms of Application-Level Security

Application-level security is among the most critical challenges faced by developers and organizations today. Vulnerabilities in the design, implementation, or configuration of applications (including weak password storage) make them susceptible to attacks such as cross-site scripting, unauthorized API access, and abuse of services through excessive requests.

Understanding and mitigating common application security risks is crucial for building secure and resilient systems. Targeted strategies are needed to address risks effectively.

Common Vulnerabilities and Exposures (CVE): Tracking Known Risks

The *Common Vulnerabilities and Exposures (CVE)* system provides a standardized list of known security vulnerabilities, along with unique identifiers (CVE IDs) and severity ratings (CVE scores). This system helps organizations stay informed about emerging threats and prioritize their mitigation efforts based on the severity of each vulnerability.

Regularly monitoring the CVE database allows security teams to identify vulnerabilities that could affect their systems and apply patches or workarounds promptly. Automated tools, such as vulnerability scanners, can integrate with the CVE database to streamline the process of

identifying and addressing risks. By staying proactive, organizations can reduce their exposure to known threats and maintain stronger security.

API Authentication: Ensuring Authorized Access

API authentication is essential for ensuring that only authorized users or systems can access APIs. Without proper authentication, APIs are vulnerable to unauthorized access, data breaches, and abuse.

To enhance API security, developers should enforce strong authentication mechanisms and regularly rotate credentials such as API keys. Role-based access control (RBAC) ensures that users or systems have access only to the resources and actions permitted by their roles. Additionally, logging and monitoring API requests helps detect suspicious activities, enabling timely responses to potential threats.

Password Hashing and Salting: Securing User Credentials

Storing passwords securely is a fundamental aspect of application security. Plain-text storage of passwords is highly vulnerable to data breaches, exposing users to significant risks. To mitigate this, developers should use password hashing and salting.

When effective hashing and salting are in place, an attacker who gains access to the hashed passwords cannot easily reverse-engineer the original credentials. Developers should also enforce strong password policies, requiring users to create complex passwords that are resistant to brute-force attacks. Multi-factor authentication (MFA) further enhances security by adding an additional layer of verification beyond passwords.

Rate Limiting: Preventing Abuse of Services

Rate limiting prevents the abuse of APIs and services by restricting the number of requests a user or system can make within a specific timeframe. Without rate limiting, attackers can overwhelm APIs with excessive requests, leading to denial of service, resource exhaustion, or data leakage.

To implement rate limits effectively, developers should define thresholds based on typical usage patterns and enforce them at the application or infrastructure level.

Supply Chain Security and Dependencies on Foreign Code

Modern software development relies on third-party libraries, frameworks, and APIs to accelerate development and enhance functionality. While these dependencies bring significant benefits, they also introduce risks related to supply chain security. Vulnerabilities or malicious code in third-

party components can compromise the integrity of an application, making it essential to understand and mitigate these risks.

Verbose Error Reports: Exposing Sensitive Information

One common risk in software development is the use of verbose error reports, which provide detailed information about system failures. While these reports are helpful for debugging during development, they can inadvertently expose sensitive information when enabled in production environments. For example, verbose error messages may reveal database schemas, server paths, or authentication mechanisms, giving attackers valuable insights into the application's architecture.

To mitigate this risk, developers should configure applications to display generic error messages in production environments while logging detailed errors securely on the server side. Additionally, regular audits of error handling mechanisms can help ensure that no sensitive information is leaked through error reports. By adopting a “least privilege” approach to error reporting, organizations can minimize the risk of exposing critical details to potential attackers.

API Access, Permissions, Verbosity, and Rate Limits: Securing External Integrations

Applications frequently rely on APIs to interact with third-party services, but insecure API access can introduce significant vulnerabilities. Misconfigured permissions, excessive verbosity in responses, and lack of rate limiting can all lead to abuse or exploitation by malicious actors.

When it comes to permissions, it is critical to ensure that APIs are accessed with the minimum privileges necessary to perform their intended function. Granting overly permissive access increases the risk that an attacker could exploit the API to perform unauthorized operations, compromise data, or alter system behavior. Properly implementing least privilege in access controls helps contain the blast radius in the event of a breach and supports better overall security hygiene.

Another important consideration is verbosity. APIs should be designed to return only the essential data required by the client, avoiding exposure of unnecessary internal details. Excessive verbosity can inadvertently reveal sensitive information about the system's structure, database schema, or debugging data that attackers could use for reconnaissance and exploitation.

Rate limiting is a key defensive mechanism to prevent abuse. By restricting the number of API requests allowed per user or system within a specific timeframe, rate limits protect services from being overwhelmed by malicious traffic or brute-force penetration attacks. Without rate limiting, APIs are more vulnerable to denial of service (DoS) attacks and uncontrolled data scraping, which

can degrade performance and expose sensitive data.

CORS Headers: Defining Access Policies

Cross-origin resource sharing (CORS) headers play a critical role in securing web applications by defining which external domains are allowed access to resources. Misconfigured CORS headers can inadvertently expose sensitive data to unauthorized domains, enabling cross-origin attacks.

For example, overly permissive CORS policies that allow access from any domain (`Access-Control-Allow-Origin: *`) can enable malicious websites to retrieve sensitive information from the application. To mitigate this risk, developers should explicitly specify trusted domains in CORS headers and avoid using wildcard configurations unless absolutely necessary. CORS policies should be reviewed periodically to ensure that they align with the application's security requirements and do not introduce unintended vulnerabilities.

CSRF Tokens: Preventing Unauthorized Requests

Cross-site request forgery (CSRF) attacks occur when attackers trick users into performing unintended actions on a web application where they are authenticated. For example, an attacker might craft a malicious link that submits a form or initiates a transaction on behalf of the user.

To prevent CSRF attacks, applications can use CSRF tokens, which are unique, unpredictable values embedded in forms or requests. These tokens validate that the request originates from a legitimate source and not from a malicious actor. When a request is received, the server verifies the token before processing the action. Implementing CSRF tokens alongside other security measures, such as SameSite cookies, further strengthens protection against unauthorized requests.

Securing Dependencies: A Holistic Approach

Supply chain security extends beyond individual components like error reports, APIs, CORS headers, and CSRF tokens. It requires a holistic approach to managing third-party dependencies and ensuring their integrity throughout the development lifecycle.

Dependency management plays a crucial role in maintaining application security. Third-party libraries must be reviewed regularly, and updated when necessary, to address known vulnerabilities that might exist in older versions. Over time, security flaws are often discovered in widely used packages. Failing to update them can leave systems exposed. Using automated tools such as dependency scanners helps teams identify outdated or risky components and prioritize remediation based on severity and usage.

Code reviews are another important layer of defense, especially when integrating external code. Thoroughly reviewing any third-party contributions ensures they follow secure coding practices and do not introduce hidden vulnerabilities. This process should include checks for improper input handling, hardcoded credentials, and any potentially dangerous logic that could compromise the integrity of the application.

Establishing vendor trust is equally important. Before integrating third-party services or code, organizations should assess the reputation and security posture of the vendor. This includes reviewing their security policies, history of vulnerability disclosures, and responsiveness to incidents. Trustworthy vendors are more likely to maintain secure products, provide timely updates, and follow responsible disclosure practices—making them safer partners in your application ecosystem.

Guided Exercises

1. You're deploying a web application that is expected to experience unpredictable spikes in traffic. How could a load balancer help maintain performance and reliability during these spikes?

2. An organization wants to protect its application from SQL injection and cross-site scripting (XSS) attacks. Would a load balancer be sufficient, or should they use an application gateway? Why?

3. What are the risks of not applying regular security updates to servers and applications in a production environment?

4. You are deploying a microservices-based application where each service communicates over HTTP. How would packet filtering contribute to the security of this system?

Explorational Exercises

1. You're deploying a healthcare application that handles sensitive patient data. The system is composed of several microservices and uses third-party libraries for encryption and authentication. The team is considering skipping regular updates to these dependencies due to time constraints. As an engineer responsible for security, how would you argue against this decision, and what measures would you propose instead?

Summary

This lesson explored how load balancers maintain service availability under high traffic, how application gateways provide intelligent traffic control and protection, how packet filtering enforces network boundaries, and why regularly applying security updates is a non-negotiable part of maintaining system integrity. Common attacks on both IT infrastructure and applications were described, with measures that help mitigate them. Together, these elements form the backbone of resilient infrastructure—enabling organizations to support reliable applications while minimizing risk.

Answers to Guided Exercises

1. You're deploying a web application that is expected to experience unpredictable spikes in traffic. How could a load balancer help maintain performance and reliability during these spikes?

A load balancer can distribute incoming traffic across multiple servers, preventing any single server from becoming overwhelmed. During traffic spikes, this helps maintain consistent response times and reduces the risk of server crashes. Additionally, the load balancer can detect server failures and reroute traffic to healthy servers, ensuring high availability.

2. An organization wants to protect its application from SQL injection and cross-site scripting (XSS) attacks. Would a load balancer be sufficient, or should they use an application gateway? Why?

Although a load balancer distributes traffic, it typically doesn't inspect the content of requests in detail. An application gateway operates at the application layer and can inspect data payloads, such as HTTP requests, for malicious payloads. This makes an application gateway uniquely suitable for defending against application-level attacks such as XSS and SQL injection. Therefore, the organization should use an application gateway to provide deeper inspections and policy enforcement.

3. What are the risks of not applying regular security updates to servers and applications in a production environment?

Failing to apply security updates can leave known vulnerabilities unpatched, making the system an easy target for attackers. This increases the risk of exploitation through service exploits, data breaches, or system compromise. Unpatched systems are a common vector for ransomware and other malware attacks. Regular updates are critical to maintaining a secure and resilient environment.

4. You are deploying a microservices-based application where each service communicates over HTTP. How would packet filtering contribute to the security of this system?

Packet filtering helps control which traffic is allowed to enter or leave the system based on IP addresses, ports, and protocols. In a microservices environment, packet filtering can limit external access to endpoints that have been exposed externally, while preventing unauthorized internal communication. This reduces the attack surface and ensures that only expected and authorized traffic flows between services.

Answers to Explorational Exercises

1. You're deploying a healthcare application that handles sensitive patient data. The system is composed of several microservices and uses third-party libraries for encryption and authentication. The team is considering skipping regular updates to these dependencies due to time constraints. As an engineer responsible for security, how would you argue against this decision, and what measures would you propose instead?

Skipping regular updates to third-party dependencies poses a significant security risk, especially in systems handling sensitive data such as healthcare records. Vulnerabilities in outdated libraries can be exploited by attackers to gain unauthorized access, compromise data integrity, or disrupt service availability. In regulated industries, failing to patch known issues can also lead to non-compliance with security standards like HIPAA, opening an organization to expensive legal fees and lawsuits.

To mitigate this, a security advocate would propose implementing automated dependency scanning tools to monitor for known vulnerabilities (e.g., using CVE databases). Additionally, adopting a regular update schedule, combined with a staging environment for safe testing before deployment, helps balance security and stability. By prioritizing critical updates and maintaining visibility into third-party code, the system remains secure without disrupting development velocity.



**Linux
Professional
Institute**

Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	704 Security and Observability
Objective:	704.1 Cloud Native Security
Lesson:	2 of 2

Introduction

Securing communication channels and validating identities are more critical than ever. Whether you are sending sensitive data over a network, logging into a secure system, or integrating applications via APIs, robust security mechanisms are required to prevent unauthorized access, data breaches, and impersonation attacks.

At the core of this secure infrastructure lies asymmetric cryptography, a technique that relies on a pair of mathematically related keys to encrypt, decrypt, sign, and verify data. This foundational concept ensures confidentiality, integrity, and authenticity in digital communications, and underlies technologies such as HTTPS, TLS, and secure email. The lesson also introduces digital certificates based on the X.509 standard, which enable trust between parties through trusted entities known as certificate authorities (CAs).

In addition to securing data in transit, verifying who is accessing systems—and what they are allowed to do—is equally essential. That is where authentication and authorization standards come into play. This lesson explores OAuth2, which facilitates secure delegated access to APIs without exposing passwords, and OpenID Connect (OIDC), which adds a robust identity layer on top of OAuth2 to authenticate users in web and mobile applications. The lesson also describes

SAML, an XML-based standard often used in enterprise single sign-on (SSO) environments, particularly where centralized identity management is required.

Finally, this lesson offers best practices for managing user credentials, such as secure password storage and rotation, and introduces several advanced authentication technologies.

Concepts of Asymmetric Cryptography and Digital Certificates

Securing communications over networks is paramount in order to protect sensitive data from interception, tampering, or unauthorized access. At the heart of secure communication lie asymmetric cryptography and digital certificates.

Asymmetric Cryptography: Public and Private Keys

Asymmetric cryptography, also known as *public-key cryptography*, uses a pair of mathematically related keys: a *public key* and a *private key*.

The public key can be shared openly and is used for encryption or verifying digital signatures. The private key is kept secret by its owner and is used for decryption or creating digital signatures. This dual-key system ensures confidentiality, integrity, and authenticity in communications.

Asymmetric cryptography supports confidentiality by allowing data to be encrypted with the recipient's public key, and ensures that only the recipient, who holds the corresponding private key, can decrypt and access the message. This guarantees that sensitive information remains protected during transmission.

Additionally, asymmetric cryptography provides integrity and authenticity through the use of digital signatures. A sender can sign a message using their private key, and the recipient can verify the signature with the sender's public key, confirming both the origin of the message and that its contents have been safe from tampering.

Together, these properties make asymmetric encryption the foundation of many security protocols, such as SSL/TLS and PGP, which rely on *public key infrastructure* (PKI) to establish trust and protect data in motion.

X.509 Certificates: Verifying Identities

X.509 is a standard format for digital certificates used to verify the identity of entities (such as websites, servers, or individuals) in secure communications. These certificates bind a public key to an identity and are issued by trusted third parties known as *certificate authorities* (CAs).

An X.509 certificate typically contains:

- The entity's public key.
- Identifying information about the entity (e.g., domain name, organization name).
- The CA's digital signature, which validates the authenticity of the certificate.
- Expiration dates to ensure certificates remain time-bound.

When a client connects to a server, the server presents its X.509 certificate. The client verifies the certificate by checking the CA's signature and ensuring that the certificate has not expired or been revoked. This process establishes trust and enables secure communication.

Certificate Authorities (CAs): Establishing Trust

A *certificate authority* (CA) is a trusted entity responsible for issuing and validating digital certificates. CAs play a critical role in the public key infrastructure (PKI) by acting as intermediaries that vouch for the authenticity of entities.

There are two main types of CAs: *root CAs* and *intermediate CAs*. Root CAs are the top-level authorities whose certificates are trusted by default and pre-installed in operating systems and web browsers. Root CAs typically do not issue certificates directly; instead, they delegate this task to intermediate CAs. These subordinate CAs operate under the authority of the root and are responsible for issuing certificates to end entities, forming what is known as a *chain of trust*.

Transport Layer Security (TLS): Securing Communication

Transport Layer Security (TLS) is a cryptographic protocol that provides encryption, authentication, and data integrity for secure communication over a network. It is widely used to secure web traffic (HTTPS), email, and other internet protocols. It is the successor to the familiar *Secure Sockets Layer* (SSL) protocol.

TLS establishes a secure connection by using asymmetric cryptography during the initial handshake, where the server presents an X.509 certificate and it is verified by the client through a trusted CA. After verification, a symmetric session key is securely exchanged and then used for all subsequent encrypted communication. This hybrid approach ensures both strong authentication and efficient data transfer.

Transport Encryption: Protecting Data in Transit

Transport encryption refers to the encryption of data during transmission to protect it from interception or eavesdropping. Protocols such as TLS implement transport encryption by encrypting data packets before they are sent over the network and decrypting them upon receipt.

Principles of Common Standards for Authentication and Authorization

Several standards and protocols have been developed to streamline authentication (verifying who a user is) and authorization (granting access to specific resources).

Single Sign-On (SSO): Simplifying Authentication

Single sign-on (SSO) allows users to authenticate once and gain seamless access to multiple systems or applications without needing to log in repeatedly. SSO improves the user experience by reducing the need to remember multiple credentials, while simultaneously enhancing security by centralizing authentication.

SSO works by using a trusted *identity provider* (IdP) that authenticates the user and issues a token or session cookie. When the user attempts to access a connected system, the system verifies the token with the IdP instead of prompting the user to log in again. For example, logging into a corporate portal might grant access to email, file storage, and project management tools without requiring separate logins.

While SSO simplifies access, it also requires robust security measures to protect the centralized authentication process. *Multi-factor authentication* (MFA) and secure token management are often used to mitigate risks associated with SSO.

OAuth2: Enabling Token-Based Authorization

OAuth2 is a widely used protocol for token-based authorization, particularly to grant applications access to APIs or user data without exposing sensitive credentials. OAuth2 operates on the principle of granting limited, scoped access to resources through short-lived tokens.

The flow begins when a user requests access to a protected resource, such as allowing an app to view their calendar. The user is redirected to an authorization server, where they authenticate and grant consent. The server then issues an access token to the app, which can use it to access the resource securely.

OAuth2 supports different grant types—such as authorization code, implicit, and client credentials—making it suitable for a variety of contexts, including web, mobile, and server-to-server communication. Its token-based model enhances security by keeping user passwords out of third-party hands.

OpenID Connect (OIDC) extends OAuth2 by adding an identity layer, allowing applications not just to access resources, but also to verify the user's identity. OIDC uses ID tokens in the form of *JSON*

web tokens (JWTs) to share authenticated user information, such as their name and email. This is particularly useful when applications need both authentication and authorization.

For example, a website might allow users to log in using a social media account. In this case, the social media provider serves as the identity provider and issues an ID token for authentication, alongside an OAuth2 access token for authorized API access.

OIDC's integration with OAuth2 enables developers to implement secure, standards-based login systems with minimal complexity.

Security Assertion Markup Language (SAML): Exchanging Authentication Data

Security Assertion Markup Language (SAML) is an XML-based standard for exchanging authentication and authorization data between parties, typically an identity provider (IdP) and a service provider (SP). SAML is commonly used in enterprise environments to enable single sign-on (SSO) across web applications.

In a typical SAML flow, the user initiates access to a service provider, such as a corporate web application. The service provider redirects the user to an identity provider (IdP), which is responsible for authenticating the user. Upon successful authentication, the IdP generates a SAML assertion—a digitally signed XML document—that contains the user's identity and relevant attributes. This assertion is returned to the service provider, which validates its authenticity and grants access based on the information received.

Although SAML offers strong security through digital signatures and encryption, its XML-based format makes it more heavyweight than newer protocols like OpenID Connect.

Comparison of Authentication Protocols

When deciding which standard to use for identity and access management, it is important to match the protocol to the application's requirements:

SSO

is ideal for simplifying user access across multiple internal systems.

OAuth2

excels in API security and delegated access scenarios, allowing users to grant applications limited access to their data without sharing credentials.

OpenID Connect (OIDC)

builds on OAuth2 to support authentication as well, making it a versatile choice for modern

web and mobile applications.

SAML

remains a strong option for traditional enterprise systems that rely on robust, centralized authentication.

Ultimately, the right choice depends on the architecture, security needs, and integration goals of the organization.

How to Manage User Credentials and Use Advanced Authentication Technologies

Managing user credentials securely is a cornerstone of cybersecurity. Weak or improperly managed credentials are a leading cause of data breaches and unauthorized access. To address this challenge, organizations must adopt secure credential management practices and employ advanced authentication technologies such as two-factor authentication (2FA), multi-factor authentication (MFA), one-time passwords (OTP), and time-based one-time passwords (TOTP).

Managing User Credentials: Best Practices

Strong password policies are foundational for securing user accounts. Users should be required to create complex passwords that include a mix of uppercase and lowercase letters, numbers, and special characters, while avoiding easily guessable phrases or patterns.

However, simply requiring strong passwords is not enough. Secure password storage is equally important: passwords must never be stored in plain text. Instead, they should be hashed using secure algorithms like bcrypt, Argon2, or PBKDF2, with a unique “salt” added to each password before hashing to prevent attacks such as rainbow table lookups.

To further reduce the risk of credential-based attacks, organizations should encourage regular password rotation, prompting users to update their credentials periodically. In addition, locking a visitor out of the site after a defined number of failed login attempts can help prevent brute-force attacks by slowing down or blocking repeated guesses.

But while these techniques form a strong baseline, they are often not sufficient. The technologies in the following sections supplement passwords with other powerful authentication technologies.

Two-Factor Authentication (2FA): Adding an Extra Layer of Security

Two-factor authentication (2FA) requires users to provide two forms of verification to gain access to a system. Typically, this involves something the user knows (e.g., a password) and something

the user has (e.g., a one-time password generated by a device).

Typically, 2FA works as follows: A user enters their username and password. The system sends the user a code through a separate channel (SMS, email, or an authenticator app). The site also prompts the user to enter the code. Only after providing both pieces of information does the user get the desired access.

2FA significantly reduces the risk of unauthorized access, even if a user's password is compromised. An attacker would need to steal both a password and a physical device to bypass 2FA, which is far less likely than compromising a single factor. However, reliance on SMS-based codes can introduce vulnerabilities, such as SIM swapping attacks, so more secure methods such as authenticator apps are recommended.

Multi-Factor Authentication (MFA): Extending 2FA

Multi-factor authentication (MFA) builds on 2FA by requiring multiple independent factors for authentication. These factors typically fall into three categories:

Something you know

Passwords, PINs, or security questions.

Something you have

Physical devices such as smartphones, hardware tokens, or smart cards.

Something you are

Biometric data such as fingerprints, facial recognition, or voice patterns.

Relying on multiple factors provides a higher level of security, making it significantly harder for attackers to compromise all required elements.

One-Time Passwords (OTP): Secure, Single-Use Codes

One-time passwords (OTP) are passwords valid for a single login session or transaction. Unlike traditional passwords, OTPs cannot be reused, making them highly resistant to replay attacks. OTPs are commonly used in conjunction with 2FA or MFA to verify user identity.

OTPs can be delivered through various methods such as SMS or email (the server sends a code directly to the user's phone or email address), or authenticator applications (tools such as Google Authenticator or Microsoft Authenticator generate OTPs locally on the user's device).

While SMS-based OTPs are convenient, they are vulnerable to interception. Authenticator apps are a more secure alternative, as they do not rely on external communication channels.

Time-Based One-Time Passwords (TOTP): Synchronized Security

Time-based one-time passwords (TOTP) are a specific type of OTP generated based on the current time and a shared secret key. TOTP ensures synchronization between the user's device and the authentication server, because both use the same algorithm and secret to generate codes.

A shared secret key is generated when the user sets up 2FA or MFA. The authenticator app uses the secret key and the current time to calculate a code. The server performs the same calculation and compares the result with the code provided by the user. If the codes match, authentication is successful.

TOTP is widely regarded as one of the most secure methods for generating OTPs, because it eliminates the risks associated with SMS-based delivery and ensures that codes are valid for only a short period (typically 30 seconds).

Authenticator Applications: Generating TOTP Locally

Authenticator applications, such as Google Authenticator, Microsoft Authenticator, and Authy, generate TOTP locally on a user's device. These apps do not require an internet connection, making them resilient to network-based attacks. Additionally, since the shared secret key is stored securely on the device, authenticator apps are immune to the vulnerability of code interception, unlike SMS-based OTPs.

Setting up an authenticator app is a straightforward yet powerful way to enhance account security using two-factor authentication (2FA). The process usually begins by a service presenting a QR code during the 2FA setup phase. The user scans this QR code using the authenticator app, which stores a shared secret securely on the device.

Once configured, the app continuously generates a six-digit or eight-digit code based on the time-based one-time password (TOTP) algorithm, changing the code every 30 seconds. During future login attempts, after providing their password, the user is prompted to enter the current code from the app. Because the codes are time-sensitive and not transmitted over the network, authenticator apps provide a strong second layer of protection against unauthorized access. Their broad compatibility with a wide range of services, including email, social media, cloud platforms, and development tools, makes them a widely adopted and secure option for implementing 2FA.

Guided Exercises

1. During the TLS handshake, what three main steps are performed to establish a secure connection?

2. How does OpenID Connect (OIDC) extend the OAuth2 framework?

3. What is the function of root CAs and intermediate CAs in the public key infrastructure (PKI)?

4. What essential components must an X.509 certificate contain?

5. Why is TOTP-based two-factor authentication considered more secure than SMS-based codes?

Explorational Exercises

1. You are setting up a secure website and plan to use HTTPS. You decide to generate a self-signed X.509 certificate instead of getting one from a trusted CA. What problems might users face when visiting your site, and how can you resolve them?

2. You are inspecting a TLS connection and notice that the X.509 certificate presented by the server is missing the common name (CN) or subject alternative name (SAN) that matches the domain you are connecting to. What could be the result of this mismatch?

Summary

This lesson explored the foundational concepts behind secure digital communication and identity verification. It explained how asymmetric cryptography and X.509 digital certificates ensure confidentiality, integrity, and authenticity, forming the backbone of protocols such as TLS. It explained how CAs establish a chain of trust and how TLS secures data in transit using a hybrid encryption model. The lesson also introduced modern authentication and authorization standards such as OAuth2, OpenID Connect (OIDC), and SAML, highlighting their roles in securing API access, enabling single sign-on (SSO), and verifying user identities. Finally, it covered best practices for managing user credentials and advanced authentication technologies including 2FA, MFA, and OTP, helping to build a secure and resilient access management strategy.

Answers to Guided Exercises

1. During the TLS handshake, what three main steps are performed to establish a secure connection?

The secure communication process begins when the server presents its X.509 certificate to the client. This certificate contains the server's public key and is signed by a trusted CA. The client then verifies the authenticity of the certificate using the CA's public key, thus ensuring that the server is legitimate and the communication can be trusted. Once the certificate is validated, the client and server use asymmetric encryption to securely exchange a symmetric session key, which encrypts all subsequent data transmitted between them.

2. How does OpenID Connect (OIDC) extend the OAuth2 framework?

OIDC adds an identity layer to OAuth2 by issuing ID tokens that provide user identity information, enabling authentication in addition to OAuth2's authorization capabilities.

3. What is the function of root CAs and intermediate CAs in the public key infrastructure (PKI)?

Root CAs are highly trusted and form the top of the trust chain, but rarely issue certificates directly. Intermediate CAs are delegated by root CAs to issue certificates to end entities, establishing a chain of trust.

4. What essential components must an X.509 certificate contain?

An X.509 certificate must contain several essential components to function properly in secure communications. These include the subject's public key, which will be used for encryption or signature verification, and the subject's identity information, such as the domain name or organization name. The certificate must also include the issuer's digital signature, which is created using the CA's private key and allows clients to verify the certificate's authenticity using the CA's public key. Additionally, the certificate contains a serial number, validity period (start and expiration dates), and information about the algorithms used. These elements together ensure trust, integrity, and secure identification of the certificate holder.

5. Why is TOTP-based two-factor authentication considered more secure than SMS-based codes?

TOTP codes are generated locally on the user's device and are not transmitted over a network, making them less susceptible to interception or SIM swapping attacks than SMS-based codes.

Answers to Explorational Exercises

1. You are setting up a secure website and plan to use HTTPS. You decide to generate a self-signed X.509 certificate instead of getting one from a trusted CA. What problems might users face when visiting your site, and how can you resolve them?

If you use a self-signed certificate, most browsers will display a warning indicating that the connection is not secure because the certificate is not issued by a trusted CA. Users will have to manually accept the risk and bypass the warning, which can harm your site's credibility and discourage visitors. To resolve this, you should obtain a certificate from a trusted CA whose root certificate is pre-installed in major browsers and operating systems. This ensures that users see a valid, trusted connection without warnings.

2. You are inspecting a TLS connection and notice that the X.509 certificate presented by the server is missing the common name (CN) or subject alternative name (SAN) that matches the domain you are connecting to. What could be the result of this mismatch?

If the domain in the certificate doesn't match the domain the client is connecting to, the client will treat the certificate as invalid. This mismatch could trigger a browser or application error, warning the user that the site might not be secure. To avoid this issue, the certificate must include a common name (CN) or a subject alternative name (SAN) that exactly matches the domain being accessed. This allows proper hostname verification and ensures the authenticity of the server.



704.2 Prometheus Monitoring

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 704.2

Weight

6

Key knowledge areas

- Understand goals of IT operations and service provisioning, including nonfunctional properties such as availability, latency, responsiveness
- Understand and identify metrics and indicators to monitor and measure the technical functionality of a service
- Understand and identify metrics and indicators to monitor and measure the logical functionality of a service
- Understand the concepts of Prometheus, including Exporters, Pushgateway, Alertmanager and Grafana
- Understand the architecture of Prometheus
- Set up Prometheus and configure file based service discovery
- Monitor containers and microservices using Prometheus
- Use PromQL to retrieve log data
- Aggregate metrics for specific labels
- Aggregate metrics over time
- Awareness of common exporters
- Awareness of application instrumentation

- Awareness of Thanos

Partial list of the used files, terms and utilities

- Prometheus, Exporters, AlertManager, Grafana
- Label selectors
- Instant vectors and aggregate functions
- Range vectors and aggregate functions
- Node Exporter and Blackbox Exporter



**Linux
Professional
Institute**

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	704 Security and Observability
Objective:	704.2 Prometheus Monitoring
Lesson:	1 of 3

Introduction

IT operations are not just about keeping systems running—they are vital to the success, reputation, and competitiveness of any organization. Whether it is an e-commerce platform, a financial service, a healthcare system, or an educational application, users expect services to be fast, available, and reliable at all times. A single delay, failure, or security lapse can lead to lost revenue, damaged trust, and missed opportunities.

This lesson focuses on the key goals of IT operations, especially the requirements of availability, latency, and responsiveness that go beyond basic functionality. Understanding and monitoring these metrics is essential in order to anticipate and prevent problems, scale efficiently, and align technical performance with business goals. By mastering these principles, IT professionals can ensure that technology truly supports the mission of the organization—serving users with speed, reliability, and confidence.

IT Operational Goals and Service Monitoring

IT operations play a pivotal role in ensuring that services are delivered reliably and efficiently to meet both business and user expectations. The following properties are essential for delivering a

seamless user experience, maintaining customer trust, and supporting business continuity.

Availability

Availability refers to a service or system remaining accessible and fully operational over a given period of time. It is one of the most critical indicators of system reliability and user satisfaction, especially for services that support business-critical operations. If users cannot reach a service when they need it—whether due to hardware failure, network disruption, or software crashes—the credibility and effectiveness of the entire system is compromised.

High availability (HA) means designing systems to minimize downtime and maximize uptime, even in the face of failures, unexpected load spikes, or infrastructure issues. This is essential not only for maintaining uninterrupted service delivery, but also for fulfilling *service level agreements* (SLAs) and maintaining business continuity.

Redundancy is foundational to high availability. It involves deploying multiple, identical instances of critical components—such as application servers, databases, load balancers, and storage systems—so that if one instance fails, another can immediately take over. This architecture eliminates single points of failure, which are among the most common causes of service outages.

Failover mechanisms complement redundancy by automatically detecting failures and rerouting traffic or restarting services without human intervention. This ensures that users might not even notice when a failure occurs, as the system remains responsive and operational.

Proactive monitoring is another crucial element of good performance. By continuously observing the health and performance of systems, using tools such as Prometheus, IT teams can detect anomalies early and resolve them before they affect users. Proactive monitoring enables rapid incident response and supports predictive maintenance, further enhancing uptime.

To prepare for catastrophic events, organizations must also develop and regularly test *disaster recovery* (DR) plans. These plans outline procedures for restoring services following major disruptions, such as data center failures, cyberattacks, or natural disasters. A well-designed DR plan includes offsite backups, secondary data centers, and automated restoration tools to minimize recovery time and data loss.

Availability is commonly expressed as a percentage of uptime, for example:

- 99.9% availability allows about 8.76 hours of downtime per year.
- 99.99% availability reduces this to about 52 minutes per year.
- 99.999% availability known as “five nines,” translates to just over 5 minutes of downtime per year.

Reaching these levels often requires not just technical measures, but also cultural and operational maturity. The organizational requirements include well-trained personnel, clear escalation procedures, and a commitment to continuous improvement. In environments where every second of downtime can mean lost revenue or customer trust, availability is not just a metric—it is a mission-critical priority.

Latency

Latency is the amount of time it takes for a system to process a request and return a response. It is one of the most critical performance metrics in IT operations because it directly affects how fast and responsive a service feels to end users. Whether a visitor is loading a webpage, streaming a video, interacting with a mobile app, or placing a trade on a financial platform, low latency is essential for delivering a smooth and seamless user experience.

Several key factors influence latency, each contributing to potential delays if not properly optimized. One of the most prominent factors is network performance, which includes issues like network congestion, the physical distance between client and server, and inefficient routing paths. For instance, a user in Brazil accessing a service hosted in Singapore may experience longer latency simply due to the distance the data has to travel. Network latency can be further exacerbated by outdated infrastructure, misconfigured routing, or overloaded links between data centers.

In addition to network-related delays, server-side processing time is another major contributor to latency. This is the time it takes for the server to interpret the request, run any business logic, access data sources, and generate a response. Slow latency can occur when servers are overloaded, when the application code is not optimized, or when third-party APIs are slow to respond. Complex microservices architectures can also introduce additional latency, if services depend heavily on one another in a serial chain.

Another important component is *storage access latency*. This refers to the delay experienced while reading from or writing to storage systems such as databases, file systems, or object stores. Latency in this area can result from slow disk speeds, high query complexity, contention over shared storage, or inefficient indexing. Applications that perform frequent I/O operations or rely on large data transfers are particularly sensitive to this type of latency.

To reduce latency and improve responsiveness, organizations adopt a range of infrastructure and architectural strategies. These include deploying *content delivery networks* (CDNs) to bring static content physically closer to users, implementing caching mechanisms at multiple layers (e.g., browser cache, reverse proxy cache, database cache), and employing geographically distributed data centers or edge computing to reduce the distance between the user and the service. On the back end, techniques such as parallel processing, asynchronous operations, and code optimization

can significantly reduce processing delays.

Ultimately, minimizing latency is not only about raw speed: it is about creating a system that feels fast, responsive, and reliable, no matter where or how the user connects. In customer-facing applications, even milliseconds of delay can affect conversion rates, user retention, and overall satisfaction, making latency a key focus for both developers and IT operations teams.

Responsiveness

Responsiveness reflects how quickly a system reacts to user inputs or events. While related to latency, responsiveness encompasses the overall user experience, including the efficiency of the underlying infrastructure and the design of the application itself. Responsiveness can be achieved through thoughtful application design, efficient resource management, and optimized infrastructure.

Responsiveness is a critical aspect of system performance that directly affects user experience. Several key factors contribute to how quickly a system responds to user interactions. Latency plays a central role: systems that respond with minimal delay are perceived as faster and more efficient. Reducing latency, through optimized network and processing performance, enhances overall responsiveness.

Another crucial factor is *resource allocation*. Ensuring that the system has adequate CPU, memory, and disk resources allows it to process user requests without delays or contention.

In addition, application design significantly determines responsiveness; well-structured code and streamlined workflows help avoid unnecessary processing steps and eliminate bottlenecks.

Lastly, the user interface (UI) contributes by providing real-time feedback, such as loading indicators or progress bars. Even if the process takes time, such feedback reassures users that the system is working, thereby improving the perceived speed and responsiveness of the application.

Responsive systems not only improve user satisfaction but also increase productivity and engagement, making them a cornerstone of modern IT operations.

Availability, latency, and responsiveness together form the foundation of a reliable and user-friendly service. They ensure that systems are accessible, high-performing, and capable of meeting user expectations in real time.

For example, an e-commerce platform with high availability allows customers to shop without interruptions, even during peak traffic periods. A video streaming service with low latency delivers smooth playback, reducing buffering and improving viewer satisfaction. A responsive mobile app provides instant feedback, enhancing the user experience and encouraging continued

use.

Metrics for Monitoring Technical Functionality

Ensuring the technical functionality of a service requires continuous monitoring of critical metrics that reflect the health, capacity, and performance of the underlying infrastructure. These metrics provide real-time visibility into how services behave under load, reveal potential issues before they affect users, and guide infrastructure scaling and optimization. Without this visibility, even minor anomalies can evolve into major service disruptions. Monitoring is therefore not just a best practice — it is a core pillar of modern IT operations.

By tracking a well-defined set of metrics across compute, memory, storage, and network resources, organizations can ensure that systems remain efficient, resilient, and responsive, even as demand fluctuates or failures occur.

CPU Usage

CPU usage measures the percentage of available processor resources currently in use by system and application processes. It is one of the most immediate indicators of system stress or inefficiency.

High CPU usage can result from increased user demand, inefficient code, resource contention, or even a malfunctioning process running in a loop. If sustained, it can lead to sluggish system response, application crashes, and even infrastructure outages. Conversely, unusually low CPU usage might indicate that resources are underutilized, which could be a result of overprovisioning or idle workloads.

Regularly monitoring CPU usage helps IT teams:

- Detect hotspots that require scaling.
- Identify abnormal patterns that suggest performance issues or malicious activity.
- Optimize resource allocation across workloads.

Threshold-based alerts can trigger proactive interventions before the user experience degrades.

Memory Usage

Memory usage tracks the amount of physical RAM being consumed by the system and running applications. Unlike CPU, which can fluctuate rapidly, memory issues often accumulate over time and can have long-lasting impacts.

When memory usage is consistently high:

- The system might begin swapping memory to disk (using virtual memory), causing dramatic performance degradation.
- Applications might face out-of-memory (OOM) errors and terminate unexpectedly.
- Garbage-collected languages (such as Java or Python) may suffer from frequent pauses due to memory cleanup cycles.

Monitoring memory usage enables teams to:

- Detect memory leaks in long-running applications.
- Set proper memory limits and requests in containerized environments.
- Validate that autoscaling rules react appropriately to memory-intensive workloads.

Disk I/O

Disk I/O (input/output) measures how often and how quickly a system can read from or write to storage devices. It is a critical metric for understanding the performance of databases, logging systems, and any applications that depend on frequent data access.

Slow disk I/O can be caused by:

- Hardware bottlenecks (e.g., slow HDDs or saturated SSDs)
- Inefficient database queries or poor indexing
- Storage misconfiguration or excessive logging

High disk I/O can lead to slow API responses, application timeouts, or unacceptable transaction delays. Monitoring disk I/O allows teams to:

- Detect storage constraints before they cause service degradation.
- Pinpoint workloads with abnormal read/write patterns.
- Optimize database queries and caching strategies.

Network Traffic

Network traffic monitoring provides insights into the volume, type, and direction of data flowing across services, users, and external systems. It helps IT teams evaluate latency, bandwidth usage, and packet-level behavior.

Key aspects include:

- Inbound/outbound throughput detects whether a system is receiving or transmitting data too slowly.
- Packet loss and retransmissions identify underlying connectivity issues.
- Latency spikes reveal routing inefficiencies or overloaded links.

Monitoring this data ensures efficient communication between services and helps detect:

- DDoS attacks or data exfiltration attempts.
- Service misconfigurations, such as unintentional open ports or faulty ingress rules.
- Traffic spikes caused by increased user demand or batch jobs.

Tools for Monitoring

Modern IT environments require automated, scalable, and flexible tools to collect and visualize technical metrics. Among the most widely adopted solutions are Prometheus and Grafana, open source tools that work together to provide easy-to-read dashboards and charts about system performance.

Prometheus is a monitoring toolkit for time-series data. It collects metrics from targets via pull or push models and supports custom queries (through a special language called PromQL), alerting rules, and integration with Kubernetes.

Grafana is a powerful visualization platform that works seamlessly with Prometheus. It allows IT teams to create interactive dashboards, set up real-time alerts, and share metrics across teams.

Together, Prometheus and Grafana form a robust foundation for observability, allowing teams to move from reactive troubleshooting to proactive optimization.

Metrics for Monitoring Logical Functionality

While technical metrics provide insights into the health of infrastructure and systems, *logical functionality* should also be monitored to ensure that a service fulfills its intended purpose effectively. Logical functionality focuses on how well a service aligns with business objectives and user expectations. By tracking indicators such as error rates, transaction success rates, uptime/downtime, and user experience, organizations can identify issues that might not be revealed by purely technical data. This holistic approach ensures that services not only operate efficiently but also deliver value to end users.

Error Rate

One critical indicator of logical functionality is *error rates*, which measure the frequency of errors encountered during service execution. For example, an e-commerce platform with frequent checkout errors risks losing customer trust and revenue.

High error rates can indicate underlying problems in application logic, integration points, or external dependencies. Monitoring error rates allows teams to pinpoint problematic areas, prioritize fixes, and ensure smoother operations.

Transaction Success Rate

Another key metric is the *transaction success rate*, which tracks the percentage of successful transactions compared to total attempts. This is particularly important for services whose functionality relies on completing user actions, such as processing payments, submitting forms, or executing API calls. A low transaction success rate can signal inefficiencies or failures in the system's ability to handle user requests. By analyzing this metric, organizations can identify patterns, optimize workflows, and improve the overall reliability of their services.

Uptime and Downtime

Uptime and *downtime* are also essential indicators of logical functionality. Uptime measures the duration during which a service remains available, while downtime tracks periods of unavailability or outages. These metrics have a direct impact on user satisfaction and business continuity. For instance, even if a system performs well technically, prolonged downtime can lead to lost opportunities, damaged reputations, and financial losses.

Tracking uptime and downtime helps teams evaluate service reliability and implement strategies to minimize disruptions, such as redundancy and failover mechanisms.

User Experience

User experience metrics provide valuable insights into how end users perceive and interact with a service. Examples include page load times, API response times, and other measures that directly affect usability.

Slow load times or delayed responses can frustrate users, leading to higher bounce rates or abandoned transactions. By monitoring these metrics, organizations can ensure that their services are not only functional but also intuitive and responsive, fostering positive user experiences.

Bridging Technical and Logical Metrics

Monitoring these logical functionality indicators complements technical metrics by addressing the broader context of service performance. While technical metrics focus on the “how” of system operation, logical metrics emphasize the “why” and “what”—ensuring that services meet business goals and user needs.

Together, these metrics create a comprehensive view of service health, enabling organizations to address issues proactively, optimize performance, and deliver services that align with both operational and strategic objectives.

Guided Exercises

1. What does “high availability” mean in the context of IT operations, and which strategies are typically used to achieve it?

2. List and briefly explain three key factors that influence latency in a system.

3. What are two methods organizations can use to reduce latency and improve system responsiveness?

4. Why is it important to monitor CPU and memory usage, and what issues can high usage indicate?

5. How does proactive monitoring contribute to high availability and system reliability?

Explorational Exercises

1. You are monitoring a web application used by customers across different regions. Recently, users in South America started complaining about slow response times, while users in Europe reported no problems. What factors would you investigate to understand and resolve the latency issue for South American users?

2. You've been asked to improve the availability of a healthcare application that must be operational 24/7, even during maintenance or system failures. What architectural and operational strategies would you propose to achieve this level of high availability?

Summary

In modern IT environments, operations teams are tasked with maintaining the reliability, performance, and availability of digital services. The goals have expanded beyond simply keeping systems running — they now have to ensure seamless user experiences, minimize downtime, and proactively detect and resolve issues.

This lesson explored the foundational goals of IT operations and the metrics used to monitor both technical and logical aspects of system functionality. By understanding concepts such as availability, latency, and responsiveness, and by learning how to track performance indicators such as CPU usage, error rates, and user experience metrics, professionals can ensure that their infrastructure supports business continuity and aligns with user expectations.

Answers to Guided Exercises

1. What does “high availability” mean in the context of IT operations, and which strategies are typically used to achieve it?

High availability refers to a system remaining accessible and operational with minimal downtime. It is typically achieved using strategies such as redundancy (deploying multiple instances of critical components), failover mechanisms (automatically switching to backups when failures occur), proactive monitoring (to detect and resolve issues early), and disaster recovery plans.

2. List and briefly explain three key factors that influence latency in a system.

Network performance

Delays due to congestion, long physical distances, or inefficient routing.

Processing time

The time the server takes to execute business logic and generate a response.

Storage access

Delays in reading from or writing to databases or storage systems

3. What are two methods organizations can use to reduce latency and improve system responsiveness?

Content delivery networks (CDNs)

Serve content closer to the user to reduce travel distance.

Caching mechanisms

Store frequently accessed data to avoid repeated processing or database access.

4. Why is it important to monitor CPU and memory usage, and what issues can high usage indicate?

Monitoring CPU and memory helps identify resource bottlenecks and potential system instability. High CPU usage may indicate excessive processing demand or inefficient application behavior, which can lead to slow response times or service interruptions. Similarly, high memory usage can signal memory leaks or poorly optimized resource allocation, potentially resulting in swapping, performance degradation, or out-of-memory failures.

5. How does proactive monitoring contribute to high availability and system reliability?

Proactive monitoring allows IT teams to detect and address issues before they affect users. By continuously tracking system health (e.g., CPU load, memory usage, response times), teams can respond to anomalies early, prevent outages, and ensure that system reliability and uptime remain high.

Answers to Explorational Exercises

1. You are monitoring a web application used by customers across different regions. Recently, users in South America started complaining about slow response times, while users in Europe reported no problems. What factors would you investigate to understand and resolve the latency issue for South American users?

To investigate the latency issue, first check the physical distance between users and the data center. If the servers are hosted far from South America (e.g., in Europe or Asia), the long distance could increase latency. Look also at network congestion, routing paths, and whether a content delivery network (CDN) is in place to cache content closer to users. If not, implementing a CDN or adding edge servers closer to South America could reduce the latency and improve performance for those users.

2. You've been asked to improve the availability of a healthcare application that must be operational 24/7, even during maintenance or system failures. What architectural and operational strategies would you propose to achieve this level of high availability?

To achieve high availability, deploy redundant instances of all critical components (e.g., servers, databases, load balancers) to avoid single points of failure. Implement automatic failover mechanisms to redirect traffic if a service becomes unavailable.

Proactive monitoring tools should be used to detect and address problems early. Develop and test a disaster recovery plan to ensure that the system can be restored quickly in case of major disruptions. Aim for geographically distributed infrastructure to protect against regional outages.



Linux
Professional
Institute

Lesson 2

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	704 Security and Observability
Objective:	704.2 Prometheus Monitoring
Lesson:	2 of 3

Introduction

Prometheus is an open source toolkit that monitors systems and issues alerts. It was originally developed at SoundCloud in 2012. Designed for reliability and scalability, Prometheus has since become one of the most widely adopted tools for time-series monitoring in cloud-native environments.

In 2016, Prometheus became the second project to join the Cloud Native Computing Foundation (CNCF), following Kubernetes. This move emphasized its independence from any single company and helped establish a strong, community-driven governance model. Today, Prometheus is maintained by a global community of contributors and is used by organizations of all sizes to monitor infrastructure, applications, containers, and services.

Prometheus Concepts and Architecture

Prometheus collects and stores metrics as time-series data, using a multidimensional data model that represents each metric by its name and a set of key-value pairs known as *labels*. Each time series consists of:

- a metric name
- a timestamp indicating when the measurement was taken
- an optional set of labels that add dimensional context to the metric (e.g., `instance="localhost:9100"`, `job="node_exporter"`)

This approach provides a highly flexible and expressive data model, allowing metrics to be monitored and queried with precision. It enables detailed analysis and aggregation on the basis of any label or combination of labels. For example:

```
http_requests_total{method="GET", handler="/api/users"}
```

can be distinguished from

```
http_requests_total{method="POST", handler="/api/login"}
```

by the differing values in the labels, even though the two metrics share the same metric name.

This structure allows for efficient compression, retention, and retrieval, even for large volumes of high-cardinality data.

Prometheus also supports data retention policies, and older data can be written to remote, long-term storage systems if desired.

Prometheus offers a rich set of features that make it one of the most powerful monitoring tools available for modern infrastructure. The following sections describes some of the most important features.

PromQL

Prometheus comes with a built-in, powerful query language called *PromQL* (*Prometheus Query Language*), designed to exploit the multidimensional data model. It supports operations such as:

- Filtering by label
- Aggregation (e.g., `sum`, `avg`, `max`)
- Rate calculations
- Time range comparisons
- Alerting
- Plotting time-series graphs and visualizations

PromQL is used not only for ad-hoc querying but also for powering dashboards (via Grafana) and generating alert conditions.

Standalone Servers

Each Prometheus server operates independently, without the need for external databases or distributed storage systems. This design choice simplifies deployment, minimizes complexity, and isolates failure domains, ensuring that issues in one node do not affect others. By operating autonomously, the Prometheus server enhances reliability, reduces operational overhead, and makes it easier to maintain and scale monitoring systems — especially in dynamic or cloud-native environments.

The Prometheus server is the central component that performs the following critical functions:

Scraping

Pulling metrics from configured targets over HTTP.

Storage

Storing the scraped metrics locally in a highly efficient time-series database.

Querying

Accepting queries through the PromQL interface for metric analysis and alert generation.

Evaluation

Evaluating alert rules and triggering alerts when defined conditions are met.

Service Discovery

Instead of requiring manual configuration for each target, Prometheus can dynamically discover endpoints to monitor using built-in service discovery mechanisms. This enables Prometheus to adapt in real-time to changes in modern, elastic environments.

Supported service discovery mechanisms include:

- Kubernetes
- Consul
- Cloud services, including Amazon.com's EC2, Google Compute Engine, and Microsoft Azure
- File-based static configurations
- DNS-based discovery

This ensures seamless integration with dynamic infrastructure, such as containerized workloads and microservices.

Pull-Based Collection Model

Prometheus pulls its data over HTTP to scrape metrics endpoints on target systems regularly. This design simplifies debugging, offers full control over when and how data is collected, and avoids the need for agents that push data (though pushing is also supported).

Push Support via the Pushgateway

Another important component in the Prometheus ecosystem is the *Pushgateway*. Although Prometheus is built around a pull-based model to scrape metrics periodically from predefined targets, this model doesn't work well for short-lived jobs or ephemeral batch processes that might start and finish execution between two scrape intervals. As a result, important metrics from these transient tasks could be missed entirely, creating blind spots in observability.

The Pushgateway solves this problem by providing a temporary metrics storage endpoint to which short-lived jobs can push their metrics upon completion. These metrics are then stored in memory by the Pushgateway and made available for Prometheus to scrape during its next collection cycle. This approach ensures that even brief, one-time tasks contribute to the monitoring data and can be analyzed later. This mechanism are particularly useful for jobs such as:

- Automated backups
- Report generation
- Data migrations
- CI/CD pipeline tasks
- Short-lived batch jobs
- Cron tasks
- Ephemeral processes that finish before Prometheus's scrape interval

It is important to note that the Pushgateway is not intended for long-lived or continuously running applications. Because it doesn't automatically expire old data, teams must carefully manage the lifecycle of pushed metrics and clean up old data to avoid stale or misleading information. This can be handled by the pushing job itself, or by additional automation that deletes obsolete metrics as needed.

By incorporating the Pushgateway into a monitoring setup, organizations ensure that every

job—no matter how short-lived—is observed, and that its outcomes are measurable. This capability helps achieve complete observability, provides valuable insights into automated workflows, and supports auditing, performance tracking, and incident forensics.

Built-In Graphing and Dashboard Support

Prometheus supports:

- A built-in expression browser for ad-hoc queries and graphs
- Integration with Grafana for rich, interactive dashboards
- Multiple export formats for other tools and external visualizations

Exporters: Bridging the Metrics Gap

Exporters are among the foundational components of Prometheus. Exporters are specialized programs or services that collect and expose metrics from third-party systems in a format that Prometheus can scrape—typically through an HTTP endpoint exposing metrics in a plain-text format, following Prometheus’s standards.

Since many applications and services do not natively provide metrics in a Prometheus-compatible format, exporters serve as adapters that bridge the gap between these systems and the Prometheus monitoring ecosystem. Exporters are vital for enabling observability across a diverse set of infrastructure, from operating systems and hardware to databases, web servers, and custom applications.

Node Exporter

One of the most widely used exporters is the *node exporter*, which gathers low-level system metrics such as:

- CPU load
- Memory consumption
- Disk I/O
- Filesystem operations
- Network interface operations

These metrics are essential for understanding the health and performance of the host operating system, especially in environments running containers or virtual machines.

Blackbox Exporter

The *blackbox exporter* monitors remote resources by performing external checks. It operates at a higher level of abstraction than the node exporter, carrying out synthetic monitoring.

Rather than pulling internal metrics from an application, the blackbox exporter probes endpoints using such protocols as HTTP, HTTPS, TCP, ICMP (ping), and DNS. The exporter simulates external access to verify availability, latency, and response codes. This is particularly useful for monitoring public-facing APIs, websites, or internal services from the perspective of an external client, without requiring access to the target's source code or internal metrics.

Custom Exporters

Thanks to a growing ecosystem, exporters are available for nearly every commonly used service and platform, including MySQL, PostgreSQL, MongoDB, Redis, HAProxy, Nginx, and even hardware-level metrics from devices such as GPUs and baseboard management controllers (BMCs). Moreover, developers can create custom exporters to expose application-specific metrics, making Prometheus highly extensible.

Exporters standardize observability across a wide range of systems, including databases (e.g., MySQL, PostgreSQL), web servers (e.g., Nginx), cloud services, and custom applications.

By incorporating exporters, Prometheus becomes a centralized observability hub, aggregating data from across the entire stack. It enables unified monitoring, troubleshooting, alerting, and performance optimization. Understanding and deploying exporters is a key step in building a comprehensive monitoring strategy for any modern IT environment.

Alertmanager: Intelligent Alert Routing and Management

Monitoring needs actionable alerts to help operators respond in a timely manner to problems in production environments. Prometheus addresses this need through *alerting rules*, which allow operators to define threshold-based conditions that trigger alerts when systems behave abnormally. For example, rules can detect when CPU usage exceeds 90%, when a service becomes unavailable, or when a disk is nearing capacity. When the specified condition is met, Prometheus generates an alert and forwards it to a dedicated component: the *Alertmanager*.

The Alertmanager is responsible for intelligent alert handling and serves as the central hub for managing the lifecycle and delivery of alerts. Its core features include:

Deduplication

Alertmanager automatically filters out duplicate alerts to avoid overwhelming responders with redundant notifications.

Grouping

It clusters together related alerts (e.g., multiple services failing in the same region) to provide contextual clarity.

Routing

Directs alerts to the appropriate teams or communication channels based on customizable rules and escalation policies. Supported destinations include email, Slack, Microsoft Teams, PagerDuty, Opsgenie, and other incident-management platforms.

Silencing

It temporarily suppresses notifications for known issues during maintenance windows or ongoing investigations.

Inhibition rules

It prevents less important alerts from being sent when a higher-priority alert is already active.

Escalation

It ensures that unresolved alerts are escalated to higher levels of responsibility over time.

This robust alerting architecture ensures that the right people are informed at the right time, without overwhelming teams with noise. By combining custom alert definitions, flexible routing, and context-aware suppression, Prometheus and Alertmanager together help organizations maintain high availability, rapid incident response, and operational focus, reducing both alert fatigue and response time.

Installing Prometheus

Prometheus can be installed in various ways, depending on the operating system or environment in use.

Using the Tarball

The Prometheus official website recommends downloading the latest release of Prometheus for the appropriate platform (<https://prometheus.io/download/>) and then extracting the files. An example installation follows. It assumes that version 3.3.0, release candidate 0, of Prometheus has been downloaded to the local system.

```
$ tar xvzf prometheus-3.3.0-rc.0.linux-amd64.tar.gz
prometheus-3.3.0-rc.0.linux-amd64/
prometheus-3.3.0-rc.0.linux-amd64/LICENSE
```

```
prometheus-3.3.0-rc.0.linux-amd64/promtool
[...]  
$ cd prometheus-3.3.0-rc.0.linux-amd64
```

Start Prometheus in the background with the default configuration:

```
$ ./prometheus --config.file=prometheus.yml &
```

Running in Docker Container

Prometheus can also start up in a Docker environment for quick setup and testing. Use the following command to start a container with a custom configuration file:

```
$ docker run -p 9090:9090 \  
-v /path/to/prometheus.yml:/etc/prometheus/prometheus.yml \  
prom/prometheus
```

Running Through Helm

Add the `prometheus-community` Helm repository:

```
$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts  
"prometheus-community" has been added to your repositories
```

Update the local Helm repository index:

```
$ helm repo update  
Hang tight while we grab the latest from your chart repositories...  
...Successfully got an update from the "prometheus-community" chart repository  
...Successfully got an update from the "bitnami" chart repository  
Update Complete. ☐Happy Helming!☐
```

Install Prometheus:

```
$ helm install prometheus prometheus-community/prometheus  
NAME: prometheus  
LAST DEPLOYED: Fri Apr 4 21:00:12 2025  
NAMESPACE: default  
STATUS: deployed
```

REVISION: 1
TEST SUITE: None

If customization is needed, use the `-f` option with a custom `values.yaml` file:

```
$ helm install prometheus prometheus-community/prometheus -f my-values.yaml
```

The `prometheus.yaml` Configuration File

Prometheus uses a YAML configuration file named `prometheus.yaml` to control its behavior. This file contains key settings such as:

Scrape intervals

How frequently Prometheus collects metrics.

Targets

The endpoints from which Prometheus scrapes metrics.

Jobs

Logical groupings of targets (e.g., `node_exporter`, `web_app`, `database`).

A sample configuration file follows:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  # - "first.rules"
  # - "second.rules"

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'file-sd-example'
    file_sd_configs:
      - files:
          - ./file_sd_targets.json
        refresh_interval: 30s
```

The Prometheus configuration file is typically divided into three main blocks: `global`, `rule_files`, and `scrape_configs`. Each block is described in the following sections.

The `global` Block

The `global` section defines configuration settings that apply across the entire Prometheus server. Two key options commonly appear here:

`scrape_interval`

Specifies how frequently Prometheus scrapes metrics from configured targets. This value can be overridden for individual jobs, but the global setting serves as a default. In the example configuration, it is set to `15s`, meaning Prometheus scrapes metrics every 15 seconds.

`evaluation_interval`

Determines how often Prometheus evaluates its rules. Rules are used to generate alerts and to create new time series based on existing data.

The `rule_files` Block

The `rule_files` section lists rule files that Prometheus should load. These files contain recording and alerting rules written in YAML. In the default setup, this block is typically empty. If a custom rule file is needed, add it here.

The `scrape_configs` Block

The `scrape_configs` section defines the actual monitoring targets—the endpoints from which Prometheus collects metrics.

For instance, by default, Prometheus is configured to monitor itself. This is done via a job called `prometheus`, which scrapes the Prometheus server's own metrics endpoint:

```
scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'file-sd-example'
    file_sd_configs:
      - files:
          - ./file_sd_targets.json
        refresh_interval: 30s
```

Some of the notable specifications are:

job_name: 'file-sd-example'

Sets the name of the scrape job. It groups metrics and labels them under this job name in Prometheus.

file_sd_configs

Tells Prometheus to use file-based service discovery to find targets dynamically.

files

Lists the paths to JSON or YAML files that contain the list of targets. In this case, Prometheus is using a local file named `./file_sd_targets.json`.

Prometheus expects each target to expose its metrics at the `/metrics` path. So in this default setup, Prometheus scrapes data from `http://localhost:9090/metrics`.

The resulting time series data provides insights into the state and performance of the Prometheus server itself—such as CPU usage, active time series count, and scrape durations—which is crucial for debugging and capacity planning.

Starting Prometheus

To launch Prometheus, navigate to the directory containing the Prometheus binary and run the following command. The configuration file used in the command is the default `prometheus.yml`:

```
$ ./prometheus --config.file=prometheus.yml
```

Once started, Prometheus begins collecting metrics.

To use Prometheus at this point, visit its web interface at `http://localhost:9090`. So far, no scraping has occurred ([Prometheus web interface](#)).

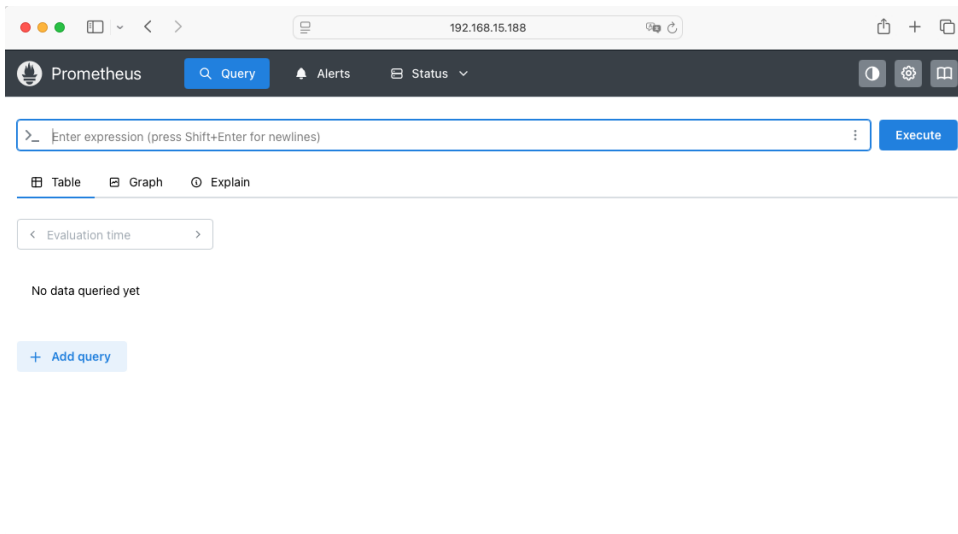


Figure 7. Prometheus web interface

Give the metrics endpoint about 30 seconds to scrape and collect initial metrics from its own HTTP endpoint, then visit the interface at <http://localhost:9090/metrics> (Metrics endpoint):

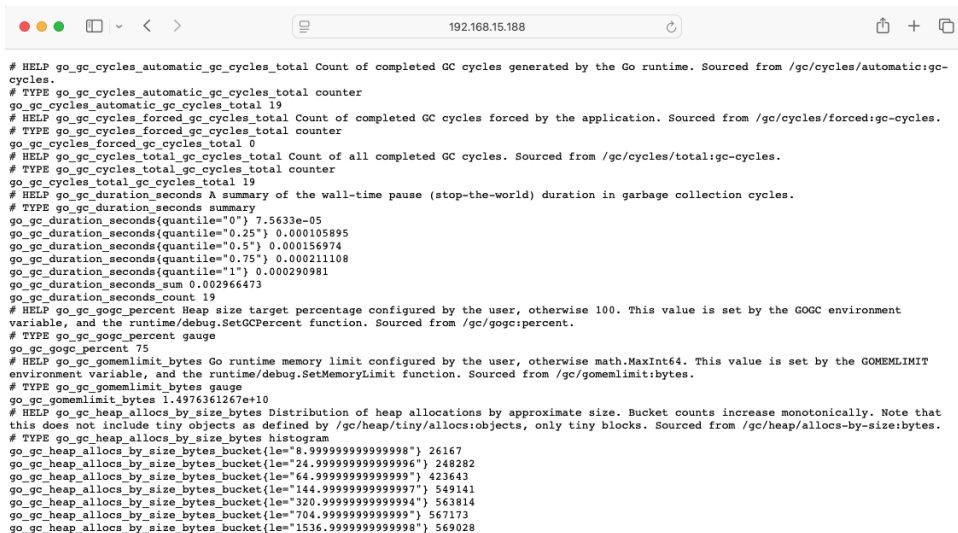


Figure 8. Metrics endpoint

Exploring Metrics With the Expression Browser

Prometheus provides a built-in expression browser for exploring collected metrics. To gain access to it, go to the <http://localhost:9090/graph> web page and switch to the “Table” view within the “Graph” tab.

Try querying a sample metric:

```
promhttp_metric_handler_requests_total
```

Query for `promhttp_metric_handler_requests_total` displays what has been collected so far on this metric, which shows the total number of `/metrics` requests served by Prometheus. Multiple time series are returned, each with different labels representing HTTP status codes and handler paths.

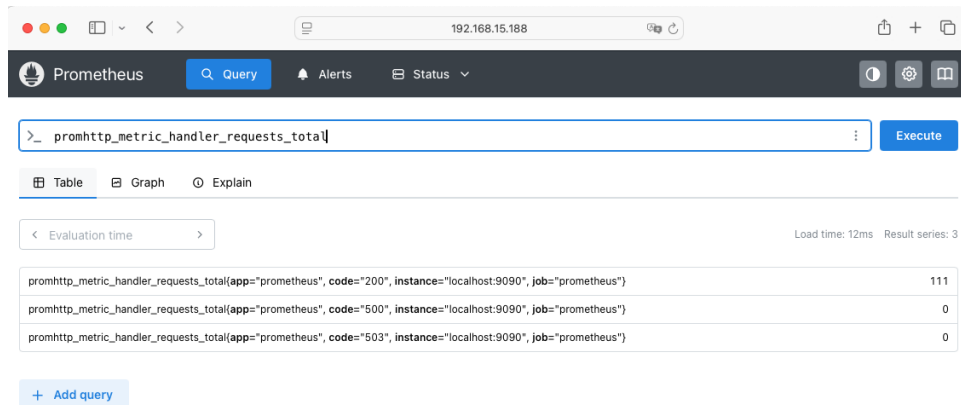


Figure 9. Query for `promhttp_metric_handler_requests_total`

To filter for only successful requests (HTTP 200), use:

```
promhttp_metric_handler_requests_total{code="200"}
```

To count how many different time series this metric includes, use:

```
count(promhttp_metric_handler_requests_total)
```

Using the Graph Interface

Prometheus can display metric data in visual form directly in its user interface. To experiment with this interface, on `http://localhost:9090/graph`, switch to the “Graph” tab.

For instance, to view a per-second rate of successful HTTP requests (HTTP responses of 200), enter:

```
rate(promhttp_metric_handler_requests_total{code="200"}[1m])
```

Results are shown in Graph for

`rate(promhttp_metric_handler_requests_total{code="200"}[1m])`.

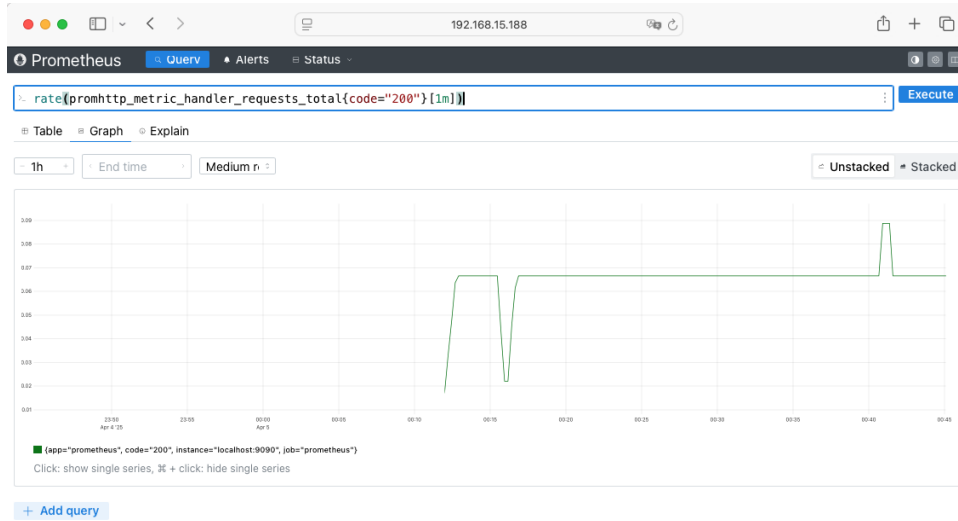


Figure 10. Graph for `rate(promhttp_metric_handler_requests_total{code="200"}[1m])`

The time range and resolution can be adjusted to explore trends and spikes interactively.

Guided Exercises

1. How does Prometheus represent its metrics, and what are labels used for?

2. What is an exporter in Prometheus? Give an example.

3. Write a PromQL query to return all metrics related to HTTP requests that resulted in HTTP 500 errors.

4. You are setting up Prometheus to monitor various services in your infrastructure. The `prometheus.yml` Prometheus configuration file allows you to define how Prometheus discovers and scrapes metrics from those services. Which section of the Prometheus configuration is used to define scrape jobs, including job names, target discovery, and scrape intervals?

5. You are configuring Prometheus to monitor a cluster of servers. In the `prometheus.yml` file, you need to specify which targets (such as nodes) are monitored. Which section of the Prometheus configuration defines which nodes are monitored?

Explorational Exercises

1. Imagine you are tasked with building a Grafana dashboard for a web application that experiences traffic spikes during marketing campaigns. What metrics would you include in the dashboard to monitor the health and performance of the application? How would you structure the panels, and what visualizations would you choose?

Summary

Prometheus is a widely adopted, open source monitoring and alerting toolkit. Its design centers on reliability, simplicity, and powerful time-series data handling. Prometheus collects data using a pull-based model, stores it in an efficient local time-series database, and allows users to query it using PromQL, a flexible and expressive query language. With native support for service discovery, exporters, and visualization through Grafana, Prometheus offers a robust solution for monitoring applications, infrastructure, and ephemeral workloads across dynamic environments.

Answers to Guided Exercises

1. How does Prometheus represent its metrics, and what are labels used for?

Prometheus stores data as time series, each consisting of a metric name, a timestamp, and key-value pairs called labels. Labels add dimensional context (e.g., instance, job, method) that allow powerful filtering and aggregation with PromQL.

2. What is an exporter in Prometheus? Give an example.

An exporter is a component that exposes metrics from external systems in a Prometheus-compatible format. For example, the node exporter provides system-level metrics such as CPU, memory, and disk usage.

3. Write a PromQL query to return all metrics related to HTTP requests that resulted in HTTP 500 errors.

This following query requests time series for the metric `promhttp_metric_handler_requests_total` with the label `code="500"`.

```
promhttp_metric_handler_requests_total{code="500"}
```

4. You are setting up Prometheus to monitor various services in your infrastructure. The `prometheus.yml` Prometheus configuration file allows you to define how Prometheus discovers and scrapes metrics from those services. Which section of the Prometheus configuration is used to define scrape jobs, including job names, target discovery, and scrape intervals?

```
scrape_configs
```

5. You are configuring Prometheus to monitor a cluster of servers. In the `prometheus.yml` file, you need to specify which targets (such as nodes) are monitored. Which section of the Prometheus configuration defines which nodes are monitored?

```
targets
```

Answers to Explorational Exercises

1. Imagine you are tasked with building a Grafana dashboard for a web application that experiences traffic spikes during marketing campaigns. What metrics would you include in the dashboard to monitor the health and performance of the application? How would you structure the panels, and what visualizations would you choose?

For a web application that experiences traffic spikes during marketing campaigns, you can create a Grafana dashboard structured around the following categories:

- Traffic and load

http_requests_total

Line graph showing requests per second, filtered by HTTP status codes.

rate(http_requests_total[1m])

Groups output by method and status.

- Latency

histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

Tracks 95th percentile latency.

Bar or line graph for latency over time.

- Application health

up

Shows which services are currently responding.

prometheus_target_interval_length_seconds

Verifies scrape health.

- Error monitoring

http_requests_total{status!~"2.."}

Limits output to errors (e.g., 4xx, 5xx) using a stacked bar chart.

- System resources — CPU and memory usage from node_exporter via:

node_cpu_seconds_total and **node_memory_MemAvailable_bytes**

- Real-time indicators — single-stat panels for:
 - Active users
 - Queue length
 - Current success rate



**Linux
Professional
Institute**

Lesson 3

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	704 Security and Observability
Objective:	704.2 Prometheus Monitoring
Lesson:	3 of 3

Introduction

In modern IT environments, driven by microservices, containers, and dynamic infrastructure, observability has become mission-critical. Tools such as Prometheus and Grafana form a robust, open-source monitoring stack that empowers organizations to collect, analyze, visualize, and act on metrics data in real time. Prometheus offers a powerful pull-based model for scraping time-series metrics. It integrates seamlessly with Kubernetes and Docker, and supports advanced querying through PromQL, the Prometheus Query Language.

Exporters such as the node exporter, blackbox exporter, and MySQL exporter help bridge the gap between Prometheus and systems that don't natively expose metrics. Together, these tools enable the monitoring of containers, Nginx servers, MySQL databases, and Kubernetes nodes with precision. By instrumenting applications, collecting metrics from various sources, and querying the metrics intelligently using PromQL, teams gain deep insight into system performance, availability, and reliability—empowering proactive incident response, capacity planning, and continuous improvement.

Grafana: Visualizing Time-Series Metrics

While Prometheus excels at collecting, storing, and querying time-series data, it lacks built-in tools for intuitive data visualization. This is where Grafana shines—as a powerful, open source analytics and monitoring platform that transforms raw Prometheus metrics into interactive, insightful dashboards.

Grafana lets teams create customizable dashboards that bring observability to life. Through seamless integration with PromQL, Grafana allows users to write precise queries and visualize the results in real time. Its rich library of visual components includes:

Line charts

tracking metrics such as CPU usage, memory consumption, and request latency over time.

Bar graphs

comparing usage across multiple nodes or services.

Single-value panels

highlighting key metrics such as uptime, error rate, or active user count.

Heatmaps

displaying the intensity or frequency of events across time intervals.

Built-in alerting

triggering visual and notification-based alerts when thresholds are breached, enabling proactive responses.

Grafana lets IT, DevOps, and Site Reliability Engineering (SRE) teams monitor system health at a glance, identify trends before they become issues, and drill down into anomalies for root cause analysis. For instance, a Grafana dashboard might reveal a sudden spike in error rates aligned with a traffic surge, providing immediate visibility into performance bottlenecks.

By pairing Prometheus's robust data collection with Grafana's dynamic visualization, organizations gain a comprehensive observability stack that supports faster troubleshooting, better performance tuning, and data-driven operational decisions.

To download Grafana, visit the Grafana download page (<https://grafana.com/grafana/download>). After installation on Ubuntu or Debian, run Grafana through:

```
$ sudo systemctl start grafana-server
$ sudo systemctl enable grafana-server
```

To access the Grafana web interface, open `http://localhost:3000` in the browser. To connect Grafana to Prometheus, visit `http://localhost:3000` and log in with username: `admin` and password: `admin` (Prometheus will advise changing the password upon first login).

Now add Prometheus as a data source:

1. In the left sidebar, click the gear icon (Configuration).
2. Click “Data Sources”.
3. Click the blue “Add data source” button.
4. Select “Prometheus” from the list.
5. In the HTTP URL field, enter `http://localhost:9090`. If Prometheus is on another machine, replace `localhost` with the server IP address or hostname.)
6. Click “Save & Test”.

A message “Data source is working” should appear in green.

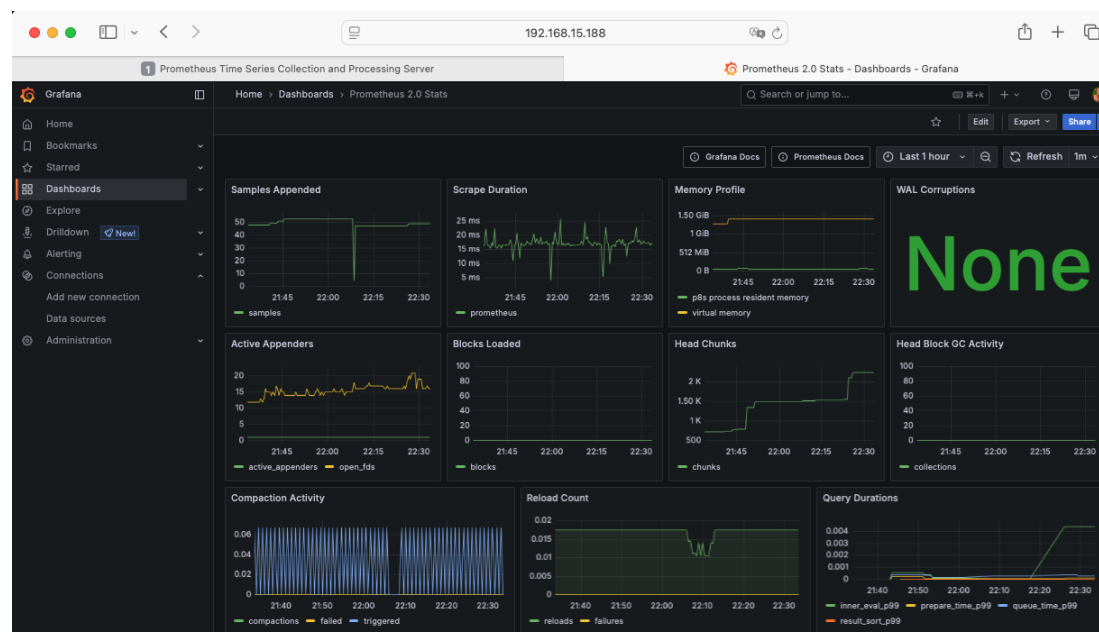


Figure 11. Sample Grafana dashboard with Prometheus 2.0 statistics

Monitoring Containers and Microservices Using Prometheus

In modern IT environments, containers and microservices have become the backbone of scalable and dynamic applications. Monitoring these components is essential to ensure their reliability, performance, and availability. Prometheus excels in this domain by integrating seamlessly with container orchestration platforms like Kubernetes, and by supporting service discovery

mechanisms. Prometheus can help monitor containers and microservices effectively, providing insights into their behavior and supporting smooth operation.

Collecting Nginx Metrics in Prometheus

To illustrate monitoring through Prometheus and Grafana, the following example creates an Nginx container with the Stub Status module enabled — a lightweight Nginx feature that exposes basic metrics such as active connections, the requests handled, and states (reading/writing/waiting). The Stub Status module is ideal for Prometheus monitoring.

Start by creating a custom `nginx.conf` file to enable `stub_status`:

```
events {}

http {
    server {
        listen 80;

        location /nginx_status {
            stub_status;
            allow all;
        }
    }
}
```

Next, run the Nginx container and bind the custom `nginx.conf` file to `/etc/nginx/nginx.conf` inside the container to apply the configuration:

```
$ docker run -d --name nginx \
  -p 8080:80 \
  -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro \
  nginx
```

Now Nginx metrics can be viewed through the web interface:

```
$ curl http://localhost:8080/nginx_status
Active connections: 1
server accepts handled requests
 4 4 4
Reading: 0 Writing: 1 Waiting: 0
```

Run a container that acts as an exporter: it fetches the raw Nginx metrics exposed at `/nginx_status` and converts them into a Prometheus-compatible format, making them available at a new HTTP endpoint that Prometheus can scrape:

```
$ docker run -d --name nginx-exporter \
  -p 9113:9113 \
  --link nginx \
  nginx/nginx-prometheus-exporter:latest \
  -nginx.scrape-uri http://nginx/nginx_status
```

Edit the `prometheus.yml` configuration file to add a new scrape job for the Nginx exporter. This tells Prometheus where to find the exporter's metrics endpoint (e.g., `localhost:9113`) so it can periodically collect and store the Nginx metrics in its time-series database.

Under the `scrape_configs` section, add a new block with `job_name: 'nginx'`. Since this is a YAML file, make sure the indentation is correct.

```
scrape_configs:
  - job_name: 'nginx'
    static_configs:
      - targets: ['localhost:9113']
```

Restart Prometheus if it is already running, or start it manually:

```
$ ./prometheus --config.file=prometheus.yml
```

Now open a browser and navigate to `http://localhost:9090`. In the Prometheus interface, use the search bar to verify that the following metrics `nginx_connections_active` and `nginx_http_requests_total` are available.

Results should look something like Prometheus nginx connections active metric.

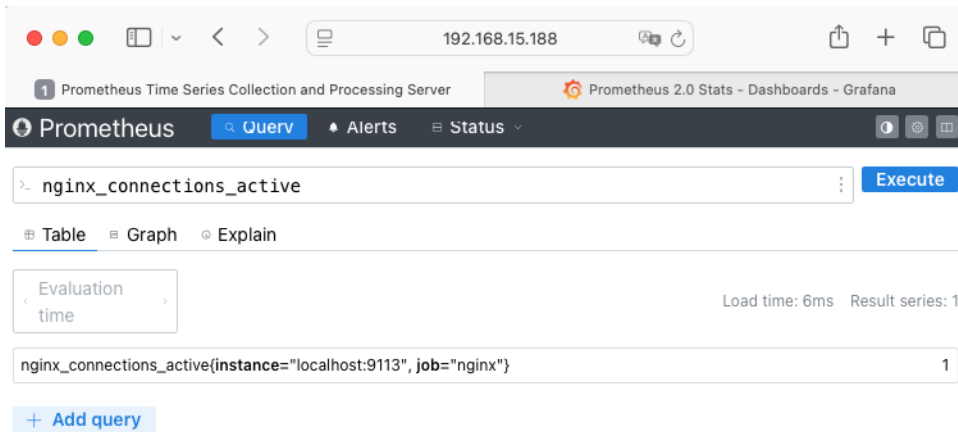


Figure 12. Prometheus `nginx_connections_active` metric

Now add the Grafana dashboard in `http://localhost:3000` as follows:

1. Add Prometheus as a data source (if is not one already).
2. Import a dashboard:
 - a. Click the “New” button, then “Import”.
 - b. Use Dashboard ID: 12708 (a popular Nginx dashboard).
 - c. Select the Prometheus data source and import it.

Nginx metrics should appear on the dashboard as shown in [Nginx connections in graphical form](#).

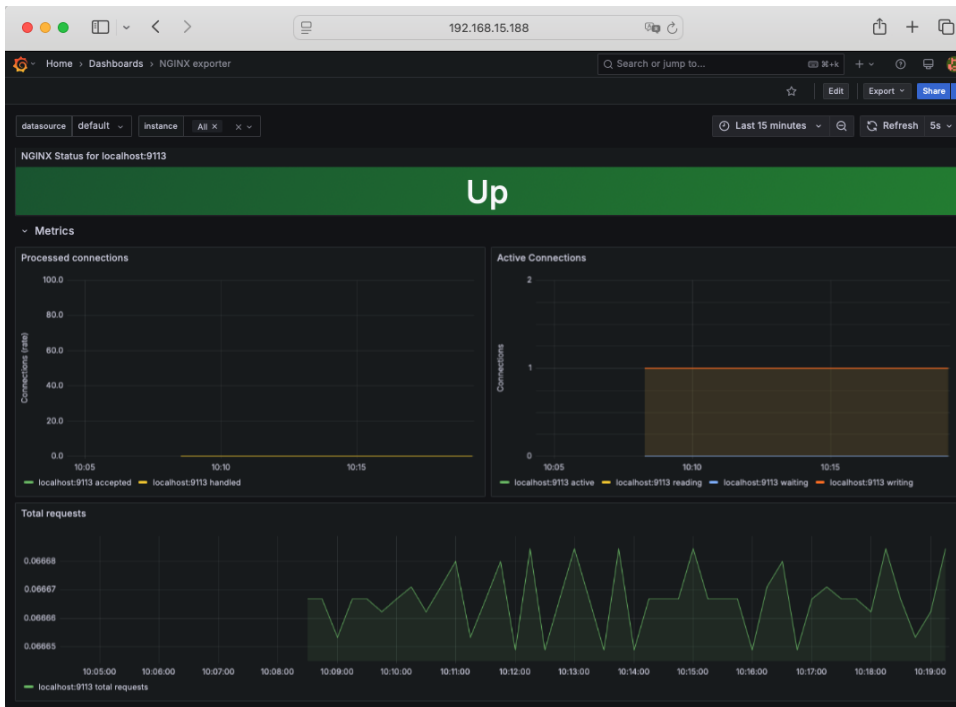


Figure 13. Nginx connections in graphical form

Prometheus and Grafana are now monitoring Nginx inside Docker.

Collecting MySQL Metrics in Prometheus

The next example deploys a MySQL database using Helm, and monitors it with Prometheus by using a MySQL exporter—a lightweight service that connects to the MySQL server, collects internal metrics (queries, connections, slow queries, etc.), and exposes them in a Prometheus-compatible format.

Start by adding the Bitnami Helm repository, if it is not already present in Helm. Bitnami provides a well-maintained MariaDB Helm chart with built-in support for Prometheus metrics.

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami
$ helm repo update
```

Then install MySQL:

```
$ helm install my-mysql bitnami/mysql \
  --set auth.rootPassword=your_password \
  --set primary.service.type=ClusterIP
NAME: my-mysql
```

Take note of the root password (`your_password`) and the service name (likely `my-mysql.default.svc.cluster.local` inside the cluster).

Now create a MySQL user for monitoring. To do so, access the MySQL pod and execute `mysql -u root -p` inside the shell. Specify user `root` with password `your_password`:

```
$ kubectl exec -it $(kubectl get pods -l app.kubernetes.io/name=mysql -o name) -- bash
Defaulted container "mysql" out of: mysql, preserve-logs-symlinks (init)
I have no name!@my-mysql-0:/$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 44
mysql>
```

Create a user `exporter` with password `exporter_password` and access to metrics:

```
mysql>
CREATE USER 'exporter'@'%' IDENTIFIED BY 'exporter_password' WITH MAX_USER_CONNECTIONS 3;
GRANT PROCESS, REPLICATION CLIENT, SELECT ON *. TO 'exporter'@'%' ;*
FLUSH PRIVILEGES;
```

The following example shows the previous SQL commands with the expected output:

```
mysql> CREATE USER 'exporter'@'%' IDENTIFIED BY 'exporter_password' WITH
MAX_USER_CONNECTIONS 3;
Query OK, 0 rows affected (0.05 sec)

mysql> GRANT PROCESS, REPLICATION CLIENT, SELECT ON *. TO 'exporter'@'%' ;*
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> exit
Bye

/$ exit
```

To deploy the `mysqld_exporter`, add the Prometheus community Helm charts:

```
$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
$ helm repo update
```

Create a file named `mysql-exporter-values.yaml` with the MySQL connection details:

```
mysql:
  db: ""
  host: "my-mysql"
  user: "exporter"
  pass: "exporter_password"
  port: 3306
  protocol: ""
```

Now install the `prometheus-mysql-exporter` Helm chart:

```
$ helm upgrade --install mysql-exporter prometheus-community/prometheus-mysql-exporter -f
mysql-exporter-values.yaml
```

It is time to add MySQL to Prometheus targets. Update `prometheus.yml`, just below `scrape_configs`, with:

```
scrape_configs:
  - job_name: 'mysql'
    static_configs:
      - targets: ['localhost:9104']
```

Visit `http://<your-prometheus>:9090` in a browser to test the monitoring. Search for metrics like:

- `mysql_global_status_threads_connected`
- `mysql_global_status_queries`
- `mysql_up`

An example of results from one Prometheus query appear in [Prometheus mysql global status queries metric](#).

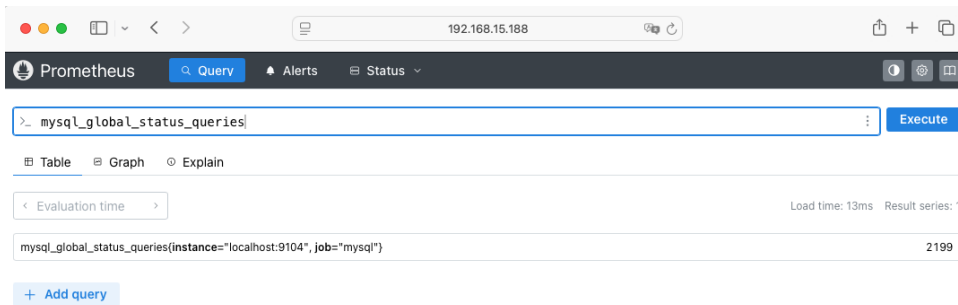


Figure 14. Prometheus `mysql_global_status_queries` metric

Now, add the Grafana dashboard in `http://localhost:3000` as follows:

1. Add Prometheus as a data source (if is not one already).
2. Import a dashboard:
 - a. Click the “New” button, then “Import.”
 - b. Use Dashboard ID: 14057 (a popular MySQL overview dashboard).
 - c. Select the Prometheus data source and import it.

MySQL metrics should now appear on the dashboard as shown in [MySQL statistics in graphical form](#).

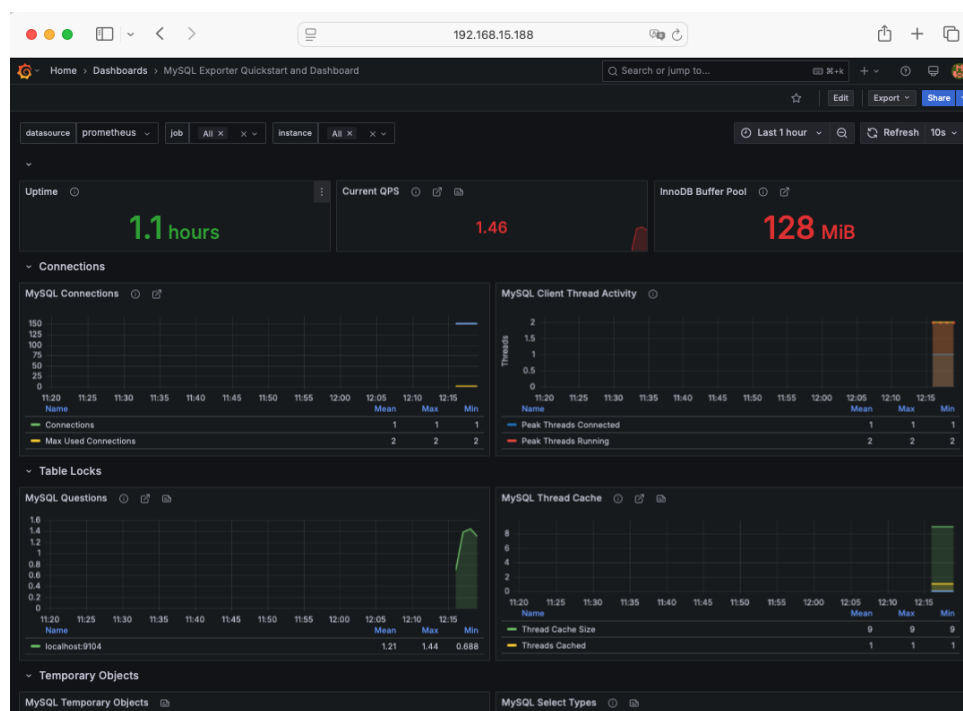


Figure 15. MySQL statistics in graphical form

Prometheus and Grafana are now monitoring MySQL inside Kubernetes.

Collecting CPU, Memory, and Disk Metrics from Kubernetes Nodes in Prometheus

The following example uses Prometheus's node exporter to show a node's behavior in Kubernetes. The example runs in a Minikube environment for convenient testing.

Create a file named `node-exporter.yaml` with the following content:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  labels:
    app: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
```

```
    app: node-exporter
spec:
  hostNetwork: true
  hostPID: true
  containers:
    - name: node-exporter
      image: prom/node-exporter:latest
      ports:
        - containerPort: 9100
          hostPort: 9100
      resources:
        limits:
          memory: 50Mi
          cpu: 100m
      securityContext:
        privileged: true
      volumeMounts:
        - name: proc
          mountPath: /host/proc
          readOnly: true
        - name: sys
          mountPath: /host/sys
          readOnly: true
        - name: root
          mountPath: /rootfs
          readOnly: true
      args:
        - '--path.procfs=/host/proc'
        - '--path.sysfs=/host/sys'
        - '--path.rootfs=/rootfs'
  volumes:
    - name: proc
      hostPath:
        path: /proc
    - name: sys
      hostPath:
        path: /sys
    - name: root
      hostPath:
        path: /
```

Apply the configuration as follows:

```
$ kubectl apply -f node.yaml
daemonset.apps/node-exporter created
```

In Minikube, the service can be accessed like this:

```
$ minikube service list
```

If the node exporter doesn't appear (because it's using `hostPort`), just forward it to a port on the host:

```
$ kubectl port-forward daemonset/node-exporter 9100 &
```

Edit the Prometheus `prometheus.yml` configuration, just below `scrape_configs`, as follows. Replace `localhost` with the node's IP address. Since we are using Minikube, the node exporter is running on the same host as Prometheus:

```
scrape_configs:
- job_name: 'node-exporter'
  static_configs:
    - targets: ['localhost:9100']
```

Restart the Prometheus server to reload the targets.

The following PromQL queries the node exporter for CPU usage:

```
rate(node_cpu_seconds_total{mode!="idle"}[5m])
```

And the following queries the node exporter for memory usage:

```
(node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
```

The following queries the node exporter for disk usage:

```
node_filesystem_avail_bytes / node_filesystem_size_bytes
```

Common Exporters in Prometheus

Prometheus exporters play a critical role in enabling the monitoring of systems, services, and applications that do not natively expose metrics in a format compatible with Prometheus. Exporters act as bridges, collecting metrics from their particular sources and exposing them in a way that Prometheus can scrape. Understanding the purpose and functionality of common exporters is essential for building a comprehensive monitoring strategy.

Some of the exporters appeared in the previous lesson; the following subsections list the most useful one from a DevOps perspective.

Node exporter

The node exporter is one of the most widely used Prometheus exporters. It collects hardware and operating system-level metrics from Linux or Unix hosts. These metrics include CPU usage, memory consumption, disk I/O, network statistics, and filesystem information.

Blackbox exporter

The blackbox exporter probes endpoints to monitor their availability and responsiveness. It supports multiple probing protocols, including HTTP, HTTPS, TCP, ICMP, and DNS.

MySQL exporter

The MySQL exporter collects metrics from MySQL databases, providing insights into database performance, query execution, and resource utilization.

PostgreSQL exporter

The PostgreSQL exporter is similar to the MySQL exporter but acts on PostgreSQL databases. It collects metrics related to query performance, replication status, and database connections.

Redis exporter

The Redis exporter collects metrics from Redis instances, providing visibility into memory usage, command execution, and client connections.

Kafka exporter

The Kafka exporter collects metrics from Apache Kafka clusters, focusing on topic partitions, consumer groups, and broker performance.

Nginx exporter

The Nginx exporter collects metrics from Nginx web servers, providing insights into request rates, response times, and error rates.

Windows exporter

The Windows exporter collects metrics from Windows-based systems, including CPU, memory, disk, and network usage.

SNMP exporter

The SNMP exporter collects metrics from devices that support the Simple Network Management Protocol (SNMP), such as routers, switches, and printers.

Advanced PromQL: Metrics Aggregation and Querying over Time and Labels

This section shows how to use PromQL to extract valuable insights from monitoring data, by writing expressive queries that reveal system behavior in real time. It also shows how to apply aggregate functions to summarize data meaningfully.

Instant Vectors

An *instant vector* is the default result type in Prometheus. It returns the latest value of a metric at the current moment.

The following example, involving requests to an Nginx server, returns the current value of the counter for all labels.

```
nginx_http_requests_total
```

Aggregating Instant Vectors

This section refers to the same metric as the previous example, but shows how to get common calculations such as sum and average.

To sum the current request counts for each job, enter:

```
sum(nginx_http_requests_total) by (job)
```

To average the current request counts for each job, enter:

```
avg(nginx_http_requests_total) by (job)
```

Range Vectors

A *range vector* is a set of values over a period of time, used for calculating trends, rates, and statistical aggregates. It uses a time selector like `[5m]` for five minutes or `[1h]` for one hour.

The following example calculates a per-second rate of HTTP requests over the most recent 5 minutes:

```
rate/nginx_http_requests_total[5m])
```

The following example uses an aggregating range vector to get average HTTP requests over the most recent 10 minutes.

```
avg_over_time/nginx_http_requests_total[10m])
```

Aggregate Metrics for Specific Labels

Prometheus can group metrics by one or more labels using the `by ()` syntax.

The following example calculates the per-second HTTP request rate per instance:

```
sum(rate/nginx_http_requests_total[1m])) by (instance)
```

Application Instrumentation

Observability is no longer a luxury—it is a necessity. *Application instrumentation* is the practice of embedding monitoring code directly into applications to expose critical metrics about performance, usage, and internal behavior. While infrastructure monitoring indicates whether a server is up or a container is running, instrumentation reveals what the code is doing. Application instrumentation can report how many requests are being handled, how long they take, or how often errors occur. This instrumentation plays an invaluable role as the bridge between the code and the monitoring system.

The first consideration in application instrumentation is choosing what should be measured. Developers should think in terms of *key performance indicators* (KPIs) relevant to their application's purpose. For instance, an e-commerce app might track cart abandonment rates, checkout errors, or API latency. A streaming platform might instrument metrics like buffer time, video start delay, or stream failures. By proactively identifying these metrics, teams can expose them through standardized endpoints (like `/metrics` in Prometheus) and gain insight into the

health and behavior of the app in real-world conditions.

Moreover, developers need to be aware of how metrics are structured and labeled. Prometheus encourages a dimensional model, where metrics include labels (key-value pairs) for contextual filtering and aggregation. Misuse of labels — such as using unique user IDs — can lead to overly high cardinality and performance bottlenecks. Therefore, teams should establish guidelines for metric naming, label usage, and documentation. A clear and consistent approach to instrumentation not only improves observability, but also makes dashboards, queries, and alerts easier to manage and interpret.

Ultimately, cultivating awareness around instrumentation leads to more reliable, maintainable, and scalable systems. Instrumentation empowers developers to detect anomalies before they escalate, respond to incidents more effectively, and continuously improve application performance.

In short, instrumentation is not just a technical practice — it is a mindset. Teams who prioritize it are better equipped to build resilient applications that meet both business goals and user expectations.

Thanos

As organizations scale their monitoring infrastructure, Prometheus alone may no longer suffice for long-term storage, high availability, or cross-cluster visibility. Thanos is an open source project that extends Prometheus by adding a global viewpoint, unlimited storage, and reliable high availability — all without compromising Prometheus’s native efficiency.

At its core, Thanos introduces a *sidecar architecture* that runs alongside each Prometheus instance. The sidecar uploads time-series data to an object storage backend such as S3, GCS, or Azure Blob Storage, enabling the long-term retention of metrics beyond Prometheus’s local storage limits. This is crucial for teams who need to retain metrics for months or even years for auditing, capacity planning, or compliance.

Further powerful capabilities provided by Thanos include horizontal scalability and global querying. Components such as the *Thanos Querier* can query across multiple Prometheus instances or regions from a single API endpoint, achieving a federated view of metrics. This is especially valuable in multicluster or hybrid cloud environments, where Prometheus servers are distributed.

Additionally, the *Thanos Store Gateway* retrieves data from object storage to serve historical queries, while the *Compactor* optimizes and down-samples metrics to reduce storage costs.

Guided Exercises

1. After running an Nginx container with `stub_status` enabled and an Nginx exporter, what configuration do you need to add to `prometheus.yml`?

2. Which PromQL query should you use in the Prometheus expression browser to view the number of active Nginx connections?

3. What query would you use to view the total number of handled HTTP requests with the label `code="200"` in Nginx?

4. Which PromQL expression gives you the per-second rate of Nginx HTTP requests over the most recent 5 minutes?

Explorational Exercises

1. You suspect that your Nginx server experiences a significant increase in traffic during lunch hours (12:00 PM to 2:00 PM). How would you use PromQL to explore this trend and validate your assumption using the metric `nginx_http_requests_total`?

2. Your users report that the website is “slow to respond” during peak hours. Which combination of Nginx metrics would you explore using Prometheus or Grafana to investigate possible bottlenecks in the response pipeline?

Summary

This lesson explored how Prometheus and Grafana provide a powerful, integrated solution for monitoring modern systems, containers, and microservices. It began by introducing Grafana's role in visualizing Prometheus time-series data through customizable dashboards and alerting. It then demonstrated how to monitor Nginx and MySQL using exporters, including setup steps with Docker and Helm, and explained how to collect system metrics from Kubernetes nodes using the node exporter. Finally, it covered the most common Prometheus exporters and illustrated how to use PromQL to query and aggregate metrics over time, empowering teams to gain actionable insights and build a robust strategy for observability.

Answers to Guided Exercises

1. After running an Nginx container with `stub status` enabled and an Nginx exporter, what configuration do you need to add to `prometheus.yml`?

```
scrape_configs:
  - job_name: 'nginx'
    static_configs:
      - targets: ['localhost:9113']
```

2. Which PromQL query should you use in the Prometheus expression browser to view the number of active Nginx connections?

```
nginx_connections_active
```

3. What query would you use to view the total number of handled HTTP requests with the label `code="200"` in Nginx?

```
nginx_http_requests_total{code="200"}
```

4. Which PromQL expression gives you the per-second rate of Nginx HTTP requests over the most recent 5 minutes?

```
rate(nginx_http_requests_total[5m])
```

Answers to Explorational Exercises

1. You suspect that your Nginx server experiences a significant increase in traffic during lunch hours (12:00 PM to 2:00 PM). How would you use PromQL to explore this trend and validate your assumption using the metric `nginx_http_requests_total`?

To explore traffic patterns, you could use a time range query in the Prometheus interface or Grafana over a historical period that includes multiple days:

```
rate(nginx_http_requests_total[5m])
```

Then, in Grafana, set the time window to several days and focus on the timeframe between 12:00 and 14:00. Look for consistent spikes in the rate during those hours. This visual analysis can confirm whether traffic increases during lunch and help in planning capacity scaling.

2. Your users report that the website is “slow to respond” during peak hours. Which combination of Nginx metrics would you explore using Prometheus or Grafana to investigate possible bottlenecks in the response pipeline?

Explore the following metrics to identify bottlenecks:

`nginx_connections_active`

To check the total number of open connections.

`nginx_connections_writing`

To see how many responses are currently being sent.

`nginx_connections_waiting`

To find how many clients are idle, possibly waiting for a response.

Analyzing these over time during peak hours may reveal patterns such as an increase in waiting connections without a corresponding increase in writing connections — which could indicate that your Nginx backend or upstream services are slow or overloaded.



704.3 Log Management and Analysis

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 704.3

Weight

2

Key knowledge areas

- Understand how application and system logging works
- Understand the architecture and features of commonly used open source logging stacks
- Awareness of syslogd and systemd-journald

Partial list of the used files, terms and utilities

- Elasticsearch and OpenSearch
- Logstash and filebeat
- Fluentd and FluentBit
- Kibana
- Loki and promtail
- Grafana
- Greylog2



Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	704 Security and Observability
Objective:	704.3 Log Management and Analysis
Lesson:	1 of 1

Introduction

Logging is a foundational component of observability. Whether the task is building cloud-native microservices, managing legacy infrastructure, or operating at scale, the ability to capture, store, search, and analyze logs is crucial for ensuring system reliability, security, and performance.

Application and system logs act as detailed chronological records of events occurring within a software system. They serve as invaluable tools for developers, system administrators, DevOps engineers, and security professionals. When properly implemented, logging enables teams to monitor application behavior, detect anomalies, troubleshoot issues, audit access and operations, and even predict failures before they affect users.

This lesson explains how logging works at both the system and application levels. The lesson explores open source logging stacks like Elasticsearch, OpenSearch, Logstash, Filebeat, Fluentd, Fluent Bit, Kibana, Loki, promtail, Grafana, and Graylog2. It also examines popular tools for log collection, parsing, visualization, and management.

Application and System Logging

Application and system logging are distinct but complementary activities. Application logging focuses on capturing events specific to the operation of software, whereas system logging is concerned with the behavior and state of the operating system and underlying infrastructure.

Application Logs

Application logs are typically generated by developers through deliberate code instrumentation, which means embedding logging statements directly into the application's source code. This can be achieved using dedicated logging libraries or frameworks that provide structured ways to output messages, such as Python's logging module, Java's Log4j or SLF4J, JavaScript's Winston, or Go's log and zap.

These logs play a critical role during both development and production. They capture key runtime information such as:

- Request identifiers (IDs) that track each user's interaction across multiple microservices.
- User sessions to correlate user behavior and state.
- Error traces or stack traces that detail where and why exceptions occurred.
- Performance metrics, like the time taken to execute a database query or render a page.
- Business events, such as the successful completion of a payment or user registration.

Logs can be emitted at various severity levels—such as `DEBUG`, `INFO`, `WARNING`, `ERROR`, and `CRITICAL`—allowing teams to filter and prioritize messages based on their importance. For instance, `DEBUG` logs might be used only during development, while `ERROR` and `CRITICAL` logs are actively monitored in production.

Structured logs, especially those in JSON format, make it easier to parse, index, and analyze logs using tools like Elasticsearch or Loki. Moreover, contextual metadata—such as timestamps, environment variables, function names, and user IDs—enables better log correlation, facilitating tasks such as distributed tracing in microservice architectures.

In essence, application logs are more than just print statements—they are a window into the application's real-time behavior, helping engineers identify bugs, monitor usage, investigate failures, ensure compliance, and improve the overall user experience.

System Logs

System logs, unlike application logs, are automatically generated by the operating system and its

core services, often without requiring any intervention from developers. These logs serve as the foundation of infrastructure-level observability, providing insight into everything that happens across the system from the kernel to user-space services.

In Linux environments, system logging is typically handled by logging daemons such as:

syslog

The traditional logging system that collects and stores messages from the kernel, system services, and applications. It can forward logs to remote servers and supports filtering by severity and facility.

rsyslog

An enhanced and more powerful version of `syslog`, offering better performance, filtering, and structured logging support.

journald

A modern logging service (part of `systemd`) that stores logs in a binary format and provides richer metadata, such as process IDs (PIDs), user IDs (UIDs), and service names. It allows for precise querying via the `journalctl` command.

System logs capture a wide range of critical operational events, including:

- User logins and logouts, along with failed or successful authentication attempts (often tracked via `/var/log/auth.log`).
- Kernel messages, such as hardware errors, system panics, or memory warnings (viewed with `dmesg` or `journalctl -k`).
- Device failures, such as disk errors, bad blocks, or network interface issues.
- Service events, including daemon startups, shutdowns, restarts, crashes, and status changes (e.g., tracked in `/var/log/syslog`, `systemctl status`, or `journalctl -u servicename`).

These logs are indispensable for system administrators, site reliability engineers (SREs), and security teams. They provide essential visibility into the inner workings of an operating system, enabling professionals to maintain system stability and security. One of the primary uses of system logs is to detect unauthorized access attempts or suspicious login behavior, which is crucial for maintaining security and compliance.

In addition, system logs are used to investigate performance degradation and identify situations where system resources, such as CPU, memory, or disk I/O, are being exhausted. This allows teams to take proactive steps before users are affected. Logs also help diagnose and respond to hardware failures or kernel-level faults by capturing low-level events such as disk errors, memory warnings,

or kernel panics.

Another important function of system logs is to monitor the health and behavior of critical services. Administrators rely on these logs to ensure that essential processes are running as expected, restarting automatically if they crash, and reporting meaningful status messages when something goes wrong. In this way, system logs act as a vital source of truth for understanding and maintaining the operational integrity of servers and infrastructure.

System logs can also be centralized and aggregated using log forwarders like Fluentd, Logstash, or Vector, making them accessible in log analysis platforms such as Elasticsearch, Splunk, or Graylog2.

In short, system logging forms the nervous system of any modern IT infrastructure, enabling visibility, security, and stability across all operating layers.

Architecture and Features of Open Source Logging Stacks

Managing logs from applications and systems is critical for monitoring performance, troubleshooting issues, and ensuring compliance. Open source logging stacks provide powerful tools to collect, process, store, and visualize logs efficiently. By understanding these tools and their roles, teams can design scalable and cost-efficient logging solutions tailored to their needs.

A typical logging stack is built around three foundational components that work together to ensure that logs are efficiently collected, stored, and analyzed.

Log Collection

The first component is *log collection*. This involves tools that automatically gather log data from various sources, such as application code, system processes, containers, and network devices. These collectors can be agents installed on the host system or integrated into the application itself. They ensure that log data, regardless of its origin, is reliably captured and forwarded to a centralized system for further handling.

Log Processing and Storage

The second critical component is *log processing and storage*. Once logs are collected, they often need to be parsed, filtered, enriched, and transformed into a structured format like JSON. This processing stage helps normalize logs from diverse sources and makes them easier to index and search later. The processed logs are then stored in a database optimized for time-series or text-based data, allowing for high-performance querying and long-term retention. Systems like Elasticsearch or Loki are commonly used in this stage because they support fast, full-text search

and indexing of large volumes of log data.

Log Visualization and Analysis

The third component is *log visualization and analysis*. This is where tools like Kibana, Grafana, or Graylog2 come into play. These interfaces allow users to query logs, build dashboards, set up alerts, and detect anomalies. They provide powerful visual representations and search capabilities that make it easier for developers, operators, and security analysts to monitor system behavior, troubleshoot issues, and gain actionable insights from raw log data. Together, these three components form a comprehensive logging pipeline that supports observability, incident response, and compliance across modern IT environments.

Open Source Logging Stacks

The following subsections examine each tool, with examples to illustrate their use.

Elasticsearch and OpenSearch

Elasticsearch and its fork, *OpenSearch*, are distributed search and analytics engines designed for storing and querying large volumes of log data. They index logs to enable fast searches and aggregations, making them ideal for log analysis at scale. Features include:

Scalability

Both Elasticsearch and OpenSearch support horizontal scaling, allowing them to handle massive amounts of log data.

Full-text search

Logs can be queried using powerful text search capabilities.

Real-time analytics

Real-time indexing ensures that logs are immediately available for analysis.

As an example use case, imagine a web application that generates thousands of HTTP request logs daily. Filebeat can collect these logs and send them to Elasticsearch for storage. Once indexed, the following JSON example filters the logs to find all requests with a **500** status code, helping to identify server errors quickly.

```
GET /logs/_search
{
  "query": {
    "match": {
```

```

        "status_code": "500"
    }
}
}

```

Logstash and Filebeat

Logstash and Filebeat are part of the Elastic stack and work together to ingest, process, and forward logs.

Logstash is a powerful and flexible data processing pipeline that handles logs and event data. It ingests logs from various sources, applies sophisticated filtering mechanisms — such as parsing unstructured text, enriching fields with metadata, or transforming formats — and then routes the processed data to a storage destination such as Elasticsearch. Its plugin-based architecture allows for extensive customization and integration, making it ideal for environments that require complex log transformations and centralized processing.

Filebeat, on the other hand, is a lightweight and efficient log shipper designed to collect log data directly from log files on disk. It is commonly deployed on endpoints or servers where applications generate logs, and it forwards this data to Logstash or Elasticsearch. Filebeat is optimized for minimal resource usage, responds to backpressure, and includes built-in modules for popular log types (like Nginx, Apache, and system logs), making it an excellent choice for distributed log collection at scale.

As an example use case, imagine a microservices architecture where each service writes logs to separate files. Filebeat can be deployed on each server to collect the logs and forward them to Logstash for processing. The following YAML example specifies the logs as input:

```

filebeat.inputs:
  - type: log
    paths:
      - /var/log/service/*.log

output.logstash:
  hosts: ["logstash-server:5044"]

```

Logstash can parse logs into structured formats before sending them to Elasticsearch:

```

input {
  beats {
    port => 5044
  }
}

```

```

    }
  }
  filter {
    grok {
      match => { "message" => "%{TIMESTAMP_ISO8601:timestamp} %{LOGLEVEL:level}
%{GREEDYDATA:message}" }
    }
  }
  output {
    elasticsearch {
      hosts => ["http://elasticsearch:9200"]
      index => "logs-%{+YYYY.MM.dd}"
    }
  }
}

```

This pipeline parses logs into the `timestamp`, `level`, and `message` fields, improving their usability for analysis.

The *Grok* filter is a powerful Logstash plugin that extracts structured fields from unstructured log data. Instead of writing raw regular expressions, Grok uses reusable patterns that are built on top of regex. These patterns match and label specific portions of a log line — such as timestamps, log levels, program names, or messages. This transformation makes raw logs easier to search, visualize, and analyze in systems like Elasticsearch and Kibana, enabling more effective monitoring and troubleshooting.

Here some examples of Grok patterns and what kinds of data they extract:

Grok pattern

```
%{SYSLOGBASE}
```

Extracted field

```
<timestamp> <hostname> <program>[<pid>]:
```

Description

Extracts in `syslog` format

Example value

```
Apr 12 14:23:01 webserver sshd[1234]: Failed password for invalid user
admin from 192.168.1.100 port 22 ssh2
```

Grok pattern

```
%{TIMESTAMP_ISO8601:timestamp}
```

Extracted field

timestamp

Description

Extracts ISO 8601 timestamp (YYYY-MM-DDTHH:MM:SS)

Example value

2025-03-31T12:34:56

Grok pattern

%{LOGLEVEL:level}

Extracted field

level

Description

Extracts log level such as INFO, DEBUG, ERROR

Example value

ERROR

Grok pattern

%{GREEDYDATA:message}

Extracted field

message

Description

Extracts the entire remainder of the log line

Example value

User failed login attempt from 192.168

Grok pattern

%{DATA:program}

Extracted field

program

Description

Extracts the name of the program generating the log

Example value

```
nginx, sshd, mysql
```

Grok pattern

```
%{SYSLOGTIMESTAMP:timestamp}
```

Extracted field

```
timestamp
```

Description

Extracts syslog-style timestamp (Mar 31 10:12:45)

Example value

```
Mar 31 10:12:45
```

Fluentd and Fluent Bit

Fluentd is a highly versatile and reliable data collector for unified logging. It supports complex workflows by collecting, filtering, transforming, and routing logs to various destinations such as Elasticsearch, Kafka, or cloud storage services. Fluentd's plugin ecosystem enables seamless integration with a wide range of data sources and outputs, making it ideal for environments that demand advanced log enrichment, routing logic, and scalability. Its ability to handle large volumes of data reliably makes it a popular choice in production-grade logging pipelines.

Fluent Bit is a lightweight, high-performance log processor and forwarder built specifically for environments with limited resources, such as Kubernetes clusters, edge computing devices, and IoT systems. Despite its small footprint, Fluent Bit offers powerful features like in-memory buffering, built-in filtering, and support for structured log formats such as JSON. It is often used as a sidecar container in Kubernetes deployments to collect and forward container logs to centralized logging systems. Fluent Bit is also fully compatible with Fluentd, allowing Fluent Bit to serve as a lightweight agent that feeds into more complex Fluentd pipelines.

As an example use case, to collect logs from a Kubernetes cluster, Fluent Bit can be deployed as a DaemonSet. The following Kubernetes configuration for this DaemonSet collects logs from containers and forwards them to Elasticsearch:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
spec:
```

```

selector:
  matchLabels:
    name: fluent-bit
template:
  metadata:
    labels:
      name: fluent-bit
  spec:
    containers:
      - name: fluent-bit
        image: fluent/fluent-bit:latest
        volumeMounts:
          - name: varlog
            mountPath: /var/log
    volumes:
      - name: varlog
        hostPath:
          path: /var/log

```

The Fluent Bit configuration handling the logs follows:

```

[INPUT]
  Name tail
  Path /var/log/containers/*.log
  Parser docker

[OUTPUT]
  Name es
  Match *
  Host elasticsearch
  Port 9200
  Index logs

```

This setup ensures that all container logs are collected and stored in Elasticsearch for analysis.

Kibana

Kibana is a visualization tool that integrates with Elasticsearch to create dashboards, charts, and reports. It provides an intuitive interface for exploring logs and metrics as well as interactive dashboards to monitor key metrics, such as error rates or request latency. It filters raw logs on specific criteria and offers sophisticated alerting mechanisms based on log patterns or thresholds.

As an example use case, Kibana can create a dashboard to monitor HTTP request logs. Such a dashboard might show:

- A pie chart showing the distribution of HTTP status codes.
- A time-series graph tracking the number of requests over time.
- A table listing recent logs with errors.

Visualizing logs in this way can quickly identify trends and anomalies in an application's behavior.

Loki and Promtail

Loki is a cost-efficient log aggregation system inspired by Prometheus. Instead of indexing the entire log content, Loki indexes only metadata (e.g., labels), reducing storage costs. *Promtail* is the agent that collects logs and sends them to Loki. Features of these tools include integration with Grafana for log exploration and visualization.

As an example use case, to monitor a Kubernetes cluster with Loki, Promtail can be deployed as a DaemonSet to collect logs and send them to Loki. An example configuration for Promtail follows:

```
server:
  http_listen_port: 9080

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: system
    static_configs:
      - targets:
          - localhost
        labels:
          job: varlogs
          __path__: /var/log/*.log
```

Once logs are stored in Loki, they can be queried in Grafana using LogQL, Loki's query language. The following query retrieves all logs containing the word "error," helping to troubleshoot issues quickly.

```
{job="varlogs"} |= "error"
```

Graylog2

Graylog2 is a centralized logging platform that combines log collection, storage, and analysis. Features include alerting, custom dashboards to visualize logs and metrics, and parsers, extracting structured data from unstructured logs.

As an example use case, to monitor security events, Graylog2 can collect logs from firewalls and intrusion detection systems. Its alerting feature can set up notifications for suspicious activities, such as repeated failed login attempts.

In combination, the tools in this lesson can create end-to-end logging pipelines tailored to the team's needs. Whether monitoring a small application or a large-scale distributed system, these open source solutions provide the foundation for effective observability and troubleshooting.

Guided Exercises

1. If Filebeat is installed and Logstash is running, what should you do to make Filebeat send logs from a new file?

2. Which configuration section in Fluent Bit defines where logs should be sent?

3. How can Fluentd be used to transform logs before forwarding them?

4. How do you define a data source in Kibana?

5. Which component in Graylog2 is responsible for ingesting log messages?

Explorational Exercises

1. A recently installed application writes log data to `/opt/app/log/info.log`. If Filebeat is already installed and set up for communication with a remote Logstash instance, what has to be done to submit the log data of the new application to Logstash?

2. You are working with Logstash and using the following Grok filter to parse log messages:

```
grok {
  match => ["message", "%{SYSLOGBASE} new node %{IPORHOST:node}"]
}
```

Given this pattern, write a complete log message that would be correctly parsed by the filter above. Your message should include:

- A standard `syslog` prefix (timestamp, hostname, and program name with optional PID).
- The literal string `new node`.
- A valid IP address or hostname to be captured as the field `node`.

Summary

Application and system logging are fundamental components for achieving observability, maintaining performance, and ensuring security. Application logs, generated through code instrumentation using libraries like Log4j or Python's logging module, provide insights into user activity, errors, and business logic execution. System logs, on the other hand, are automatically generated by the operating system and services, capturing critical events such as kernel messages, authentication attempts, and service status changes. These logs are essential for developers, system administrators, and security teams to detect anomalies, troubleshoot issues, monitor infrastructure health, and ensure operational compliance.

To manage the growing complexity and volume of logs, organizations employ open source logging stacks that include tools such as Elasticsearch, Fluentd, Logstash, Kibana, Loki, and Graylog2. These stacks typically involve three core components: log collection, log processing and storage, and log visualization. Collectors like Fluent Bit and Filebeat gather logs from various sources. Processors like Logstash or Fluentd transform and route data. Finally, visualization platforms like Kibana or Grafana allow users to explore logs and identify patterns.

Answers to Guided Exercises

1. If Filebeat is installed and Logstash is running, what should you do to make Filebeat send logs from a new file?

Add the file path to Filebeat's `paths` section in `filebeat.inputs`, and restart the Filebeat service.

2. Which configuration section in Fluent Bit defines where logs should be sent?

The `[OUTPUT]` section.

3. How can Fluentd be used to transform logs before forwarding them?

Use filters such as `record_transformer` to add or modify fields in log records.

4. How do you define a data source in Kibana?

Create an index pattern that matches the name of the Elasticsearch indices you want to explore.

5. Which component in Graylog2 is responsible for ingesting log messages?

The `Input` component.

Answers to Explorational Exercises

1. A recently installed application writes log data to `/opt/app/log/info.log`. If Filebeat is already installed and set up for communication with a remote Logstash instance, what has to be done to submit the log data of the new application to Logstash?

Filebeat uses inputs (formerly called “prospectors”) in its configuration file (`filebeat.yml`) to define which files to monitor.

To include the new log data, edit the Filebeat configuration file (usually `/etc/filebeat/filebeat.yml`). Under the appropriate `filebeat.inputs` section, add or update the `paths` setting to include the new log file:

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /var/log/*.log
    - /opt/app/log/info.log # <== Add this line
```

Restart Filebeat for the changes to take effect:

```
$ sudo systemctl restart filebeat`
```

2. You are working with Logstash and using the following Grok filter to parse log messages:

```
grok {
  match => ["message", "%{SYSLOGBASE} new node %{IPORHOST:node}"]
}
```

Given this pattern, write a complete log message that would be correctly parsed by the filter above. Your message should include:

- A standard `syslog` prefix (timestamp, hostname, and program name with optional PID).
- The literal string `new node`.
- A valid IP address or hostname to be captured as the field `node`.

Let's break down the Grok filter pattern:

```
%{SYSLOGBASE} new node %{IPORHOST:node}
```

The `%{SYSLOGBASE}` is a predefined grok pattern that matches standard syslog log headers, such as:

```
<timestamp> <hostname> <program>[<pid>]:
```

Therefore, a sample message is:

```
Apr 12 14:23:01 webserver sshd[1234]: Failed password for invalid user admin from  
192.168.1.100 port 22 ssh2
```




704.4 Tracing

Reference to LPI objectives

DevOps Tools Engineer v2.0, Exam 701, Objective 704.4

Weight

2

Key knowledge areas

- Understanding the concepts of tracing
- Understanding the concepts of OpenTelemetry
- Awareness of commonly used open source telemetry analysis tools
- Awareness of application instrumentation

Partial list of the used files, terms and utilities

- OpenTelemetry
- Spans and Distributed Traces
- Contexts, Span and Trace IDs
- Span attributes, events, links, status and kind
- Grafana Tempo
- Jaeger



**Linux
Professional
Institute**

Lesson 1

Certificate:	DevOps Tools Engineer
Version:	2.0
Topic:	704 Security and Observability
Objective:	704.4 Tracing
Lesson:	1 of 1

Introduction

In today's cloud-native and microservices-based architectures, requests often travel across multiple services, containers, and infrastructure layers. Understanding what happens during each step of this journey is critical for ensuring performance, reliability, and maintainability. In short, *tracing* is essential.

Tracing makes distributed systems more visible, by capturing detailed insights about each request's path, timing, and dependencies. It complements metrics and logs by showing how and where delays or failures occur, making it a key component of modern observability.

Concepts of Tracing

Tracing gathers data about the path of a single request as it moves through different components of a system. While metrics and logs provide snapshots and isolated messages, tracing offers an end-to-end view of how components work together to handle a request.

Tracing provides critical observability into distributed applications, allowing developers to understand how requests flow through systems, identify bottlenecks, and reduce latency.

Concepts like spans, traces, and context propagation form the building blocks of effective tracing.

Distributed Tracing and Spans

Distributed tracing follows requests across service boundaries, typically in a microservices environment. When a request spans multiple services, each service generates its own trace data. These trace fragments, called *spans*, are collected and linked together to form a complete trace of the request lifecycle.

A span captures the details of a specific operation, such as a database query, an HTTP API call, a background job, or a message sent to a queue. A span, therefore, is the fundamental building block of a distributed trace. Each span represents a single, discrete unit of work performed as part of a larger request or operation.

Each span contains metadata that makes it easier to analyze the flow of requests and identify performance bottlenecks or errors. Components of a span include:

Operation name

A high-level description of what the span represents. Examples include:

- `GET /api/users`
- Database Query: `SELECT * FROM users`

Duration

The time that elapsed between the start and end timestamps. Duration is a critical metric for identifying slow operations or performance issues.

Tags

Key-value pairs that provide additional context about the span. Tags help categorize and filter spans during analysis. They are often used to include metadata and appear in the form:

- `http.method=GET`
- `http.status_code=200`
- `db.statement=SELECT * FROM users`

Logs or events

Timestamped annotations within a span that capture specific moments during the operation. These logs provide granular details about what happened during the operation. Think of logs and events as breadcrumbs or log entries that explain what's happening inside a span at specific points in time. Examples include:

- Query execution started
- Response received from database
- Payment Initiated (marking the moment a payment process begins)

A *trace* is a collection of spans and represents the full journey of a request. It starts at the entry point (e.g., an API gateway) and continues across all the services involved in the request. A trace allows developers to see the entire flow of execution and how long each part took.

Context Propagation

To correlate spans across distributed services, tracing relies on *context propagation*. This ensures that trace IDs and span metadata are passed along with a request as it moves from one service to another.

When a span is created, it is assigned a unique *span ID* that identifies that specific unit of work. It distinguishes each operation and allows nesting (parent-child relationships).

Each span belongs to a larger trace, which is identified by a *trace ID*. These identifiers are propagated between services using protocols such as HTTP headers (e.g., *traceparent* or *b3*) and link together all spans involved in the same request.

A *parent span ID* points to the span that triggered the current span, forming a hierarchy.

Links

In tracing, a *link* connects a span to another span that is not its direct parent. In basic traces, spans are organized in a tree-like structure with parent-child relationships. But in more complex workflows, such as asynchronous jobs, message queues, or fan-in/fan-out patterns, a span could relate to multiple sources or originate from parallel operations. These non-hierarchical relationships can be preserved and represented through links.

Status

The *status* of a span provides a high-level summary of whether the operation represented by the span was successful, failed, or encountered an error. This makes it easier to filter and identify problematic operations within a trace. Common status codes include:

status.code = OK

Indicates that the operation completed successfully.

status.code = ERROR

Indicates that the operation failed due to an exception, timeout, or other error condition.

Kind

The *kind* of a span describes its role within a distributed trace. Available kinds include:

INTERNAL

Default value, used for operations that happen inside a process, like executng a function or processing data.

SERVER

Marks spans that represent incoming requests handled by a server (e.g., receiving an API call).

CLIENT

Represents outgoing requests made to another service (e.g., calling a REST endpoint or querying a database).

PRODUCER

Indicates that a message or event is being produced and sent (e.g., publishing to a message queue).

CONSUMER

Used when a service is consuming or receiving a message or event (e.g., reading from a queue or topic).

Understanding the kinds of spans helps to interpret traces more accurately, especially when analyzing the interaction between services. For instance, a **CLIENT** span followed by a **SERVER** span indicates a service-to-service call. A **PRODUCER** followed later by a **CONSUMER** are most likely part of a message-passing architecture.

Example of Distributed Tracing in Action

Distributed tracing is a powerful technique for monitoring and analyzing requests as they flow through a distributed system. The following example helps show how tracing works.

Imagine that a user visits an e-commerce platform and searches for a product. The request flows through multiple services, including the API gateway, product service, inventory service, and database. Each step in this process is represented by a **span**, and the entire journey is captured as a **trace**.

The process begins when the user sends a `GET /products/456` request to the platform. This request is the starting point of the trace and is represented by a root span.

The API gateway receives the request and processes it, creating a child span to represent the gateway's processing. This span includes metadata such as the HTTP method (GET) and the endpoint (`/products/456`).

After processing the request, the API gateway forwards it to the product service. Another child span is created to represent this operation: `Fetch Product Details`.

The product service queries the database to retrieve details about product 456. This database query is represented by yet another child span: `Database Query: SELECT * FROM products WHERE id = 456`.

After all operations are complete, the API gateway compiles the results and sends the response back to the user. The root span is marked as complete, and the entire trace is stored for analysis.

When visualized in a tool like Jaeger or Grafana Tempo, the trace appears as a hierarchical timeline of spans:

```
Trace: GET /products/456
  └─ Span: API Gateway Processing
    └─ Span: Fetch Product Details
      └─ Span: Database Query: SELECT * FROM products WHERE id = 456
    └─ Span: Check Inventory for Product 456
```

This visualization makes it easy to identify which operations took the longest or encountered errors. Each span includes its start and end times, duration, and any associated metadata.

Application Instrumentation

To enable distributed tracing, applications must be *instrumented* to generate trace data. This data is critical for observability, allowing developers, DevOps engineers, and SREs to understand how applications behave in real time, identify performance issues, and gain visibility into user interactions. Application instrumentation involves adding code or using libraries that automatically create and propagate spans while requests flow through services. Instrumentation is a fundamental requirement for building reliable, scalable, and debuggable systems.

How Instrumentation Works

Instrumentation can be done manually or in automated fashion.

Manual instrumentation requires developers to explicitly add telemetry code to their application using logging frameworks, metrics libraries, or tracing SDKs.

For example, a developer might add code to log when a database query is executed or to start a span when an HTTP request is made. This approach provides full control and customization, allowing engineers to record business-specific events and add contextual metadata.

Automatic instrumentation, on the other hand, uses middleware, agents, or instrumentation frameworks to insert telemetry code without modifying the application's source code. Tools like OpenTelemetry offer auto-instrumentation packages for popular frameworks such as Node.js Express, Java Spring, and Python's Flask. This enables teams to begin collecting traces and metrics with minimal effort.

Instrumentation libraries typically provide APIs for logging (`logger.info()`), defining metrics (`histogram.observe()`), and managing spans (`tracer.startSpan()`). When instrumented properly, applications can expose metrics such as request duration, error rate, CPU usage, and throughput. This data can then be collected by observability backends such as Prometheus, Grafana, Tempo, and Jaeger.

One of the most important developments in application instrumentation is the rise of *OpenTelemetry*, a vendor-neutral and open source framework for collecting and exporting telemetry data. OpenTelemetry supports traces, metrics, and logs, offering SDKs and auto-instrumentation packages for many programming languages.

Other widely-used instrumentation libraries include Prometheus Client Libraries (e.g., `prom-client` for Node.js, `prometheus_client` for Python), Datadog's APM SDKs, and Zipkin libraries. These tools allow developers to instrument custom events, define latency histograms, and capture high-cardinality data like user IDs or endpoint names.

OpenTelemetry

OpenTelemetry is a vendor-neutral, open source observability framework for collecting, processing, and exporting telemetry data such as traces, metrics, and logs. OpenTelemetry standardizes the way telemetry data is collected, processed, and exported across distributed systems. It originated as a unification of two projects: OpenTracing and OpenCensus.

OpenTelemetry provides:

- Instrumentation libraries for many programming languages
- APIs and SDKs to create spans and traces
- Auto-instrumentation agents for popular web frameworks and databases

- Exporters to send telemetry data to back ends like Jaeger, Zipkin, and Grafana Tempo

With OpenTelemetry, developers can instrument their code using a consistent API and export the data to multiple backends without rewriting the instrumentation logic. For example, an application using OpenTelemetry can send traces to Jaeger, metrics to Prometheus, and logs to a centralized logging system — all with minimal configuration.

The OpenTelemetry architecture is built on several core components, each playing a vital role in ensuring that observability data is consistently captured and made available for analysis, regardless of the underlying platform or technology stack.

As explained earlier, a crucial part of distributed tracing is context propagation, which ensures that trace context — such as the trace ID and span ID — is maintained as a request flows through multiple services. Context propagation enables full trace reconstruction across services, allowing teams to visualize end-to-end request flows. OpenTelemetry implements standardized propagation formats (such as W3C Trace Context) and APIs to inject and extract this metadata across communication protocols (such as HTTP, gRPC, and message queues).

The OpenTelemetry Specification

The OpenTelemetry specification defines the cross-language requirements and expectations that all OpenTelemetry implementations must follow. It ensures consistency, interoperability, and clarity across different programming languages and environments.

Rather than just defining terminology, the specification provides a detailed structure around three core components:

API

This layer has language-agnostic behavior but is implemented separately in each programming language, providing developers with a consistent and minimal surface to instrument their applications.

SDK

The SDK is responsible for efficiently managing and exporting telemetry data, while also being customizable and extensible for different environments and use cases.

Data

This part of the specification defines the *OpenTelemetry Protocol* (OTLP), which is a standardized, high-performance, and extensible protocol for transmitting traces, metrics, and logs between collectors, agents, and observability backends. It also defines semantic conventions, which provide a set of vendor-agnostic names and values for attributes (e.g.,

`http.method`, `db.system`, `net.peer.ip`). These conventions promote consistency and make telemetry data easier to query, analyze, and understand.

Collector

The *Collector* is at the heart of OpenTelemetry's data pipeline. It is a vendor-neutral agent or gateway that can receive telemetry data from multiple sources—including instrumented applications, agents, or other collectors—and apply processing logic before exporting the data to observability backends. The Collector is modular and extensible, featuring three key stages:

- Receivers ingest data from various sources.
- Processors modify, filter, or enrich data.
- Exporters send data to external systems.

This modularity enables scalable and centralized telemetry pipelines without tying applications to specific vendors.

Back ends are observability platforms that receive telemetry data for storage, querying, visualization, and alerting.

Instrumentation

Instrumentation enables applications to emit telemetry data. OpenTelemetry supports automatic instrumentation via agents or hooks, and manual instrumentation using language-specific SDKs (e.g., Java, Python, Go, JavaScript).

Developers can embed trace spans, record logs, and capture performance metrics along with contextual metadata such as request IDs, session details, and response times. This data helps with root cause analysis, performance tuning, and debugging.

Distributed Tracing with Jaeger and Grafana Tempo

As systems grow more complex and distributed—especially in microservices architectures—understanding the flow of requests across services becomes increasingly difficult. This is where distributed tracing tools like Jaeger and Grafana Tempo play a crucial role. Both are open source tracing back ends designed to ingest, store, and visualize traces. They make it easier for developers and operators to diagnose latency issues, debug failures, and monitor service interactions.

Jaeger: Open Source Distributed Tracing System

Jaeger is a distributed tracing system originally developed by Uber and now part of the Cloud Native Computing Foundation (CNCF). It allows users to track a request as it travels through a microservice-based system. This includes visualizing spans, identifying performance bottlenecks, and understanding service dependencies. Jaeger supports standards like OpenTracing and OpenTelemetry and can be used with many programming languages and frameworks.

A typical Jaeger architecture includes instrumentation in applications, often via OpenTelemetry SDKs. These applications send trace data to an OpenTelemetry Collector or directly to a Jaeger Agent. The data then flows into the Jaeger Collector, is stored in a backend like Elasticsearch or Cassandra, and can be queried via the Jaeger UI. This interface allows users to explore traces based on service name, duration, operation, or custom tags.

For example, in an e-commerce application, a user request to place an order might involve multiple services: the frontend, order processor, inventory manager, and payment gateway. Each of these generates a span, and all spans are grouped into a trace.

Using Jaeger, an engineer can trace the request end-to-end, identify which component introduced latency, and drill down to the exact cause — be it a slow database call or an error in a remote API.

Grafana Tempo: Lightweight and Scalable Tracing

Grafana Tempo is a high-scale, cost-efficient tracing backend developed by Grafana Labs. Unlike traditional tracing systems, Tempo avoids indexing the contents of trace data. Instead, it just indexes trace IDs and stores trace payloads in object storage solutions such as S3, GCS, or Azure Blob. This design drastically reduces infrastructure cost and simplifies scaling.

Tempo integrates natively with Grafana, enabling seamless correlation of logs, metrics, and traces within a single user interface. It works well with the OpenTelemetry Collector, which receives spans from instrumented services and forwards them to Tempo.

Once stored, traces can be retrieved using trace IDs found in logs (e.g., from Loki) or dashboards (e.g., from Prometheus metrics). This workflow enhances root cause analysis and supports efficient incident response.

Consider a scenario where a video streaming platform receives complaints about long load times. Using Grafana and Tempo together, a developer can first identify the affected user session through logs in Loki, extract the trace ID, and use it to pull the full trace from Tempo. The trace might reveal that the video service suffered a cache miss, which caused additional latency in fetching content from a remote database. This insight allows the developer to pinpoint the problem quickly and resolve it with minimal disruption.

Guided Exercises

1. A request goes through the following services: Frontend - Order Service - Payment Service - Third-Party Gateway. Each step is tracked using OpenTelemetry. Which OpenTelemetry concept describes this full request path, and what is the role of each service's tracking unit?
2. You have a span representing a database query operation. What are three types of metadata that might be included in this span to help with analysis?
3. A span is generated when Service A sends a request to Service B, and another span is created when Service B receives it. What are the span kinds for each?
4. In Grafana Tempo, you notice a spike in request latency. You find a trace where a span for the Payment Gateway takes much longer than others. What might be a next step in troubleshooting the problem?
5. You're using Jaeger to troubleshoot an e-commerce app. You search by service name and find a trace with multiple spans. One span has `status.code = ERROR`. What should you look at next?

Explorational Exercises

1. You are investigating a user complaint in a ride-sharing application. The user reports that confirming a ride took too long. You search for the trace using the user's request ID and observe the following in Grafana Tempo:

- The Front End span took 20 ms.
- The Ride Matching Service span took 80 ms.
- The Payment Service span took 3.2 seconds, with a logged event `Waiting for external provider`.
- The Notification Service span took 100 ms.

What can you infer from the trace about the root cause of the delay? What follow-up actions would you take?

2. You are tasked with instrumenting a microservice that handles customer orders. It includes the following actions:

- Receiving the order via HTTP.
- Validating input.
- Writing to a database.
- Sending a confirmation email.

You are using OpenTelemetry with automatic instrumentation for HTTP and database layers. Which parts of the service might still need manual instrumentation, and what specific information would you capture in custom spans?

Summary

In modern cloud-native and microservices-based architectures, distributed tracing is an essential observability practice that provides visibility into how requests propagate through complex systems. Unlike metrics and logs that offer isolated or aggregated views, tracing connects the dots across services, showing the complete journey of a request in real time.

To implement and analyze traces effectively, tools such as OpenTelemetry, Jaeger, and Grafana Tempo are widely used. OpenTelemetry provides the standard instrumentation libraries and propagation mechanisms needed to generate trace data.

Answers to Guided Exercises

1. A request goes through the following services: Front End - Order Service - Payment Service - Third-Party Gateway. Each step is tracked using OpenTelemetry. Which OpenTelemetry concept describes this full request path, and what is the role of each service's tracking unit?

The full request path is called a trace, and each individual tracking unit within a service is a span. Together, the trace is composed of spans from the Frontend, Order Service, Payment Service, and Third-Party Gateway, showing the end-to-end flow.

2. You have a span representing a database query operation. What are three types of metadata that might be included in this span to help with analysis?

The span might include:

- Tags such as `db.system=mysql` or `db.statement=SELECT * FROM users`.
 - Start and end timestamps to calculate duration.
 - Events such as `Query Started` or `Cache Miss`, to give more context during execution.
3. A span is generated when Service A sends a request to Service B, and another span is created when Service B receives it. What are the span kinds for each?

Service A's span is of kind `CLIENT`.

Service B's span is of kind `SERVER`.

This helps define directionality and communication roles in the trace.

4. In Grafana Tempo, you notice a spike in request latency. You find a trace where a span for the Payment Gateway takes much longer than others. What might be a next step in troubleshooting the problem?

Drill down into the Payment Gateway span to view its events, duration, and status. Check for tags like `error=true` or events like `Timeout Warning`. This helps identify whether the delay was caused by a network issue, external service delay, or misconfiguration.

5. You're using Jaeger to troubleshoot an e-commerce app. You search by service name and find a trace with multiple spans. One span has `status.code = ERROR`. What should you look at next?

You should examine:

- The `error` message or log associated with that span.

- Its **parent span**, to see what triggered it.
- Its **duration**, to check whether it timed out.

This analysis will help determine the cause and scope of the failure.

Answers to Explorational Exercises

1. You are investigating a user complaint in a ride-sharing application. The user reports that confirming a ride took too long. You search for the trace using the user's request ID and observe the following in Grafana Tempo:

- The Front End span took 20 ms.
- The Ride Matching Service span took 80 ms.
- The Payment Service span took 3.2 seconds, with a logged event `Waiting for external provider`.
- The Notification Service span took 100 ms.

What can you infer from the trace about the root cause of the delay? What follow-up actions would you take?

The Payment Service span shows a significant delay (3.2 seconds), especially compared to other components. The logged event `Waiting for external provider` indicates that the issue likely lies with the third-party payment provider.

Follow-up actions include:

- Check the events and status of the Payment Service span for more details.
 - Look at the external request latency metrics and retries.
 - Coordinate with the external provider to investigate their service availability.
 - Consider implementing circuit breakers or fallbacks to reduce user impact in the future.
2. You are tasked with instrumenting a microservice that handles customer orders. It includes the following actions:
- Receiving the order via HTTP
 - Validating input
 - Writing to a database
 - Sending a confirmation email

You are using OpenTelemetry with automatic instrumentation for HTTP and database layers. Which parts of the service might still need manual instrumentation, and what specific information would you capture in custom spans?

The input validation and email sending steps might not be captured by automatic instrumentation. You should add manual spans for these steps using OpenTelemetry SDKs.

In the custom spans, capture:

- Operation name such as `Validate Order`, `Send Email`.
- Tags like `validation_result=success` or `email_provider=SendGrid`.
- Events such as `Validation Started`, `Email Dispatched`.
- Error status if validation fails or the email service returns an error.

This approach ensures full trace coverage and enables better debugging and visibility into business logic.

Imprint

© 2025 by Linux Professional Institute: Learning Materials, “DevOps Tools Engineer (Version 2.0)”.

PDF generated: 2025-11-25

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0). To view a copy of this license, visit

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

While Linux Professional Institute has used good faith efforts to ensure that the information and instructions contained in this work are accurate, Linux Professional Institute disclaims all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

The LPI Learning Materials are an initiative of Linux Professional Institute (<https://lpi.org>). Learning Materials and their translations can be found at <https://learning.lpi.org>.

For questions and comments on this edition as well as on the entire project write an email to: learning@lpi.org.